

WetInk Next

Полный снимок исходного кода: P0-P32

P32: документы проектов, autosave изменённых tile, GPU-превью слоёв и проекта, файловое хранилище с миграцией старых .wetink и дублирование проектов. Снимок включает движок, UI, persistence и unit-тесты текущей рабочей копии.

Включено файлов: 155. Сгенерировано из текущей рабочей копии проекта.

1. app/build.gradle.kts

```
1  plugins {
2      id("com.android.application")
3      id("org.jetbrains.kotlin.android")
4      id("org.jetbrains.kotlin.plugin.compose")
5      id("org.jetbrains.kotlin.plugin.serialization")
6  }
7
8  android {
9      namespace = "com.wetinknext"
10     compileSdk = 36
11
12     defaultConfig {
13         applicationId = "com.wetinknext"
14         minSdk = 26
15         targetSdk = 36
16         versionCode = 1
17         versionName = "0.1.0"
18         testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
19     }
20
21     buildTypes {
22         release {
23             isMinifyEnabled = false
24         }
25     }
26
27     compileOptions {
28         sourceCompatibility = JavaVersion.VERSION_17
29         targetCompatibility = JavaVersion.VERSION_17
30     }
31
32     kotlinOptions {
33         jvmTarget = "17"
34     }
35
36     buildFeatures {
37         compose = true
38         buildConfig = true
39     }
40 }
41
42 dependencies {
43     val composeBom = platform("androidx.compose:compose-bom:2024.09.03")
44     implementation(composeBom)
45     androidTestImplementation(composeBom)
46
47     implementation("androidx.core:core-ktx:1.13.1")
48     implementation("androidx.activity:activity-compose:1.9.2")
49     implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.8.7")
50     implementation("androidx.compose.ui:ui")
51     implementation("androidx.compose.ui:ui-tooling-preview")
52     implementation("androidx.compose.material3:material3")
53     implementation("androidx.compose.material:material-icons-extended")
54     implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.6.3")
55
56     testImplementation("junit:junit:4.13.2")
57
58     debugImplementation("androidx.compose.ui:ui-tooling")
59     debugImplementation("androidx.compose.ui:ui-test-manifest")
60 }
61 }
```

2. app/src/main/AndroidManifest.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android">
3     <application
4         android:allowBackup="true"
5         android:icon="@mipmap/ic_launcher"
6         android:roundIcon="@mipmap/ic_launcher_round"
7         android:label="@string/app_name"
8         android:supportRtl="true"
9         android:theme="@style/Theme.WetInkNext">
10        <activity
11            android:name=".app.MainActivity"
12            android:exported="true">
13            <intent-filter>
14                <action android:name="android.intent.action.MAIN" />
15                <category android:name="android.intent.category.LAUNCHER" />
16            </intent-filter>
17        </activity>
18    </application>
19 </manifest>
```

3. app/src/main/java/com/wetinknext/app/AppViewModel.kt

```
1 package com.wetinknext.app
2
3 import android.content.Context
4 import android.content.SharedPreferences
5 import androidx.compose.runtime.getValue
6 import androidx.compose.runtime.mutableStateOf
7 import androidx.compose.runtime.setValue
8 import androidx.lifecycle.ViewModel
9 import androidx.lifecycle.ViewModelProvider
10 import androidx.lifecycle.viewModelScope
11 import com.wetinknext.data.project.CreateProjectRequest
12 import com.wetinknext.data.project.AutosaveCoordinator
13 import com.wetinknext.data.project.FileProjectRepository
14 import com.wetinknext.data.project.ProjectRepository
15 import com.wetinknext.data.project.ProjectSummary
16 import com.wetinknext.data.project.TrashedProjectSummary
17 import com.wetinknext.domain.document.ProjectDocument
18 import com.wetinknext.engine.undo.TileCoord
19 import com.wetinknext.engine.core.ThumbnailCapture
20 import com.wetinknext.data.project.ThumbnailEncoder
21 import com.wetinknext.engine.thumbnail.ThumbnailBuildResult
22 import com.wetinknext.domain.document.LoadedWetInkProject
23 import java.io.File
24 import kotlinx.coroutines.launch
25 import kotlinx.coroutines.Dispatchers
26 import kotlinx.coroutines.withContext
27
28 sealed interface AppRoute {
29     data object Home : AppRoute
30     data class Editor(val projectId: String) : AppRoute
31 }
32
33 data class AppUiState(
34     val projects: List<ProjectSummary> = emptyList(),
35     val trashedProjects: List<TrashedProjectSummary> = emptyList(),
36     val isLoading: Boolean = true,
37     val recoveryProject: ProjectSummary? = null,
38     val errorMessage: String? = null,
39     val isDirty: Boolean = false,
40     val isSaving: Boolean = false,
41 )
42
43 /** Owns app navigation and project catalog state, never GL resources. */
44 class AppViewModel(
45     private val repository: ProjectRepository,
46     private val session: SharedPreferences,
47 ) : ViewModel() {
48     var route: AppRoute by mutableStateOf(AppRoute.Home)
49     private set
50     var uiState: AppUiState by mutableStateOf(AppUiState())
51     private set
52     var openedDocument: ProjectDocument? by mutableStateOf(null)
53     private set
54     var openedProject: LoadedWetInkProject? by mutableStateOf(null)
55     private set
56     private val autosaveCoordinator = AutosaveCoordinator(viewModelScope)
57     private var pendingSaveDocument: ProjectDocument? = null
58     private var pendingLayerPayloads: Map<Long, ByteArray> = emptyMap()
59     private var pendingTilesAcknowledgement: (() -> Unit)? = null
60     private var pendingThumbnailWebp: ByteArray? = null
61     private var pendingLayerPreviewsWebp: Map<Long, ByteArray> = emptyMap()
62     private var latestThumbnailSequence = 0L
63
64     init {
65         refreshProjects()
66     }
67
68     fun refreshProjects() {
69         viewModelScope.launch {
70             uiState = uiState.copy(isLoading = true, errorMessage = null)
71             runCatching { repository.list() to repository.listTrash() }
72                 .onSuccess { (projects, trashedProjects) ->
73                     val interruptedId = session.getString(PREF_ACTIVE_PROJECT_ID, null)
74                     val shouldRecover = session.getBoolean(PREF_EDITOR_SESSION_OPEN, false)
75                     uiState = AppUiState(
76                         projects = projects,
77                         trashedProjects = trashedProjects,
78                         isLoading = false,
79                         recoveryProject = if (shouldRecover) {
80                             projects.firstOrNull { it.id == interruptedId }
81                         } else {
82                             null
83                         },
84                     )
85                 }
86                 .onFailure { error ->
87                     uiState = uiState.copy(isLoading = false, errorMessage = error.message)
88                 }
89         }
90     }
```

```

91
92 fun createProject(request: CreateProjectRequest) {
93     viewModelScope.launch {
94         uiState = uiState.copy(isLoading = true, errorMessage = null)
95         runCatching {
96             val summary = repository.create(request)
97             summary to repository.openForEditor(summary.id)
98         }.onSuccess { (summary, loadedProject) ->
99             openedProject = loadedProject
100             openedDocument = loadedProject.document
101             route = AppRoute.Editor(summary.id)
102             markEditorSession(summary.id)
103             uiState = uiState.copy(
104                 projects = (uiState.projects + summary).sortedByDescending(ProjectSummary::updatedAt),
105                 isLoading = false,
106                 recoveryProject = null,
107             )
108         }.onFailure { error ->
109             uiState = uiState.copy(isLoading = false, errorMessage = error.message)
110         }
111     }
112 }
113
114 fun openProject(projectId: String) {
115     viewModelScope.launch {
116         uiState = uiState.copy(isLoading = true, errorMessage = null)
117         runCatching {
118             val recovery = repository.openRecovery(projectId)
119             if (recovery != null) {
120                 repository.save(recovery.document, recovery.assets.layerTiles)
121                 repository.clearRecovery(projectId)
122                 recovery
123             } else {
124                 repository.openForEditor(projectId)
125             }
126         }
127         .onSuccess { loadedProject ->
128             openedProject = loadedProject
129             openedDocument = loadedProject.document
130             route = AppRoute.Editor(projectId)
131             markEditorSession(projectId)
132             uiState = uiState.copy(isLoading = false, recoveryProject = null)
133         }
134         .onFailure { error ->
135             uiState = uiState.copy(isLoading = false, errorMessage = error.message)
136         }
137     }
138 }
139
140 fun renameProject(projectId: String, name: String) {
141     viewModelScope.launch {
142         runCatching { repository.rename(projectId, name) }
143         .onSuccess { refreshProjects() }
144         .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
145     }
146 }
147
148 fun duplicateProject(projectId: String) {
149     viewModelScope.launch {
150         runCatching { repository.duplicate(projectId) }
151         .onSuccess { refreshProjects() }
152         .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
153     }
154 }
155
156 fun moveProjectToTrash(projectId: String) {
157     viewModelScope.launch {
158         runCatching { repository.moveToTrash(projectId) }
159         .onSuccess { refreshProjects() }
160         .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
161     }
162 }
163
164 fun restoreProject(projectId: String) {
165     viewModelScope.launch {
166         runCatching { repository.restore(projectId) }
167         .onSuccess { refreshProjects() }
168         .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
169     }
170 }
171
172 fun deleteProjectForever(projectId: String) {
173     viewModelScope.launch {
174         runCatching { repository.deleteForever(projectId) }
175         .onSuccess { refreshProjects() }
176         .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
177     }
178 }
179
180 fun dismissRecovery() {
181     session.edit().remove(PREF_ACTIVE_PROJECT_ID).putBoolean(PREF_EDITOR_SESSION_OPEN, false).apply()
182     uiState = uiState.copy(recoveryProject = null)

```

```

183     }
184
185     /** Coalesces document metadata changes from the GL thread. */
186     fun scheduleAutosave(document: ProjectDocument) {
187         if (document.id != openedDocument?.id) return
188         openedDocument = document
189         pendingSaveDocument = document
190         schedulePendingSave()
191     }
192
193     fun scheduleTileAutosave(
194         document: ProjectDocument,
195         payloads: Map<Long, ByteArray>,
196         dirty: Map<Long, Set<TileCoord>>,
197         onSave: () -> Unit,
198     ) {
199         if (document.id != openedDocument?.id) return
200         openedDocument = document
201         pendingSaveDocument = document
202         // LayerTileStore keeps tiles dirty until this acknowledgement. Therefore
203         // the newest snapshot always contains every cancelled autosave's tiles.
204         pendingLayerPayloads = payloads
205         pendingTilesAcknowledgement = onSave
206         schedulePendingSave()
207     }
208
209     fun submitThumbnail(image: ThumbnailCapture.Rgba) {
210         val document = openedDocument ?: return
211         val projectId = document.id
212         val sequence = ++latestThumbnailSequence
213         viewModelScope.launch {
214             val encoded = withContext(Dispatchers.Default) {
215                 ThumbnailEncoder.encodeWebp(image)
216             }
217             // Encoders may finish out of order. An older stroke must never
218             // replace the preview captured for a newer committed stroke.
219             if (sequence != latestThumbnailSequence) return@launch
220             if (projectId != openedDocument?.id) {
221                 // The editor may have just been closed while encoding was in
222                 // flight. Persist this last valid preview by itself instead of
223                 // dropping it and leaving the gallery with a stale card. Read
224                 // current metadata first: the captured callback must not
225                 // overwrite a later layer/property save with stale metadata.
226                 runCatching {
227                     val currentDocument = repository.open(projectId)
228                     repository.save(currentDocument, emptyMap(), encoded)
229                 }
230                 .onSuccess { refreshProjects() }
231                 .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
232                 return@launch
233             }
234             pendingThumbnailWebp = encoded
235             pendingSaveDocument = openedDocument
236             schedulePendingSave()
237         }
238     }
239
240     /**
241     * Receives files already encoded by ThumbnailBuildScheduler. Reading their
242     * bytes on Dispatchers.IO keeps Android's main thread free; autosave then
243     * publishes the previews together with document.json in one .wetink write.
244     */
245     fun submitThumbnailBuild(result: ThumbnailBuildResult) {
246         val document = openedDocument ?: return
247         val projectId = document.id
248         viewModelScope.launch {
249             val previews = withContext(Dispatchers.IO) {
250                 val project = result.projectPreview
251                 ?.takeIf(File::isFile)
252                 ?.readBytes()
253                 val layers = result.layerPreviews.mapNotNull { (layerId, file) ->
254                     file.takeIf(File::isFile)?.readBytes()?.let { layerId to it }
255                 }.toMap()
256                 project to layers
257             }
258             if (projectId != openedDocument?.id) return@launch
259
260             previews.first?.let { pendingThumbnailWebp = it }
261             if (previews.second.isNotEmpty()) {
262                 pendingLayerPreviewsWebp = pendingLayerPreviewsWebp + previews.second
263             }
264             pendingSaveDocument = openedDocument
265             schedulePendingSave()
266         }
267     }
268
269     /**
270     * One coalesced save owns document metadata, changed layer payloads and the
271     * latest canvas thumbnail. Restarting its debounce is intentional: it keeps
272     * those three representations in the same atomic .wetink publication.
273     */
274     private fun schedulePendingSave() {

```

```

275     val document = pendingSaveDocument ?: return
276     uiState = uiState.copy(isDirty = true)
277     autosaveCoordinator.schedule {
278         val recoveryPayloads = pendingLayerPayloads
279         if (recoveryPayloads.isNotEmpty()) {
280             runCatching { repository.saveRecovery(document, recoveryPayloads) }
281                 .onFailure { error -> uiState = uiState.copy(errorMessage = error.message) }
282         }
283
284         val documentToSave = pendingSaveDocument ?: return@schedule
285         val payloadsToSave = pendingLayerPayloads
286         val thumbnailToSave = pendingThumbnailWebp
287         val layerPreviewsToSave = pendingLayerPreviewsWebp
288         val acknowledge = pendingTilesAcknowledgement
289
290         uiState = uiState.copy(isSaving = true)
291         runCatching {
292             repository.save(
293                 document = documentToSave,
294                 changedLayerTiles = payloadsToSave,
295                 thumbnailWebp = thumbnailToSave,
296                 layerPreviewsWebp = layerPreviewsToSave,
297             )
298         }
299         .onSuccess {
300             if (pendingSaveDocument == documentToSave) {
301                 pendingSaveDocument = null
302                 pendingLayerPayloads = emptyMap()
303                 pendingTilesAcknowledgement = null
304                 if (thumbnailToSave != null) pendingThumbnailWebp = null
305                 if (layerPreviewsToSave.isNotEmpty()) pendingLayerPreviewsWebp = emptyMap()
306             }
307             repository.clearRecovery(documentToSave.id)
308             uiState = uiState.copy(isDirty = false, isSaving = false)
309             acknowledge?.invoke()
310         }
311         .onFailure { error ->
312             uiState = uiState.copy(isSaving = false, errorMessage = error.message)
313         }
314     }
315 }
316
317 fun closeEditor() {
318     autosaveCoordinator.cancel()
319     val document = openedDocument
320     val payloads = pendingLayerPayloads
321     val thumbnail = pendingThumbnailWebp
322     val layerPreviews = pendingLayerPreviewsWebp
323     val acknowledge = pendingTilesAcknowledgement
324     document?.let { openDocument ->
325         viewModelScope.launch {
326             uiState = uiState.copy(isSaving = true)
327             runCatching { repository.save(openDocument, payloads, thumbnail, layerPreviews) }
328                 .onSuccess {
329                     repository.clearRecovery(openDocument.id)
330                     acknowledge?.invoke()
331                     uiState = uiState.copy(isDirty = false, isSaving = false)
332                     refreshProjects()
333                 }
334                 .onFailure { error -> uiState = uiState.copy(isSaving = false, errorMessage = error.message) }
335         }
336     }
337     pendingSaveDocument = null
338     pendingLayerPayloads = emptyMap()
339     pendingTilesAcknowledgement = null
340     pendingThumbnailWebp = null
341     pendingLayerPreviewsWebp = emptyMap()
342     openedDocument = null
343     openedProject = null
344     route = AppRoute.Home
345     dismissRecovery()
346     refreshProjects()
347 }
348
349 private fun markEditorSession(projectId: String) {
350     session.edit()
351         .putString(PREF_ACTIVE_PROJECT_ID, projectId)
352         .putBoolean(PREF_EDITOR_SESSION_OPEN, true)
353         .apply()
354 }
355
356 companion object {
357     private const val PREFS_NAME = "wetinknext_project_session"
358     private const val PREF_ACTIVE_PROJECT_ID = "active_project_id"
359     private const val PREF_EDITOR_SESSION_OPEN = "editor_session_open"
360 }
361
362 class Factory(context: Context) : ViewModelProvider.Factory {
363     private val appContext = context.applicationContext
364
365     @Suppress("UNCHECKED_CAST")
366     override fun <T : ViewModel> create(modelClass: Class<T>): T {

```

```
367         check(modelClass.isAssignableFrom(AppViewModel::class.java))
368     return AppViewModel(
369         repository = FileProjectRepository(File(appContext.filesDir, "projects")),
370         session = appContext.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE),
371     ) as T
372 }
373 }
374 }
```

4. app/src/main/java/com/wetinknext/app/EditorScreen.kt

```
1 package com.wetinknext.app
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.border
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.lazy.grid.GridCells
8 import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
9 import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.automirrored.filled.ArrowBack
11 import androidx.compose.runtime.*
12 import androidx.compose.ui.Alignment
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.graphics.Color
15 import androidx.compose.ui.graphics.toArgb
16 import androidx.compose.ui.draw.clip
17 import androidx.compose.foundation.shape.RoundedCornerShape
18 import androidx.compose.ui.unit.dp
19 import androidx.compose.ui.viewinterop.AndroidView
20 import androidx.compose.ui.platform.LocalContext
21 import com.wetinknext.domain.document.ProjectDocument
22 import com.wetinknext.engine.brush.BrushPreset
23 import com.wetinknext.engine.brush.BrushLibrary
24 import com.wetinknext.engine.core.EditorUiState
25 import com.wetinknext.engine.core.PaintSurfaceView
26 import com.wetinknext.ui.animation.AnimationTimelineToolbar
27 import com.wetinknext.ui.color.ColorPanel
28 import com.wetinknext.ui.color.GlesColorState
29 import com.wetinknext.ui.components.*
30 import com.wetinknext.ui.state.LayerState
31 import com.wetinknext.ui.theme.WetInkTheme
32 import com.wetinknext.ui.theme.AppTheme
33 import com.wetinknext.ui.theme.AppThemes
34 import com.wetinknext.ui.theme.ThemeController
35 import com.wetinknext.ui.theme.rememberThemeController
36
37 private enum class EditorPanel {
38     NONE,
39     BRUSH,
40     LAYERS,
41     COLOR,
42     SELECTION,
43     TRANSFORM,
44     ADJUSTMENTS,
45     ANIMATION,
46     SETTINGS,
47 }
48
49 @Composable
50 fun EditorScreen(
51     projectId: String,
52     document: ProjectDocument,
53     layerTiles: Map<Long, ByteArray> = emptyMap(),
54     layerPreviews: Map<Long, ByteArray> = emptyMap(),
55     saveStatus: String = "Сохранено",
56     onDocumentChanged: (ProjectDocument) -> Unit = {},
57     onDirtyLayerTiles: (ProjectDocument, Map<Long, ByteArray>, Map<Long, Set<com.wetinknext.engine.undo.TileCoord>>, () ->
Unit) -> Unit = { _, _, _, _ -> },
58     onThumbnailCaptured: (com.wetinknext.engine.core.ThumbnailCapture.Rgba) -> Unit = {},
59     onThumbnailBuildSaved: (com.wetinknext.engine.thumbnail.ThumbnailBuildResult) -> Unit = {},
60     onBack: () -> Unit = {}
61 ) {
62     val context = LocalContext.current
63     val themeController = rememberThemeController()
64     val theme = themeController.current
65     var uiState by remember { mutableStateOf(EditorUiState.empty) }
66     var openPanel by remember { mutableStateOf(EditorPanel.NONE) }
67     var isEraser by remember { mutableStateOf(false) }
68     var documentLoading by remember(projectId) { mutableStateOf(true) }
69
70     val colorState = remember { GlesColorState(context) }
71     val brushColor by remember { mutableStateOf(Color.Black) }
72     var selectedBrush by remember { mutableStateOf<BrushPreset?>(BrushLibrary.pencil6B) }
73
74     val layerState = remember { LayerState() }
75
76     val surface = remember(projectId, document.id) {
77         PaintSurfaceView(
78             context = context,
79             document = document,
80             layerTiles = layerTiles,
81             layerPreviews = layerPreviews,
82         ).also { view ->
83             view.onEditorStateChange = { uiState = it }
84             view.onProjectDocumentChange = onDocumentChanged
85             view.onDirtyLayerTiles = { changed, payloads, dirty -> onDirtyLayerTiles(changed, payloads, dirty) { view.
acknowledgeSavedTiles(dirty) } }
86             view.onThumbnailCaptured = onThumbnailCaptured
87             view.onThumbnailBuildSaved = onThumbnailBuildSaved
88             view.onDocumentSessionLoaded = { documentLoading = false }
```

```

89     }
90 }
91
92 DisposableEffect(surface, onDocumentChanged) {
93     surface.onProjectDocumentChange = onDocumentChanged
94     surface.onDirtyLayerTiles = { changed, payloads, dirty -> onDirtyLayerTiles(changed, payloads, dirty) { surface.
acknowledgeSavedTiles(dirty) } }
95     surface.onThumbnailCaptured = onThumbnailCaptured
96     surface.onThumbnailBuildSaved = onThumbnailBuildSaved
97     surface.onDocumentSessionLoaded = { documentLoading = false }
98     onDispose {
99         surface.onProjectDocumentChange = null
100         surface.onDirtyLayerTiles = null
101         surface.onThumbnailCaptured = null
102         surface.onThumbnailBuildSaved = null
103         surface.onDocumentSessionLoaded = null
104     }
105 }
106
107 LaunchedEffect(uiState) {
108     layerState.syncFromEditor(uiState)
109 }
110
111 // Initialize state
112 LaunchedEffect(surface) {
113     surface.requestState()
114     surface.applyBrushPreset(BrushLibrary.pencil6B)
115     selectedBrush = BrushLibrary.pencil6B
116 }
117
118 LaunchedEffect(theme.id) {
119     surface.setCanvasBackdrop(theme.canvasBackdrop.toArgb(), theme.canvasGrid.toArgb())
120 }
121
122 WetInkTheme(theme = theme, fontMode = themeController.font) {
123     Box(
124         modifier = Modifier
125             .fillMaxSize()
126             .background(theme.appBg),
127     ) {
128         AndroidView(
129             modifier = Modifier.fillMaxSize(),
130             factory = { surface },
131         )
132
133         if (documentLoading) {
134             Box(
135                 modifier = Modifier.fillMaxSize().background(theme.appBg.copy(alpha = .92f)),
136                 contentAlignment = Alignment.Center,
137             ) {
138                 androidx.compose.material3.Text("Загрузка документа...", color = theme.textPrimary)
139             }
140         }
141
142         // Save status remains in state for diagnostics; it must not overlap the canvas.
143
144         Box(
145             modifier = Modifier
146                 .align(Alignment.TopStart)
147                 .padding(12.dp),
148         ) {
149             CanvasActionButton(
150                 icon = Icons.AutoMirrored.Filled.ArrowBack,
151                 description = "На главную",
152                 theme = theme,
153                 onClick = onBack,
154             )
155         }
156
157         // Top Toolbar
158         Box(
159             modifier = Modifier
160                 .align(Alignment.TopEnd)
161         ) {
162             TopToolbar(
163                 theme = theme,
164                 currentColor = brushColor,
165                 canUndo = uiState.canUndo,
166                 canRedo = uiState.canRedo,
167                 isSelectionActive = openPanel == EditorPanel.SELECTION,
168                 isTransformActive = openPanel == EditorPanel.TRANSFORM,
169                 isAnimationActive = openPanel == EditorPanel.ANIMATION,
170                 isAdjustmentsActive = openPanel == EditorPanel.ADJUSTMENTS,
171                 onUndoClick = { surface.undo() },
172                 onRedoClick = { surface.redo() },
173                 onSelectionClick = {
174                     openPanel = if (openPanel == EditorPanel.SELECTION) EditorPanel.NONE else EditorPanel.SELECTION
175                 },
176                 onTransformClick = {
177                     openPanel = if (openPanel == EditorPanel.TRANSFORM) EditorPanel.NONE else EditorPanel.TRANSFORM
178                 },
179                 onAnimationClick = {

```

```

180         openPanel = if (openPanel == EditorPanel.ANIMATION) EditorPanel.NONE else EditorPanel.ANIMATION
181     },
182     onAdjustmentsClick = {
183         openPanel = if (openPanel == EditorPanel.ADJUSTMENTS) EditorPanel.NONE else EditorPanel.ADJUSTMENTS
184     },
185     onSettingsClick = {
186         openPanel = if (openPanel == EditorPanel.SETTINGS) EditorPanel.NONE else EditorPanel.SETTINGS
187     },
188     onLayersClick = {
189         openPanel = if (openPanel == EditorPanel.LAYERS) EditorPanel.NONE else EditorPanel.LAYERS
190     },
191     onColorClick = {
192         openPanel = if (openPanel == EditorPanel.COLOR) EditorPanel.NONE else EditorPanel.COLOR
193     },
194 )
195 }
196
197 // Left controls keep the same compact stack as the reference editor.
198 Column(
199     modifier = Modifier
200         .align(Alignment.TopStart)
201         .padding(top = 100.dp),
202     horizontalAlignment = Alignment.Start,
203 ) {
204     SideToolbar(
205         theme = theme,
206         brushSize = uiState.brushSizePx,
207         brushOpacity = uiState.brushOpacity,
208         isEraser = isEraser,
209         onBrushSizeChange = { surface.setBrushSize(it) },
210         onBrushOpacityChange = { surface.setBrushOpacity(it) },
211         onBrushClick = {
212             openPanel = if (openPanel == EditorPanel.BRUSH) EditorPanel.NONE else EditorPanel.BRUSH
213         },
214         onEraserClick = { isEraser = it }
215     )
216     LeftCanvasActions(
217         theme = theme,
218         onClearLayerClick = { surface.clearActiveLayer() },
219         onMirrorClick = { surface.toggleCanvasMirror() },
220     )
221 }
222
223 // Panel Host
224 EditorPanelHost(
225     panel = openPanel,
226     theme = theme,
227     surface = surface,
228     layerState = layerState,
229     colorState = colorState,
230     brushColor = brushColor,
231     selectedBrush = selectedBrush,
232     themeController = themeController,
233     onBrushSelected = { preset ->
234         selectedBrush = preset
235         surface.applyBrushPreset(preset)
236     },
237     onColorChange = {
238         brushColor = it
239         surface.setBrushColor(it)
240     },
241     onDismiss = { openPanel = EditorPanel.NONE }
242 )
243 }
244 }
245 }
246
247 @Composable
248 private fun EditorPanelHost(
249     panel: EditorPanel,
250     theme: com.wetinknext.ui.theme.AppTheme,
251     surface: PaintSurfaceView?,
252     layerState: LayerState,
253     colorState: GlesColorState,
254     brushColor: Color,
255     selectedBrush: BrushPreset?,
256     themeController: ThemeController,
257     onBrushSelected: (BrushPreset) -> Unit,
258     onColorChange: (Color) -> Unit,
259     onDismiss: () -> Unit
260 ) {
261     Box(modifier = Modifier.fillMaxSize()) {
262         when (panel) {
263             EditorPanel.NONE -> Unit
264
265             EditorPanel.SETTINGS -> {
266                 Box(modifier = Modifier.align(Alignment.TopEnd).padding(top = 70.dp, end = 16.dp)) {
267                     ThemeSettingsPanel(
268                         theme = theme,
269                         themeController = themeController,
270                         onDismiss = onDismiss,
271

```

```

272     }
273 }
274
275 EditorPanel.BRUSH -> {
276     Box(modifier = Modifier.align(Alignment.CenterStart).padding(start = 72.dp)) {
277         BrushPanel(
278             currentBrush = selectedBrush ?: BrushLibrary.pencil6B,
279             onBrushSelect = onBrushSelected,
280             onBrushStudioOpen = { /* TODO */ },
281             theme = theme,
282             onDismiss = onDismiss,
283             onDuplicate = { /* TODO */ },
284             onDelete = { /* TODO */ }
285         )
286     }
287 }
288
289 EditorPanel.LAYERS -> {
290     Box(modifier = Modifier.align(Alignment.TopEnd).padding(top = 80.dp, end = 16.dp)) {
291         LayersPanel(
292             state = layerState,
293             theme = theme,
294             onDismiss = onDismiss,
295             onAddLayer = { surface?.addLayer() },
296             onSelectLayer = { surface?.setActiveLayer(it) },
297             onVisibleChange = { id, visible -> surface?.setLayerVisible(id, visible) },
298             onOpacityChange = { id, opacity -> surface?.setLayerOpacity(id, opacity) },
299             onRemoveLayer = { id -> surface?.removeLayer(id) }
300         )
301     }
302 }
303
304 EditorPanel.COLOR -> {
305     Box(modifier = Modifier.align(Alignment.TopEnd).padding(top = 80.dp, end = 16.dp)) {
306         ColorPanel(
307             state = colorState,
308             color = brushColor,
309             theme = theme,
310             onColorChange = onColorChange,
311             onAddFromPhoto = { /* TODO */ },
312             onDismiss = onDismiss
313         )
314     }
315 }
316
317 EditorPanel.SELECTION -> {
318     Box(modifier = Modifier.align(Alignment.BottomCenter).padding(bottom = 32.dp)) {
319         SelectionToolbar(
320             theme = theme,
321             currentShape = SelectionShapeUi.FREEHAND,
322             onShapeChange = { /* TODO */ },
323             onDone = onDismiss
324         )
325     }
326 }
327
328 EditorPanel.TRANSFORM -> {
329     Box(modifier = Modifier.align(Alignment.BottomCenter).padding(bottom = 32.dp)) {
330         TransformMenuView(
331             theme = theme,
332             currentMode = TransformModeUi.UNIFORM,
333             actions = object : TransformActions {
334                 override fun reset() { /* TODO */ }
335                 override fun cancel() { onDismiss() }
336                 override fun apply() { onDismiss() }
337                 override fun flipHorizontal() { /* TODO */ }
338                 override fun flipVertical() { /* TODO */ }
339                 override fun setMode(mode: TransformModeUi) { /* TODO */ }
340             }
341         )
342     }
343 }
344
345 EditorPanel.ADJUSTMENTS -> {
346     Box(modifier = Modifier.align(Alignment.TopEnd).padding(top = 80.dp, end = 16.dp)) {
347         AdjustmentsPanel(
348             theme = theme,
349             onClose = onDismiss,
350             onAction = { /* TODO */ }
351         )
352     }
353 }
354
355 EditorPanel.ANIMATION -> {
356     Box(modifier = Modifier.align(Alignment.BottomCenter)) {
357         AnimationTimelineToolbar(
358             theme = theme,
359             document = com.wetinknext.domain.animation.AnimationDocument(enabled = true),
360             currentFrameId = 0L,
361             onFrameSelect = {},
362             onFrameMove = { _, _ -> },
363             onAddFrame = {},

```

```

364         onDuplicateFrame = {},
365         onDeleteFrame = {},
366         onHoldChange = { _, _ -> },
367         onRoleToggle = { _, _ -> },
368         onAssembleLayers = {},
369         onLayersClick = {},
370         onClose = onDismiss
371     )
372 }
373 }
374 }
375 }
376 }
377
378 @Composable
379 private fun ThemeSettingsPanel(
380     theme: AppTheme,
381     themeController: ThemeController,
382     onDismiss: () -> Unit,
383 ) {
384     var preferencesOpen by remember { mutableStateOf(false) }
385     Column(
386         modifier = Modifier
387             .width(320.dp)
388             .clip(RoundedCornerShape(22.dp))
389             .background(theme.panelBg)
390             .padding(16.dp),
391         verticalArrangement = Arrangement.spacedBy(12.dp),
392     ) {
393         Row(verticalAlignment = Alignment.CenterVertically) {
394             androidx.compose.material3.Text(
395                 text = if (preferencesOpen) "Предпочтения" else "Меню",
396                 color = theme.textPrimary,
397                 style = androidx.compose.material3.MaterialTheme.typography.titleLarge,
398                 modifier = Modifier.weight(1f),
399             )
400             androidx.compose.material3.Text(
401                 text = "x",
402                 color = theme.textSecondary,
403                 style = androidx.compose.material3.MaterialTheme.typography.headlineSmall,
404                 modifier = Modifier.size(28.dp).clickable(onClick = onDismiss),
405             )
406         }
407
408         Row(
409             modifier = Modifier.fillMaxWidth().clip(RoundedCornerShape(10.dp))
410                 .background(theme.panelBgVariant.copy(alpha = .5f)).padding(3.dp),
411             horizontalArrangement = Arrangement.spacedBy(2.dp),
412         ) {
413             listOf("Меню", "Холст", "Экспорт", "Сведения").forEachIndexed { index, label ->
414                 Box(
415                     modifier = Modifier
416                         .weight(1f).clip(RoundedCornerShape(8.dp))
417                         .background(if (index == 0) theme.panelBgVariant else Color.Transparent)
418                         .padding(vertical = 8.dp),
419                     contentAlignment = Alignment.Center,
420                 ) { androidx.compose.material3.Text(label, color = if (index == 0) theme.textPrimary else theme.textSecondary,
421                     style = androidx.compose.material3.MaterialTheme.typography.labelSmall) }
422             }
423
424             if (!preferencesOpen) {
425                 androidx.compose.material3.Text(
426                     text = "Предпочтения",
427                     color = theme.textPrimary,
428                     modifier = Modifier.fillMaxWidth().clickable { preferencesOpen = true }.padding(vertical = 14.dp, horizontal
429 = 4.dp),
430                 )
431             } else {
432                 androidx.compose.material3.Text("Тема", color = theme.textSecondary, style = androidx.compose.material3.
433                 MaterialTheme.typography.labelMedium)
434                 val choices = AppThemes.all + themeController.customTheme.copy(id = "custom", displayName = "Своя")
435                 choices.chunked(5).forEach { row ->
436                     Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween) {
437                         row.forEach { candidate ->
438                             val selected = candidate.id == themeController.current.id
439                             Box(
440                                 modifier = Modifier.size(48.dp).clip(androidx.compose.foundation.shape.CircleShape)
441                                     .background(if (selected) Color.White else Color.Transparent)
442                                     .clickable { if (candidate.id == "custom") themeController.selectCustom() else
443                                     themeController.select(candidate) }
444                                 .padding(2.dp),
445                                 contentAlignment = Alignment.Center,
446                             ) {
447                                 Box(
448                                     modifier = Modifier.fillMaxSize().clip(androidx.compose.foundation.shape.CircleShape)
449                                         .background(if (candidate.id == "custom") theme.panelBgVariant else candidate.appBg),
450                                     contentAlignment = Alignment.Center,
451                                 ) {
452                                     if (candidate.id == "custom") androidx.compose.material3.Text("Своя", color = Color.White,
453                                     style = androidx.compose.material3.MaterialTheme.typography.labelSmall)
454                                     else Box(Modifier.size(24.dp).clip(androidx.compose.foundation.shape.CircleShape).background(

```

```

candidate.accent))
451         }
452     }
453 }
454     repeat(5 - row.size) { Spacer(Modifier.size(48.dp)) }
455 }
456 }
457     androidx.compose.material3.Text(themeController.current.displayName, color = theme.textPrimary)
458 }
459 }
460 }
461
462 private fun PaintSurfaceView.applyBrushPreset(
463     preset: BrushPreset
464 ) {
465     val settings = preset.settings
466     applyBrush(settings)
467
468     val grainPath = settings.grainAssetPath
469         ?.trim()
470         ?.takeIf { it.isNotEmpty() }
471
472     if (grainPath == null) {
473         clearGrainTexture()
474     } else {
475         loadGrainTexture(
476             path = grainPath,
477             scale = settings.grainScale,
478             canvasLocked = settings.grainCanvasLocked,
479             depth = settings.textureDepth,
480             contrast = settings.textureContrast,
481         )
482     }
483 }
484
485 val shapePath = settings.shapeAssetPath?.trim()?.takeIf { it.isNotEmpty() }
486 if (shapePath == null) {
487     clearShapeTexture()
488 } else {
489     loadShapeTexture(path = shapePath, rgbToAlpha = settings.rgbToAlpha)
490 }
491 }

```

5. app/src/main/java/com/wetinknext/app/MainActivity.kt

```
1 package com.wetinknext.app
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.viewModels
7 import androidx.core.view.WindowCompat
8 import androidx.core.view.WindowInsetsCompat
9 import androidx.core.view.WindowInsetsControllerCompat
10
11 class MainActivity : ComponentActivity() {
12     private val appViewModel: AppViewModel by viewModels {
13         AppViewModel.Factory(this)
14     }
15
16     override fun onCreate(savedInstanceState: Bundle?) {
17         super.onCreate(savedInstanceState)
18         WindowCompat.setDecorFitsSystemWindows(window, false)
19         WindowInsetsControllerCompat(window, window.decorView).apply {
20             hide(WindowInsetsCompat.Type.statusBars())
21             systemBarsBehavior = WindowInsetsControllerCompat.BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE
22         }
23         setContent { AppRoot(appViewModel) }
24     }
25 }
```

6. app/src/main/java/com/wetinknext/app/WetInkApp.kt

```
1 package com.wetinknext.app
2
3 import androidx.activity.compose.BackHandler
4 import androidx.compose.material3.AlertDialog
5 import androidx.compose.material3.Text
6 import androidx.compose.material3.TextButton
7 import androidx.compose.runtime.Composable
8 import androidx.compose.runtime.LaunchedEffect
9 import androidx.compose.ui.text.font.FontWeight
10 import com.wetinknext.data.project.CreateProjectRequest
11 import com.wetinknext.data.project.ProjectSummary
12 import com.wetinknext.ui.home.HomeProject
13 import com.wetinknext.ui.home.HomeScreen
14 import com.wetinknext.ui.home.DeletedHomeProject
15 import com.wetinknext.ui.theme.WetInkTheme
16 import com.wetinknext.ui.theme.rememberThemeController
17
18 @Composable
19 fun AppRoot(viewModel: AppViewModel) {
20     when (val currentRoute = viewModel.route) {
21         AppRoute.Home -> ProjectHomeRoute(viewModel)
22         is AppRoute.Editor -> {
23             val document = viewModel.openedDocument
24             val loadedProject = viewModel.openedProject
25             if (document?.id == currentRoute.projectId && loadedProject?.document?.id == currentRoute.projectId) {
26                 BackHandler { viewModel.closeEditor() }
27                 EditorScreen(
28                     projectId = currentRoute.projectId,
29                     document = document,
30                     layerTiles = loadedProject.assets.layerTiles,
31                     layerPreviews = loadedProject.assets.layerPreviewsWebp,
32                     saveStatus = when {
33                         viewModel.uiState.isSaving -> "Сохраняется..."
34                         viewModel.uiState.isDirty -> "Есть несохранённые изменения"
35                         else -> "Сохранено"
36                     },
37                     onDocumentChanged = viewModel::scheduleAutosave,
38                     onDirtyLayerTiles = viewModel::scheduleTileAutosave,
39                     onThumbnailCaptured = viewModel::submitThumbnail,
40                     onThumbnailBuildSaved = viewModel::submitThumbnailBuild,
41                     onBack = viewModel::closeEditor,
42                 )
43             } else {
44                 LaunchedEffect(currentRoute.projectId) {
45                     viewModel.openProject(currentRoute.projectId)
46                 }
47             }
48         }
49     }
50 }
51
52 /** Kept for existing previews and callers while MainActivity uses AppRoot. */
53 @Composable
54 fun WetInkApp(viewModel: AppViewModel) = AppRoot(viewModel)
55
56 @Composable
57 private fun ProjectHomeRoute(viewModel: AppViewModel) {
58     val themeController = rememberThemeController()
59     val theme = themeController.current
60     val state = viewModel.uiState
61
62     WetInkTheme(theme = theme, fontMode = themeController.font) {
63         HomeScreen(
64             theme = theme,
65             projects = state.projects.map(ProjectSummary::toHomeProject),
66             deletedProjects = state.trashedProjects.map { trashed ->
67                 DeletedHomeProject(
68                     project = trashed.project.toHomeProject(),
69                     deletedAtMs = trashed.deletedAt,
70                 )
71             },
72             onOpenProject = { viewModel.openProject(it.id) },
73             onCreateProject = { name, width, height, dpi, colorProfile ->
74                 viewModel.createProject(
75                     CreateProjectRequest(
76                         name = name,
77                         width = width,
78                         height = height,
79                         dpi = dpi,
80                         colorProfile = colorProfile,
81                     ),
82                 )
83             },
84             onRenameProject = { project, name -> viewModel.renameProject(project.id, name) },
85             onDuplicateProject = { project -> viewModel.duplicateProject(project.id) },
86             onDeleteProject = { project -> viewModel.moveProjectToTrash(project.id) },
87             onRestoreProject = { deleted -> viewModel.restoreProject(deleted.project.id) },
88             onDeleteForever = { deleted -> viewModel.deleteProjectForever(deleted.project.id) },
89         )
90     }
```

```

91     state.recoveryProject?.let { project ->
92         AlertDialog(
93             onDismissRequest = viewModel::dismissRecovery,
94             title = { Text("Восстановить работу?" ) },
95             text = { Text("Приложение было закрыто во время работы с «${project.name}»." ) },
96             confirmButton = {
97                 TextButton(onClick = { viewModel.openProject(project.id) }) {
98                     Text("Открыть", fontWeight = FontWeight.SemiBold)
99                 }
100             },
101             dismissButton = {
102                 TextButton(onClick = viewModel::dismissRecovery) {
103                     Text("Не сейчас")
104                 }
105             },
106         )
107     }
108 }
109 }
110
111 private fun ProjectSummary.toHomeProject(): HomeProject = HomeProject(
112     id = id,
113     name = name,
114     canvasWidth = width,
115     canvasHeight = height,
116     updatedAtMs = updatedAt,
117     thumbnailBytes = thumbnailBytes,
118     thumbnailVersion = thumbnailVersion,
119 )

```

7. app/src/main/java/com/wetinknext/data/project/AutosaveCoordinator.kt

```
1 package com.wetinknext.data.project
2
3 import kotlinx.coroutines.CoroutineScope
4 import kotlinx.coroutines.Job
5 import kotlinx.coroutines.delay
6 import kotlinx.coroutines.launch
7
8 /**
9  * Coalesces completed document edits into one disk publication.
10  *
11  * Callers only schedule after transactional changes (UP, clear, layer or
12  * property operations), never for high-frequency MOVE events.
13  */
14 class AutosaveCoordinator(
15     private val scope: CoroutineScope,
16     private val debounceMillis: Long = DEFAULT_DEBOUNCE_MILLIS,
17 ) {
18     private var pendingJob: Job? = null
19
20     fun schedule(save: suspend () -> Unit) {
21         pendingJob?.cancel()
22         pendingJob = scope.launch {
23             delay(debounceMillis)
24             save()
25         }
26     }
27
28     fun cancel() {
29         pendingJob?.cancel()
30         pendingJob = null
31     }
32
33     companion object {
34         const val DEFAULT_DEBOUNCE_MILLIS = 700L
35     }
36 }
```

8. app/src/main/java/com/wetinknext/data/project/DirectoryProjectStore.kt

```
1 package com.wetinknext.data.project
2
3 import com.wetinknext.domain.document.LoadedWetInkProject
4 import com.wetinknext.domain.document.ProjectAssets
5 import com.wetinknext.domain.document.ProjectDocument
6 import com.wetinknext.domain.document.ProjectManifest
7 import com.wetinknext.domain.document.StoredLayer
8 import java.io.File
9 import java.io.IOException
10 import java.nio.file.AtomicMoveNotSupportedException
11 import java.nio.file.Files
12 import java.nio.file.StandardCopyOption
13 import kotlinx.serialization.decodeFromString
14 import kotlinx.serialization.encodeToString
15 import kotlinx.serialization.json.Json
16
17 /** New on-disk project format: directory metadata plus independently replaceable layer files. */
18 class DirectoryProjectStore {
19     private val clock: () -> Long = System::currentTimeMillis,
20     private val json: Json = Json { encodeDefaults = true; ignoreUnknownKeys = true; prettyPrint = true },
21 } {
22     fun save(directory: File, document: ProjectDocument, assets: ProjectAssets): LoadedWetInkProject {
23         require(directory.name == document.id) { "Project directory/id mismatch" }
24         val expected = document.layers.map { it.id }.toSet()
25         require(assets.layerTiles.keys == expected) { "Every layer needs one payload" }
26         ensureDirectory(directory)
27         val timestamp = clock()
28         val saved = document.copy(
29             updatedAt = timestamp,
30             thumbnailVersion = if (assets.thumbnailWebp != null) timestamp else document.thumbnailVersion,
31         )
32         saved.layers.forEach { layer -> writeAtomic(File(directory, layer.pixelFile), checkNotNull(assets.layerTiles[layer.id])) }
33         assets.thumbnailWebp?.let { writeAtomic(File(directory, THUMBNAIL), it) }
34         assets.layerPreviewsWebp.forEach { (id, bytes) -> writeAtomic(File(directory, previewPath(id)), bytes) }
35         deleteRemovedLayerFiles(directory, saved.layers.map { it.id }.toSet())
36         writeAtomic(File(directory, DOCUMENT), json.encodeToString(saved).encodeToByteArray())
37         val manifest = ProjectManifest(
38             documentId = saved.id, name = saved.name, width = saved.width, height = saved.height,
39             thumbnailPath = assets.thumbnailWebp?.let { THUMBNAIL }, thumbnailVersion = saved.thumbnailVersion,
40             createdAt = saved.createdAt, updatedAt = saved.updatedAt,
41             layers = saved.layers.map { StoredLayer(it.id, it.pixelFile, checkNotNull(assets.layerTiles[it.id]).size.toLong()) }
42         ),
43         writeAtomic(File(directory, MANIFEST), json.encodeToString(manifest).encodeToByteArray())
44         return LoadedWetInkProject(saved, manifest, assets)
45     }
46
47     fun load(directory: File): LoadedWetInkProject {
48         val manifest = readManifest(directory)
49         val document = readDocument(directory)
50         require(manifest.documentId == document.id) { "Manifest/document id mismatch" }
51         val tiles = document.layers.associate { layer ->
52             val bytes = requireFile(File(directory, layer.pixelFile)).readBytes()
53             val stored = manifest.layers.firstOrNull { it.id == layer.id } ?: error("Missing layer metadata")
54             require(stored.byteSize == bytes.size.toLong()) { "Layer size mismatch" }
55             layer.id to bytes
56         }
57         val previews = document.layers.mapNotNull { layer ->
58             File(directory, previewPath(layer.id)).takeIf(File::isFile)?.readBytes()?.let { layer.id to it }
59         }.toMap()
60         val thumbnail = manifest.thumbnailPath?.let { File(directory, it).takeIf(File::isFile)?.readBytes() }
61         return LoadedWetInkProject(document, manifest, ProjectAssets(tiles, thumbnail, previews))
62     }
63
64     fun readManifest(directory: File): ProjectManifest =
65         json.decodeFromString(requireFile(File(directory, MANIFEST)).readText())
66
67     fun readDocument(directory: File): ProjectDocument =
68         json.decodeFromString(requireFile(File(directory, DOCUMENT)).readText())
69
70     fun readThumbnail(directory: File): ByteArray? = File(directory, THUMBNAIL).takeIf(File::isFile)?.readBytes()
71
72     fun delete(directory: File) {
73         require(directory.isDirectory) { "Project directory does not exist" }
74         directory.walkBottomUp().forEach { if (!it.delete()) throw IOException("Cannot delete ${it.path}") }
75     }
76
77     private fun deleteRemovedLayerFiles(directory: File, active: Set<Long>) {
78         listOf(LAYERS, PREVIEWS).forEach { folder ->
79             File(directory, folder).listFiles().orEmpty().forEach { file ->
80                 val id = file.name.removePrefix("layer-").substringBefore('.').toLongOrNull()
81                 ?: file.name.substringBefore('.').toLongOrNull()
82                 if (id != null && id !in active) file.delete()
83             }
84         }
85     }
86
87     private fun writeAtomic(file: File, bytes: ByteArray) {
88         ensureDirectory(checkNotNull(file.parentFile))
89     }
90 }
```

```

89         val tmp = File(checkNotNull(file.parentFile), "${file.name}.tmp")
90         try {
91             tmp.outputStream().use { it.write(bytes) }
92             try { Files.move(tmp.toPath(), file.toPath(), StandardCopyOption.ATOMIC_MOVE, StandardCopyOption.
REPLACE_EXISTING) }
93             catch (_: AtomicMoveNotSupportedException) { Files.move(tmp.toPath(), file.toPath(), StandardCopyOption.
REPLACE_EXISTING) }
94             } finally { if (tmp.exists()) tmp.delete() }
95         }
96
97         private fun ensureDirectory(file: File) {
98             if (!file.exists() && !file.mkdirs()) throw IOException("Cannot create ${file.path}")
99             require(file.isDirectory) { "Expected directory: ${file.path}" }
100         }
101
102         private fun requireFile(file: File): File = file.also { require(it.isFile) { "Missing ${it.path}" } }
103
104         private companion object {
105             const val MANIFEST = "manifest.json"
106             const val DOCUMENT = "document.json"
107             const val THUMBNAIL = "thumbnail.webp"
108             const val LAYERS = "layers"
109             const val PREVIEWS = "thumbnails"
110             fun previewPath(layerId: Long) = "$PREVIEWS/layer-$layerId.webp"
111         }
112     }

```

9. app/src/main/java/com/wetinknext/data/project/FileProjectRepository.kt

```
1 package com.wetinknext.data.project
2
3 import com.wetinknext.domain.document.ProjectAssets
4 import com.wetinknext.domain.document.ProjectDocument
5 import com.wetinknext.domain.document.ProjectManifest
6 import com.wetinknext.domain.document.WetInkProjectContainer
7 import java.io.File
8 import java.io.IOException
9 import java.nio.file.AtomicMoveNotSupportedException
10 import java.nio.file.Files
11 import java.nio.file.StandardCopyOption
12 import java.util.Base64
13 import java.util.UUID
14
15 /** File-backed project catalog. Each project remains one independent .wetink file. */
16 class FileProjectRepository(
17     private val projectsDirectory: File,
18     private val container: WetInkProjectContainer = WetInkProjectContainer(),
19     private val directoryProjectStore: DirectoryProjectStore = DirectoryProjectStore(),
20 ) : ProjectRepository {
21     override suspend fun create(request: CreateProjectRequest): ProjectSummary {
22         val document = ProjectDocument.newUntitled(
23             name = request.name,
24             width = request.width,
25             height = request.height,
26             dpi = request.dpi,
27             colorProfile = request.colorProfile,
28         )
29         val assets = ProjectAssets(
30             layerTiles = document.layers.associate { it.id to ByteArray(0) },
31             thumbnailWebp = DEFAULT_THUMBNAIL_WEBP,
32         )
33         val saved = directoryStore.save(projectDirectory(document.id), document, assets)
34         return saved.manifest.toSummary()
35     }
36
37     override suspend fun list(): List<ProjectSummary> {
38         ensureDirectories()
39         return listProjects(projectsDirectory)
40     }
41
42     override suspend fun listTrash(): List<TrashedProjectSummary> {
43         ensureDirectories()
44         return trashDirectory.listFiles()
45             .orEmpty()
46             .asSequence()
47             .filter { file -> file.isDirectory || (file.isFile && file.extension.equals(FILE_EXTENSION, ignoreCase = true) &&
!file.name.endsWith(".recovery.$FILE_EXTENSION")) }
48             .mapNotNull { file ->
49                 runCatching {
50                     TrashedProjectSummary(readSummary(file), file.lastModified())
51                 }.getOrNull()
52             }
53             .sortedByDescending(TrashedProjectSummary::deletedAt)
54             .toList()
55     }
56
57     override suspend fun open(id: String): ProjectDocument = loadProject(id).document
58
59     override suspend fun openForEditor(id: String) = loadProject(id)
60
61     override suspend fun save(document: ProjectDocument) {
62         val previous = loadProject(document.id)
63         saveProject(document, previous.assets.retainDocumentLayers(document))
64     }
65
66     override suspend fun save(document: ProjectDocument, changedLayerTiles: Map<Long, ByteArray>) {
67         if (changedLayerTiles.isEmpty()) return save(document)
68         val previous = loadProject(document.id)
69         val allowed = document.layers.map { it.id }.toSet()
70         require(changedLayerTiles.keys.all(allowed::contains)) { "Unknown layer tile payload" }
71         saveProject(
72             document,
73             previous.assets.retainDocumentLayers(document).copy(
74                 layerTiles = previous.assets.layerTiles
75                     .filterKeys(allowed::contains)
76                     .plus(changedLayerTiles),
77             ),
78         )
79     }
80
81     override suspend fun save(
82         document: ProjectDocument,
83         changedLayerTiles: Map<Long, ByteArray>,
84         thumbnailWebp: ByteArray?,
85         layerPreviewsWebp: Map<Long, ByteArray>,
86     ) {
87         val previous = loadProject(document.id)
88         val layerIds = document.layers.map { it.id }.toSet()
89         saveProject(
```

```

90         document,
91         previous.assets.copy(
92             layerTiles = (previous.assets.layerTiles + changedLayerTiles)
93                 .filterKeys(layerIds::contains),
94             thumbnailWebp = thumbnailWebp ?: previous.assets.thumbnailWebp,
95             layerPreviewsWebp = (previous.assets.layerPreviewsWebp + layerPreviewsWebp)
96                 .filterKeys(layerIds::contains),
97         ),
98     )
99 }
100
101 override suspend fun saveRecovery(document: ProjectDocument, changedLayerTiles: Map<Long, ByteArray>) {
102     val current = loadProject(document.id)
103     container.save(recoveryFile(document.id), document, current.assets.copy(layerTiles = current.assets.layerTiles +
104 changedLayerTiles))
105 }
106
107 override suspend fun clearRecovery(id: String) {
108     val file = recoveryFile(id)
109     if (file.exists() && !file.delete()) throw IOException("Cannot clear recovery snapshot")
110 }
111
112 override suspend fun openRecovery(id: String) = recoveryFile(id).takeIf(File::isFile)?.let(container::load)
113
114 override suspend fun rename(id: String, name: String) {
115     val document = open(id)
116     save(document.copy(name = name))
117 }
118
119 /** Creates a separate editable copy; undo history intentionally is not persisted. */
120 override suspend fun duplicate(id: String): ProjectSummary {
121     val source = loadProject(id)
122     val copyId = UUID.randomUUID().toString()
123     val copiedDocument = source.document.copy(
124         id = copyId,
125         name = duplicateName(source.document.name),
126         createdAt = System.currentTimeMillis(),
127         updatedAt = System.currentTimeMillis(),
128     )
129     val saved = directoryStore.save(projectDirectory(copyId), copiedDocument, source.assets)
130     return saved.manifest.toSummary(saved.assets.thumbnailWebp)
131 }
132
133 override suspend fun moveToTrash(id: String) {
134     val source = projectLocation(id)
135     require(source.exists()) { "Project does not exist: $id" }
136     val target = trashedProjectLocation(id, source.isDirectory)
137     atomicMove(source, target)
138     target.setLastModified(System.currentTimeMillis())
139 }
140
141 override suspend fun restore(id: String) {
142     val source = trashedProjectLocation(id, true).takeIf(File::exists) ?: trashedProjectLocation(id, false)
143     require(source.exists()) { "Trashed project does not exist: $id" }
144     atomicMove(source, if (source.isDirectory) projectDirectory(id) else legacyProjectFile(id))
145 }
146
147 override suspend fun deleteForever(id: String) {
148     val file = trashedProjectLocation(id, true).takeIf(File::exists) ?: trashedProjectLocation(id, false)
149     require(file.exists()) { "Trashed project does not exist: $id" }
150     if (file.isDirectory) directoryStore.delete(file) else if (!file.delete()) throw IOException("Cannot delete project
151 permanently: ${file.path}")
152 }
153
154 private fun projectDirectory(id: String): File {
155     ensureDirectories()
156     return File/projectsDirectory, safeId(id))
157 }
158
159 private fun legacyProjectFile(id: String): File = File/projectsDirectory, "${safeId(id)}.FILE_EXTENSION")
160
161 private fun trashedProjectLocation(id: String, directory: Boolean): File {
162     ensureDirectories()
163     return File(trashDirectory, if (directory) safeId(id) else "${safeId(id)}.FILE_EXTENSION")
164 }
165
166 private fun projectLocation(id: String): File = projectDirectory(id).takeIf(File::exists) ?: legacyProjectFile(id)
167 private fun recoveryFile(id: String): File = File/projectsDirectory, "${safeId(id)}.recovery.FILE_EXTENSION")
168
169 private fun ensureDirectories() {
170     if (!projectsDirectory.exists() && !projectsDirectory.mkdirs()) {
171         throw IOException("Cannot create projects directory: ${projectsDirectory.path}")
172     }
173     if (!trashDirectory.exists() && !trashDirectory.mkdirs()) {
174         throw IOException("Cannot create project trash: ${trashDirectory.path}")
175     }
176 }
177
178 private fun listProjects(directory: File): List<ProjectSummary> =
179     directory.listFiles()
180         .orEmpty()
181         .asSequence()

```

```

180         .filter { file -> file.isDirectory || (file.isFile && file.extension.equals(FILE_EXTENSION, ignoreCase = true) &&
!file.name.endsWith(".recovery.$FILE_EXTENSION")) }
181     }.mapNotNull { file -> runCatching { readSummary(file) }.getOrNull() }
182     }.sortedByDescending(ProjectSummary::updatedAt)
183     }.toList()
184
185     private fun atomicMove(source: File, target: File) {
186         try {
187             Files.move(
188                 source.toPath(),
189                 target.toPath(),
190                 StandardCopyOption.ATOMIC_MOVE,
191                 StandardCopyOption.REPLACE_EXISTING,
192             )
193         } catch (error: AtomicMoveNotSupportedException) {
194             throw IOException("Atomic project move is not supported", error)
195         }
196     }
197
198     private fun loadProject(id: String) = projectDirectory(id).takeIf(File::isDirectory)
199     ?.let(directoryStore::load)
200     ?: container.load(legacyProjectFile(id))
201
202     /** First regular save migrates an old ZIP project to the directory format. */
203     private fun saveProject(document: ProjectDocument, assets: ProjectAssets) =
204         directoryStore.save(projectDirectory(document.id), document, assets).also {
205             legacyProjectFile(document.id).takeIf(File::isFile)?.delete()
206         }
207
208     private fun readSummary(location: File): ProjectSummary = if (location.isDirectory) {
209         directoryStore.readManifest(location).toSummary(directoryStore.readThumbnail(location))
210     } else {
211         container.readManifest(location).toSummary(container.readThumbnail(location))
212     }
213
214     private fun safeId(id: String): String {
215         require(id.isNotBlank() && id.all { it.isLetterOrDigit() || it == '-' || it == '_' }) {
216             "Unsafe project id"
217         }
218         return id
219     }
220
221     private fun duplicateName(name: String): String = "$name copy".take(80)
222
223     /** Drops binary assets for layers removed from the persisted document. */
224     private fun ProjectAssets.retainDocumentLayers(document: ProjectDocument): ProjectAssets {
225         val allowed = document.layers.map { it.id }.toSet()
226         return copy(
227             layerTiles = layerTiles.filterKeys(allowed::contains),
228             layerPreviewsWebp = layerPreviewsWebp.filterKeys(allowed::contains),
229         )
230     }
231
232     private fun ProjectManifest.toSummary(thumbnail: ByteArray? = null): ProjectSummary = ProjectSummary(
233         id = documentId,
234         name = name,
235         width = width,
236         height = height,
237         updatedAt = updatedAt,
238         thumbnailPath = thumbnailPath,
239         thumbnailBytes = thumbnail,
240         thumbnailVersion = thumbnailVersion,
241     )
242
243     private val trashDirectory: File
244     get() = File(projectsDirectory, TRASH_DIRECTORY)
245
246     private companion object {
247         const val FILE_EXTENSION = "wetink"
248         const val TRASH_DIRECTORY = ".trash"
249         // Valid transparent 1x1 WebP until a GL-generated project preview is available.
250         val DEFAULT_THUMBNAIL_WEBP: ByteArray = Base64.getDecoder().decode(
251             "UklGRiIAAABXRJQVIA4IBYAAADQAwCdASoBAAEAAUAmJaQAA3AA/vuUAAA=",
252         )
253     }
254 }

```

10. app/src/main/java/com/wetinknext/data/project/ProjectRepository.kt

```
1 package com.wetinknext.data.project
2
3 import com.wetinknext.domain.document.ProjectDocument
4 import com.wetinknext.domain.document.LoadedWetInkProject
5 import com.wetinknext.domain.document.ProjectAssets
6
7 interface ProjectRepository {
8     suspend fun create(request: CreateProjectRequest): ProjectSummary
9     suspend fun list(): List<ProjectSummary>
10    suspend fun listTrash(): List<TrashedProjectSummary>
11    suspend fun open(id: String): ProjectDocument
12    /** Reads document metadata and all layer payloads needed to build a GPU session. */
13    suspend fun openForEditor(id: String): LoadedWetInkProject
14    suspend fun save(document: ProjectDocument)
15    /** Atomically replaces only the supplied layer payloads; other layer data is retained. */
16    suspend fun save(document: ProjectDocument, changedLayerTiles: Map<Long, ByteArray>)
17    suspend fun save(
18        document: ProjectDocument,
19        changedLayerTiles: Map<Long, ByteArray>,
20        thumbnailWebp: ByteArray?,
21        layerPreviewsWebp: Map<Long, ByteArray> = emptyMap(),
22    )
23    suspend fun saveRecovery(document: ProjectDocument, changedLayerTiles: Map<Long, ByteArray>)
24    suspend fun clearRecovery(id: String)
25    suspend fun openRecovery(id: String): LoadedWetInkProject?
26    suspend fun rename(id: String, name: String)
27    suspend fun duplicate(id: String): ProjectSummary
28    suspend fun moveToTrash(id: String)
29    suspend fun restore(id: String)
30    suspend fun deleteForever(id: String)
31 }
32
33 data class CreateProjectRequest(
34     val name: String,
35     val width: Int = ProjectDocument.DEFAULT_WIDTH,
36     val height: Int = ProjectDocument.DEFAULT_HEIGHT,
37     val dpi: Int = ProjectDocument.DEFAULT_DPI,
38     val colorProfile: String = ProjectDocument.SRGB_PROFILE,
39 )
40
41 /** Metadata for the gallery. No layer tile data is loaded to create this item. */
42 data class ProjectSummary(
43     val id: String,
44     val name: String,
45     val width: Int,
46     val height: Int,
47     val updatedAt: Long,
48     val thumbnailPath: String?,
49     val thumbnailBytes: ByteArray? = null,
50     /** Cache version for the gallery WebP; metadata loading remains tile-free. */
51     val thumbnailVersion: Long = 0L,
52 )
53
54 data class TrashedProjectSummary(
55     val project: ProjectSummary,
56     val deletedAt: Long,
57 )
```

11. app/src/main/java/com/wetinknext/data/project/ThumbnailEncoder.kt

```
1 package com.wetinknext.data.project
2
3 import android.graphics.Bitmap
4 import android.os.Build
5 import com.wetinknext.engine.core.ThumbnailCapture
6 import java.io.ByteArrayOutputStream
7
8 /** CPU-only encoder. Call from a background coroutine, never from the GL thread. */
9 object ThumbnailEncoder {
10     fun encodeWebp(image: ThumbnailCapture.Rgba): ByteArray {
11         val pixels = IntArray(image.width * image.height)
12         for (destinationY in 0 until image.height) {
13             // ThumbnailRenderer already flips glReadPixels into Bitmap order.
14             var source = destinationY * image.width * 4
15             val destination = destinationY * image.width
16             for (x in 0 until image.width) {
17                 val r = image.pixels[source++].toInt() and 0xFF
18                 val g = image.pixels[source++].toInt() and 0xFF
19                 val b = image.pixels[source++].toInt() and 0xFF
20                 val a = image.pixels[source++].toInt() and 0xFF
21                 pixels[destination + x] = (a shl 24) or (r shl 16) or (g shl 8) or b
22             }
23         }
24         val bitmap = Bitmap.createBitmap(pixels, image.width, image.height, Bitmap.Config.ARGB_8888)
25         return ByteArrayOutputStream().use { output ->
26             check(bitmap.compress(webpFormat(), 88, output)) { "WebP encoding failed" }
27             bitmap.recycle()
28             output.toByteArray()
29         }
30     }
31
32     @Suppress("DEPRECATION")
33     private fun webpFormat(): Bitmap.CompressFormat =
34         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) {
35             Bitmap.CompressFormat.WEBP_LOSSY
36         } else {
37             Bitmap.CompressFormat.WEBP
38         }
39 }
```

12. app/src/main/java/com/wetinknext/domain/animation/AnimationModels.kt

```
1 package com.wetinknext.domain.animation
2
3 import kotlinx.serialization.Serializable
4
5 @Serializable
6 enum class AnimationPlaybackMode {
7     LOOP,
8     PING_PONG,
9     ONE_SHOT,
10 }
11
12 @Serializable
13 enum class AnimationFrameRole {
14     NORMAL,
15     BACKGROUND,
16     FOREGROUND,
17 }
18
19 @Serializable
20 data class OnionSkinSettings(
21     val enabled: Boolean = false, // Disabled by default until renderer supports it
22     val previousFrames: Int = 1,
23     val nextFrames: Int = 1,
24     val opacity: Float = 0.35f,
25     val previousColorArgb: Int = 0xFFFFF4FA3.toInt(),
26     val nextColorArgb: Int = 0xFF4A8DFF.toInt(),
27     val blendPrimaryFrame: Boolean = false,
28 )
29
30 @Serializable
31 data class AnimationFrame(
32     val id: Long,
33     val layerIds: List<Long>,
34     val holdFrames: Int = 1,
35     val role: AnimationFrameRole = AnimationFrameRole.NORMAL,
36 )
37
38 @Serializable
39 data class AnimationDocument(
40     val enabled: Boolean = false,
41     val framesPerSecond: Int = 12,
42     val playbackMode: AnimationPlaybackMode = AnimationPlaybackMode.LOOP,
43     val onionSkin: OnionSkinSettings = OnionSkinSettings(),
44     val frames: List<AnimationFrame> = emptyList(),
45 )
```

13. app/src/main/java/com/wetinknext/domain/animation/AnimationRenderPlan.kt

```
1 package com.wetinknext.domain.animation
2
3 /**
4  * Defines which frames and roles should be rendered in the current view.
5  */
6 data class AnimationRenderPlan(
7     val backgroundFrame: AnimationFrame?,
8     val currentFrame: AnimationFrame?,
9     val foregroundFrame: AnimationFrame?,
10    val previousOnionFrames: List<AnimationFrame>,
11    val nextOnionFrames: List<AnimationFrame>,
12 )
13
14 /**
15  * Logic to decide which layers are visible based on the animation document.
16  * This is a pure function for easy testing.
17  */
18 fun buildAnimationRenderPlan(
19     document: AnimationDocument,
20     currentFrameId: Long,
21 ): AnimationRenderPlan {
22     if (!document.enabled) {
23         return AnimationRenderPlan(null, null, null, emptyList(), emptyList())
24     }
25
26     val frames = document.frames
27     val currentIndex = frames.indexOfFirst { it.id == currentFrameId }
28
29     val bgFrame = frames.find { it.role == AnimationFrameRole.BACKGROUND }
30     val fgFrame = frames.find { it.role == AnimationFrameRole.FOREGROUND }
31     val current = if (currentIndex >= 0) frames[currentIndex] else null
32
33     val prevOnions = mutableListOf<AnimationFrame>()
34     val nextOnions = mutableListOf<AnimationFrame>()
35
36     if (document.onionSkin.enabled && currentIndex >= 0) {
37         // Previous frames
38         for (i in 1..document.onionSkin.previousFrames) {
39             val idx = currentIndex - i
40             if (idx >= 0) {
41                 val f = frames[idx]
42                 if (f.role == AnimationFrameRole.NORMAL) prevOnions.add(f)
43             }
44         }
45         // Next frames
46         for (i in 1..document.onionSkin.nextFrames) {
47             val idx = currentIndex + i
48             if (idx < frames.size) {
49                 val f = frames[idx]
50                 if (f.role == AnimationFrameRole.NORMAL) nextOnions.add(f)
51             }
52         }
53     }
54
55     return AnimationRenderPlan(
56         backgroundFrame = bgFrame,
57         currentFrame = current,
58         foregroundFrame = fgFrame,
59         previousOnionFrames = prevOnions,
60         nextOnionFrames = nextOnions
61     )
62 }
```

14. app/src/main/java/com/wetinknext/domain/animation/AnimationTimeline.kt

```

1 package com.wetinknext.domain.animation
2
3 /**
4  * Creates a default set of animation frames from a list of layer IDs.
5  * Each layer becomes its own frame. System "Background" (usually ID 1) should be filtered out by the caller.
6  */
7 fun createFramesFromLayers(layerIds: List<Long>): List<AnimationFrame> {
8     return layerIds.map { layerId ->
9         AnimationFrame(
10             id = layerId, // Use layerId as initial frame ID for simplicity in 1:1 mapping
11             layerIds = listOf(layerId)
12         )
13     }
14 }
15
16 /**
17  * Ensures the animation document is valid given the currently existing layers.
18  * Removes dead layer references, empty frames, and enforces range limits.
19  */
20 fun normalizedAnimationDocument(
21     document: AnimationDocument,
22     existingLayerIds: Set<Long>,
23 ): AnimationDocument {
24     return normalizedAnimationDocument(
25         document = document,
26         layersInDocumentOrder = existingLayerIds.map { AnimationLayerInfo(id = it) },
27     )
28 }
29
30 /** Minimal layer data needed by the timeline. Kept independent from OpenGL resources. */
31 data class AnimationLayerInfo(
32     val id: Long,
33     val isCanvasBackgroundLayer: Boolean = false,
34     val isClipping: Boolean = false,
35 )
36
37 /**
38  * Normalizes frames against the real document order. Canvas background never becomes a frame.
39  * Layer IDs inside every frame are also kept in document order, which is required for clipping.
40  */
41 fun normalizedAnimationDocument(
42     document: AnimationDocument,
43     layersInDocumentOrder: List<AnimationLayerInfo>,
44 ): AnimationDocument {
45     val order = layersInDocumentOrder.mapIndexed { index, layer -> layer.id to index }.toMap()
46     val animatableIds = layersInDocumentOrder
47         .filterNot { it.isCanvasBackgroundLayer }
48         .map { it.id }
49     val usedLayerIds = mutableSetOf<Long>()
50
51     val normalizedFrames = document.frames.mapNotNull { frame ->
52         val validLayerIds = frame.layerIds
53             .filter { id -> id in order && id !in usedLayerIds }
54             .sortedBy { order.getOrDefault(it, 0) }
55
56         if (validLayerIds.isEmpty()) {
57             null
58         } else {
59             usedLayerIds.addAll(validLayerIds)
60             frame.copy(
61                 layerIds = validLayerIds,
62                 holdFrames = frame.holdFrames.coerceIn(1, 120)
63             )
64         }
65     }
66     // A layer can belong to only one frame. Any newly created document layer gets its own frame.
67     val missingLayerIds = animatableIds.filter { it !in usedLayerIds }
68     val newFrames = missingLayerIds.map { layerId ->
69         AnimationFrame(id = layerId, layerIds = listOf(layerId))
70     }
71     return document.copy(
72         framesPerSecond = document.framesPerSecond.coerceIn(1, 60),
73         frames = normalizedFrames + newFrames
74     )
75 }
76
77 /**
78  * Combines multiple layers into a single animation frame.
79  */
80 fun groupLayersIntoFrame(
81     document: AnimationDocument,
82     layerIdsToGroup: List<Long>,
83     newFrameId: Long = System.nanoTime()
84 ): AnimationDocument {
85     val orderedIds = layerIdsToGroup.distinct()
86     val idsSet = orderedIds.toSet()
87     if (idsSet.isEmpty()) return document
88
89     val insertionIndex = document.frames.indexOfFirst { frame -> frame.layerIds.any { it in idsSet } }
90     val cleanedFrames = document.frames.map { frame ->

```

```

91     frame.copy(layerIds = frame.layerIds.filter { it !in idsSet })
92   }.filter { it.layerIds.isNotEmpty() }.toMutableList()
93   val newFrame = AnimationFrame(id = newFrameId, layerIds = orderedIds)
94   val targetIndex = if (insertionIndex < 0) cleanedFrames.size else insertionIndex.coerceAtMost(cleanedFrames.size)
95   cleanedFrames.add(targetIndex, newFrame)
96   return document.copy(frames = cleanedFrames)
97 }
98
99 sealed interface AnimationFrameGroupingValidation {
100   data class Valid(val layerIdsInDocumentOrder: List<Long>) : AnimationFrameGroupingValidation
101   data object EmptySelection : AnimationFrameGroupingValidation
102   data class MissingClippingBase(val clippingLayerId: Long) : AnimationFrameGroupingValidation
103   data class MissingClippingLayer(val baseLayerId: Long, val clippingLayerId: Long) : AnimationFrameGroupingValidation
104 }
105
106 /**
107  * A clipping layer and its base must live in one animation frame: otherwise a frame would render
108  * with a different compositing result than the document. The selection is returned in document order.
109  */
110 fun validateAnimationFrameGrouping(
111   selectedLayerIds: Set<Long>,
112   layersInDocumentOrder: List<AnimationLayerInfo>,
113 ): AnimationFrameGroupingValidation {
114   val layers = layersInDocumentOrder.filterNot { it.isCanvasBackgroundLayer }
115   val selected = selectedLayerIds.intersect(layers.map { it.id }.toSet())
116   if (selected.isEmpty()) return AnimationFrameGroupingValidation.EmptySelection
117
118   val baseByClippingId = mutableMapOf<Long, Long>()
119   layers.forEachIndexed { index, layer ->
120     if (layer.isClipping) {
121       val base = layers.subList(0, index).lastOrNull { !it.isClipping }
122       ?: return AnimationFrameGroupingValidation.MissingClippingBase(layer.id)
123       baseByClippingId[layer.id] = base.id
124       if (layer.id in selected && base.id !in selected) {
125         return AnimationFrameGroupingValidation.MissingClippingLayer(layer.id, base.id)
126       }
127     }
128   }
129   baseByClippingId.forEach { (clippingId, baseId) ->
130     if (baseId in selected && clippingId !in selected) {
131       return AnimationFrameGroupingValidation.MissingClippingLayer(baseId, clippingId)
132     }
133   }
134   return AnimationFrameGroupingValidation.Valid(layers.filter { it.id in selected }.map { it.id })
135 }
136
137 /**
138  * Expands frames into a flat list of frame IDs to be played back, accounting for holdFrames.
139  */
140 fun expandedPlaybackFrameIds(
141   frames: List<AnimationFrame>,
142 ): List<Long> {
143   return frames.flatMap { frame ->
144     List(frame.holdFrames) { frame.id }
145   }
146 }
147
148 data class PlaybackStep(
149   val nextIndex: Int,
150   val nextDirection: Int, // 1 for forward, -1 for backward
151   val shouldStop: Boolean = false
152 )
153
154 /**
155  * Calculates the next index in the playback sequence.
156  */
157 fun nextPlaybackIndex(
158   currentIndex: Int,
159   direction: Int,
160   frameCount: Int,
161   mode: AnimationPlaybackMode,
162 ): PlaybackStep {
163   if (frameCount <= 0) return PlaybackStep(0, 1, true)
164   if (frameCount == 1) return PlaybackStep(0, 1, mode == AnimationPlaybackMode.ONE_SHOT)
165
166   var nextDir = direction
167   var nextIdx = currentIndex + direction
168   var stop = false
169
170   when (mode) {
171     AnimationPlaybackMode.LOOP -> {
172       if (nextIdx >= frameCount) nextIdx = 0
173       if (nextIdx < 0) nextIdx = frameCount - 1
174     }
175     AnimationPlaybackMode.PING_PONG -> {
176       if (nextIdx >= frameCount) {
177         nextIdx = (frameCount - 2).coerceAtLeast(0)
178         nextDir = -1
179       } else if (nextIdx < 0) {
180         nextIdx = (1).coerceAtMost(frameCount - 1)
181         nextDir = 1
182       }
183     }
184   }

```

```
183     }
184     AnimationPlaybackMode.ONE_SHOT -> {
185         if (nextIdx >= frameCount) {
186             nextIdx = frameCount - 1
187             stop = true
188         }
189     }
190 }
191
192 return PlaybackStep(nextIdx, nextDir, stop)
193 }
```

15. app/src/main/java/com/wetinknext/domain/document/ProjectDocument.kt

```

1 package com.wetinknext.domain.document
2
3 import com.wetinknext.domain.animation.AnimationDocument
4 import com.wetinknext.engine.brush.BrushPreset
5 import com.wetinknext.engine.canvas.BlendMode
6 import kotlinx.serialization.Serializable
7 import java.util.UUID
8
9 /**
10  * Persisted project structure. Pixel data deliberately lives outside this JSON
11  * document: [LayerDocument.pixelFile] points at its future tile/container data.
12  */
13 @Serializable
14 data class ProjectDocument(
15     val id: String,
16     val name: String,
17     val width: Int,
18     val height: Int,
19     val dpi: Int,
20     val colorProfile: String,
21     val createdAt: Long,
22     val updatedAt: Long,
23     val layers: List<LayerDocument>,
24     val brushes: List<BrushPreset> = emptyList(),
25     val animation: AnimationDocument = AnimationDocument(),
26     val activeLayerId: Long? = null,
27     /** Changes only when the project-wide preview is rebuilt. */
28     val thumbnailVersion: Long = 0L,
29     val version: Int = CURRENT_VERSION,
30 ) {
31     init {
32         require(id.isNotBlank()) { "Project id must not be blank" }
33         require(name.isNotBlank()) { "Project name must not be blank" }
34         require(width > 0 && height > 0) { "Project dimensions must be positive" }
35         require(dpi in 1..2400) { "Project DPI must be within 1..2400" }
36         require(colorProfile.isNotBlank()) { "Project color profile must not be blank" }
37         require(layers.isNotEmpty()) { "A project must contain at least one layer" }
38         require(layers.map(LayerDocument::id).distinct().size == layers.size) {
39             "Layer ids must be unique"
40         }
41         require(activeLayerId == null || layers.any { it.id == activeLayerId }) {
42             "Active layer must belong to the project"
43         }
44         require(version >= 1) { "Project version must be positive" }
45         require(thumbnailVersion >= 0L) { "Project thumbnail version must not be negative" }
46     }
47
48     fun withUpdatedTimestamp(nowMillis: Long): ProjectDocument = copy(updatedAt = nowMillis)
49
50     companion object {
51         const val CURRENT_VERSION = 1
52         const val DEFAULT_WIDTH = 1500
53         const val DEFAULT_HEIGHT = 2000
54         const val DEFAULT_DPI = 300
55         const val SRGB_PROFILE = "sRGB IEC61966-2.1"
56         const val DISPLAY_P3_PROFILE = "Display P3"
57         const val MIN_DIMENSION = 256
58         const val MAX_DIMENSION = 8192
59         const val MIN_DPI = 72
60         const val MAX_DPI = 1200
61         const val MAX_INITIAL_GPU_BYTES = 256L * 1024L * 1024L
62         private const val INITIAL_RENDER_TARGET_COUNT = 3L
63         private const val RGBA16F_BYTES_PER_PIXEL = 8L
64         val SUPPORTED_COLOR_PROFILES = setOf(SRGB_PROFILE, DISPLAY_P3_PROFILE)
65
66         fun newUntitled(
67             name: String = "Untitled",
68             width: Int = DEFAULT_WIDTH,
69             height: Int = DEFAULT_HEIGHT,
70             dpi: Int = DEFAULT_DPI,
71             colorProfile: String = SRGB_PROFILE,
72             nowMillis: Long = System.currentTimeMillis(),
73             id: String = UUID.randomUUID().toString(),
74         ): ProjectDocument {
75             val normalizedProfile = normalizeColorProfile(colorProfile)
76             validateNewCanvas(width, height, dpi, normalizedProfile)
77             return ProjectDocument(
78                 id = id,
79                 name = name,
80                 width = width,
81                 height = height,
82                 dpi = dpi,
83                 colorProfile = normalizedProfile,
84                 createdAt = nowMillis,
85                 updatedAt = nowMillis,
86                 layers = listOf(
87                     LayerDocument(
88                         id = BACKGROUND_LAYER_ID,
89                         name = "Фон",
90                         visible = true,

```

```

91         locked = true,
92         opacity = 1f,
93         blendMode = BlendMode.NORMAL,
94         pixelFile = pixelFileFor(BACKGROUND_LAYER_ID),
95     ),
96     LayerDocument(
97         id = FIRST_PAINT_LAYER_ID,
98         name = "Слой 1",
99         visible = true,
100        locked = false,
101        opacity = 1f,
102        blendMode = BlendMode.NORMAL,
103        pixelFile = pixelFileFor(FIRST_PAINT_LAYER_ID),
104    ),
105 ),
106 activeLayerId = FIRST_PAINT_LAYER_ID,
107 )
108 }
109
110 /** Validates the initial two layer targets plus the transient stroke target. */
111 fun validateNewCanvas(width: Int, height: Int, dpi: Int, colorProfile: String) {
112     require(width in MIN_DIMENSION..MAX_DIMENSION) { "Canvas width must be within $MIN_DIMENSION..$MAX_DIMENSION px" }
113     require(height in MIN_DIMENSION..MAX_DIMENSION) { "Canvas height must be within $MIN_DIMENSION..$MAX_DIMENSION
px" }
114     require(dpi in MIN_DPI..MAX_DPI) { "Canvas DPI must be within $MIN_DPI..$MAX_DPI" }
115     require(normalizeColorProfile(colorProfile) in SUPPORTED_COLOR_PROFILES) { "Unsupported color profile:
$colorProfile" }
116     val estimatedBytes = width.toLong() * height * INITIAL_RENDER_TARGET_COUNT * RGBA16F_BYTES_PER_PIXEL
117     require(estimatedBytes <= MAX_INITIAL_GPU_BYTES) {
118         "Canvas requires ${estimatedBytes / (1024 * 1024)} MB GPU memory; limit is ${MAX_INITIAL_GPU_BYTES / (1024 *
1024)} MB"
119     }
120 }
121
122 /** Keeps older UI values (`sRGB`) compatible with the persisted profile identifier. */
123 fun normalizeColorProfile(value: String): String = when (value.trim()) {
124     "sRGB", SRGB_PROFILE -> SRGB_PROFILE
125     "Display P3", DISPLAY_P3_PROFILE -> DISPLAY_P3_PROFILE
126     else -> value.trim()
127 }
128
129 fun pixelFileFor(layerId: Long): String = "layers/$layerId.tiles"
130
131 private const val BACKGROUND_LAYER_ID = 1L
132 private const val FIRST_PAINT_LAYER_ID = 2L
133 }
134 }
135
136 @Serializable
137 data class LayerDocument(
138     val id: Long,
139     val name: String,
140     val visible: Boolean,
141     val locked: Boolean,
142     val opacity: Float,
143     val blendMode: BlendMode,
144     val pixelFile: String,
145     /** Increments whenever the runtime layer pixels change; useful for preview invalidation. */
146     val thumbnailVersion: Long = 0L,
147 ) {
148     init {
149         require(id >= 0L) { "Layer id must not be negative" }
150         require(name.isNotBlank()) { "Layer name must not be blank" }
151         require(opacity in 0f..1f) { "Layer opacity must be within 0..1" }
152         require(pixelFile.isNotBlank()) { "Layer pixel file must not be blank" }
153         require(thumbnailVersion >= 0L) { "Layer thumbnail version must not be negative" }
154     }
155 }

```

16. app/src/main/java/com/wetinknext/domain/document/WetInkProjectContainer.kt

```

1  package com.wetinknext.domain.document
2
3  import kotlinx.serialization.Serializable
4  import kotlinx.serialization.decodeFromString
5  import kotlinx.serialization.encodeToString
6  import kotlinx.serialization.json.Json
7  import java.io.BufferedInputStream
8  import java.io.BufferedOutputStream
9  import java.io.ByteArrayOutputStream
10 import java.io.File
11 import java.io.IOException
12 import java.nio.file.AtomicMoveNotSupportedException
13 import java.nio.file.Files
14 import java.nio.file.StandardCopyOption
15 import java.util.zip.ZipEntry
16 import java.util.zip.ZipFile
17 import java.util.zip.ZipOutputStream
18
19 @Serializable
20 data class ProjectManifest(
21     val format: String = FORMAT,
22     val formatVersion: Int = FORMAT_VERSION,
23     val documentId: String,
24     val name: String,
25     val width: Int,
26     val height: Int,
27     val documentPath: String = DOCUMENT_ENTRY,
28     val thumbnailPath: String? = null,
29     val thumbnailVersion: Long = 0L,
30     val createdAt: Long,
31     val updatedAt: Long,
32     val layers: List<StoredLayer>,
33 ) {
34     companion object {
35         const val FORMAT = "wetink"
36         const val FORMAT_VERSION = 1
37         const val DOCUMENT_ENTRY = "document.json"
38     }
39 }
40
41 @Serializable
42 data class StoredLayer(
43     val id: Long,
44     val path: String,
45     val byteSize: Long,
46 )
47
48 /** Binary payloads deliberately sit next to the JSON document, not inside it. */
49 data class ProjectAssets(
50     val layerFiles: Map<Long, ByteArray>,
51     val thumbnailWebp: ByteArray? = null,
52     val layerPreviewsWebp: Map<Long, ByteArray> = emptyMap(),
53 )
54
55 data class LoadedWetInkProject(
56     val document: ProjectDocument,
57     val manifest: ProjectManifest,
58     val assets: ProjectAssets,
59 )
60
61 /**
62  * Project ZIP reader/writer. A project is published only by an atomic rename
63  * from `name.wetink.tmp` to `name.wetink`, so a failed save cannot corrupt the
64  * previously published project.
65  */
66 class WetInkProjectContainer(
67     private val clockMillis: () -> Long = System::currentTimeMillis,
68     private val json: Json = Json {
69         encodeDefaults = true
70         ignoreUnknownKeys = true
71         prettyPrint = true
72     },
73 ) {
74     fun save(
75         projectFile: File,
76         document: ProjectDocument,
77         assets: ProjectAssets,
78     ): LoadedWetInkProject {
79         require(projectFile.extension.equals(FILE_EXTENSION, ignoreCase = true)) {
80             "Project file must use .$FILE_EXTENSION extension"
81         }
82         validateAssets(document, assets)
83
84         val savedTimestamp = clockMillis()
85         val savedDocument = document.copy(
86             updatedAt = savedTimestamp,
87             thumbnailVersion = if (assets.thumbnailWebp != null) savedTimestamp else document.thumbnailVersion,
88         )
89         val manifest = ProjectManifest(
90             documentId = savedDocument.id,

```

```

91         name = savedDocument.name,
92         width = savedDocument.width,
93         height = savedDocument.height,
94         thumbnailPath = assets.thumbnailWebp?.let { THUMBNAIL_ENTRY },
95         thumbnailVersion = savedDocument.thumbnailVersion,
96         createdAt = savedDocument.createdAt,
97         updatedAt = savedDocument.updatedAt,
98         layers = savedDocument.layers.map { layer ->
99             StoredLayer(
100                 id = layer.id,
101                 path = layer.pixelFile,
102                 byteSize = checkNotNull(assets.layerTiles[layer.id]).size.toLong(),
103             )
104         },
105     )
106
107     val parent = projectFile.parentFile ?: throw IOException("Project file requires a parent folder")
108     if (!parent.exists() && !parent.mkdirs()) throw IOException("Cannot create ${parent.path}")
109     val temporaryFile = File(parent, "${projectFile.name}.tmp")
110     if (temporaryFile.exists() && !temporaryFile.delete()) {
111         throw IOException("Cannot replace stale temporary project ${temporaryFile.path}")
112     }
113
114     var published = false
115     try {
116         BufferedOutputStream(temporaryFile.outputStream()).use { output ->
117             ZipOutputStream(output).use { zip ->
118                 // Write large layer data first; metadata becomes visible only
119                 // after every payload has been written and the ZIP is closed.
120                 savedDocument.layers.forEach { layer ->
121                     writeEntry(zip, layer.pixelFile, checkNotNull(assets.layerTiles[layer.id]))
122                 }
123                 writeEntry(zip, ProjectManifest.DOCUMENT_ENTRY, json.encodeToString(savedDocument).encodeToByteArray())
124                 assets.thumbnailWebp?.let { writeEntry(zip, THUMBNAIL_ENTRY, it) }
125                 assets.layerPreviewsWebp.forEach { (layerId, preview) ->
126                     writeEntry(zip, previewPathFor(layerId), preview)
127                 }
128                 writeEntry(zip, MANIFEST_ENTRY, json.encodeToString(manifest).encodeToByteArray())
129             }
130         }
131         if (temporaryFile.length() <= 0L) throw IOException("Temporary project is empty")
132         atomicPublish(temporaryFile, projectFile)
133         published = true
134     } finally {
135         if (!published && temporaryFile.exists()) temporaryFile.delete()
136     }
137
138     return LoadedWetInkProject(savedDocument, manifest, assets)
139 }
140
141 fun load(projectFile: File): LoadedWetInkProject {
142     require(projectFile.isFile) { "Project does not exist: ${projectFile.path}" }
143     ZipFile(projectFile).use { zip ->
144         val manifest = readJson<ProjectManifest>(zip, MANIFEST_ENTRY)
145         require(manifest.format == ProjectManifest.FORMAT) { "Unsupported project format" }
146         require(manifest.formatVersion == ProjectManifest.FORMAT_VERSION) { "Unsupported project version" }
147
148         val document = readJson<ProjectDocument>(zip, manifest.documentPath)
149         require(document.id == manifest.documentId) { "Manifest/document id mismatch" }
150         require(document.updatedAt == manifest.updatedAt) { "Manifest/document timestamp mismatch" }
151         require(manifest.layers.map { it.id } == document.layers.map { it.id }) {
152             "Manifest/document layer order mismatch"
153         }
154
155         val tiles = document.layers.associate { layer ->
156             val stored = manifest.layers.first { it.id == layer.id }
157             require(stored.path == layer.pixelFile) { "Layer path mismatch for ${layer.id}" }
158             val bytes = readEntry(zip, layer.pixelFile)
159             require(bytes.size.toLong() == stored.byteSize) { "Layer size mismatch for ${layer.id}" }
160             layer.id to bytes
161         }
162         val previews = document.layers.mapNotNull { layer ->
163             val entry = zip.getEntry(previewPathFor(layer.id)) ?: return@mapNotNull null
164             layer.id to readEntry(zip, entry)
165         }.toMap()
166         val thumbnail = manifest.thumbnailPath?.let { readEntry(zip, it) }
167
168         return LoadedWetInkProject(
169             document = document,
170             manifest = manifest,
171             assets = ProjectAssets(tiles, thumbnail, previews),
172         )
173     }
174 }
175
176 /** Reads just the compact manifest; no document JSON or tile payload is loaded. */
177 fun readManifest(projectFile: File): ProjectManifest {
178     require(projectFile.isFile) { "Project does not exist: ${projectFile.path}" }
179     ZipFile(projectFile).use { zip ->
180         val manifest = readJson<ProjectManifest>(zip, MANIFEST_ENTRY)
181         require(manifest.format == ProjectManifest.FORMAT) { "Unsupported project format" }
182         require(manifest.formatVersion == ProjectManifest.FORMAT_VERSION) { "Unsupported project version" }

```

```

183         return manifest
184     }
185 }
186
187 fun readThumbnail(projectFile: File): ByteArray? {
188     ZipFile(projectFile).use { zip ->
189         val manifest = readJson<ProjectManifest>(zip, MANIFEST_ENTRY)
190         return manifest.thumbnailPath?.let { readEntry(zip, it) }
191     }
192 }
193
194 /** Opens only project metadata. Tile bytes remain in the ZIP until requested. */
195 fun readDocument(projectFile: File): ProjectDocument {
196     require(projectFile.isFile) { "Project does not exist: ${projectFile.path}" }
197     ZipFile(projectFile).use { zip ->
198         val manifest = readJson<ProjectManifest>(zip, MANIFEST_ENTRY)
199         require(manifest.format == ProjectManifest.FORMAT) { "Unsupported project format" }
200         require(manifest.formatVersion == ProjectManifest.FORMAT_VERSION) { "Unsupported project version" }
201         val document = readJson<ProjectDocument>(zip, manifest.documentPath)
202         require(document.id == manifest.documentId) { "Manifest/document id mismatch" }
203         require(document.updatedAt == manifest.updatedAt) { "Manifest/document timestamp mismatch" }
204         return document
205     }
206 }
207
208 private fun validateAssets(document: ProjectDocument, assets: ProjectAssets) {
209     val expectedIds = document.layers.map { it.id }.toSet()
210     require(assets.layerTiles.keys == expectedIds) { "Every document layer requires one tile payload" }
211     require(assets.layerPreviewsWebp.keys.all(expectedIds::contains)) {
212         "Preview references an unknown layer"
213     }
214     document.layers.forEach { layer ->
215         require(isSafeLayerPath(layer.pixelFile)) { "Unsafe layer path: ${layer.pixelFile}" }
216     }
217 }
218
219 private fun atomicPublish(temporaryFile: File, projectFile: File) {
220     try {
221         Files.move(
222             temporaryFile.toPath(),
223             projectFile.toPath(),
224             StandardCopyOption.ATOMIC_MOVE,
225             StandardCopyOption.REPLACE_EXISTING,
226         )
227     } catch (error: AtomicMoveNotSupportedException) {
228         throw IOException("Atomic project publish is not supported for ${projectFile.path}", error)
229     }
230 }
231
232 private fun writeEntry(zip: ZipOutputStream, path: String, bytes: ByteArray) {
233     zip.putNextEntry(ZipEntry(path))
234     zip.write(bytes)
235     zip.closeEntry()
236 }
237
238 private inline fun <reified T> readJson(zip: ZipFile, path: String): T =
239     json.decodeFromString(readEntry(zip, path).decodeToString())
240
241 private fun readEntry(zip: ZipFile, path: String): ByteArray {
242     val entry = zip.getEntry(path) ?: throw IOException("Missing project entry: $path")
243     return readEntry(zip, entry)
244 }
245
246 private fun readEntry(zip: ZipFile, entry: ZipEntry): ByteArray {
247     if (entry.size > MAX_ENTRY_BYTES) throw IOException("Project entry is too large: ${entry.name}")
248     BufferedInputStream(zip.getInputStream(entry)).use { input ->
249         val output = ByteArrayOutputStream(entry.size.coerceAtLeast(0L).coerceAtMost(MAX_ENTRY_BYTES).toInt())
250         val buffer = ByteArray(READ_BUFFER_BYTES)
251         var total = 0L
252         while (true) {
253             val read = input.read(buffer)
254             if (read < 0) break
255             total += read
256             if (total > MAX_ENTRY_BYTES) {
257                 throw IOException("Project entry is too large: ${entry.name}")
258             }
259             output.write(buffer, 0, read)
260         }
261         return output.toByteArray()
262     }
263 }
264
265 private fun isSafeLayerPath(path: String): Boolean =
266     path.startsWith("layers/") && !path.contains("..") && !path.startsWith('/')
267
268 private companion object {
269     const val FILE_EXTENSION = "wetink"
270     const val MANIFEST_ENTRY = "manifest.json"
271     const val THUMBNAIL_ENTRY = "thumbnail.webp"
272     const val MAX_ENTRY_BYTES = 256L * 1024L * 1024L
273     const val READ_BUFFER_BYTES = 16 * 1024
274 }

```

```
275         fun previewPathFor(layerId: Long): String = "previews/$layerId.webp"
276     }
277 }
```

17. app/src/main/java/com/wetinknext/engine/brush/BlendController.kt

```
1 package com.wetinknext.engine.brush
2
3 import android.opengl.GLES30
4
5 class BlendController {
6
7     fun begin(policy: BlendPolicy) {
8         GLES30.glEnable(GLES30.GL_BLEND)
9         // NON_BUILDUP needs a dedicated accumulation mask. Until that pass is
10        // introduced, both policies use the correct premultiplied source-over.
11        GLES30.glBlendEquation(GLES30.GL_FUNC_ADD)
12        GLES30.glBlendFunc(
13            GLES30.GL_ONE,
14            GLES30.GL_ONE_MINUS_SRC_ALPHA,
15        )
16    }
17
18    fun end() {
19        GLES30.glBlendEquation(GLES30.GL_FUNC_ADD)
20        GLES30.glBlendFunc(
21            GLES30.GL_ONE,
22            GLES30.GL_ONE_MINUS_SRC_ALPHA,
23        )
24        GLES30.glDisable(GLES30.GL_BLEND)
25    }
26 }
```

18. app/src/main/java/com/wetinknext/engine/brush/BrushDynamics.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlin.math.pow
4 import kotlin.math.sqrt
5
6 object BrushDynamics {
7
8     fun tiltMagnitude(
9         tiltX: Float,
10        tiltY: Float,
11    ): Float {
12        return sqrt(
13            tiltX * tiltX +
14            tiltY * tiltY,
15        ).coerceIn(0f, 1f)
16    }
17
18    /**
19     * Resolves the final radius and opacity for a single dab based on input.
20     */
21    fun resolve(
22        settings: BrushSettings,
23        pressure: Float,
24        tiltX: Float,
25        tiltY: Float,
26        out: ResolvedDab,
27    ) {
28        val p = pressure
29            .coerceIn(0f, 1f)
30            .let { it.toDouble().pow(settings.pressureGamma.toDouble()).toFloat() }
31
32        val tilt = tiltMagnitude(tiltX, tiltY)
33
34        val pressureSize = if (settings.pressureToSize) {
35            settings.minSizeRatio + (1f - settings.minSizeRatio) * p
36        } else {
37            1f
38        }
39        val tiltSize = 1f + settings.tiltToSize * tilt
40        out.radius = (
41            settings.baseRadiusPx * pressureSize * tiltSize
42        ).coerceAtLeast(0.25f)
43
44        val pressureOpacity = if (settings.pressureToOpacity) p else 1f
45
46        // Tilt must not make a dab more opaque.
47        val tiltOpacity = 1f - (0.45f * settings.tiltToOpacity * tilt)
48        out.coverage = (pressureOpacity * tiltOpacity).coerceIn(0f, 1f)
49
50        // Local dab alpha only. Global brush opacity is applied during stroke blit.
51        out.opacity = out.coverage
52    }
53 }
54
55 data class ResolvedDab(
56     var radius: Float = 0f,
57     var opacity: Float = 1f,
58     var coverage: Float = 1f,
59 )
```

19. app/src/main/java/com/wetinknext/engine/brush/BrushLibrary.kt

```
1 package com.wetinknext.engine.brush
2
3 import androidx.compose.material.icons.Icons
4 import androidx.compose.material.icons.filled.Brush
5 import androidx.compose.material.icons.filled.Create
6 import androidx.compose.material.icons.filled.Star
7 import androidx.compose.ui.graphics.vector.ImageVector
8
9 data class BrushUiCategory(
10     val name: String,
11     val icon: ImageVector,
12     val brushes: List<BrushPreset>,
13 )
14
15 object BrushLibrary {
16     val pencil6B = BrushPreset(
17         id = "pencil_6b",
18         settings = BrushSettings(
19             name = "Pencil 6B",
20             category = "Pencil",
21             renderMode = BrushRenderMode.STAMP,
22             baseRadiusPx = 8f,
23             opacity = 0.82f,
24             flow = 0.65f,
25             spacing = 0.08f,
26             spacingUsesDiameter = true,
27             hardness = 0.72f,
28             smoothing = 0.08f,
29             streamline = 0.04f,
30             pressureToSize = true,
31             pressureToOpacity = true,
32             pressureGamma = 1.25f,
33             minSizeRatio = 0.30f,
34             tiltToSize = 0.55f,
35             grainAssetPath = "asset:brush/pencil_6b_grain.png",
36             grainCanvasLocked = true,
37             grainScale = 5.5f,
38             textureDepth = 0.65f,
39             textureContrast = 1.35f,
40             blendPolicy = BlendPolicy.NORMAL_BUILDUP,
41         ),
42     )
43
44     val gPen = BrushPreset(
45         id = "g_pen",
46         settings = BrushSettings(
47             name = "G-Pen",
48             category = "Ink",
49             renderMode = BrushRenderMode.RIBBON,
50             baseRadiusPx = 4.5f,
51             opacity = 1f,
52             flow = 1f,
53             smoothing = 0.35f,
54             streamline = 0.22f,
55             pressureToSize = true,
56             pressureToOpacity = false,
57             pressureGamma = 1.7f,
58             minSizeRatio = 0.06f,
59             blendPolicy = BlendPolicy.NON_BUILDUP,
60         )
61     )
62
63     val allCategories = listOf(
64         BrushUiCategory(
65             name = "Pencil",
66             icon = Icons.Default.Create,
67             brushes = listOf(pencil6B)
68         ),
69         BrushUiCategory(
70             name = "Ink",
71             icon = Icons.Default.Brush,
72             brushes = listOf(gPen)
73         ),
74         BrushUiCategory(
75             name = "Favorites",
76             icon = Icons.Default.Star,
77             brushes = emptyList()
78         )
79     )
80 }
```

20. app/src/main/java/com/wetinknext/engine/brush/BrushPreset.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlinx.serialization.Serializable
4
5 @Serializable
6 data class BrushPreset(
7     val id: String,
8     val settings: BrushSettings,
9 )
```

21. app/src/main/java/com/wetinknext/engine/brush/BrushRepository.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlinx.serialization.encodeToString
4 import kotlinx.serialization.json.Json
5
6 class BrushRepository(
7     private val json: Json = defaultJson,
8 ) {
9     fun encode(preset: BrushPreset): String =
10         json.encodeToString(preset)
11
12     fun decode(value: String): BrushPreset =
13         json.decodeFromString<BrushPreset>(value).let { preset ->
14             preset.copy(settings = preset.settings.resolved())
15         }
16
17     companion object {
18         val defaultJson = Json {
19             encodeDefaults = true
20             ignoreUnknownKeys = true
21             explicitNulls = false
22         }
23     }
24 }
```

22. app/src/main/java/com/wetinknext/engine/brush/BrushSettings.kt

```

1  package com.wetinknext.engine.brush
2
3  import kotlinx.serialization.Serializable
4
5  @Serializable
6  enum class BrushRenderMode { STAMP, RIBBON, WET }
7
8  @Serializable
9  enum class BlendPolicy { NORMAL_BUILDUP, NON_BUILDUP }
10
11 @Serializable
12 enum class RibbonCap { ROUND, BUTT }
13
14 @Serializable
15 enum class RibbonJoin { MITER, ROUND, BEVEL }
16
17 @Serializable
18 data class RibbonSettings(
19     val minWidthRatio: Float = 0.06f,
20     val taperStartPx: Float = 8f,
21     val taperEndPx: Float = 14f,
22     val miterLimit: Float = 3f,
23     val aaWidthPx: Float = 1f,
24     val cap: RibbonCap = RibbonCap.ROUND,
25     val join: RibbonJoin = RibbonJoin.ROUND,
26     val minPointDistancePx: Float = 0.5f,
27 )
28
29 @Serializable
30 data class WetSettings(val wetness: Float = 0f, val spread: Float = 0f, val bleed: Float = 0f)
31
32 @Serializable
33 data class BrushSettings(
34     val name: String = "Debug Stamp",
35     val category: String = "Debug",
36     val renderMode: BrushRenderMode = BrushRenderMode.STAMP,
37     /**
38      * Internal brush radius in document pixels.
39      * UI displays diameter = baseRadiusPx * 2.
40      */
41     val baseRadiusPx: Float = 14f,
42     val opacity: Float = 1f,
43     val flow: Float = 1f,
44     val spacing: Float = 0.16f,
45     val spacingUsesDiameter: Boolean = true,
46     val hardness: Float = 1f,
47     val colorArgb: Long = 0xFF000000L,
48     val smoothing: Float = 0f,
49     val streamline: Float = 0f,
50     val antiAliasLevel: Int = 2,
51     val useTempStrokeBuffer: Boolean = false,
52     val pressureToSize: Boolean = true,
53     val pressureToOpacity: Boolean = true,
54     val pressureGamma: Float = 1f,
55     val minSizeRatio: Float = 0.05f,
56     val velocityToSize: Float = 0f,
57     val velocityToOpacity: Float = 0f,
58     val tiltToSize: Float = 0f,
59     val tiltToOpacity: Float = 0f,
60     val tiltToRotation: Float = 0f,
61     val rotationJitter: Float = 0f,
62     val sizeJitter: Float = 0f,
63     val opacityJitter: Float = 0f,
64     val scatter: Float = 0f,
65     val shapeAssetPath: String? = null,
66     val grainAssetPath: String? = null,
67     val grainScale: Float = 1f,
68     val grainCanvasLocked: Boolean = true,
69     val rgbToAlpha: Boolean = false,
70     val textureContrast: Float = 1f,
71     val textureDepth: Float = 1f,
72     val pixelPen: Boolean = false,
73     val squareStroke: Boolean = false,
74     val noAntialias: Boolean = false,
75     val blendPolicy: BlendPolicy = BlendPolicy.NORMAL_BUILDUP,
76     val ribbon: RibbonSettings = RibbonSettings(),
77     val wet: WetSettings = WetSettings(),
78 ) {
79     /** Returns a settings copy safe for every engine path. */
80     fun resolved(): BrushSettings {
81         val safeRadius = baseRadiusPx.coerceIn(0.25f, 4096f)
82         val safeOpacity = opacity.coerceIn(0f, 1f)
83         val safeFlow = flow.coerceIn(0f, 1f)
84         val safeSpacing = spacing.coerceIn(0.001f, 4f)
85         val safeGamma = pressureGamma.coerceIn(0.05f, 8f)
86         val safeMinSize = minSizeRatio.coerceIn(0.01f, 1f)
87
88         val safeVelocityToSize = velocityToSize.coerceIn(0f, 1f)
89         val safeVelocityToOpacity = velocityToOpacity.coerceIn(0f, 1f)
90         val safeTiltToSize = tiltToSize.coerceIn(0f, 1f)

```

```

91     val safeTiltToOpacity = tiltToOpacity.coerceIn(0f, 1f)
92     val safeTiltToRotation = tiltToRotation.coerceIn(0f, 1f)
93     val safeSizeJitter = sizeJitter.coerceIn(0f, 1f)
94     val safeOpacityJitter = opacityJitter.coerceIn(0f, 1f)
95     val safeRotationJitter = rotationJitter.coerceIn(0f, 1f)
96     val safeScatter = scatter.coerceAtLeast(0f)
97
98     val safeGrainScale = grainScale.coerceIn(0.0001f, 256f)
99     val safeTextureDepth = textureDepth.coerceIn(0f, 1f)
100    val safeTextureContrast = textureContrast.coerceIn(0f, 4f)
101    val safeAa = antiAliasLevel.coerceIn(0, 3)
102
103    val safeRibbon = ribbon.copy(
104        minWidthRatio = ribbon.minWidthRatio.coerceIn(0.01f, 1f),
105        taperStartPx = ribbon.taperStartPx.coerceAtLeast(0f),
106        taperEndPx = ribbon.taperEndPx.coerceAtLeast(0f),
107        miterLimit = ribbon.miterLimit.coerceIn(1f, 32f),
108        aaWidthPx = ribbon.aaWidthPx.coerceIn(0f, 16f),
109        minPointDistancePx = ribbon.minPointDistancePx.coerceIn(0.05f, 32f),
110    )
111    val safeWet = wet.copy(
112        wetness = wet.wetness.coerceIn(0f, 1f),
113        spread = wet.spread.coerceIn(0f, 1f),
114        bleed = wet.bleed.coerceIn(0f, 1f),
115    )
116
117    return copy(
118        baseRadiusPx = safeRadius,
119        opacity = safeOpacity,
120        flow = safeFlow,
121        spacing = safeSpacing,
122        pressureGamma = safeGamma,
123        minSizeRatio = safeMinSize,
124        velocityToSize = safeVelocityToSize,
125        velocityToOpacity = safeVelocityToOpacity,
126        tiltToSize = safeTiltToSize,
127        tiltToOpacity = safeTiltToOpacity,
128        tiltToRotation = safeTiltToRotation,
129        sizeJitter = safeSizeJitter,
130        opacityJitter = safeOpacityJitter,
131        rotationJitter = safeRotationJitter,
132        scatter = safeScatter,
133        grainScale = safeGrainScale,
134        textureDepth = safeTextureDepth,
135        textureContrast = safeTextureContrast,
136        antiAliasLevel = safeAa,
137        ribbon = safeRibbon,
138        wet = safeWet,
139    )
140 }
141 }

```

23. app/src/main/java/com/wetinknext/engine/brush/BrushTexture.kt

```
1 package com.wetinknext.engine.brush
2
3 import android.opengl.GLES30
4 import com.wetinknext.engine.gl.GlCheck
5 import java.nio.ByteBuffer
6
7 class BrushTexture {
8     var textureId: Int = 0
9     private set
10
11     var width: Int = 0
12     private set
13
14     var height: Int = 0
15     private set
16
17     fun createFromRgba(
18         width: Int,
19         height: Int,
20         rgba: ByteBuffer,
21     ) {
22         GlCheck.checkOnGThread()
23         release()
24
25         val ids = IntArray(1)
26         GLES30.glGenTextures(1, ids, 0)
27         textureId = ids[0]
28
29         check(textureId != 0) { "Failed to create brush texture" }
30
31         this.width = width
32         this.height = height
33
34         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, textureId)
35
36         GLES30.glTexParameteri(
37             GLES30.GL_TEXTURE_2D,
38             GLES30.GL_TEXTURE_MIN_FILTER,
39             GLES30.GL_LINEAR,
40         )
41         GLES30.glTexParameteri(
42             GLES30.GL_TEXTURE_2D,
43             GLES30.GL_TEXTURE_MAG_FILTER,
44             GLES30.GL_LINEAR,
45         )
46         GLES30.glTexParameteri(
47             GLES30.GL_TEXTURE_2D,
48             GLES30.GL_TEXTURE_WRAP_S,
49             GLES30.GL_REPEAT,
50         )
51         GLES30.glTexParameteri(
52             GLES30.GL_TEXTURE_2D,
53             GLES30.GL_TEXTURE_WRAP_T,
54             GLES30.GL_REPEAT,
55         )
56
57         GLES30.glPixelStorei(
58             GLES30.GL_UNPACK_ALIGNMENT,
59             1,
60         )
61
62         GLES30.glTexImage2D(
63             GLES30.GL_TEXTURE_2D,
64             0,
65             GLES30.GL_RGBA,
66             width,
67             height,
68             0,
69             GLES30.GL_RGBA,
70             GLES30.GL_UNSIGNED_BYTE,
71             rgba,
72         )
73
74         GLES30.glPixelStorei(
75             GLES30.GL_UNPACK_ALIGNMENT,
76             4,
77         )
78
79         GLES30.glBindTexture(
80             GLES30.GL_TEXTURE_2D,
81             0,
82         )
83     }
84
85     fun release() {
86         GlCheck.checkOnGThread()
87         if (textureId != 0) {
88             GLES30.glDeleteTextures(
89                 1,
90                 intArrayOf(textureId),
91             )
92         }
93     }
94 }
```

```
91         0,  
92     )  
93 }  
94  
95     textureId = 0  
96     width = 0  
97     height = 0  
98 }  
99  
100 }
```

24. app/src/main/java/com/wetinknext/engine/brush/CapsuleEmitter.kt

```

1 package com.wetinknext.engine.brush
2
3 import com.wetinknext.engine.input.InputBatch
4 import kotlin.math.hypot
5 import kotlin.math.max
6 import kotlin.math.pow
7
8 /**
9  * Преобразует поток InputSample в цепочку капсул.
10  *
11  * Один сегмент:
12  *   A = x0, y0, radius0
13  *   B = x1, y1, radius1
14  *
15  * Капсула с одинаковыми радиусами является обычной capsule.
16  * При разном давлении shader строит round-cone.
17  *
18  * Важно:
19  * - не хранит InputSample;
20  * - не создаёт List/FloatArray на MOVE;
21  * - reusable state принадлежит GL/render pipeline;
22  * - pointerId фиксируется на DOWN;
23  * - давление применяется к радиусу;
24  * - opacity намеренно не применяется здесь, она применяется одним blit-pass.
25  */
26 class CapsuleEmitter(
27     private var settings: BrushSettings,
28 ) {
29     private var strokeSettings: BrushSettings? = null
30     private val stabilizer = Stabilizer()
31     private val pressureFilter = PressureFilter()
32     private val resolvedDab = ResolvedDab()
33
34     private var active = false
35     private var pointerId = -1
36
37     private var hasLastPoint = false
38     private var lastX = 0f
39     private var lastY = 0f
40     private var lastRadius = 0f
41     private var lastCoverage = 1f
42
43     // Окно точек для интерполяции P0, P1, P2, P3
44     private var p0x = 0f; private var p0y = 0f; private var p0r = 0f
45     private var p1x = 0f; private var p1y = 0f; private var p1r = 0f
46     private var p2x = 0f; private var p2y = 0f; private var p2r = 0f
47     private var p3x = 0f; private var p3y = 0f; private var p3r = 0f
48     private var p0c = 1f; private var p1c = 1f
49     private var p2c = 1f; private var p3c = 1f
50     private var pointsInWindow = 0
51
52     private var emittedSegments = 0
53
54     var hasStroke: Boolean = false
55         private set
56
57     var totalLength: Float = 0f
58         private set
59
60     var minX: Float = 0f
61         private set
62
63     var minY: Float = 0f
64         private set
65
66     var maxX: Float = 0f
67         private set
68
69     var maxY: Float = 0f
70         private set
71
72     var hasBounds: Boolean = false
73         private set
74
75     fun updateSettings(value: BrushSettings) {
76         // Active stroke reads strokeSettings; this value is for the next begin().
77         settings = value.resolved()
78     }
79
80     private fun activeSettings(): BrushSettings {
81         check(active) { "Capsule emitter has no active stroke" }
82         return strokeSettings ?: error("Missing stroke settings")
83     }
84
85     /**
86     * Полный сброс текущего stroke.
87     * overflowCount рендера здесь не сбрасывается.
88     */
89     fun reset() {
90         active = false

```

```

91         strokeSettings = null
92         pointerId = -1
93
94         hasLastPoint = false
95         lastX = 0f
96         lastY = 0f
97         lastRadius = 0f
98         lastCoverage = 1f
99
100        pointsInWindow = 0
101
102        emittedSegments = 0
103        hasStroke = false
104        totalLength = 0f
105
106        minX = 0f
107        minY = 0f
108        maxX = 0f
109        maxY = 0f
110        hasBounds = false
111
112        stabilizer.reset()
113        pressureFilter.reset()
114    }
115
116    /**
117     * Начало stroke.
118     *
119     * Первый sample добавляется как вырожденная капсула.
120     * В fragment shader она превращается в чистый диск.
121     */
122    fun begin(
123        batch: InputBatch,
124        out: CapsuleStrokeRenderer,
125        strokeSettings: BrushSettings = settings,
126    ) {
127        reset()
128        out.beginStroke()
129        val resolvedSettings = strokeSettings.resolved()
130        this.strokeSettings = resolvedSettings
131
132        if (batch.sampleCount <= 0) return
133
134        val sample = batch.samples[0]
135
136        active = true
137        pointerId = sample.pointerId
138
139        stabilizer.strength = (
140            resolvedSettings.smoothing * SMOOTHING_WEIGHT +
141            resolvedSettings.streamline * STREAMLINE_WEIGHT
142        ).coerceIn(0f, 1f)
143
144        stabilizer.process(
145            timestampNanos = sample.timestampNanos,
146            rawX = sample.canvasX,
147            rawY = sample.canvasY,
148        )
149
150        val x = stabilizer.x
151        val y = stabilizer.y
152        val filteredPressure = pressureFilter.filter(sample.timestampNanos, sample.pressure)
153        BrushDynamics.resolve(resolvedSettings, filteredPressure, sample.tiltX, sample.tiltY, resolvedDab)
154        val radius = resolvedDab.radius
155        val coverage = resolvedDab.coverage
156
157        lastX = x
158        lastY = y
159        lastRadius = radius
160        lastCoverage = coverage
161        hasLastPoint = true
162        hasStroke = true
163
164        // Инициализация окна
165        p0x = x; p0y = y; p0r = radius
166        p1x = x; p1y = y; p1r = radius
167        p2x = x; p2y = y; p2r = radius
168        p0c = coverage; p1c = coverage; p2c = coverage
169        pointsInWindow = 1
170
171        // Вырожденный сегмент: круглый dab/tap.
172        out.addSegment(
173            x0 = x,
174            y0 = y,
175            radius0 = radius,
176            coverage0 = coverage,
177            x1 = x,
178            y1 = y,
179            radius1 = radius,
180            coverage1 = coverage,
181        )
182        emittedSegments++

```

```

183         includeBounds(x, y, radius)
184     }
185 }
186
187 /**
188  * Обрабатывает MOVE batch.
189  *
190  * Historical samples уже должны находиться внутри batch
191  * раньше текущего sample, это обеспечивает StrokeInputCapturer.
192  */
193 fun append(
194     batch: InputBatch,
195     out: CapsuleStrokeRenderer,
196 ) {
197     if (!active || batch.sampleCount <= 0) return
198     val settings = activeSettings()
199
200     for (index in 0 until batch.sampleCount) {
201         val sample = batch.samples[index]
202
203         if (sample.pointerId != pointerId) {
204             continue
205         }
206
207         stabilizer.process(
208             timestampNanos = sample.timestampNanos,
209             rawX = sample.canvasX,
210             rawY = sample.canvasY,
211         )
212
213         val x = stabilizer.x
214         val y = stabilizer.y
215         val filteredPressure = pressureFilter.filter(sample.timestampNanos, sample.pressure)
216         BrushDynamics.resolve(settings, filteredPressure, sample.tiltX, sample.tiltY, resolvedDab)
217         val radius = resolvedDab.radius
218         val coverage = resolvedDab.coverage
219
220         emitPoint(
221             x = x,
222             y = y,
223             radius = radius,
224             coverage = coverage,
225             out = out,
226         )
227     }
228 }
229
230 /**
231  * Завершает stroke.
232  *
233  * Последний текущий sample должен быть передан через append()
234  * до вызова finish().
235  */
236 fun finish(
237     out: CapsuleStrokeRenderer,
238     cancel: Boolean,
239 ) {
240     if (!active) return
241     val settings = activeSettings()
242
243     if (cancel) {
244         reset()
245         out.clearStrokeData()
246         return
247     }
248
249     // Принудительно завершаем кривую, используя последнюю точку как P3
250     if (pointsInWindow >= 2 && settings.smoothing > 1e-4f) {
251         interpolate(
252             p0x, p0y, p0r,
253             p0c,
254             p1x, p1y, p1r, p1c,
255             p2x, p2y, p2r, p2c,
256             p2x, p2y, p2r, p2c,
257             out,
258         )
259     }
260
261     active = false
262     pointerId = -1
263 }
264
265 private fun emitPoint(
266     x: Float,
267     y: Float,
268     radius: Float,
269     coverage: Float,
270     out: CapsuleStrokeRenderer,
271 ) {
272     val settings = activeSettings()
273     val dx = x - lastX
274     val dy = y - lastY

```

```

275     val distance = hypot(dx, dy)
276
277     if (distance < minimumPointDistance()) {
278         return
279     }
280
281     if (settings.smoothing <= 1e-4f) {
282         // Raw mode: прямые сегменты без задержки на окно
283         out.addSegment(lastX, lastY, lastRadius, lastCoverage, x, y, radius, coverage)
284         emittedSegments++
285         totalLength += distance
286         includeBounds(lastX, lastY, lastRadius)
287         includeBounds(x, y, radius)
288     } else {
289         // Interpolated mode: Catmull-Rom через окно
290         when (pointsInWindow) {
291             1 -> {
292                 p2x = x; p2y = y; p2r = radius; p2c = coverage
293                 pointsInWindow = 2
294             }
295             2 -> {
296                 p3x = x; p3y = y; p3r = radius; p3c = coverage
297                 interpolate(p0x, p0y, p0r, p0c, p1x, p1y, p1r, p1c, p2x, p2y, p2r, p2c, p3x, p3y, p3r, p3c, out)
298                 // Сдвиг окна
299                 p0x = p1x; p0y = p1y; p0r = p1r; p0c = p1c
300                 p1x = p2x; p1y = p2y; p1r = p2r; p1c = p2c
301                 p2x = p3x; p2y = p3y; p2r = p3r; p2c = p3c
302             }
303         }
304     }
305
306     hasStroke = true
307     lastX = x
308     lastY = y
309     lastRadius = radius
310     lastCoverage = coverage
311 }
312
313 /**
314  * Адаптивная интерполяция сегмента P1-P2 с использованием P0 и P3 как контрольных точек.
315  */
316 private fun interpolate(
317     p0x: Float, p0y: Float, p0r: Float, p0c: Float,
318     p1x: Float, p1y: Float, p1r: Float, p1c: Float,
319     p2x: Float, p2y: Float, p2r: Float, p2c: Float,
320     p3x: Float, p3y: Float, p3r: Float, p3c: Float,
321     out: CapsuleStrokeRenderer
322 ) {
323     val dist = hypot(p2x - p1x, p2y - p1y)
324     if (dist < 1e-4f) return
325
326     // Вычисляем контрольные точки для кубической кривой Безье (Catmull-Rom -> Bezier)
327     val c1x = p1x + (p2x - p0x) / 6f
328     val c1y = p1y + (p2y - p0y) / 6f
329     val c2x = p2x - (p3x - p1x) / 6f
330     val c2y = p2y - (p3y - p1y) / 6f
331
332     // Адаптивное количество шагов: примерно каждые 0.25..0.5 радиуса
333     val avgRadius = (p1r + p2r) * 0.5f
334     val stepPx = max(1f, avgRadius * INTERPOLATION_STEP_RATIO)
335     val steps = max(1, (dist / stepPx).toInt().coerceAtMost(MAX_INTERPOLATION_STEPS))
336
337     var prevX = p1x
338     var prevY = p1y
339     var prevR = p1r
340     var prevCoverage = p1c
341
342     for (i in 1..steps) {
343         val t = i.toFloat() / steps
344         val mt = 1f - t
345
346         // Кубическое Безье для позиции
347         val x = mt * mt * mt * p1x + 3f * mt * mt * t * c1x + 3f * mt * t * t * c2x + t * t * t * p2x
348         val y = mt * mt * mt * p1y + 3f * mt * mt * t * c1y + 3f * mt * t * t * c2y + t * t * t * p2y
349
350         // Линейная интерполяция радиуса
351         val r = p1r + (p2r - p1r) * t
352         val coverage = p1c + (p2c - p1c) * t
353
354         out.addSegment(prevX, prevY, prevR, prevCoverage, x, y, r, coverage)
355         emittedSegments++
356         totalLength += hypot(x - prevX, y - prevY)
357         includeBounds(x, y, r)
358
359         prevX = x
360         prevY = y
361         prevR = r
362         prevCoverage = coverage
363     }
364 }
365
366 /**

```

```

367      * Для капсул не нужен такой плотный шаг, как для stamp-кисти:
368      * segment shader сам покрывает весь отрезок между А и В.
369      */
370      private fun minimumPointDistance(): Float {
371          val settings = activeSettings()
372          val configured = settings.ribbon.minPointDistancePx
373              .coerceIn(MIN_POINT_DISTANCE_MIN, MAX_POINT_DISTANCE)
374
375          val radiusBased = settings.baseRadiusPx * POINT_STEP_RADIUS_RATIO
376
377          return max(
378              configured,
379              radiusBased.coerceAtMost(MAX_POINT_DISTANCE),
380          )
381      }
382
383      private fun includeBounds(
384          x: Float,
385          y: Float,
386          radius: Float,
387      ) {
388          val left = x - radius
389          val top = y - radius
390          val right = x + radius
391          val bottom = y + radius
392
393          if (!hasBounds) {
394              minX = left
395              minY = top
396              maxX = right
397              maxY = bottom
398              hasBounds = true
399              return
400          }
401
402          if (left < minX) minX = left
403          if (top < minY) minY = top
404          if (right > maxX) maxX = right
405          if (bottom > maxY) maxY = bottom
406      }
407
408      companion object {
409          private const val SMOOTHING_WEIGHT = 0.7f
410          private const val STREAMLINE_WEIGHT = 0.3f
411
412          private const val MIN_RADIUS_PX = 0.25f
413
414          private const val MIN_POINT_DISTANCE_MIN = 0.05f
415          private const val MAX_POINT_DISTANCE = 32f
416          private const val POINT_STEP_RADIUS_RATIO = 0.08f
417
418          private const val INTERPOLATION_STEP_RATIO = 0.4f
419          private const val MAX_INTERPOLATION_STEPS = 64
420      }
421  }

```

25. app/src/main/java/com/wetinknext/engine/brush/CapsuleStrokeRenderer.kt

```

1  package com.wetinknext.engine.brush
2
3  import android.opengl.GLES30
4  import com.wetinknext.BuildConfig
5  import com.wetinknext.engine.gl.GlCheck
6  import com.wetinknext.engine.gl.GlProgram
7  import com.wetinknext.engine.gl.RenderTarget
8  import com.wetinknext.engine.gl.ShaderLib
9  import java.nio.ByteBuffer
10 import java.nio.ByteOrder
11
12 /**
13  * GPU-renderer для круглого штриха.
14  *
15  * Один instance:
16  *   x0, y0, radius0,
17  *   x1, y1, radius1
18  *
19  * Геометрия каждого сегмента является AABB, а настоящая форма
20  * рассчитывается в capsuleFragment через signed distance.
21  *
22  * Важно:
23  * - target должен быть очищен перед началом нового штриха;
24  * - рисование использует premultiplied source-over blending;
25  * - цвет должен быть постоянным для всего strokeTarget;
26  * - выход shader: premultiplied linear RGBA.
27  */
28 class CapsuleStrokeRenderer(
29     private val maxSegments: Int = DEFAULT_MAX_SEGMENTS,
30 ) {
31     private var program: GlProgram? = null
32
33     private var vaoId = 0
34     private var cornerVboId = 0
35     private var instanceVboId = 0
36
37     private var uCanvasToClip = -1
38     private var uColorLinear = -1
39     private val blendController = BlendController()
40     private var uCanvasSize = -1
41     private var uGrainTex = -1
42     private var uGrainActive = -1
43     private var uGrainScale = -1
44     private var uGrainCanvasLocked = -1
45     private var uTextureDepth = -1
46     private var uTextureContrast = -1
47     private var uFlow = -1
48
49     private var grainTextureId = 0
50     private var grainScale = 1f
51     private var grainCanvasLocked = true
52     private var textureDepth = 1f
53     private var textureContrast = 1f
54
55     private val checkGlErrors: Boolean
56         get() = BuildConfig.DEBUG
57
58     /**
59     * Формат:
60     *   x0, y0, r0, x1, y1, r1
61     */
62     private val instanceData =
63         ByteBuffer
64             .allocateDirect(
65                 maxSegments * FLOATS_PER_SEGMENT * Float.SIZE_BYTES,
66             )
67             .order(ByteOrder.nativeOrder())
68             .asFloatBuffer()
69
70     var segmentCount: Int = 0
71         private set
72
73     private var uploadedSegmentCount = 0
74
75     /**
76     * Не сбрасывается на каждый новый stroke.
77     * Иначе диагностика потерь снова будет скрыта.
78     */
79     var overflowCount: Long = 0L
80         private set
81
82     val isEmpty: Boolean
83         get() = segmentCount == 0
84
85     val hasPendingSegments: Boolean
86         get() = segmentCount > uploadedSegmentCount
87
88     fun create() {
89         GlCheck.checkOnGlThread()
90         release()

```

```

91
92     program = GLProgram(
93         vertexSource = ShaderLib.capsuleVertex,
94         fragmentSource = ShaderLib.capsuleFragment,
95     )
96
97     val currentProgram = checkNotNull(program)
98     currentProgram.use()
99
100    uCanvasToClip = GLES30.glGetUniformLocation(
101        currentProgram.id,
102        "uCanvasToClip",
103    )
104    uColorLinear = GLES30.glGetUniformLocation(
105        currentProgram.id,
106        "uColorLinear",
107    )
108    uCanvasSize = GLES30.glGetUniformLocation(
109        currentProgram.id,
110        "uCanvasSize",
111    )
112    uGrainTex = GLES30.glGetUniformLocation(
113        currentProgram.id,
114        "uGrainTex",
115    )
116    uGrainScale = GLES30.glGetUniformLocation(
117        currentProgram.id,
118        "uGrainScale",
119    )
120    uGrainActive = GLES30.glGetUniformLocation(
121        currentProgram.id,
122        "uGrainActive",
123    )
124    uGrainCanvasLocked = GLES30.glGetUniformLocation(
125        currentProgram.id,
126        "uGrainCanvasLocked",
127    )
128    uTextureDepth = GLES30.glGetUniformLocation(
129        currentProgram.id,
130        "uTextureDepth",
131    )
132    uTextureContrast = GLES30.glGetUniformLocation(
133        currentProgram.id,
134        "uTextureContrast",
135    )
136    uFlow = GLES30.glGetUniformLocation(
137        currentProgram.id,
138        "uFlow",
139    )
140
141    check(uCanvasToClip >= 0) {
142        "Capsule shader uniform uCanvasToClip is missing"
143    }
144    check(uColorLinear >= 0) {
145        "Capsule shader uniform uColorLinear is missing"
146    }
147    check(uCanvasSize >= 0) {
148        "Capsule shader uniform uCanvasSize is missing"
149    }
150    check(uFlow >= 0) {
151        "Capsule shader uniform uFlow is missing"
152    }
153
154    val corners = floatArrayOf(
155        -1f, -1f,
156        1f, -1f,
157        -1f, 1f,
158        1f, 1f,
159    )
160
161    val cornerData =
162        ByteBuffer
163            .allocateDirect(corners.size * Float.SIZE_BYTES)
164            .order(ByteOrder.nativeOrder())
165            .asFloatBuffer()
166
167    cornerData.put(corners)
168    cornerData.position(0)
169
170    val ids = IntArray(1)
171
172    GLES30.glGenVertexArrays(1, ids, 0)
173    vaoId = ids[0]
174
175    GLES30.glGenBuffers(1, ids, 0)
176    cornerVboId = ids[0]
177
178    GLES30.glGenBuffers(1, ids, 0)
179    instanceVboId = ids[0]
180
181    check(vaoId != 0) { "Failed to create capsule VAO" }
182    check(cornerVboId != 0) { "Failed to create capsule corner VBO" }

```

```

183         check(instanceVboId != 0) { "Failed to create capsule instance VBO" }
184
185         GLES30.glBindVertexArray(vaoId)
186
187         // Углы AABB-квада.
188         GLES30.glBindBuffer(
189             GLES30.GL_ARRAY_BUFFER,
190             cornerVboId,
191         )
192         GLES30.glBufferData(
193             GLES30.GL_ARRAY_BUFFER,
194             corners.size * Float.SIZE_BYTES,
195             cornerData,
196             GLES30.GL_STATIC_DRAW,
197         )
198         GLES30.glEnableVertexAttribArray(ATTR_CORNER)
199         GLES30.glVertexAttribPointer(
200             ATTR_CORNER,
201             2,
202             GLES30.GL_FLOAT,
203             false,
204             2 * Float.SIZE_BYTES,
205             0,
206         )
207
208         // Данные сегментов.
209         GLES30.glBindBuffer(
210             GLES30.GL_ARRAY_BUFFER,
211             instanceVboId,
212         )
213         GLES30.glBufferData(
214             GLES30.GL_ARRAY_BUFFER,
215             maxSegments * STRIDE_BYTES,
216             null,
217             GLES30.GL_DYNAMIC_DRAW,
218         )
219
220         // A: x0, y0, radius0
221         GLES30.glEnableVertexAttribArray(ATTR_SEGMENT_A)
222         GLES30.glVertexAttribPointer(
223             ATTR_SEGMENT_A,
224             4,
225             GLES30.GL_FLOAT,
226             false,
227             STRIDE_BYTES,
228             0,
229         )
230         GLES30.glVertexAttribDivisor(
231             ATTR_SEGMENT_A,
232             1,
233         )
234
235         // B: x1, y1, radius1
236         GLES30.glEnableVertexAttribArray(ATTR_SEGMENT_B)
237         GLES30.glVertexAttribPointer(
238             ATTR_SEGMENT_B,
239             4,
240             GLES30.GL_FLOAT,
241             false,
242             STRIDE_BYTES,
243             4 * Float.SIZE_BYTES,
244         )
245         GLES30.glVertexAttribDivisor(
246             ATTR_SEGMENT_B,
247             1,
248         )
249
250         GLES30.glBindBuffer(
251             GLES30.GL_ARRAY_BUFFER,
252             0,
253         )
254         GLES30.glBindVertexArray(0)
255
256         GLES30.glUseProgram(0)
257
258         checkNoGLError("CapsuleStrokeRenderer.create")
259     }
260
261     /**
262     * Начинает новый stroke.
263     *
264     * Старый target должен быть очищен отдельно:
265     * strokeTarget.clear(0f, 0f, 0f, 0f)
266     */
267     fun beginStroke() {
268         segmentCount = 0
269         uploadedSegmentCount = 0
270
271         instanceData.clear()
272         instanceData.limit(
273             maxSegments * FLOATS_PER_SEGMENT,
274         )

```

```

275     }
276
277     /**
278     * Добавляет один сегмент в CPU-buffer.
279     *
280     * Если capacity исчерпана, сегмент не теряется молча:
281     * overflowCount увеличивается.
282     */
283     fun addSegment(
284         x0: Float,
285         y0: Float,
286         radius0: Float,
287         coverage0: Float,
288         x1: Float,
289         y1: Float,
290         radius1: Float,
291         coverage1: Float,
292     ): Boolean {
293         if (segmentCount >= maxSegments) {
294             overflowCount++
295             return false
296         }
297
298         val offset = segmentCount * FLOATS_PER_SEGMENT
299
300         instanceData.put(offset, x0)
301         instanceData.put(offset + 1, y0)
302         instanceData.put(offset + 2, radius0.coerceAtLeast(0.01f))
303         instanceData.put(offset + 3, coverage0.coerceIn(0f, 1f))
304
305         instanceData.put(offset + 4, x1)
306         instanceData.put(offset + 5, y1)
307         instanceData.put(offset + 6, radius1.coerceAtLeast(0.01f))
308         instanceData.put(offset + 7, coverage1.coerceIn(0f, 1f))
309
310         segmentCount++
311         return true
312     }
313
314     /**
315     * Загружает и рисует только новые сегменты.
316     *
317     * Используется для preview во время MOVE.
318     * Перед первым вызовом target должен быть очищен.
319     */
320     fun drawPending(
321         target: RenderTarget,
322         width: Int,
323         height: Int,
324         canvasToClip: FloatArray,
325         colorLinear: FloatArray,
326         blendPolicy: BlendPolicy,
327         flow: Float,
328     ) {
329         val first = uploadedSegmentCount
330         val count = segmentCount - uploadedSegmentCount
331
332         if (count <= 0) return
333
334         val drawn = drawRange(
335             target = target,
336             width = width,
337             height = height,
338             canvasToClip = canvasToClip,
339             colorLinear = colorLinear,
340             firstSegment = first,
341             count = count,
342             blendPolicy = blendPolicy,
343             flow = flow,
344         )
345
346         if (drawn) {
347             uploadedSegmentCount = segmentCount
348         }
349     }
350
351     /**
352     * Полностью рисует все накопленные сегменты.
353     *
354     * Используется при commit, если target был очищен.
355     */
356     fun drawAll(
357         target: RenderTarget,
358         width: Int,
359         height: Int,
360         canvasToClip: FloatArray,
361         colorLinear: FloatArray,
362         blendPolicy: BlendPolicy,
363         flow: Float,
364     ) {
365         if (segmentCount <= 0) return
366     }

```

```

367         drawRange(
368             target = target,
369             width = width,
370             height = height,
371             canvasToClip = canvasToClip,
372             colorLinear = colorLinear,
373             firstSegment = 0,
374             count = segmentCount,
375             blendPolicy = blendPolicy,
376             flow = flow,
377         )
378     }
379
380     /**
381     * Полностью перерисовывает сегменты в диапазоне [firstSegment, firstSegment + count).
382     *
383     * Важный момент: glVertexAttribPointer вызывается здесь повторно,
384     * потому что offset у instanced-атрибутов меняется для каждого диапазона.
385     */
386     private fun drawRange(
387         target: RenderTarget,
388         width: Int,
389         height: Int,
390         canvasToClip: FloatArray,
391         colorLinear: FloatArray,
392         firstSegment: Int,
393         count: Int,
394         blendPolicy: BlendPolicy,
395         flow: Float,
396     ): Boolean {
397         val currentProgram = program ?: return false
398
399         require(canvasToClip.size >= 16) {
400             "canvasToClip must contain at least 16 floats"
401         }
402         require(colorLinear.size >= 3) {
403             "colorLinear must contain at least 3 floats"
404         }
405         require(firstSegment >= 0)
406         require(count >= 0)
407         require(firstSegment + count <= segmentCount)
408
409         if (count == 0) return true
410
411         target.bind()
412
413         GLES30.glViewport(
414             0,
415             0,
416             width,
417             height,
418         )
419
420         GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
421
422         blendController.begin(blendPolicy)
423
424         currentProgram.use()
425
426         GLES30.glUniformMatrix4fv(
427             uCanvasToClip,
428             1,
429             false,
430             canvasToClip,
431             0,
432         )
433
434         GLES30.glUniform3f(
435             uColorLinear,
436             colorLinear[0].coerceIn(0f, 1f),
437             colorLinear[1].coerceIn(0f, 1f),
438             colorLinear[2].coerceIn(0f, 1f),
439         )
440
441         GLES30.glUniform2f(
442             uCanvasSize,
443             width.toFloat(),
444             height.toFloat(),
445         )
446
447         GLES30.glUniform1f(
448             uGrainScale,
449             grainScale,
450         )
451
452         GLES30.glUniform1i(
453             uGrainActive,
454             if (grainTextureId != 0) 1 else 0,
455         )
456
457         GLES30.glUniform1i(
458             uGrainCanvasLocked,

```

```

459         if (grainCanvasLocked) 1 else 0,
460     )
461
462     GLES30.glUniform1f(
463         uTextureDepth,
464         textureDepth,
465     )
466
467     GLES30.glUniform1f(
468         uTextureContrast,
469         textureContrast,
470     )
471
472     GLES30.glUniform1f(
473         uFlow,
474         flow.coerceIn(0f, 1f),
475     )
476
477     if (grainTextureId != 0) {
478         GLES30.glActiveTexture(GLES30.GL_TEXTURE2)
479         GLES30.glBindTexture(
480             GLES30.GL_TEXTURE_2D,
481             grainTextureId,
482         )
483         GLES30.glUniform1i(uGrainTex, 2)
484     }
485
486     /*
487     * FloatBuffer должен содержать только передаваемый диапазон.
488     * glBufferSubData прочитает remaining() элементов.
489     */
490     val sourceStart = firstSegment * FLOATS_PER_SEGMENT
491     val sourceEnd =
492         sourceStart + count * FLOATS_PER_SEGMENT
493
494     instanceData.position(sourceStart)
495     instanceData.limit(sourceEnd)
496
497     GLES30.glBindVertexArray(vaoId)
498     GLES30.glBindBuffer(
499         GLES30.GL_ARRAY_BUFFER,
500         instanceVboId,
501     )
502
503     GLES30.glBufferSubData(
504         GLES30.GL_ARRAY_BUFFER,
505         firstSegment * STRIDE_BYTES,
506         count * STRIDE_BYTES,
507         instanceData,
508     )
509
510     /*
511     * Для ES 3.0 нет glDrawArraysInstancedBaseInstance.
512     * Поэтому offset атрибутов сдвигается вручную.
513     *
514     * VAO должен быть привязан во время glVertexAttribPointer.
515     */
516     GLES30.glVertexAttribPointer(
517         ATTR_SEGMENT_A,
518         4,
519         GLES30.GL_FLOAT,
520         false,
521         STRIDE_BYTES,
522         firstSegment * STRIDE_BYTES,
523     )
524
525     GLES30.glVertexAttribPointer(
526         ATTR_SEGMENT_B,
527         4,
528         GLES30.GL_FLOAT,
529         false,
530         STRIDE_BYTES,
531         firstSegment * STRIDE_BYTES + 4 * Float.SIZE_BYTES,
532     )
533
534     GLES30.glDrawArraysInstanced(
535         GLES30.GL_TRIANGLE_STRIP,
536         0,
537         CORNER_VERTEX_COUNT,
538         count,
539     )
540
541     if (grainTextureId != 0) {
542         GLES30.glActiveTexture(GLES30.GL_TEXTURE2)
543         GLES30.glBindTexture(
544             GLES30.GL_TEXTURE_2D,
545             0,
546         )
547     }
548
549     GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
550

```

```

551     GLES30.glBindBuffer(
552         GLES30.GL_ARRAY_BUFFER,
553         0,
554     )
555     GLES30.glBindVertexArray(0)
556
557     blendController.end()
558
559     GLES30.glBindFramebuffer(
560         GLES30.GL_FRAMEBUFFER,
561         0,
562     )
563
564     instanceData.clear()
565     instanceData.limit(
566         maxSegments * FLOATS_PER_SEGMENT,
567     )
568
569     if (checkGLErrors) {
570         checkNoGLError("CapsuleStrokeRenderer.drawRange")
571     }
572     return true
573 }
574
575 /**
576  * Сбрасывает только GPU/CPU состояние текущего renderer.
577  * Счётчик overflow сохраняется.
578  */
579 fun clearStrokeData() {
580     segmentCount = 0
581     uploadedSegmentCount = 0
582     instanceData.clear()
583     instanceData.limit(
584         maxSegments * FLOATS_PER_SEGMENT,
585     )
586 }
587
588 fun setGrainTexture(
589     textureId: Int,
590     scale: Float,
591     canvasLocked: Boolean,
592     depth: Float,
593     contrast: Float,
594 ) {
595     grainTextureId = textureId
596     grainScale = scale.coerceAtLeast(0.0001f)
597     grainCanvasLocked = canvasLocked
598     textureDepth = depth.coerceIn(0f, 1f)
599     textureContrast = contrast.coerceIn(0f, 2f)
600 }
601
602 fun clearGrainTexture() {
603     grainTextureId = 0
604     grainScale = 1f
605     grainCanvasLocked = true
606     textureDepth = 1f
607     textureContrast = 1f
608 }
609
610 /**
611  * Все GL-объекты удаляются только на GL-потоке.
612  */
613 fun release() {
614     GLCheck.checkOnGLThread()
615     program?.release()
616     program = null
617
618     if (instanceVboId != 0) {
619         GLES30.glDeleteBuffers(
620             1,
621             intArrayOf(instanceVboId),
622             0,
623         )
624     }
625
626     if (cornerVboId != 0) {
627         GLES30.glDeleteBuffers(
628             1,
629             intArrayOf(cornerVboId),
630             0,
631         )
632     }
633
634     if (vaoId != 0) {
635         GLES30.glDeleteVertexArrays(
636             1,
637             intArrayOf(vaoId),
638             0,
639         )
640     }
641
642     vaoId = 0

```

```

643         cornerVboId = 0
644         instanceVboId = 0
645
646         uCanvasToClip = -1
647         uColorLinear = -1
648         uCanvasSize = -1
649         uGrainTex = -1
650         uGrainScale = -1
651         uGrainActive = -1
652         uGrainCanvasLocked = -1
653         uTextureDepth = -1
654         uTextureContrast = -1
655         uFlow = -1
656         grainTextureId = 0
657
658         segmentCount = 0
659         uploadedSegmentCount = 0
660
661         instanceData.clear()
662         instanceData.limit(
663             maxSegments * FLOATS_PER_SEGMENT,
664         )
665     }
666
667     private fun checkNoGLError(label: String) {
668         val error = GLES30.glGetError()
669         check(error == GLES30.GL_NO_ERROR) {
670             "$label: GL error 0x${error.toString(16)}"
671         }
672     }
673
674     companion object {
675
676         private const val ATTR_CORNER = 0
677         private const val ATTR_SEGMENT_A = 1
678         private const val ATTR_SEGMENT_B = 2
679
680         private const val CORNER_VERTEX_COUNT = 4
681
682         const val FLOATS_PER_SEGMENT = 8
683         const val STRIDE_BYTES =
684             FLOATS_PER_SEGMENT * Float.SIZE_BYTES
685
686         const val DEFAULT_MAX_SEGMENTS = 16_384
687     }
688 }

```

26. app/src/main/java/com/wetinknext/engine/brush/ClosureDebug.kt

```
1 package com.wetinknext.engine.brush
2
3 import android.util.Log
4 import com.wetinknext.BuildConfig
5
6 /** One debug log per finished stroke; never used from the render hot path. */
7 object ClosureDebug {
8     const val TAG = "RibbonClosure"
9     fun publish(distance: Float, threshold: Float, closed: Boolean, samples: Int) {
10         if (!BuildConfig.DEBUG) return
11         runCatching {
12             Log.d(TAG, "dist=%.2f thr=%.2f closed=%s n=%d".format(distance, threshold, closed, samples))
13         }
14     }
15 }
```

27. app/src/main/java/com/wetinknext/engine/brush/ColorSpaces.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlin.math.pow
4
5 object ColorSpaces {
6     fun srgbToLinear(value: Float): Float { val v=value.coerceIn(0f,1f); return if(v<=.04045f)v/12.92f else ((v+.055f)/1.
055f).pow(2.4f) }
7     fun linearToSrgb(value: Float): Float { val v=value.coerceIn(0f,1f); return if(v<=.0031308f)v*12.92f else 1.055f*v.pow(1f/
2.4f)-.055f }
8     fun srgb8ToLinear(Argb: Long,out:FloatArray){require(out.size>=3);out[0]=srgbToLinear(((Argb shr 16)and 0xFF)/255f);out[
1]=srgbToLinear(((Argb shr 8)and 0xFF)/255f);out[2]=srgbToLinear((Argb and 0xFF)/255f)}
9 }
```

28. app/src/main/java/com/wetinknext/engine/brush/DabBuffer.kt

```

1  package com.wetinknext.engine.brush
2
3  import com.wetinknext.engine.core.DirtyRect
4  import java.nio.ByteBuffer
5  import java.nio.ByteOrder
6
7  class DabBuffer(val capacity: Int = DEFAULT_CAPACITY) {
8      init { require(capacity > 0) }
9      val floats = ByteBuffer.allocateDirect(capacity * FLOATS_PER_DAB * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).
asFloatBuffer()
10     var count = 0; private set; var overflowCount = 0L; private set
11     fun clear() { count = 0; overflowCount = 0L; floats.clear() }
12     fun add(x: Float, y: Float, radius: Float, rotation: Float, coverage: Float, flow: Float): Boolean { if (count >=
capacity) { overflowCount++; return false }; floats.put(x); floats.put(y); floats.put(radius); floats.put(rotation); floats.put(
coverage.coerceIn(0f, 1f)); floats.put(flow.coerceIn(0f, 1f)); count++; return true }
13     fun prepareForUpload() { floats.position(0); floats.limit(count * FLOATS_PER_DAB) }
14     fun prepareForUpload(firstDab: Int, dabCount: Int) {
15         require(firstDab >= 0)
16         require(dabCount >= 0)
17         require(firstDab + dabCount <= count)
18         floats.position(firstDab * FLOATS_PER_DAB)
19         floats.limit((firstDab + dabCount) * FLOATS_PER_DAB)
20     }
21     /** Restores the direct buffer for appending after a GL upload. */
22     fun finishUpload() {
23         floats.limit(capacity * FLOATS_PER_DAB)
24         floats.position(count * FLOATS_PER_DAB)
25     }
26
27     /**
28      * Includes the exact circular footprint of each emitted dab plus a caller
29      * supplied safety fringe for analytic AA and textured-tip filtering.
30      */
31     fun includeDirtyRect(rect: DirtyRect, extraMargin: Float) {
32         val safeExtraMargin = extraMargin.coerceAtLeast(0f)
33         for (index in 0 until count) {
34             val offset = index * FLOATS_PER_DAB
35             val x = floats.get(offset)
36             val y = floats.get(offset + 1)
37             val radius = floats.get(offset + 2).coerceAtLeast(0f) + safeExtraMargin
38             rect.include(
39                 left = x - radius,
40                 top = y - radius,
41                 right = x + radius,
42                 bottom = y + radius,
43             )
44         }
45     }
46     companion object { const val FLOATS_PER_DAB = 6; const val DEFAULT_CAPACITY = 8192 }
47 }

```

29. app/src/main/java/com/wetinknext/engine/brush/DabRenderer.kt

```

1  package com.wetinknext.engine.brush
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.GlCheck
5  import com.wetinknext.engine.gl.GlProgram
6  import com.wetinknext.engine.gl.RenderTarget
7  import com.wetinknext.engine.gl.ShaderLib
8  import java.nio.ByteBuffer
9  import java.nio.ByteOrder
10
11 /** Instanced circular dab renderer. All colour values are premultiplied linear output. */
12 class DabRenderer(private val maxDabs: Int) {
13     private var program: GlProgram? = null
14     private var vaoId = 0
15     private var quadBufferId = 0
16     private var instanceBufferId = 0
17     private var uCanvasToClip = -1
18     private var uColorLinear = -1
19     private var uCanvasSize = -1
20     private var uGrainTex = -1
21     private var uGrainActive = -1
22     private var uGrainScale = -1
23     private var uGrainCanvasLocked = -1
24     private var uTextureDepth = -1
25     private var uTextureContrast = -1
26     private var uStrokeOpacity = -1
27     private var uShapeTex = -1
28     private var uShapeActive = -1
29     private var uReverseShape = -1
30     private var uRgbToAlpha = -1
31
32     private var grainTextureId = 0
33     private var grainScale = 1f
34     private var grainCanvasLocked = true
35     private var textureDepth = 1f
36     private var textureContrast = 1f
37     private var shapeTextureId = 0
38     private var reverseShape = false
39     private var rgbToAlpha = false
40
41     private var uploadedDabCount = 0
42     private val blendController = BlendController()
43
44     /** Number of dabs already written into the current stroke target. */
45     val uploadedCount: Int get() = uploadedDabCount
46
47     fun create() {
48         GlCheck.checkOnGUIThread()
49         release()
50         uploadedDabCount = 0
51         program = GlProgram(ShaderLib.dabVertex, ShaderLib.dabFragment)
52         val currentProgram = checkNotNull(program)
53         currentProgram.use()
54         uCanvasToClip = GLES30.glGetUniformLocation(currentProgram.id, "uCanvasToClip")
55         uColorLinear = GLES30.glGetUniformLocation(currentProgram.id, "uColorLinear")
56         uCanvasSize = GLES30.glGetUniformLocation(currentProgram.id, "uCanvasSize")
57         uGrainTex = GLES30.glGetUniformLocation(currentProgram.id, "uGrainTex")
58         uGrainActive = GLES30.glGetUniformLocation(currentProgram.id, "uGrainActive")
59         uGrainScale = GLES30.glGetUniformLocation(currentProgram.id, "uGrainScale")
60         uGrainCanvasLocked = GLES30.glGetUniformLocation(currentProgram.id, "uGrainCanvasLocked")
61         uTextureDepth = GLES30.glGetUniformLocation(currentProgram.id, "uTextureDepth")
62         uTextureContrast = GLES30.glGetUniformLocation(currentProgram.id, "uTextureContrast")
63         uStrokeOpacity = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeOpacity")
64         uShapeTex = GLES30.glGetUniformLocation(currentProgram.id, "uShapeTex")
65         uShapeActive = GLES30.glGetUniformLocation(currentProgram.id, "uShapeActive")
66         uReverseShape = GLES30.glGetUniformLocation(currentProgram.id, "uReverseShape")
67         uRgbToAlpha = GLES30.glGetUniformLocation(currentProgram.id, "uRgbToAlpha")
68
69         check(uCanvasToClip >= 0 && uColorLinear >= 0 && uStrokeOpacity >= 0) { "Dab shader uniforms missing" }
70
71         val quad = floatArrayOf(-1f, -1f, 1f, -1f, -1f, 1f, 1f, 1f)
72         val quadData = ByteBuffer.allocateDirect(quad.size * Float.SIZE_BYTES)
73             .order(ByteOrder.nativeOrder())
74             .asFloatBuffer()
75         quadData.put(quad).position(0)
76
77         val ids = IntArray(1)
78         GLES30.glGenVertexArrays(1, ids, 0)
79         vaoId = ids[0]
80         GLES30.glGenBuffers(1, ids, 0)
81         quadBufferId = ids[0]
82         GLES30.glGenBuffers(1, ids, 0)
83         instanceBufferId = ids[0]
84
85         GLES30.glBindVertexArray(vaoId)
86         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, quadBufferId)
87         GLES30.glBufferData(
88             GLES30.GL_ARRAY_BUFFER,
89             quad.size * Float.SIZE_BYTES,
90             quadData,

```

```

91         GLES30.GL_STATIC_DRAW,
92     )
93     GLES30.glEnableVertexAttribArray(0)
94     GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
95
96     GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, instanceBufferId)
97     GLES30.glBufferData(
98         GLES30.GL_ARRAY_BUFFER,
99         maxDabs * DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES,
100        null,
101        GLES30.GL_DYNAMIC_DRAW,
102    )
103     GLES30.glEnableVertexAttribArray(1)
104     GLES30.glVertexAttribPointer(1, 4, GLES30.GL_FLOAT, false, DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES, 0)
105     GLES30.glVertexAttribDivisor(1, 1)
106     GLES30.glEnableVertexAttribArray(2)
107     GLES30.glVertexAttribPointer(2, 2, GLES30.GL_FLOAT, false, DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES, 16)
108     GLES30.glVertexAttribDivisor(2, 1)
109     GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0)
110     GLES30.glBindVertexArray(0)
111 }
112
113 /** Draws all buffered dabs into a document-space RenderTarget. */
114 fun drawInto(
115     target: RenderTarget,
116     width: Int,
117     height: Int,
118     canvasToFbo: FloatArray,
119     dabs: DabBuffer,
120     colorLinear: FloatArray,
121     blendPolicy: BlendPolicy,
122     strokeOpacity: Float = 1f,
123 ) {
124     if (dabs.count == 0) return
125     require(colorLinear.size >= 3)
126
127     dabs.prepareForUpload(firstDab = 0, dabCount = dabs.count)
128     try {
129         drawRange(
130             target = target,
131             width = width,
132             height = height,
133             canvasToFbo = canvasToFbo,
134             dabs = dabs,
135             firstDab = 0,
136             count = dabs.count,
137             colorLinear = colorLinear,
138             blendPolicy = blendPolicy,
139             strokeOpacity = strokeOpacity,
140         )
141     } finally {
142         dabs.finishUpload()
143     }
144 }
145
146 fun release() {
147     GLCheck.checkOnGLThread()
148     program?.release()
149     program = null
150     uploadedDabCount = 0
151     grainTextureId = 0
152     shapeTextureId = 0
153     reverseShape = false
154     rgbToAlpha = false
155     if (instanceBufferId != 0) GLES30.glDeleteBuffers(1, intArrayOf(instanceBufferId), 0)
156     if (quadBufferId != 0) GLES30.glDeleteBuffers(1, intArrayOf(quadBufferId), 0)
157     if (vaoId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vaoId), 0)
158     vaoId = 0
159     quadBufferId = 0
160     instanceBufferId = 0
161     uCanvasToClip = -1
162     uColorLinear = -1
163     uStrokeOpacity = -1
164 }
165
166 fun beginStroke() {
167     uploadedDabCount = 0
168 }
169
170 fun clearStrokeData() {
171     uploadedDabCount = 0
172 }
173
174 fun setGrainTexture(
175     textureId: Int,
176     scale: Float,
177     canvasLocked: Boolean,
178     depth: Float,
179     contrast: Float,
180 ) {
181     grainTextureId = textureId
182     grainScale = scale.coerceAtLeast(0.0001f)

```

```

183         grainCanvasLocked = canvasLocked
184         textureDepth = depth.coerceIn(0f, 1f)
185         textureContrast = contrast.coerceIn(0f, 2f)
186     }
187
188     fun clearGrainTexture() {
189         grainTextureId = 0
190         grainScale = 1f
191         grainCanvasLocked = true
192         textureDepth = 1f
193         textureContrast = 1f
194     }
195
196     /** Sets a tip mask sampled once per dab in local dab coordinates. */
197     fun setShapeTexture(textureId: Int, reverse: Boolean, rgbToAlpha: Boolean) {
198         shapeTextureId = textureId
199         reverseShape = reverse
200         this.rgbToAlpha = rgbToAlpha
201     }
202
203     fun clearShapeTexture() {
204         shapeTextureId = 0
205         reverseShape = false
206         rgbToAlpha = false
207     }
208
209     fun drawPendingInto(
210         target: RenderTarget,
211         width: Int,
212         height: Int,
213         canvasToFbo: FloatArray,
214         dabs: DabBuffer,
215         colorLinear: FloatArray,
216         blendPolicy: BlendPolicy,
217         strokeOpacity: Float = 1f,
218     ) {
219         val first = uploadedDabCount
220         val count = dabs.count - first
221         if (count <= 0) return
222
223         dabs.prepareForUpload(first, count)
224         try {
225             drawRange(
226                 target = target,
227                 width = width,
228                 height = height,
229                 canvasToFbo = canvasToFbo,
230                 dabs = dabs,
231                 firstDab = first,
232                 count = count,
233                 colorLinear = colorLinear,
234                 blendPolicy = blendPolicy,
235                 strokeOpacity = strokeOpacity,
236             )
237             uploadedDabCount = dabs.count
238         } finally {
239             dabs.finishUpload()
240         }
241     }
242
243     private fun drawRange(
244         target: RenderTarget,
245         width: Int,
246         height: Int,
247         canvasToFbo: FloatArray,
248         dabs: DabBuffer,
249         firstDab: Int,
250         count: Int,
251         colorLinear: FloatArray,
252         blendPolicy: BlendPolicy,
253         strokeOpacity: Float,
254     ) {
255         val currentProgram = program ?: return
256         target.bind()
257         GLES30.glViewport(0, 0, width, height)
258         blendController.begin(blendPolicy)
259         currentProgram.use()
260         GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, canvasToFbo, 0)
261         GLES30.glUniform3f(uColorLinear, colorLinear[0], colorLinear[1], colorLinear[2])
262
263         GLES30.glUniform2f(uCanvasSize, width.toFloat(), height.toFloat())
264         GLES30.glUniform1i(uGrainActive, if (grainTextureId != 0) 1 else 0)
265         GLES30.glUniform1f(uGrainScale, grainScale)
266         GLES30.glUniform1i(uGrainCanvasLocked, if (grainCanvasLocked) 1 else 0)
267         GLES30.glUniform1f(uTextureDepth, textureDepth)
268         GLES30.glUniform1f(uTextureContrast, textureContrast)
269         GLES30.glUniform1f(uStrokeOpacity, strokeOpacity.coerceIn(0f, 1f))
270         GLES30.glUniform1i(uShapeActive, if (shapeTextureId != 0) 1 else 0)
271         GLES30.glUniform1i(uReverseShape, if (reverseShape) 1 else 0)
272         GLES30.glUniform1i(uRgbToAlpha, if (rgbToAlpha) 1 else 0)
273
274         if (grainTextureId != 0) {

```

```

275         GLES30.glActiveTexture(GLES30.GL_TEXTURE2)
276         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, grainTextureId)
277         GLES30.glUniform1i(uGrainTex, 2)
278     }
279     if (shapeTextureId != 0) {
280         GLES30.glActiveTexture(GLES30.GL_TEXTURE3)
281         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, shapeTextureId)
282         GLES30.glUniform1i(uShapeTex, 3)
283     }
284
285     GLES30.glBindVertexArray(vaoId)
286     GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, instanceBufferId)
287     GLES30.glBufferSubData(
288         GLES30.GL_ARRAY_BUFFER,
289         firstDab * DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES,
290         count * DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES,
291         dabs.floats,
292     )
293
294     // Reset offsets for instancing
295     GLES30.glVertexAttribPointer(
296         1, 4, GLES30.GL_FLOAT, false, DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES, 0
297     )
298     GLES30.glVertexAttribPointer(
299         2, 2, GLES30.GL_FLOAT, false, DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES, 16
300     )
301
302     // Manual attribute offset for ES 3.0 (since BaseInstance is 3.1+)
303     GLES30.glVertexAttribPointer(
304         1, 4, GLES30.GL_FLOAT, false, DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES,
305         (firstDab * DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES)
306     )
307     GLES30.glVertexAttribPointer(
308         2, 2, GLES30.GL_FLOAT, false, DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES,
309         (firstDab * DabBuffer.FLOATS_PER_DAB * Float.SIZE_BYTES + 16)
310     )
311
312     GLES30.glDrawArraysInstanced(GLES30.GL_TRIANGLE_STRIP, 0, 4, count)
313
314     if (grainTextureId != 0) {
315         GLES30.glActiveTexture(GLES30.GL_TEXTURE2)
316         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
317     }
318     if (shapeTextureId != 0) {
319         GLES30.glActiveTexture(GLES30.GL_TEXTURE3)
320         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
321     }
322     GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
323
324     GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0)
325     GLES30.glBindVertexArray(0)
326     blendController.end()
327     GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
328 }
329 }

```

30. app/src/main/java/com/wetinknext/engine/brush/OneEuroFilter.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlin.math.PI
4 import kotlin.math.sqrt
5
6 class OneEuroFilter(private var minCutoff: Float = 1.6f, private var beta: Float = 0.25f, private var dCutoff: Float = 1f) {
7     private var initialized = false; private var lastTimestampNanos = 0L
8     private var x = 0f; private var y = 0f; private var dx = 0f; private var dy = 0f
9     fun reset() { initialized = false; lastTimestampNanos = 0L; x = 0f; y = 0f; dx = 0f; dy = 0f }
10    fun filter(timestampNanos: Long, rawX: Float, rawY: Float, out: FloatArray) {
11        require(out.size >= 2)
12        if (!initialized) { initialized = true; lastTimestampNanos = timestampNanos; x = rawX; y = rawY; out[0] = x; out[1] =
y; return }
13        var dt = (timestampNanos - lastTimestampNanos) / 1_000_000_000f
14        if (dt < MIN_DT) dt = MIN_DT
15        lastTimestampNanos = timestampNanos
16        val rate = 1f / dt; val alphaD = alpha(rate, dCutoff)
17        dx = alphaD * ((rawX - x) * rate) + (1f - alphaD) * dx; dy = alphaD * ((rawY - y) * rate) + (1f - alphaD) * dy
18        val alpha = alpha(rate, (minCutoff + beta * sqrt(dx * dx + dy * dy)).coerceAtLeast(0.01f))
19        x = alpha * rawX + (1f - alpha) * x; y = alpha * rawY + (1f - alpha) * y; out[0] = x; out[1] = y
20    }
21    private fun alpha(rate: Float, cutoff: Float): Float { val tau = 1f / (2f * PI.toFloat() * cutoff); return 1f / (1f + tau
* rate) }
22    private companion object { const val MIN_DT = 1f / 1000f }
23 }
```

31. app/src/main/java/com/wetinknext/engine/brush/PressureFilter.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlin.math.exp
4
5 /**
6  * Time-based stylus pressure smoothing.
7  * A zero pressure sample while the pointer is still in contact is treated as a
8  * device glitch; Android sends the real zero value on UP after the final point.
9  */
10 class PressureFilter {
11     private var initialized = false
12     private var lastTimestampNanos = 0L
13     private var value = 0f
14
15     fun reset() { initialized = false; lastTimestampNanos = 0L; value = 0f }
16
17     fun filter(timestampNanos: Long, rawPressure: Float): Float {
18         val raw = rawPressure.coerceIn(0f, 1f)
19         if (!initialized) { initialized = true; lastTimestampNanos = timestampNanos; value = raw; return value }
20         val dt = ((timestampNanos - lastTimestampNanos) / NANOS_PER_SECOND).coerceIn(MIN_DT, MAX_DT)
21         lastTimestampNanos = timestampNanos
22         val stableRaw = if (raw <= ZERO_GLITCH_THRESHOLD && value > ZERO_GLITCH_THRESHOLD) value else raw
23         val tau = if (stableRaw >= value) RISE_TAU_SECONDS else FALL_TAU_SECONDS
24         val alpha = 1f - exp((-dt / tau).toDouble()).toFloat()
25         value += (stableRaw - value) * alpha
26         return value.coerceIn(0f, 1f)
27     }
28
29     private companion object {
30         const val NANOS_PER_SECOND = 1_000_000_000f
31         const val MIN_DT = 1f / 1000f
32         const val MAX_DT = 1f / 20f
33         const val RISE_TAU_SECONDS = .018f
34         const val FALL_TAU_SECONDS = .055f
35         const val ZERO_GLITCH_THRESHOLD = .01f
36     }
37 }
```

32. app/src/main/java/com/wetinknext/engine/brush/Stabilizer.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlin.math.sqrt
4
5 class Stabilizer(private val filter: OneEuroFilter = OneEuroFilter()) {
6     var strength = 0f; var x = 0f; private set; var y = 0f; private set; var velocity = 0f; private set
7     private var hasLast = false; private var lastX = 0f; private var lastY = 0f; private var lastT = 0L; private val filtered
8     = FloatArray(2)
9     fun reset() { filter.reset(); x = 0f; y = 0f; velocity = 0f; hasLast = false; lastX = 0f; lastY = 0f; lastT = 0L }
10    fun process(timestampNanos: Long, rawX: Float, rawY: Float) {
11        if (strength <= 0f) { x = rawX; y = rawY } else { filter.filter(timestampNanos, rawX, rawY, filtered); x = filtered[
12            0]; y = filtered[1] }
13        if (hasLast) { var dt = (timestampNanos - lastT) / 1_000_000_000f; if (dt < 1f / 1000f) dt = 1f / 1000f; val d = sqrt(
14            (x-lastX)*(x-lastX)+(y-lastY)*(y-lastY)); velocity += 0.2f * (d / dt - velocity) }
15        hasLast = true; lastX = x; lastY = y; lastT = timestampNanos
16    }
17 }
```

WetInk Next - source snapshot P0-P32

Page 69

```

91     }
92   }
93 }
94 }
95
96 fun finish(out: DabBuffer, cancel: Boolean) {
97   if (!active) return
98   val settings = activeSettings()
99   if (cancel) {
100     reset()
101     return
102   }
103   if (settings.smoothing > .0001f && pointCount >= 3) {
104     flushSmoothedTail(out)
105   }
106   if (!movedDuringStroke || carriedDistance > .001f) {
107     addDab(
108       x = lastX,
109       y = lastY,
110       pressure = lastPressure,
111       tiltX = lastTiltX,
112       tiltY = lastTiltY,
113       out = out,
114     )
115   }
116   reset()
117 }
118
119 private fun appendSmoothedPoint(x: Float, y: Float, pressure: Float, out: DabBuffer) {
120   p0x = p1x; p0y = p1y; p0p = p1p
121   p1x = p2x; p1y = p2y; p1p = p2p
122   p2x = p3x; p2y = p3y; p2p = p3p
123   p3x = x; p3y = y; p3p = pressure
124   pointCount = (pointCount + 1).coerceAtMost(4)
125
126   when (pointCount) {
127     2 -> addPoint(x, y, pressure, lastTiltX, lastTiltY, out)
128     4 -> interpolateCatmullRom(out)
129   }
130 }
131
132 /** Processes the last delayed Catmull-Rom segment with a duplicated endpoint. */
133 private fun flushSmoothedTail(out: DabBuffer) {
134   p0x = p1x; p0y = p1y; p0p = p1p
135   p1x = p2x; p1y = p2y; p1p = p2p
136   p2x = p3x; p2y = p3y; p2p = p3p
137   p3x = p2x; p3y = p2y; p3p = p2p
138   pointCount = 4
139   interpolateCatmullRom(out)
140 }
141
142 private fun interpolateCatmullRom(out: DabBuffer) {
143   val settings = activeSettings()
144   val dx = p2x - p1x
145   val dy = p2y - p1y
146   val distance = sqrt(dx * dx + dy * dy)
147   if (distance < .0001f) return
148
149   val steps = (distance / maxOf(1f, settings.baseRadiusPx * .25f))
150     .toInt()
151     .coerceIn(2, 64)
152
153   for (index in 1..steps) {
154     val t = index.toFloat() / steps
155     val t2 = t * t
156     val t3 = t2 * t
157     val x = .5f * (
158       2f * p1x + (-p0x + p2x) * t +
159       (2f * p0x - 5f * p1x + 4f * p2x - p3x) * t2 +
160       (-p0x + 3f * p1x - 3f * p2x + p3x) * t3
161     )
162     val y = .5f * (
163       2f * p1y + (-p0y + p2y) * t +
164       (2f * p0y - 5f * p1y + 4f * p2y - p3y) * t2 +
165       (-p0y + 3f * p1y - 3f * p2y + p3y) * t3
166     )
167     val pressure = p1p + (p2p - p1p) * t
168     addPoint(x, y, pressure, lastTiltX, lastTiltY, out)
169   }
170 }
171
172 private fun addPoint(x: Float, y: Float, p: Float, tx: Float, ty: Float, out: DabBuffer) {
173   val settings = activeSettings()
174   val dx = x - lastX
175   val dy = y - lastY
176   val distance = sqrt(dx * dx + dy * dy)
177   if (distance <= .0001f) return
178   if (distance > 1.5f) movedDuringStroke = true
179
180   var travelled = 0f
181   while (travelled < distance) {
182     val tProbe = (travelled / distance).coerceIn(0f, 1f)

```

```

183         val probePressure = lastPressure + (p - lastPressure) * tProbe
184         val probeTiltX = lastTiltX + (tx - lastTiltX) * tProbe
185         val probeTiltY = lastTiltY + (ty - lastTiltY) * tProbe
186
187         BrushDynamics.resolve(
188             settings = settings,
189             pressure = probePressure,
190             tiltX = probeTiltX,
191             tiltY = probeTiltY,
192             out = resolvedDab,
193         )
194
195         val step = spacingForRadius(resolvedDab.radius)
196         // Radius can shrink with pressure, leaving more carried distance
197         // than the new step. Emit at the current point and reset it.
198         val needed = (step - carriedDistance).coerceAtLeast(0f)
199
200         if (travelled + needed > distance) {
201             carriedDistance += distance - travelled
202             break
203         }
204
205         travelled += needed
206         val t = (travelled / distance).coerceIn(0f, 1f)
207         addDab(
208             x = lastX + dx * t,
209             y = lastY + dy * t,
210             pressure = lastPressure + (p - lastPressure) * t,
211             tiltX = lastTiltX + (tx - lastTiltX) * t,
212             tiltY = lastTiltY + (ty - lastTiltY) * t,
213             out = out,
214         )
215         carriedDistance = 0f
216     }
217     lastX = x
218     lastY = y
219     lastPressure = p
220     lastTiltX = tx
221     lastTiltY = ty
222 }
223
224 private fun spacingForRadius(radius: Float): Float {
225     val settings = activeSettings()
226     val unit = if (settings.spacingUsesDiameter) radius * 2f else radius
227     return (unit * settings.spacing).coerceIn(.25f, 256f)
228 }
229
230 private fun addDab(
231     x: Float,
232     y: Float,
233     pressure: Float,
234     tiltX: Float,
235     tiltY: Float,
236     out: DabBuffer,
237 ) {
238     val settings = activeSettings()
239     BrushDynamics.resolve(
240         settings = settings,
241         pressure = pressure,
242         tiltX = tiltX,
243         tiltY = tiltY,
244         out = resolvedDab,
245     )
246     out.add(
247         x = x,
248         y = y,
249         radius = resolvedDab.radius,
250         rotation = 0f,
251         coverage = resolvedDab.coverage,
252         flow = settings.flow.coerceIn(0f, 1f),
253     )
254 }
255 }

```

34. app/src/main/java/com/wetinknext/engine/brush/TextureLoader.kt

```

1  package com.wetinknext.engine.brush
2
3  import android.content.Context
4  import android.graphics.Bitmap
5  import android.graphics.BitmapFactory
6  import java.nio.ByteBuffer
7  import java.nio.ByteOrder
8  import java.util.concurrent.ExecutorService
9  import java.util.concurrent.Executors
10
11 data class LoadedBrushTexture(
12     val path: String,
13     val width: Int,
14     val height: Int,
15     val rgba: ByteBuffer,
16 )
17
18 class TextureLoader(
19     private val context: Context,
20     private val executor: ExecutorService =
21         Executors.newSingleThreadExecutor(),
22 ) {
23     fun loadAsync(
24         path: String,
25         onLoad: (LoadedBrushTexture) -> Unit,
26         onError: (Throwable) -> Unit,
27     ) {
28         executor.execute {
29             try {
30                 val loaded = decode(path)
31                 onLoad(loaded)
32             } catch (error: Throwable) {
33                 onError(error)
34             }
35         }
36     }
37
38     private fun decode(path: String): LoadedBrushTexture {
39         val source = decodeBitmap(path)
40
41         val bitmap = if (source.config == Bitmap.Config.ARGB_8888) {
42             source
43         } else {
44             source.copy(Bitmap.Config.ARGB_8888, false)
45         }
46
47         val width = bitmap.width
48         val height = bitmap.height
49
50         val rgba = ByteBuffer
51             .allocateDirect(width * height * 4)
52             .order(ByteOrder.nativeOrder())
53
54         // Bitmap stores ARGB_8888 pixels as packed ARGB ints. Uploading that
55         // buffer directly makes the byte order platform-dependent; GLES expects
56         // the explicit RGBA byte sequence declared in BrushTexture.
57         val pixels = IntArray(width * height)
58         bitmap.getPixels(
59             pixels,
60             0,
61             width,
62             0,
63             0,
64             width,
65             height,
66         )
67         for (pixel in pixels) {
68             rgba.put(((pixel ushr 16) and 0xFF).toByte())
69             rgba.put(((pixel ushr 8) and 0xFF).toByte())
70             rgba.put((pixel and 0xFF).toByte())
71             rgba.put(((pixel ushr 24) and 0xFF).toByte())
72         }
73         rgba.flip()
74
75         if (bitmap != source && !bitmap.isRecycled) {
76             bitmap.recycle()
77         }
78
79         if (!source.isRecycled && source != bitmap) {
80             source.recycle()
81         }
82
83         return LoadedBrushTexture(
84             path = path,
85             width = width,
86             height = height,
87             rgba = rgba,
88         )
89     }
90 }

```

```

91     private fun decodeBitmap(path: String): Bitmap {
92         val bitmap = if (path.startsWith("asset:")) {
93             val assetPath = path.removePrefix("asset:")
94
95             context.assets.open(assetPath).use { input ->
96                 BitmapFactory.decodeStream(input)
97             }
98         } else {
99             BitmapFactory.decodeFile(path)
100         }
101
102         return requireNotNull(bitmap) {
103             "Cannot decode brush texture: $path"
104         }
105     }
106
107     fun shutdown() {
108         executor.shutdown()
109     }
110 }

```

35. app/src/main/java/com/wetinknext/engine/canvas/BlendMode.kt

```
1 package com.wetinknext.engine.canvas
2
3 import kotlinx.serialization.Serializable
4
5 /** Stored per layer now; shader implementations beyond NORMAL are scheduled after P6. */
6 @Serializable
7 enum class BlendMode(val id: Int) {
8     NORMAL(0),
9     MULTIPLY(1),
10    SCREEN(2),
11    OVERLAY(3),
12    DARKEN(4),
13    LIGHTEN(5),
14    ADD(6),
15 }
```

36. app/src/main/java/com/wetinknext/engine/canvas/Compositor.kt

```

1  package com.wetinknext.engine.canvas
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.CanvasGeometry
5  import com.wetinknext.engine.gl.GlCheck
6  import com.wetinknext.engine.gl.GlProgram
7  import com.wetinknext.engine.gl.ShaderLib
8
9  /** Presents visible layers bottom-to-top, inserting the active preview at its layer position. */
10 class Compositor {
11     private var program: GlProgram? = null
12     private var uCanvasToClip = -1
13     private var uCanvasSize = -1
14     private var uLayerTex = -1
15     private var uStrokeTex = -1
16     private var uStrokeActive = -1
17     private var uOpacity = -1
18     private var uStrokeOpacity = -1
19
20     fun create() {
21         GlCheck.checkOnGltThread()
22         release()
23         program = GlProgram(ShaderLib.compositorVertex, ShaderLib.compositorFragment)
24         val currentProgram = checkNotNull(program)
25         currentProgram.use()
26         uCanvasToClip = GLES30.glGetUniformLocation(currentProgram.id, "uCanvasToClip")
27         uCanvasSize = GLES30.glGetUniformLocation(currentProgram.id, "uCanvasSize")
28         uLayerTex = GLES30.glGetUniformLocation(currentProgram.id, "uLayerTex")
29         uStrokeTex = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeTex")
30         uStrokeActive = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeActive")
31         uOpacity = GLES30.glGetUniformLocation(currentProgram.id, "uOpacity")
32         uStrokeOpacity = GLES30.glGetUniformLocation(currentProgram.id, "uStrokeOpacity")
33         check(uCanvasToClip >= 0 && uCanvasSize >= 0 && uLayerTex >= 0 &&
34             uStrokeTex >= 0 && uStrokeActive >= 0 && uOpacity >= 0 && uStrokeOpacity >= 0) {
35             "Compositor uniforms missing: canvasToClip=$uCanvasToClip, " +
36             "canvasSize=$uCanvasSize, layerTex=$uLayerTex, strokeTex=$uStrokeTex, " +
37             "strokeActive=$uStrokeActive, opacity=$uOpacity, strokeOpacity=$uStrokeOpacity"
38         }
39         GlCheck.noError("Compositor create")
40     }
41
42     fun render(
43         geometry: CanvasGeometry,
44         layers: LayerStack,
45         activeLayerId: Long,
46         strokeTextureId: Int,
47         canvasToClip: FloatArray,
48         strokeOpacity: Float,
49     ) {
50         val currentProgram = program ?: return
51         currentProgram.use()
52         GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, canvasToClip, 0)
53         GLES30.glUniform2f(uCanvasSize, layers.canvasWidth.toFloat(), layers.canvasHeight.toFloat())
54         GLES30.glEnable(GLES30.GL_BLEND)
55         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
56
57         for (layer in layers.allLayers()) {
58             if (!layer.created || !layer.isVisible) continue
59             val hasStroke = layer.id == activeLayerId && strokeTextureId != 0
60
61             GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
62             GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, layer.target.textureId)
63             GLES30.glUniform1i(uLayerTex, 0)
64             if (hasStroke) {
65                 GLES30.glActiveTexture(GLES30.GL_TEXTURE1)
66                 GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, strokeTextureId)
67                 GLES30.glUniform1i(uStrokeTex, 1)
68             }
69             GLES30.glUniform1i(uStrokeActive, if (hasStroke) 1 else 0)
70             GLES30.glUniform1f(uOpacity, layer.opacity.coerceIn(0f, 1f))
71             GLES30.glUniform1f(uStrokeOpacity, strokeOpacity.coerceIn(0f, 1f))
72             geometry.draw()
73         }
74
75         GLES30.glDisable(GLES30.GL_BLEND)
76         GLES30.glActiveTexture(GLES30.GL_TEXTURE1)
77         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
78         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
79         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
80     }
81
82     /**
83      * Draws one layer into the currently bound target.
84      *
85      * This is deliberately separate from [render]: a layer-preview needs the
86      * layer bitmap even when the layer is currently hidden in the project. Its
87      * opacity is still applied, while a blend mode has no second layer to blend
88      * against in this one-layer path.
89      */
90     fun renderLayer(

```

```

91     geometry: CanvasGeometry,
92     layer: PaintLayer,
93     canvasWidth: Int = layer.target.width,
94     canvasHeight: Int = layer.target.height,
95     canvasToClip: FloatArray,
96 ) {
97     if (!layer.created || canvasWidth <= 0 || canvasHeight <= 0) return
98     val currentProgram = program ?: return
99     currentProgram.use()
100     GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, canvasToClip, 0)
101     GLES30.glUniform2f(uCanvasSize, canvasWidth.toFloat(), canvasHeight.toFloat())
102     GLES30.glEnable(GLES30.GL_BLEND)
103     GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
104
105     GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
106     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, layer.target.textureId)
107     GLES30.glUniform1i(uLayerTex, 0)
108     GLES30.glUniform1i(uStrokeActive, 0)
109     GLES30.glUniform1f(uOpacity, layer.opacity.coerceIn(0f, 1f))
110     GLES30.glUniform1f(uStrokeOpacity, 1f)
111     geometry.draw()
112
113     GLES30.glDisable(GLES30.GL_BLEND)
114     GLES30.glActiveTexture(GLES30.GL_TEXTURE1)
115     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
116     GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
117     GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
118 }
119
120 fun release() {
121     GLCheck.checkOnGLThread()
122     program?.release()
123     program = null
124     uCanvasToClip = -1
125     uCanvasSize = -1
126     uLayerTex = -1
127     uStrokeTex = -1
128     uStrokeActive = -1
129     uOpacity = -1
130     uStrokeOpacity = -1
131 }
132 }

```

37. app/src/main/java/com/wetinknext/engine/canvas/LayerStack.kt

```

1  package com.wetinknext.engine.canvas
2
3  import com.wetinknext.engine.core.Camera
4  import com.wetinknext.engine.gl.GlCaps
5  import com.wetinknext.domain.document.ProjectDocument
6
7  /**
8   * Render-thread-owned ordered collection of document layers.
9   * Index 0 is the bottom-most layer and the final item is the top-most layer.
10  */
11  class LayerStack {
12      val camera = Camera()
13
14      private val layers = mutableListOf<PaintLayer>()
15      private var nextId = 1L
16      private var documentUsesHalfFloat = false
17
18      var activeLayerId: Long = NO_LAYER
19      private set
20      var canvasWidth: Int = 0
21      private set
22      var canvasHeight: Int = 0
23      private set
24
25      val count: Int get() = layers.size
26
27      fun allLayers(): List<PaintLayer> = layers
28
29      fun activeLayer(): PaintLayer? = findLayerById(activeLayerId)
30
31      fun findLayerById(id: Long): PaintLayer? = layers.firstOrNull { it.id == id }
32
33      /** Creates the runtime stack from persisted layer metadata. */
34      fun create(caps: GlCaps, document: ProjectDocument) {
35          release()
36          canvasWidth = document.width
37          canvasHeight = document.height
38          documentUsesHalfFloat = caps.supportsHalfFloatColorBuffer
39
40          document.layers.forEachIndexed { index, source ->
41              val layer = PaintLayer(source.id, source.name).also {
42                  it.isVisible = source.visible
43                  it.isLocked = source.locked
44                  it.opacity = source.opacity
45                  it.blendMode = source.blendMode
46                  it.version = source.thumbnailVersion
47                  it.create(canvasWidth, canvasHeight, documentUsesHalfFloat)
48              }
49              // Tile restoration is a later persistence step. Preserve the new
50              // document's white background until a stored pixel payload is loaded.
51              if (index == 0 && source.locked) {
52                  layer.target.clear(1f, 1f, 1f, 1f)
53              }
54              layers += layer
55          }
56
57          activeLayerId = document.activeLayerId ?: layers.last().id
58          nextId = (layers.maxOf { it.id } + 1L).coerceAtLeast(1L)
59      }
60
61      /** Creates the locked opaque white background and one transparent drawing layer. */
62      fun create(caps: GlCaps, width: Int, height: Int) {
63          require(width > 0 && height > 0)
64          release()
65          canvasWidth = width
66          canvasHeight = height
67          documentUsesHalfFloat = caps.supportsHalfFloatColorBuffer
68
69          val background = newLayer("Фон", documentUsesHalfFloat).also {
70              it.isLocked = true
71              it.target.clear(1f, 1f, 1f, 1f)
72          }
73          layers += background
74
75          val drawing = newLayer("Слой 1", documentUsesHalfFloat)
76          layers += drawing
77          activeLayerId = drawing.id
78      }
79
80      fun addLayer(name: String, insertAt: Int = layers.size): PaintLayer {
81          check(canvasWidth > 0 && canvasHeight > 0) { "LayerStack has not been created" }
82          val layer = newLayer(name, documentUsesHalfFloat)
83          layers.add(insertAt.coerceIn(0, layers.size), layer)
84          activeLayerId = layer.id
85          return layer
86      }
87
88      /** A document must retain at least one layer; locked layers cannot be removed. */
89      fun removeLayer(id: Long): PaintLayer? {
90          val index = layers.indexOfFirst { it.id == id }

```

```

91         if (index < 0 || layers.size <= 1) return null
92         val layer = layers[index]
93         if (layer.isLocked) return null
94
95         layers.removeAt(index)
96         layer.release()
97         if (activeLayerId == id) {
98             activeLayerId = layers[index.coerceAtMost(layers.lastIndex)].id
99         }
100         return layer
101     }
102
103     fun setActive(id: Long): Boolean {
104         if (findLayerById(id) == null) return false
105         activeLayerId = id
106         return true
107     }
108
109     fun moveLayer(id: Long, delta: Int): Boolean {
110         val index = layers.indexOfFirst { it.id == id }
111         if (index < 0) return false
112         val newIndex = (index + delta).coerceIn(0, layers.lastIndex)
113         if (newIndex == index) return false
114         val layer = layers.removeAt(index)
115         layers.add(newIndex, layer)
116         return true
117     }
118
119     fun release() {
120         layers.forEach(PaintLayer::release)
121         layers.clear()
122         activeLayerId = NO_LAYER
123         canvasWidth = 0
124         canvasHeight = 0
125         documentUsesHalfFloat = false
126     }
127
128     private fun newLayer(name: String, useHalfFloat: Boolean): PaintLayer =
129         PaintLayer(nextId++, name).also { it.create(canvasWidth, canvasHeight, useHalfFloat) }
130
131     private companion object {
132         const val NO_LAYER = -1L
133     }
134 }

```

38. app/src/main/java/com/wetinknext/engine/canvas/PaintLayer.kt

```
1 package com.wetinknext.engine.canvas
2
3 import com.wetinknext.engine.gl.RenderTarget
4
5 /** One paintable document layer and its private GPU target. */
6 class PaintLayer(
7     val id: Long,
8     var name: String,
9 ) {
10     val target = RenderTarget()
11
12     var isVisible = true
13     var isLocked = false
14     var opacity = 1f
15     var blendMode = BlendMode.NORMAL
16     var version = 0L
17
18     var created = false
19     private set
20
21     fun create(width: Int, height: Int, preferHalfFloat: Boolean) {
22         if (created && target.width == width && target.height == height) return
23         target.create(width, height, preferHalfFloat)
24         target.clear(0f, 0f, 0f, 0f)
25         created = true
26     }
27
28     fun clear() {
29         if (created) target.clear(0f, 0f, 0f, 0f)
30     }
31
32     fun release() {
33         target.release()
34         created = false
35     }
36 }
```

39. app/src/main/java/com/wetinknext/engine/canvas/StrokeBlitter.kt

```

1  package com.wetinknext.engine.canvas
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.gl.CanvasGeometry
5  import com.wetinknext.engine.gl.GlProgram
6  import com.wetinknext.engine.gl.RenderTarget
7  import com.wetinknext.engine.gl.ShaderLib
8
9  /** Переносит готовый штрих из strokeTarget в слой: premultiplied source-over * opacity. */
10 class StrokeBlitter {
11     private var program: GlProgram? = null
12     private var uCanvasToClip = -1
13     private var uCanvasSize = -1
14     private var uStrokeTex = -1
15     private var uOpacity = -1
16
17     fun create() {
18         release()
19         program = GlProgram(ShaderLib.compositorVertex, ShaderLib.strokeBlitFragment)
20         val p = checkNotNull(program)
21         p.use()
22         uCanvasToClip = GLES30.glGetUniformLocation(p.id, "uCanvasToClip")
23         uCanvasSize = GLES30.glGetUniformLocation(p.id, "uCanvasSize")
24         uStrokeTex = GLES30.glGetUniformLocation(p.id, "uStrokeTex")
25         uOpacity = GLES30.glGetUniformLocation(p.id, "uOpacity")
26         check(uCanvasToClip >= 0 && uCanvasSize >= 0 && uStrokeTex >= 0 && uOpacity >= 0) {
27             "StrokeBlitter uniforms missing"
28         }
29     }
30
31     fun blit(
32         layer: RenderTarget,
33         geometry: CanvasGeometry,
34         strokeTextureId: Int,
35         canvasToFbo: FloatArray,
36         width: Int,
37         height: Int,
38         opacity: Float,
39     ) {
40         val p = program ?: return
41         if (strokeTextureId == 0) return
42         layer.bind()
43         GLES30.glViewport(0, 0, width, height)
44         GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
45         GLES30.glEnable(GLES30.GL_BLEND)
46         GLES30.glBlendEquation(GLES30.GL_FUNC_ADD)
47         GLES30.glBlendFunc(GLES30.GL_ONE, GLES30.GL_ONE_MINUS_SRC_ALPHA)
48         p.use()
49         GLES30.glUniformMatrix4fv(uCanvasToClip, 1, false, canvasToFbo, 0)
50         GLES30.glUniform2f(uCanvasSize, width.toFloat(), height.toFloat())
51         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
52         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, strokeTextureId)
53         GLES30.glUniform1i(uStrokeTex, 0)
54         GLES30.glUniform1f(uOpacity, opacity.coerceIn(0f, 1f))
55         geometry.draw()
56         GLES30.glDisable(GLES30.GL_BLEND)
57         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
58         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
59     }
60
61     fun release() {
62         program?.release()
63         program = null
64         uCanvasToClip = -1; uCanvasSize = -1; uStrokeTex = -1; uOpacity = -1
65     }
66 }

```

40. app/src/main/java/com/wetinknext/engine/core/Camera.kt

```
1 package com.wetinknext.engine.core
2
3 import java.util.concurrent.atomic.AtomicReference
4 import kotlin.math.min
5
6 /** Thread-safe camera state shared by the UI and GL threads. */
7 class Camera(initial: ViewTransform = ViewTransform()) {
8     private val state = AtomicReference(initial)
9
10    fun snapshot(): ViewTransform = state.get()
11
12    fun set(transform: ViewTransform) {
13        state.set(transform)
14    }
15
16    fun update(transform: (ViewTransform) -> ViewTransform) {
17        while (true) {
18            val current = state.get()
19            if (state.compareAndSet(current, transform(current))) return
20        }
21    }
22
23    fun reset() {
24        state.set(ViewTransform())
25    }
26
27    fun fitCanvas(
28        canvasWidth: Int,
29        canvasHeight: Int,
30        viewWidth: Int,
31        viewHeight: Int,
32        marginRatio: Float = 0.04f,
33    ) {
34        if (canvasWidth <= 0 || canvasHeight <= 0 || viewWidth <= 0 || viewHeight <= 0) return
35        val usableWidth = viewWidth * (1f - marginRatio * 2f)
36        val usableHeight = viewHeight * (1f - marginRatio * 2f)
37        val scale = min(usableWidth / canvasWidth, usableHeight / canvasHeight)
38        .coerceIn(ViewTransform.MIN_SCALE, ViewTransform.MAX_SCALE)
39        state.set(
40            ViewTransform(
41                scale = scale,
42                translateX = (viewWidth - canvasWidth * scale) * 0.5f,
43                translateY = (viewHeight - canvasHeight * scale) * 0.5f,
44            ),
45        )
46    }
47 }
```

41. app/src/main/java/com/wetinknext/engine/core/DirtyRect.kt

```
1 package com.wetinknext.engine.core
2
3 import kotlin.math.ceil
4 import kotlin.math.floor
5 import kotlin.math.max
6 import kotlin.math.min
7
8 /** Reusable mutable dirty rectangle in canvas coordinates. */
9 class DirtyRect {
10     var left = 0f; private set
11     var top = 0f; private set
12     var right = 0f; private set
13     var bottom = 0f; private set
14     var isEmpty = true; private set
15
16     fun clear() {
17         left = 0f; top = 0f; right = 0f; bottom = 0f; isEmpty = true
18     }
19
20     fun include(x: Float, y: Float, radius: Float) = include(x - radius, y - radius, x + radius, y + radius)
21
22     fun include(left: Float, top: Float, right: Float, bottom: Float) {
23         if (right < left || bottom < top) return
24         if (isEmpty) {
25             this.left = left; this.top = top; this.right = right; this.bottom = bottom; isEmpty = false
26             return
27         }
28         this.left = min(this.left, left); this.top = min(this.top, top)
29         this.right = max(this.right, right); this.bottom = max(this.bottom, bottom)
30     }
31
32     fun include(other: DirtyRect) {
33         if (!other.isEmpty) include(other.left, other.top, other.right, other.bottom)
34     }
35
36     fun expand(pixels: Float) {
37         if (!isEmpty && pixels > 0f) {
38             left -= pixels; top -= pixels; right += pixels; bottom += pixels
39         }
40     }
41
42     fun clamp(canvasWidth: Float, canvasHeight: Float) {
43         if (isEmpty) return
44         left = left.coerceIn(0f, canvasWidth); top = top.coerceIn(0f, canvasHeight)
45         right = right.coerceIn(0f, canvasWidth); bottom = bottom.coerceIn(0f, canvasHeight)
46         if (right <= left || bottom <= top) clear()
47     }
48
49     fun copyTo(out: DirtyRect) {
50         out.left = left; out.top = top; out.right = right; out.bottom = bottom; out.isEmpty = isEmpty
51     }
52
53     fun toPixelBounds(out: IntArray) {
54         require(out.size >= 4)
55         if (isEmpty) {
56             out[0] = 0; out[1] = 0; out[2] = 0; out[3] = 0
57             return
58         }
59         out[0] = floor(left).toInt(); out[1] = floor(top).toInt()
60         out[2] = ceil(right).toInt(); out[3] = ceil(bottom).toInt()
61     }
62 }
```

42. app/src/main/java/com/wetinknext/engine/core/DocumentSession.kt

```
1 package com.wetinknext.engine.core
2
3 import com.wetinknext.domain.document.ProjectDocument
4 import com.wetinknext.engine.canvas.LayerStack
5 import com.wetinknext.engine.gl.GlCaps
6 import com.wetinknext.engine.undo.UndoManager
7
8 /** Render-thread lifetime of an opened document and its GPU layer resources. */
9 class DocumentSession(
10     val document: ProjectDocument,
11     val layerStack: LayerStack,
12     val undoManager: UndoManager,
13     private val layerTiles: Map<Long, ByteArray>,
14 ) {
15     /** Layers whose pixels must be persisted by the next autosave. GL-thread owned. */
16     val dirtyLayerIds = mutableSetOf<Long>()
17
18     /** Metadata or pixels changed since the last successful autosave. GL-thread owned. */
19     var projectDirty: Boolean = false
20     private set
21
22     fun markLayerDirty(layerId: Long) {
23         dirtyLayerIds += layerId
24         projectDirty = true
25     }
26
27     fun markProjectDirty() {
28         projectDirty = true
29     }
30
31     fun markSaved(savedLayerIds: Set<Long>) {
32         dirtyLayerIds.removeAll(savedLayerIds)
33         projectDirty = dirtyLayerIds.isNotEmpty()
34     }
35
36     fun loadIntoGpu(caps: GlCaps) {
37         layerStack.create(caps, document)
38         document.layers.forEach { layer ->
39             LayerTileCodec.uploadPersistent(
40                 target = checkNotNull(layerStack.findLayerById(layer.id)).target,
41                 payload = layerTiles[layer.id] ?: ByteArray(0),
42             )
43         }
44         check(layerStack.activeLayerId == document.activeLayerId) { "Active layer was not restored" }
45     }
46 }
```

43. app/src/main/java/com/wetinknext/engine/core/EditorState.kt

```
1 package com.wetinknext.engine.core
2
3 /** Immutable state published by the GL thread for Compose to render. */
4 data class LayerUiModel(
5     val id: Long,
6     val name: String,
7     val isVisible: Boolean,
8     val isLocked: Boolean,
9     val opacity: Float,
10    val active: Boolean,
11    val canDelete: Boolean,
12    val thumbnailPath: String? = null,
13    val thumbnailVersion: Long = 0L,
14 )
15
16 data class UndoDiagnostics(
17     val pendingJobs: Int = 0,
18     val staleResults: Int = 0,
19     val compressionFailures: Int = 0,
20     val restoreFailures: Int = 0,
21     val memoryBytes: Long = 0L,
22 )
23
24 data class EditorUiState(
25     val layers: List<LayerUiModel>,
26     val canUndo: Boolean,
27     val canRedo: Boolean,
28     val brushSizePx: Float,
29     val brushOpacity: Float,
30     val activeLayerId: Long,
31     val ready: Boolean,
32     val undoDiagnostics: UndoDiagnostics = UndoDiagnostics(),
33 ) {
34     companion object {
35         val empty = EditorUiState(
36             layers = emptyList(),
37             canUndo = false,
38             canRedo = false,
39             brushSizePx = 16f,
40             brushOpacity = 1f,
41             activeLayerId = -1L,
42             ready = false,
43         )
44     }
45 }
```

44. app/src/main/java/com/wetinknext/engine/core/EngineRenderer.kt

```

1 package com.wetinknext.engine.core
2
3 import android.opengl.GLES30
4 import android.opengl.GLSurfaceView
5 import android.util.Log
6 import android.view.MotionEvent
7 import androidx.compose.ui.graphics.toArgb
8 import com.wetinknext.BuildConfig
9 import com.wetinknext.domain.document.LayerDocument
10 import com.wetinknext.domain.document.ProjectDocument
11 import com.wetinknext.engine.brush.BrushSettings
12 import com.wetinknext.engine.brush.BrushTexture
13 import com.wetinknext.engine.brush.LoadedBrushTexture
14 import com.wetinknext.engine.brush.BrushRenderMode
15 import com.wetinknext.engine.brush.CapsuleEmitter
16 import com.wetinknext.engine.brush.CapsuleStrokeRenderer
17 import com.wetinknext.engine.brush.ColorSpaces
18 import com.wetinknext.engine.brush.DabBuffer
19 import com.wetinknext.engine.brush.DabRenderer
20 import com.wetinknext.engine.brush.StampEmitter
21 import com.wetinknext.engine.canvas.Compositor
22 import com.wetinknext.engine.canvas.LayerStack
23 import com.wetinknext.engine.canvas.PaintLayer
24 import com.wetinknext.engine.canvas.StrokeBlitter
25 import com.wetinknext.engine.gl.CanvasGeometry
26 import com.wetinknext.engine.gl.GlCaps
27 import com.wetinknext.engine.gl.GlCheck
28 import com.wetinknext.engine.gl.RenderTarget
29 import com.wetinknext.engine.input.InputAction
30 import com.wetinknext.engine.input.InputBatch
31 import com.wetinknext.engine.input.InputBatchPool
32 import com.wetinknext.engine.input.GestureRouter
33 import com.wetinknext.engine.input.StrokeInputCapturer
34 import com.wetinknext.engine.persistence.LayerTileStore
35 import com.wetinknext.engine.thumbnail.ThumbnailBuildResult
36 import com.wetinknext.engine.thumbnail.ThumbnailBuildScheduler
37 import com.wetinknext.engine.thumbnail.ThumbnailRenderer
38 import com.wetinknext.engine.undo.TileSnapshotRestore
39 import com.wetinknext.engine.undo.UndoManager
40 import java.io.File
41 import java.util.concurrent.ArrayBlockingQueue
42 import java.util.concurrent.atomic.AtomicBoolean
43 import javax.microedition.khronos.egl.EGLConfig
44 import javax.microedition.khronos.opengles.GL10
45
46 /**
47  * The P6 render-thread owner. Active dabs are accumulated only in strokeTarget;
48  * UP snapshots and merges them into the selected PaintLayer atomically.
49  */
50 class EngineRenderer(
51     private var projectDocument: ProjectDocument = ProjectDocument.newUntitled(),
52     private val initialLayerTiles: Map<Long, ByteArray> = emptyMap(),
53     private val thumbnailOutputDirectory: File? = null,
54 ) : GLSurfaceView.Renderer {
55     private var caps: GlCaps? = null
56     private val layerStack = LayerStack()
57     private val undoManager = UndoManager()
58     private val layerTileStore = LayerTileStore(initialLayerTiles)
59     private var documentSession: DocumentSession? = null
60     private val undoPipeline = UndoCompressionPipeline(
61         requestRender = { onInputRenderRequested?.invoke() },
62     )
63     private var undoRestoreFailures = 0
64     private var glThread: Thread? = null
65
66     private var geometry: CanvasGeometry? = null
67     private var compositor: Compositor? = null
68     private val thumbnailCapture = ThumbnailCapture()
69     private val layerThumbnailPaths = mutableMapOf<Long, String>()
70     /** Last layer version for which a WebP preview was successfully published. */
71     private val layerThumbnailVersions = mutableMapOf<Long, Long>()
72     private val thumbnailScheduler = thumbnailOutputDirectory?.let { outputDirectory ->
73         ThumbnailBuildScheduler(
74             outputDirectory = outputDirectory,
75             renderer = ThumbnailRenderer(),
76             requestGlFrame = { onInputRenderRequested?.invoke() },
77             onSave = { result -> onThumbnailBuildSaved?.invoke(result) },
78             onFailure = { error ->
79                 if (BuildConfig.DEBUG) Log.e(THUMBNAIL_TAG, "Thumbnail build failed", error)
80             },
81         )
82     }
83     private var dabRenderer: DabRenderer? = null
84     private var capsuleRenderer: CapsuleStrokeRenderer? = null
85     private var grainTexture: BrushTexture? = null
86     private var grainPath: String? = null
87     private var shapeTexture: BrushTexture? = null
88     private var shapePath: String? = null
89     private val strokeTarget = RenderTarget()
90     private val canvasToFboMatrix = FloatArray(16)

```

```

91     private val strokeCommitter = StrokeCommitter(
92         strokeTarget = strokeTarget,
93         canvasToFbo = canvasToFboMatrix,
94         undoPipeline = undoPipeline,
95         undoManager = undoManager,
96         onTilesCommitted = { layer, tiles ->
97             layerTileStore.markDirty(layer.id, tiles)
98             documentSession?.markLayerDirty(layer.id)
99             publishDirtyTiles()
100             markLayerThumbnailDirty(layer.id)
101         },
102     )
103     private val canvasToClipMatrix = FloatArray(16)
104
105     private var screenWidth = 1
106     private var screenHeight = 1
107     private val canvasBackdropColor = floatArrayOf(0.08f, 0.09f, 0.12f)
108     private val canvasGridColor = floatArrayOf(0.16f, 0.18f, 0.22f)
109
110     private val inputPool = InputBatchPool(batchCount = 64, maxSamplesPerBatch = 256)
111     private val inputQueue = ArrayBlockingQueue<InputBatch>(64)
112     private val inputCapturer = StrokeInputCapturer(layerStack.camera, inputPool, inputQueue)
113     private val undoGestureRequested = AtomicBoolean(false)
114     private val redoGestureRequested = AtomicBoolean(false)
115     private val resetCameraGestureRequested = AtomicBoolean(false)
116     private val gestureRouter = GestureRouter(
117         camera = layerStack.camera,
118         drawingInput = inputCapturer,
119         callbacks = object : GestureRouter.Callbacks {
120             override fun cancelDrawing() {
121                 requestCancelFromInput()
122             }
123
124             override fun undoGesture() {
125                 undoGestureRequested.set(true)
126             }
127
128             override fun redoGesture() {
129                 redoGestureRequested.set(true)
130             }
131
132             override fun resetCamera() {
133                 resetCameraGestureRequested.set(true)
134             }
135
136             override fun requestRender() {
137                 onInputRenderRequested?.invoke()
138             }
139         },
140     )
141
142     private var brushSettings = BrushSettings(
143         name = "G-Pen",
144         renderMode = BrushRenderMode.RIBBON,
145         baseRadiusPx = 9f,
146         spacing = 0.12f,
147         colorArgb = 0xFF1B1F24L,
148         smoothing = 0.35f,
149         streamline = 0.22f,
150         pressureToOpacity = false,
151         pressureGamma = 1.7f,
152         minSizeRatio = 0.06f,
153         ribbon = com.wetinknext.engine.brush.RibbonSettings(
154             cap = com.wetinknext.engine.brush.RibbonCap.ROUND,
155             join = com.wetinknext.engine.brush.RibbonJoin.ROUND,
156             miterLimit = 2.5f,
157             minPointDistancePx = 1.25f,
158             aaWidthPx = 1f,
159             taperStartPx = 16f,
160             taperEndPx = 64f,
161         ),
162     )
163     private val stampEmitter = StampEmitter(brushSettings)
164     private val capsuleEmitter = CapsuleEmitter(brushSettings)
165     private val dabBuffer = DabBuffer()
166     private val strokeColorLinear = FloatArray(3)
167     private val activeStrokeColorLinear = FloatArray(3)
168     private val strokeDirtyRect = DirtyRect()
169     private val dirtyBounds = IntArray(4)
170
171     private var strokeActive = false
172     private var activeStrokeBrush: BrushSettings? = null
173     private var strokeBlitter: StrokeBlitter? = null
174     private var capsulePreviewInitialized = false
175     private var stampPreviewInitialized = false
176     private val cancelRequested = AtomicBoolean(false)
177
178     /** Assigned by PaintSurfaceView; invoked on the GL thread. */
179     var onStateChange: ((EditorUiState) -> Unit)? = null
180
181     /** Metadata-only snapshot for persistence; invoked on the GL thread. */
182     var onProjectDocumentChange: ((ProjectDocument) -> Unit)? = null

```

```

183
184     /** Called only after document metadata and all stored layer payloads reached the GPU. */
185     var onDocumentSessionLoaded: (() -> Unit)? = null
186     var onDirtyLayerTiles: ((ProjectDocument, Map<Long, ByteArray>, Map<Long, Set<com.wetinknext.engine.undo.TileCoord>>) ->
Unit)? = null
187     var onThumbnailCaptured: ((ThumbnailCapture.Rgba) -> Unit)? = null
188     /** Posted on the UI thread after the scheduler published the current previews. */
189     var onThumbnailBuildSaved: ((ThumbnailBuildResult) -> Unit)? = null
190
191     /** Assigned by PaintSurfaceView; invoked from the UI touch thread. */
192     var onInputRenderRequested: (() -> Unit)? = null
193
194     fun setOnSecondaryPointerDown(listener: () -> Unit) {
195         inputCapturer.onSecondaryPointerDown = listener
196     }
197
198     fun onTouchEvent(event: MotionEvent): Boolean = gestureRouter.onTouchEvent(event)
199
200     fun requestCancelFromInput() {
201         cancelRequested.set(true)
202     }
203
204     /** Fits the document into the current GL surface. Must run on the GL thread. */
205     fun resetCamera() {
206         layerStack.camera.fitCanvas(
207             canvasWidth = layerStack.canvasWidth,
208             canvasHeight = layerStack.canvasHeight,
209             viewWidth = screenWidth,
210             viewHeight = screenHeight,
211         )
212     }
213
214     /** Applies theme colors to the area that surrounds the document. */
215     fun setCanvasBackdrop(backdropArgb: Int, gridArgb: Int) {
216         argbToRgb(backdropArgb, canvasBackdropColor)
217         argbToRgb(gridArgb, canvasGridColor)
218     }
219
220     /** These methods are called from GLSurfaceView.queueEvent by the UI layer. */
221     fun undo() {
222         if (undoPipeline.pendingCount > 0) return
223         resetStroke()
224         while (true) {
225             val entry = undoManager.peekUndo() ?: return
226             val layer = layerStack.findLayerById(entry.layerId)
227             if (layer == null) {
228                 undoManager.dropUndo(entry)
229                 continue
230             }
231             val restored = TileSnapshotRestore.restore(layer.target, entry.beforeTiles)
232             if (!restored) {
233                 undoRestoreFailures++
234                 publishState()
235                 return
236             }
237             if (!undoManager.commitUndo(entry)) {
238                 publishState()
239                 return
240             }
241             layer.version++
242             persistRestoredTiles(layer, entry.beforeTiles)
243             updateProjectDocument { it }
244             publishState()
245             return
246         }
247     }
248
249     fun redo() {
250         if (undoPipeline.pendingCount > 0) return
251         resetStroke()
252         while (true) {
253             val entry = undoManager.peekRedo() ?: return
254             val layer = layerStack.findLayerById(entry.layerId)
255             if (layer == null) {
256                 undoManager.dropRedo(entry)
257                 continue
258             }
259             val restored = TileSnapshotRestore.restore(layer.target, entry.afterTiles)
260             if (!restored) {
261                 undoRestoreFailures++
262                 publishState()
263                 return
264             }
265             if (!undoManager.commitRedo(entry)) {
266                 publishState()
267                 return
268             }
269             layer.version++
270             persistRestoredTiles(layer, entry.afterTiles)
271             updateProjectDocument { it }
272             publishState()
273             return

```

```

274     }
275 }
276
277 fun setActiveLayer(id: Long): Boolean {
278     resetStroke()
279     return layerStack.setActive(id).also { changed ->
280         if (changed) {
281             updateProjectDocument { it.copy(activeLayerId = id) }
282             publishState()
283         }
284     }
285 }
286
287 fun addLayer(): Long = addLayer(nextLayerName())
288
289 fun addLayer(name: String): Long {
290     resetStroke()
291     val layer = layerStack.addLayer(name)
292     updateProjectDocument { document ->
293         document.copy(
294             layers = document.layers + layerDocumentFrom(layer),
295             activeLayerId = layer.id,
296         )
297     }
298     captureThumbnail(listOf(layer.id))
299     publishState()
300     return layer.id
301 }
302
303 fun removeLayer(id: Long): Boolean {
304     checkOnGLThread()
305     resetStroke()
306     invalidatePendingUndoHistory()
307     val removed = layerStack.removeLayer(id) != null
308     if (removed) {
309         undoManager.removeEntriesForLayer(id)
310         layerThumbnailPaths.remove(id)
311         layerThumbnailVersions.remove(id)
312         thumbnailScheduler?.removeLayer(id)
313         updateProjectDocument { document ->
314             document.copy(
315                 layers = document.layers.filterNot { it.id == id },
316                 activeLayerId = layerStack.activeLayerId,
317             )
318         }
319         captureThumbnail()
320         publishState()
321     }
322     return removed
323 }
324
325 /** Moves one layer in the document order and rebuilds only the project preview. */
326 fun moveLayer(id: Long, delta: Int): Boolean {
327     checkOnGLThread()
328     resetStroke()
329     val moved = layerStack.moveLayer(id, delta)
330     if (moved) {
331         updateProjectDocument { it }
332         captureThumbnail()
333         publishState()
334     }
335     return moved
336 }
337
338 fun setLayerVisible(id: Long, visible: Boolean) {
339     resetStroke()
340     val layer = layerStack.findLayerById(id) ?: return
341     layer.isVisible = visible
342     syncLayerDocument(layer)
343     captureThumbnail()
344     publishState()
345 }
346
347 fun setLayerOpacity(id: Long, opacity: Float) {
348     val layer = layerStack.findLayerById(id) ?: return
349     layer.opacity = opacity.coerceIn(0f, 1f)
350     syncLayerDocument(layer)
351     captureThumbnail(listOf(layer.id))
352     publishState()
353 }
354
355 /** Clears only the editable active layer. The locked background is protected. */
356 fun clearActiveLayer(): Boolean {
357     checkOnGLThread()
358     resetStroke()
359     val layer = layerStack.activeLayer() ?: return false
360     val cleared = strokeCommitter.clearLayer(
361         layer = layer,
362         canvasWidth = layerStack.canvasWidth,
363         canvasHeight = layerStack.canvasHeight,
364     )
365     if (cleared) {

```

```

366         updateProjectDocument { it }
367         publishState()
368     }
369     return cleared
370 }
371
372 /** Mirrors only the camera view; document pixels and undo history stay unchanged. */
373 fun toggleCanvasMirror(): Boolean {
374     resetStroke()
375     val width = layerStack.canvasWidth
376     val height = layerStack.canvasHeight
377     if (width <= 0 || height <= 0) return false
378
379     // Flip around the document centre in screen space. Toggling the matrix
380     // sign alone flips around the camera origin and makes the canvas jump.
381     val camera = layerStack.camera
382     val current = camera.snapshot()
383     val canvasCenterX = width * .5f
384     val canvasCenterY = height * .5f
385     val before = FloatArray(2)
386     val after = FloatArray(2)
387     current.canvasToScreen(canvasCenterX, canvasCenterY, before)
388     val flipped = current.copy(flipX = !current.flipX)
389     flipped.canvasToScreen(canvasCenterX, canvasCenterY, after)
390     camera.set(
391         flipped.copy(
392             translateX = flipped.translateX + before[0] - after[0],
393             translateY = flipped.translateY + before[1] - after[1],
394         ),
395     )
396     return true
397 }
398
399 /** px = ДИАМЕТР кисти в canvas-пикселях, ровно то, что показывает UI. */
400 fun setBrushSize(px: Float) =
401     updateBrush(brushSettings.copy(baseRadiusPx = (px * .5f).coerceIn(.5f, 200f)))
402
403 fun setBrushOpacity(opacity: Float) =
404     updateBrush(brushSettings.copy(opacity = opacity.coerceIn(0f, 1f)))
405
406 fun setBrushColor(color: androidx.compose.ui.graphics.Color) =
407     updateBrush(brushSettings.copy(colorArgb = color.toArgb().toLong() and 0xFFFFFFFFL))
408
409 fun applyBrush(settings: BrushSettings) {
410     val nextSettings = settings
411         .copy(colorArgb = brushSettings.colorArgb)
412         .resolved()
413     updateBrush(nextSettings)
414 }
415
416 fun setBrushSettings(settings: BrushSettings) {
417     updateBrush(settings)
418 }
419
420 /** Правка кисти во время штриха больше не отменяет штрих: применяем на следующем DOWN. */
421 private fun updateBrush(next: BrushSettings) {
422     brushSettings = next.resolved()
423     if (!strokeActive) {
424         applyBrushToEmitters(brushSettings)
425         ColorSpaces.srgb8ToLinear(brushSettings.colorArgb, strokeColorLinear)
426     }
427     publishState()
428 }
429
430 private fun applyBrushToEmitters(settings: BrushSettings) {
431     stampEmitter.updateSettings(settings)
432     capsuleEmitter.updateSettings(settings)
433 }
434
435 fun requestState() {
436     publishState()
437 }
438
439 fun checkOnGUIThread() {
440     val current = Thread.currentThread()
441     check(current === glThread) {
442         "GL resource accessed outside GLSurfaceView render thread: current=$current, expected=$glThread"
443     }
444 }
445
446 override fun onSurfaceCreated(gl: GL10?, config: EGLConfig?) {
447     val currentThread = Thread.currentThread()
448     glThread = currentThread
449     GLCheck.setGLThread(currentThread)
450     releaseGLObjects()
451     undoPipeline.ensureRunning()
452     thumbnailScheduler?.ensureRunning()
453     val nextCaps = GLCaps.query()
454     caps = nextCaps
455     documentSession = DocumentSession(
456         document = projectDocument,
457         layerStack = layerStack,

```

```

458         undoManager = undoManager,
459         layerTiles = initialLayerTiles,
460     ).also { it.loadIntoGpu(nextCaps) }
461     strokeTarget.create(
462         projectDocument.width,
463         projectDocument.height,
464         nextCaps.supportsHalfFloatColorBuffer,
465     )
466     strokeTarget.clear(0f, 0f, 0f, 0f)
467     ViewTransform.buildCanvasToFbo(
468         projectDocument.width.toFloat(),
469         projectDocument.height.toFloat(),
470         canvasToFboMatrix,
471     )
472     geometry = CanvasGeometry().also {
473         it.create(projectDocument.width, projectDocument.height)
474     }
475     compositor = Compositor().also { it.create() }
476     dabRenderer = DabRenderer(dabBuffer.capacity).also { it.create() }
477     capsuleRenderer = CapsuleStrokeRenderer().also {
478         it.create()
479     }
480     strokeBlitter = StrokeBlitter().also { it.create() }
481     // Build both the gallery preview and the individual layer previews
482     // once the complete GPU session is available.
483     captureThumbnail(layerStack.allLayers().map(PaintLayer::id))
484     onDocumentSessionLoaded?.invoke()
485     ColorSpaces.srgb8ToLinear(brushSettings.colorArgb, strokeColorLinear)
486     publishState()
487     GLCheck.noError("P6 surface creation")
488 }
489
490 override fun onSurfaceChanged(gl: GL10?, width: Int, height: Int) {
491     screenWidth = width.coerceAtLeast(1)
492     screenHeight = height.coerceAtLeast(1)
493     GLES30.glViewport(0, 0, screenWidth, screenHeight)
494     layerStack.camera.fitCanvas(layerStack.canvasWidth, layerStack.canvasHeight, screenWidth, screenHeight)
495     GLCheck.noError("P6 surface changed")
496 }
497
498 override fun onDrawFrame(gl: GL10?) {
499     if (cancelRequested.compareAndSet(true, false)) resetStroke()
500     if (undoGestureRequested.compareAndSet(true, false)) undo()
501     if (redoGestureRequested.compareAndSet(true, false)) redo()
502     if (resetCameraGestureRequested.compareAndSet(true, false)) resetCamera()
503     drainInput()
504     if (undoPipeline.process(undoManager)) publishState()
505     thumbnailScheduler?.processCompleted()
506     thumbnailScheduler?.buildIfNeeded(layerStack)
507
508     val frameStrokeBrush = activeStrokeBrush
509     if (strokeActive && frameStrokeBrush?.renderMode == BrushRenderMode.RIBBON) {
510         renderCapsulePreview(frameStrokeBrush)
511     } else if (strokeActive && frameStrokeBrush?.renderMode == BrushRenderMode.STAMP && dabBuffer.count > 0) {
512         // No longer clearing and drawing everything here, handled incrementally in drainInput
513     }
514
515     GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
516     GLES30.glViewport(0, 0, screenWidth, screenHeight)
517     GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
518     GLES30.glDisable(GLES30.GL_BLEND)
519     GLES30.glClearColor(canvasBackdropColor[0], canvasBackdropColor[1], canvasBackdropColor[2], 1f)
520     GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT)
521     drawBackdropGrid()
522
523     layerStack.camera.snapshot().buildCanvasToClip(
524         screenWidth.toFloat(),
525         screenHeight.toFloat(),
526         canvasToClipMatrix,
527     )
528     compositor?.render(
529         geometry = geometry ?: return,
530         layers = layerStack,
531         activeLayerId = layerStack.activeLayerId,
532         strokeTextureId = if (
533             strokeActive &&
534             (
535                 dabBuffer.count > 0 ||
536                 (
537                     frameStrokeBrush?.renderMode == BrushRenderMode.RIBBON &&
538                     capsuleEmitter.hasStroke
539                 )
540             )
541         ) {
542             strokeTarget.textureId
543         } else {
544             0
545         },
546         // Both render modes write their local flow into strokeTarget. The
547         // compositor therefore applies only the snapshot's global opacity.
548         strokeOpacity = frameStrokeBrush?.opacity?.coerceIn(0f, 1f) ?: 1f,
549         canvasToClip = canvasToClipMatrix,

```

```

550     )
551 }
552
553 private fun drawBackdropGrid() {
554     val cellPx = 28
555     GLES30.glEnable(GLES30.GL_SCISSOR_TEST)
556     GLES30.glClearColor(canvasGridColor[0], canvasGridColor[1], canvasGridColor[2], 1f)
557     var x = 0
558     while (x < screenWidth) {
559         GLES30.glScissor(x, 0, 1, screenHeight)
560         GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT)
561         x += cellPx
562     }
563     var y = 0
564     while (y < screenHeight) {
565         GLES30.glScissor(0, y, screenWidth, 1)
566         GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT)
567         y += cellPx
568     }
569     GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
570 }
571
572 private fun argbToRgb(Argb: Int, out: FloatArray) {
573     out[0] = ((Argb ushr 16) and 0xFF) / 255f
574     out[1] = ((Argb ushr 8) and 0xFF) / 255f
575     out[2] = (Argb and 0xFF) / 255f
576 }
577
578 /** Invalidates worker results created before a destructive document change. */
579 private fun invalidatePendingUndoHistory() {
580     checkOnGLThread()
581     undoPipeline.invalidate()
582 }
583
584 private fun shutdownUndoExecutor() {
585     undoPipeline.shutdown()
586 }
587
588 /** Safe to call from the UI thread; cancellation is executed on the next GL frame. */
589 fun cancelActiveStroke() {
590     cancelRequested.set(true)
591 }
592
593 /** Must run on the GL thread while the context is still current. */
594 fun releaseGLObjects() {
595     checkOnGLThread()
596     invalidatePendingUndoHistory()
597     discardPendingInput()
598     resetStroke()
599     undoManager.clear()
600     shutdownUndoExecutor()
601     dabRenderer?.release()
602     dabRenderer = null
603     grainTexture?.release()
604     grainTexture = null
605     grainPath = null
606     shapeTexture?.release()
607     shapeTexture = null
608     shapePath = null
609     capsuleRenderer?.release()
610     capsuleRenderer = null
611     strokeBlitter?.release()
612     strokeBlitter = null
613     compositor?.release()
614     compositor = null
615     thumbnailScheduler?.shutdown()
616     thumbnailCapture.release()
617     documentSession = null
618     geometry?.release()
619     geometry = null
620     strokeTarget.release()
621     layerStack.release()
622     caps = null
623 }
624
625 private fun drainInput() {
626     while (true) {
627         val batch = inputQueue.poll() ?: return
628         try {
629             when (batch.action) {
630                 InputAction.DOWN -> {
631                     if (!batch.isEmpty()) {
632                         resetStroke()
633                         val strokeBrush = brushSettings.resolved()
634                         activeStrokeBrush = strokeBrush
635                         ColorSpaces.srgb8ToLinear(strokeBrush.colorArgb, activeStrokeColorLinear)
636                         activeStrokeColorLinear.copyInto(strokeColorLinear)
637                         strokeActive = true
638                     }
639                     if (strokeBrush.renderMode == BrushRenderMode.RIBBON) {
640                         capsuleEmitter.begin(
641                             batch = batch,

```

```

642             out = checkNotNull(capsuleRenderer),
643             strokeSettings = strokeBrush,
644         )
645     } else {
646         dabRenderer?.beginStroke()
647         stampEmitter.begin(batch, dabBuffer, strokeBrush)
648         strokeTarget.clear(0f, 0f, 0f, 0f)
649         stampPreviewInitialized = true
650
651         drawPendingStampPreview("DOWN")
652     }
653 }
654 }
655
656 InputAction.MOVE -> if (strokeActive) {
657     val strokeBrush = activeStrokeBrush
658     ?: error("Missing active stroke snapshot")
659     if (strokeBrush.renderMode == BrushRenderMode.RIBBON) {
660         capsuleEmitter.append(
661             batch = batch,
662             out = checkNotNull(capsuleRenderer),
663         )
664     } else {
665         stampEmitter.append(batch, dabBuffer)
666         drawPendingStampPreview("MOVE")
667     }
668 }
669
670 InputAction.UP -> if (strokeActive) {
671     val strokeBrush = activeStrokeBrush
672     ?: error("Missing active stroke snapshot")
673     if (strokeBrush.renderMode == BrushRenderMode.RIBBON) {
674         val renderer = checkNotNull(capsuleRenderer)
675
676         // UP должен сначала обработать historical samples,
677         // затем текущую точку, которую уже положил capturer.
678         capsuleEmitter.append(
679             batch = batch,
680             out = renderer,
681         )
682         capsuleEmitter.finish(
683             out = renderer,
684             cancel = false,
685         )
686         commitCapsuleStroke(strokeBrush)
687     } else {
688         stampEmitter.append(batch, dabBuffer)
689         stampEmitter.finish(dabBuffer, cancel = false)
690         commitStroke(strokeBrush)
691     }
692 }
693
694 resetStroke()
695 }
696
697 InputAction.CANCEL -> resetStroke()
698 }
699 } finally {
700     inputPool.release(batch)
701 }
702 }
703
704 private fun commitStroke(strokeBrush: BrushSettings) {
705     val layer = layerStack.activeLayer() ?: return
706     if (layer.isLocked || dabBuffer.count == 0) return
707     if (!computeDirtyPixelBounds(dirtyBounds)) return
708     val blitter = strokeBlitter ?: return
709     val strokeGeometry = geometry ?: return
710
711     // Complete the isolated local-coverage mask before it is composited.
712     // Global brush opacity is deliberately applied only by the blit.
713     drawPendingStampPreview("UP")
714
715     if (strokeCommitter.commit(
716         layer = layer,
717         geometry = strokeGeometry,
718         blitter = blitter,
719         dirtyBounds = dirtyBounds,
720         canvasWidth = layerStack.canvasWidth,
721         canvasHeight = layerStack.canvasHeight,
722         opacity = strokeBrush.opacity,
723     )) {
724         publishState()
725     }
726 }
727
728 /** Returns a clamped pixel region covering the exact emitted dabs and their AA fringe. */
729 private fun computeDirtyPixelBounds(out: IntArray): Boolean {
730     if (dabBuffer.count == 0) return false
731     strokeDirtyRect.clear()
732     dabBuffer.includeDirtyRect(
733         rect = strokeDirtyRect,

```

```

734         extraMargin = STAMP_DIRTY_MARGIN_PX,
735     )
736     strokeDirtyRect.clamp(layerStack.canvasWidth.toFloat(), layerStack.canvasHeight.toFloat())
737     strokeDirtyRect.toPixelBounds(out)
738     return out[2] > out[0] && out[3] > out[1]
739 }
740
741 private fun drawPendingStampPreview(phase: String) {
742     val renderer = dabRenderer ?: return
743     val strokeBrush = activeStrokeBrush
744     ?: error("Missing active stroke snapshot")
745     check(renderer.uploadedCount <= dabBuffer.count) {
746         "Stamp preview rewound: uploaded=${renderer.uploadedCount}, dabs=${dabBuffer.count}"
747     }
748     if (BuildConfig.DEBUG) {
749         Log.d(
750             BRUSH_DIAG_TAG,
751             "phase=$phase dabs=${dabBuffer.count} pending=${dabBuffer.count - renderer.uploadedCount}",
752         )
753     }
754     renderer.drawPendingInto(
755         target = strokeTarget,
756         width = layerStack.canvasWidth,
757         height = layerStack.canvasHeight,
758         canvasToFbo = canvasToFboMatrix,
759         dabs = dabBuffer,
760         colorLinear = activeStrokeColorLinear,
761         blendPolicy = strokeBrush.blendPolicy,
762         strokeOpacity = 1f,
763     )
764     check(renderer.uploadedCount == dabBuffer.count) {
765         "Stamp preview did not consume all dabs: uploaded=${renderer.uploadedCount}, dabs=${dabBuffer.count}"
766     }
767 }
768
769 private fun resetStroke() {
770     strokeActive = false
771     activeStrokeBrush = null
772     stampEmitter.reset()
773     dabBuffer.clear()
774     capsuleEmitter.reset()
775     capsuleRenderer?.clearStrokeData()
776     capsulePreviewInitialized = false
777     stampPreviewInitialized = false
778     dabRenderer?.clearStrokeData()
779     if (strokeTarget.framebufferId != 0) {
780         strokeTarget.clear(0f, 0f, 0f, 0f)
781     }
782 }
783
784 /** Releases pooled batches without interpreting them during shutdown/context recreation. */
785 private fun discardPendingInput() {
786     while (true) {
787         val batch = inputQueue.poll() ?: return
788         inputPool.release(batch)
789     }
790 }
791
792 /**
793  * Builds the persisted metadata from the GL-owned runtime state. UI state
794  * is never treated as the document source of truth.
795  */
796 fun buildDocumentSnapshot(): ProjectDocument {
797     checkOnGLThread()
798     return projectDocument.copy(
799         updatedAt = System.currentTimeMillis(),
800         activeLayerId = layerStack.activeLayerId,
801         layers = layerStack.allLayers().map(::layerDocumentFrom),
802     )
803 }
804
805 private fun updateProjectDocument(
806     transform: (ProjectDocument) -> ProjectDocument,
807 ) {
808     projectDocument = transform(projectDocument)
809     projectDocument = buildDocumentSnapshot()
810     documentSession?.markProjectDirty()
811     onProjectDocumentChange?.invoke(projectDocument)
812 }
813
814 fun acknowledgeSavedTiles(saved: Map<Long, Set<com.wetinknext.engine.undo.TileCoord>>) {
815     layerTileStore.acknowledge(saved)
816     documentSession?.markSaved(saved.keys)
817 }
818
819 private fun publishDirtyTiles() {
820     val dirty = layerTileStore.takeDirty()
821     if (dirty.isEmpty()) return
822     projectDocument = buildDocumentSnapshot()
823     documentSession?.markProjectDirty()
824     onDirtyLayerTiles?.invoke(projectDocument, layerTileStore.payloadsForDirty(dirty), dirty)
825 }

```

```

826
827 /** Re-captures restored GPU tiles so Undo/Redo is persisted like a stroke. */
828 private fun persistRestoredTiles(
829     layer: PaintLayer,
830     tiles: List<com.wetinknext.engine.undo.TileSnapshot>,
831 ) {
832     if (tiles.isEmpty()) return
833     val left = tiles.minOf { it.pixelLeft }
834     val top = tiles.minOf { it.pixelTop }
835     val right = tiles.maxOf { it.pixelLeft + it.pixelWidth }
836     val bottom = tiles.maxOf { it.pixelTop + it.pixelHeight }
837     val raw = com.wetinknext.engine.undo.TileSnapshotCapture.capture(
838         target = layer.target,
839         bounds = intArrayOf(left, top, right, bottom),
840     )
841     layerTileStore.markDirty(layer.id, raw)
842     publishDirtyTiles()
843     markLayerThumbnailDirty(layer.id)
844 }
845
846 private fun captureThumbnail(dirtyLayerIds: Collection<Long> = emptyList()) {
847     if (caps == null || geometry == null || compositor == null) return
848     val scheduler = thumbnailScheduler
849     if (scheduler != null) {
850         scheduler.markDirty(projectDirty = true, dirtyLayerIds = dirtyLayerIds)
851         onInputRenderRequested?.invoke()
852     } else {
853         onThumbnailCaptured?.invoke(thumbnailCapture.capture(layerStack))
854     }
855 }
856
857 /** Called only after a successful GPU edit of [layerId]. */
858 private fun markLayerThumbnailDirty(layerId: Long) {
859     val scheduler = thumbnailScheduler
860     if (scheduler != null) {
861         scheduler.markLayerDirty(layerId)
862         scheduler.markProjectDirty()
863         onInputRenderRequested?.invoke()
864     } else {
865         captureThumbnail()
866     }
867 }
868
869 /**
870  * Records published preview files on the GL thread. Scheduler generations
871  * ensure every listed layer still has the pixels used to create its image.
872  */
873 fun applyThumbnailBuild(result: ThumbnailBuildResult) {
874     checkOnGLThread()
875     result.layerPreviews.forEach { (layerId, file) ->
876         layerStack.findLayerById(layerId)?.let { layer ->
877             layerThumbnailPaths[layerId] = file.absolutePath
878             layerThumbnailVersions[layerId] = layer.version
879         }
880     }
881     layerThumbnailPaths.keys.retainAll(layerStack.allLayers().map(PaintLayer::id).toSet())
882     layerThumbnailVersions.keys.retainAll(layerStack.allLayers().map(PaintLayer::id).toSet())
883     // layerDocumentFrom copies the current PaintLayer.version into
884     // thumbnailVersion, and updateProjectDocument atomically publishes the
885     // resulting document snapshot to the autosave bridge.
886     updateProjectDocument { it }
887     publishState()
888 }
889
890 /**
891  * Makes WebP previews restored from the persisted project immediately
892  * available to Compose. An older persisted preview may never replace a
893  * newer runtime image after a stroke has incremented the layer version.
894  */
895 fun installRestoredLayerThumbnails(previews: Map<Long, File>) {
896     checkOnGLThread()
897     var changed = false
898     previews.forEach { (layerId, file) ->
899         val layer = layerStack.findLayerById(layerId) ?: return@forEach
900         val storedVersion = projectDocument.layers
901             .firstOrNull { it.id == layerId }
902             ?.thumbnailVersion
903         ?: return@forEach
904         if (file.isFile && layer.version == storedVersion && layerThumbnailVersions[layerId] == null) {
905             layerThumbnailPaths[layerId] = file.absolutePath
906             layerThumbnailVersions[layerId] = storedVersion
907             changed = true
908         }
909     }
910     if (changed) publishState()
911 }
912
913 private fun syncLayerDocument(layer: PaintLayer) {
914     updateProjectDocument { document ->
915         document.copy(
916             layers = document.layers.map { stored ->
917                 if (stored.id == layer.id) layerDocumentFrom(layer, stored.pixelFile) else stored

```

```

918         },
919         activeLayerId = layerStack.activeLayerId,
920     )
921 }
922 }
923
924 private fun layerDocumentFrom(
925     layer: PaintLayer,
926     pixelFile: String = ProjectDocument.pixelFileFor(layer.id),
927 ): LayerDocument = LayerDocument(
928     id = layer.id,
929     name = layer.name,
930     visible = layer.isVisible,
931     locked = layer.isLocked,
932     opacity = layer.opacity,
933     blendMode = layer.blendMode,
934     pixelFile = pixelFile,
935     thumbnailVersion = layerThumbnailVersions[layer.id]
936         ?: projectDocument.layers.firstOrNull { it.id == layer.id }?.thumbnailVersion
937         ?: 0L,
938 )
939
940 private fun publishState() {
941     val listener = onStateChange ?: return
942     val layers = layerStack.allLayers()
943     val activeLayerId = layerStack.activeLayerId
944     val canDeleteLayers = layers.size > 1
945     listener(
946         EditorUiState(
947             layers = layers.map { layer ->
948                 LayerUiModel(
949                     id = layer.id,
950                     name = layer.name,
951                     isVisible = layer.isVisible,
952                     isLocked = layer.isLocked,
953                     opacity = layer.opacity,
954                     active = layer.id == activeLayerId,
955                     canDelete = !layer.isLocked && canDeleteLayers,
956                     thumbnailPath = layerThumbnailPaths[layer.id],
957                     thumbnailVersion = layerThumbnailVersions[layer.id]
958                         ?: projectDocument.layers.firstOrNull { it.id == layer.id }?.thumbnailVersion
959                         ?: 0L,
960                 )
961             },
962             canUndo = undoManager.canUndo && undoPipeline.pendingCount == 0,
963             canRedo = undoManager.canRedo && undoPipeline.pendingCount == 0,
964             brushSizePx = brushSettings.baseRadiusPx * 2f,
965             brushOpacity = brushSettings.opacity,
966             activeLayerId = activeLayerId,
967             ready = layerStack.canvasWidth > 0 && layerStack.canvasHeight > 0,
968             undoDiagnostics = UndoDiagnostics(
969                 pendingJobs = undoPipeline.pendingCount,
970                 staleResults = undoPipeline.staleResultCount,
971                 compressionFailures = undoPipeline.compressionFailureCount,
972                 restoreFailures = undoRestoreFailures,
973                 memoryBytes = undoManager.memoryBytes,
974             ),
975         ),
976     )
977 }
978
979 /**
980  * Вызывается только через GLSurfaceView.queueEvent,
981  * то есть на GL-потоке с current EGL context.
982  */
983 fun applyLoadedGrain(
984     loaded: LoadedBrushTexture,
985     scale: Float,
986     canvasLocked: Boolean,
987     depth: Float,
988     contrast: Float,
989 ) {
990     val oldTexture = grainTexture
991     val newTexture = BrushTexture()
992
993     try {
994         newTexture.createFromRgba(
995             width = loaded.width,
996             height = loaded.height,
997             rgba = loaded.rgba,
998         )
999
1000     capsuleRenderer?.setGrainTexture(
1001         textureId = newTexture.textureId,
1002         scale = scale,
1003         canvasLocked = canvasLocked,
1004         depth = depth,
1005         contrast = contrast,
1006     )
1007
1008     dabRenderer?.setGrainTexture(
1009         textureId = newTexture.textureId,

```

```

1010         scale = scale,
1011         canvasLocked = canvasLocked,
1012         depth = depth,
1013         contrast = contrast,
1014     )
1015
1016     grainTexture = newTexture
1017     grainPath = loaded.path
1018
1019     oldTexture?.release()
1020 } catch (error: Throwable) {
1021     newTexture.release()
1022     throw error
1023 }
1024 }
1025
1026 fun clearGrainTexture() {
1027     capsuleRenderer?.clearGrainTexture()
1028     dabRenderer?.clearGrainTexture()
1029
1030     grainTexture?.release()
1031     grainTexture = null
1032     grainPath = null
1033 }
1034
1035 /** Must run on the GL thread. Shape masks affect STAMP dabs only. */
1036 fun applyLoadedShape(
1037     loaded: LoadedBrushTexture,
1038     reverse: Boolean,
1039     rgbToAlpha: Boolean,
1040 ) {
1041     val oldTexture = shapeTexture
1042     val newTexture = BrushTexture()
1043
1044     try {
1045         newTexture.createFromRgba(loaded.width, loaded.height, loaded.rgba)
1046         dabRenderer?.setShapeTexture(newTexture.textureId, reverse, rgbToAlpha)
1047         shapeTexture = newTexture
1048         shapePath = loaded.path
1049         oldTexture?.release()
1050     } catch (error: Throwable) {
1051         newTexture.release()
1052         throw error
1053     }
1054 }
1055
1056 fun clearShapeTexture() {
1057     dabRenderer?.clearShapeTexture()
1058     shapeTexture?.release()
1059     shapeTexture = null
1060     shapePath = null
1061 }
1062
1063 private fun nextLayerName(): String = "Слой ${layerStack.count + 1}"
1064
1065 private fun renderCapsulePreview(strokeBrush: BrushSettings) {
1066     val renderer = capsuleRenderer ?: return
1067
1068     if (!capsulePreviewInitialized) {
1069         strokeTarget.clear(
1070             red = 0f,
1071             green = 0f,
1072             blue = 0f,
1073             alpha = 0f,
1074         )
1075         capsulePreviewInitialized = true
1076     }
1077
1078     renderer.drawPending(
1079         target = strokeTarget,
1080         width = layerStack.canvasWidth,
1081         height = layerStack.canvasHeight,
1082         canvasToClip = canvasToFboMatrix,
1083         colorLinear = activeStrokeColorLinear,
1084         blendPolicy = strokeBrush.blendPolicy,
1085         flow = strokeBrush.flow,
1086     )
1087 }
1088
1089 private fun commitCapsuleStroke(strokeBrush: BrushSettings) {
1090     val layer = layerStack.activeLayer() ?: return
1091     val renderer = capsuleRenderer ?: return
1092
1093     if (layer.isLocked) return
1094     if (!capsuleEmitter.hasStroke) return
1095     if (!computeCapsuleDirtyBounds(dirtyBounds)) return
1096     val blitter = strokeBlitter ?: return
1097     val strokeGeometry = geometry ?: return
1098
1099     /*
1100     * Сначала строим stroke в отдельном offscreen target.
1101     * Старый слой и новый renderer не используют один FBO для рисования.

```

```

1102     */
1103     strokeTarget.clear(
1104         red = 0f,
1105         green = 0f,
1106         blue = 0f,
1107         alpha = 0f,
1108     )
1109
1110     renderer.drawAll(
1111         target = strokeTarget,
1112         width = layerStack.canvasWidth,
1113         height = layerStack.canvasHeight,
1114         canvasToClip = canvasToFboMatrix,
1115         colorLinear = activeStrokeColorLinear,
1116         blendPolicy = strokeBrush.blendPolicy,
1117         flow = strokeBrush.flow,
1118     )
1119
1120     if (strokeCommitter.commit(
1121         layer = layer,
1122         geometry = strokeGeometry,
1123         blitter = blitter,
1124         dirtyBounds = dirtyBounds,
1125         canvasWidth = layerStack.canvasWidth,
1126         canvasHeight = layerStack.canvasHeight,
1127         opacity = strokeBrush.opacity,
1128     )) {
1129         publishState()
1130     }
1131 }
1132
1133 private fun computeCapsuleDirtyBounds(
1134     out: IntArray,
1135 ): Boolean {
1136     if (!capsuleEmitter.hasBounds) return false
1137
1138     strokeDirtyRect.clear()
1139     strokeDirtyRect.include(
1140         left = capsuleEmitter.minX,
1141         top = capsuleEmitter.minY,
1142         right = capsuleEmitter.maxX,
1143         bottom = capsuleEmitter.maxY,
1144     )
1145     strokeDirtyRect.expand(AA_MARGIN_PX)
1146     strokeDirtyRect.clamp(
1147         layerStack.canvasWidth.toFloat(),
1148         layerStack.canvasHeight.toFloat(),
1149     )
1150     strokeDirtyRect.toPixelBounds(out)
1151
1152     return out[2] > out[0] && out[3] > out[1]
1153 }
1154
1155 companion object {
1156     private const val BRUSH_DIAG_TAG = "BrushDiag"
1157     private const val THUMBNAIL_TAG = "ThumbnailBuild"
1158     private const val AA_MARGIN_PX = 4f
1159     private const val STAMP_DIRTY_MARGIN_PX = 4f
1160 }
1161 }

```

45. app/src/main/java/com/wetinknext/engine/core/LatestValueDelivery.kt

```
1 package com.wetinknext.engine.core
2
3 import java.util.concurrent.atomic.AtomicReference
4
5 /**
6  * Retains the newest cross-thread value until a consumer is ready.
7  *
8  * GLSurfaceView can create EGL before Compose has installed its listener. A
9  * thumbnail is a replaceable snapshot, so retaining only the newest frame is
10  * both sufficient and bounded.
11  */
12 internal class LatestValueDelivery<T> {
13     private val pending = AtomicReference<T?>(null)
14
15     fun offer(value: T) {
16         pending.set(value)
17     }
18
19     fun dispatchTo(listener: ((T) -> Unit)?) {
20         val callback = listener ?: return
21         pending.getAndSet(null)?.let(callback)
22     }
23
24     fun clear() {
25         pending.set(null)
26     }
27 }
```

46. app/src/main/java/com/wetinknext/engine/core/LayerTileCodec.kt

```

1 package com.wetinknext.engine.core
2
3 import android.opengl.GLES30
4 import com.wetinknext.engine.gl.RenderTarget
5 import com.wetinknext.engine.persistence.PersistentLayerTiles
6 import java.nio.ByteBuffer
7 import java.nio.ByteOrder
8
9 /**
10  * Stable payload for a complete layer texture. Empty data means a newly-created
11  * layer; later persistence can emit this format without changing the loader.
12  */
13 object LayerTileCodec {
14     private const val MAGIC = 0x5754494C // WTLT
15     private const val VERSION = 1
16     private const val HEADER_BYTES = 20
17
18     data class Decoded(val width: Int, val height: Int, val bytesPerPixel: Int, val pixels: ByteArray)
19
20     fun encode(width: Int, height: Int, bytesPerPixel: Int, pixels: ByteArray): ByteArray {
21         require(width > 0 && height > 0 && bytesPerPixel in setOf(4, 8)) { "Invalid layer tile header" }
22         val expected = width.toLong() * height * bytesPerPixel
23         require(expected <= Int.MAX_VALUE && pixels.size == expected.toInt()) { "Invalid layer tile size" }
24         return ByteBuffer.allocate(HEADER_BYTES + pixels.size)
25             .order(ByteOrder.BIG_ENDIAN)
26             .putInt(MAGIC)
27             .putInt(VERSION)
28             .putInt(width)
29             .putInt(height)
30             .putInt(bytesPerPixel)
31             .put(pixels)
32             .array()
33     }
34
35     fun decode(bytes: ByteArray): Decoded? {
36         if (bytes.isEmpty()) return null
37         require(bytes.size >= HEADER_BYTES) { "Layer tile payload is truncated" }
38         val buffer = ByteBuffer.wrap(bytes).order(ByteOrder.BIG_ENDIAN)
39         require(buffer.int == MAGIC) { "Unknown layer tile payload" }
40         require(buffer.int == VERSION) { "Unsupported layer tile version" }
41         val width = buffer.int
42         val height = buffer.int
43         val bytesPerPixel = buffer.int
44         require(width > 0 && height > 0 && bytesPerPixel in setOf(4, 8)) { "Invalid layer tile header" }
45         val expected = width.toLong() * height * bytesPerPixel
46         require(expected <= Int.MAX_VALUE && bytes.size - HEADER_BYTES == expected.toInt()) { "Invalid layer tile size" }
47         return Decoded(width, height, bytesPerPixel, bytes.copyOfRange(HEADER_BYTES, bytes.size))
48     }
49
50     fun upload(target: RenderTarget, bytes: ByteArray) {
51         val decoded = decode(bytes) ?: return
52         require(decoded.width == target.width && decoded.height == target.height) { "Layer tile dimensions do not match
document" }
53         val expectedBpp = if (target.usesHalfFloat) 8 else 4
54         require(decoded.bytesPerPixel == expectedBpp) { "Layer tile pixel format does not match GPU target" }
55
56         val data = ByteBuffer.allocateDirect(decoded.pixels.size).order(ByteOrder.nativeOrder())
57         data.put(decoded.pixels).flip()
58         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
59         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, target.textureId)
60         GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 1)
61         try {
62             GLES30.glTexSubImage2D(
63                 GLES30.GL_TEXTURE_2D, 0, 0, 0, decoded.width, decoded.height,
64                 GLES30.GL_RGBA,
65                 if (decoded.bytesPerPixel == 8) GLES30.GL_HALF_FLOAT else GLES30.GL_UNSIGNED_BYTE,
66                 data,
67             )
68             check(GLES30.glGetError() == GLES30.GL_NO_ERROR) { "Layer texture upload failed" }
69         } finally {
70             GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 4)
71             GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
72         }
73     }
74
75     fun uploadPersistent(target: RenderTarget, payload: ByteArray) {
76         if (payload.isEmpty()) return
77         val expectedBpp = if (target.usesHalfFloat) 8 else 4
78         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
79         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, target.textureId)
80         GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 1)
81         try {
82             PersistentLayerTiles.decode(payload).forEach { tile ->
83                 require(tile.bytesPerPixel == expectedBpp && tile.pixelsLeft + tile.pixelsWidth <= target.width && tile.
pixelTop + tile.pixelsHeight <= target.height) { "Invalid persistent tile" }
84                 val data = ByteBuffer.allocateDirect(tile.bytes.size).order(ByteOrder.nativeOrder())
85                 data.put(tile.bytes).flip()
86                 GLES30.glTexSubImage2D(GLES30.GL_TEXTURE_2D, 0, tile.pixelsLeft, tile.pixelsTop, tile.pixelsWidth, tile.
pixelHeight, GLES30.GL_RGBA, if (expectedBpp == 8) GLES30.GL_HALF_FLOAT else GLES30.GL_UNSIGNED_BYTE, data)
87             }
88         }
89     }
90 }

```

```
88         check(GLES30.glGetError() == GLES30.GL_NO_ERROR) { "Persistent tile upload failed" }
89     } finally {
90         GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 4)
91         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
92     }
93 }
94 }
```

47. app/src/main/java/com/wetinknext/engine/core/PaintEngine.kt

```
1 package com.wetinknext.engine.core
2
3 import com.wetinknext.engine.canvas.LayerStack
4 import com.wetinknext.engine.gl.GlCaps
5 import com.wetinknext.engine.gl.RenderTarget
6
7 /** Lightweight document facade retained for engine callers outside the renderer. */
8 class PaintEngine {
9     val layerStack = LayerStack()
10    val camera: Camera get() = layerStack.camera
11    val canvasTarget: RenderTarget get() = checkNotNull(layerStack.activeLayer()).target
12
13    val canvasWidth: Int get() = layerStack.canvasWidth
14    val canvasHeight: Int get() = layerStack.canvasHeight
15    var initialized = false
16        private set
17
18    fun create(caps: GlCaps, width: Int, height: Int) {
19        layerStack.create(caps, width, height)
20        initialized = true
21    }
22
23    fun addLayer(name: String): Long = layerStack.addLayer(name).id
24
25    fun removeLayer(id: Long): Boolean = layerStack.removeLayer(id) != null
26
27    fun setActiveLayer(id: Long): Boolean = layerStack.setActive(id)
28
29    fun release() {
30        layerStack.release()
31        initialized = false
32    }
33 }
```

48. app/src/main/java/com/wetinknext/engine/core/PaintSurfaceView.kt

```

1  package com.wetinknext.engine.core
2
3  import android.content.Context
4  import android.opengl.GLSurfaceView
5  import android.util.AttributeSet
6  import android.view.MotionEvent
7  import com.wetinknext.domain.document.ProjectDocument
8  import com.wetinknext.engine.brush.BrushSettings
9  import com.wetinknext.engine.brush.TextureLoader
10 import com.wetinknext.engine.brush.LoadedBrushTexture
11 import com.wetinknext.engine.thumbnail.ThumbnailBuildResult
12 import com.wetinknext.engine.thumbnail.ThumbnailPreviewFiles
13 import java.io.File
14 import java.util.concurrent.ExecutorService
15 import java.util.concurrent.Executors
16
17 /** Thread boundary between Compose commands and the GL-owned paint engine. */
18 class PaintSurfaceView @JvmOverloads constructor(
19     context: Context,
20     attrs: AttributeSet? = null,
21     document: ProjectDocument = ProjectDocument.newUntitled(),
22     layerTiles: Map<Long, ByteArray> = emptyMap(),
23     layerPreviews: Map<Long, ByteArray> = emptyMap(),
24 ) : GLSurfaceView(context, attrs) {
25     private val thumbnailOutputDirectory = File(context.cacheDir, "wetink-thumbnails/${document.id}")
26     private val engineRenderer = EngineRenderer(
27         projectDocument = document,
28         initialLayerTiles = layerTiles,
29         thumbnailOutputDirectory = thumbnailOutputDirectory,
30     )
31
32     private val textureLoader = TextureLoader(context)
33     private val thumbnailRestoreExecutor: ExecutorService = Executors.newSingleThreadExecutor()
34     private var grainGeneration = 0L
35     private var shapeGeneration = 0L
36
37     var onTextureError: ((String, Throwable) -> Unit)? = null
38
39     var onEditorStateChange: ((EditorUiState) -> Unit)? = null
40     var onProjectDocumentChange: ((ProjectDocument) -> Unit)? = null
41     private var documentSessionLoaded = false
42     var onDocumentSessionLoaded: (() -> Unit)? = null
43     set(value) {
44         field = value
45         if (documentSessionLoaded && value != null) post(value)
46     }
47     var onDirtyLayerTiles: ((ProjectDocument, Map<Long, ByteArray>, Map<Long, Set<com.wetinknext.engine.undo.TileCoord>)) ->
Unit)? = null
48     private val pendingThumbnailCapture = LatestValueDelivery<ThumbnailCapture.Rgba>()
49     var onThumbnailCaptured: ((ThumbnailCapture.Rgba) -> Unit)? = null
50     set(value) {
51         field = value
52         if (value != null) {
53             // setRenderer() may initialize EGL before AndroidView's caller
54             // has installed its listener. Replay the latest capture then.
55             post(::dispatchPendingThumbnail)
56         }
57     }
58     /** File previews that were encoded on the worker and published to cache. */
59     var onThumbnailBuildSaved: ((ThumbnailBuildResult) -> Unit)? = null
60
61     init {
62         setEGLContextClientVersion(3)
63         setRenderer(engineRenderer)
64         renderMode = RENDERMODE_WHEN_DIRTY
65         engineRenderer.onStateChange = { state ->
66             post { onEditorStateChange?.invoke(state) }
67             requestRender()
68         }
69         engineRenderer.onProjectDocumentChange = { document ->
70             post { onProjectDocumentChange?.invoke(document) }
71         }
72         engineRenderer.onDocumentSessionLoaded = {
73             documentSessionLoaded = true
74             post { onDocumentSessionLoaded?.invoke() }
75         }
76         engineRenderer.onDirtyLayerTiles = { document, payloads, dirty -> post { onDirtyLayerTiles?.invoke(document, payloads,
dirty) } }
77         engineRenderer.onThumbnailCaptured = { image ->
78             pendingThumbnailCapture.offer(image)
79             post(::dispatchPendingThumbnail)
80         }
81         engineRenderer.onThumbnailBuildSaved = { result ->
82             // The worker has published files, but document metadata and layer
83             // UI state remain GL-owned. Update those first, then notify the
84             // app-level persistence bridge on the main thread.
85             post {
86                 queueEvent {
87                     engineRenderer.applyThumbnailBuild(result)
88                     requestRender()

```

```

89         post { onThumbnailBuildSaved?.invoke(result) }
90     }
91 }
92 }
93 engineRenderer.onInputRenderRequested = { requestRender() }
94 restoreStoredLayerPreviews(layerPreviews)
95 }
96
97 override fun onTouchEvent(event: MotionEvent): Boolean {
98     val handled = engineRenderer.onTouchEvent(event)
99     if (handled) requestRender()
100     return handled || super.onTouchEvent(event)
101 }
102
103 fun requestState() = queueEvent {
104     engineRenderer.requestState()
105     requestRender()
106 }
107
108 fun undo() = queueEvent {
109     engineRenderer.undo()
110     requestRender()
111 }
112
113 fun redo() = queueEvent {
114     engineRenderer.redo()
115     requestRender()
116 }
117
118 fun acknowledgeSavedTiles(saved: Map<Long, Set<com.wetinknext.engine.undo.TileCoord>>) = queueEvent {
119     engineRenderer.acknowledgeSavedTiles(saved)
120 }
121
122 fun setActiveLayer(id: Long) = queueEvent {
123     engineRenderer.setActiveLayer(id)
124     requestRender()
125 }
126
127 fun addLayer() = queueEvent {
128     engineRenderer.addLayer()
129     requestRender()
130 }
131
132 fun removeLayer(id: Long) = queueEvent {
133     engineRenderer.removeLayer(id)
134     requestRender()
135 }
136
137 fun moveLayer(id: Long, delta: Int) = queueEvent {
138     engineRenderer.moveLayer(id, delta)
139     requestRender()
140 }
141
142 fun setLayerVisible(id: Long, visible: Boolean) =
143     queueEvent {
144         engineRenderer.setLayerVisible(id, visible)
145         requestRender()
146     }
147
148 fun setLayerOpacity(id: Long, opacity: Float) =
149     queueEvent {
150         engineRenderer.setLayerOpacity(id, opacity)
151         requestRender()
152     }
153
154 fun clearActiveLayer() = queueEvent {
155     engineRenderer.clearActiveLayer()
156     requestRender()
157 }
158
159 fun toggleCanvasMirror() = queueEvent {
160     engineRenderer.toggleCanvasMirror()
161     requestRender()
162 }
163
164 fun setCanvasBackdrop(backdropArgb: Int, gridArgb: Int) = queueEvent {
165     engineRenderer.setCanvasBackdrop(backdropArgb, gridArgb)
166     requestRender()
167 }
168
169 fun setBrushSize(px: Float) = queueEvent {
170     engineRenderer.setBrushSize(px)
171     requestRender()
172 }
173
174 fun setBrushOpacity(opacity: Float) = queueEvent {
175     engineRenderer.setBrushOpacity(opacity)
176     requestRender()
177 }
178
179 fun applyBrush(settings: BrushSettings) = queueEvent {
180     engineRenderer.applyBrush(settings)

```

```

181         requestRender()
182     }
183
184     fun setBrushSettings(settings: BrushSettings) = queueEvent {
185         engineRenderer.setBrushSettings(settings)
186         requestRender()
187     }
188
189     fun setBrushColor(color: androidx.compose.ui.graphics.Color) = queueEvent {
190         engineRenderer.setBrushColor(color)
191         requestRender()
192     }
193
194     fun loadGrainTexture(
195         path: String,
196         scale: Float = 1f,
197         canvasLocked: Boolean = true,
198         depth: Float = 1f,
199         contrast: Float = 1f,
200     ) {
201         val generation = ++grainGeneration
202
203         textureLoader.loadAsync(
204             path = path,
205             onLoad = { loaded ->
206                 queueEvent {
207                     if (generation != grainGeneration) {
208                         return@queueEvent
209                     }
210
211                     engineRenderer.applyLoadedGrain(
212                         loaded = loaded,
213                         scale = scale,
214                         canvasLocked = canvasLocked,
215                         depth = depth,
216                         contrast = contrast,
217                     )
218                     requestRender()
219                 }
220             },
221             onError = { error ->
222                 post {
223                     onTextureError?.invoke(path, error)
224                 }
225             },
226         )
227     }
228
229     fun clearGrainTexture() {
230         grainGeneration++
231
232         queueEvent {
233             engineRenderer.clearGrainTexture()
234             requestRender()
235         }
236     }
237
238     fun loadShapeTexture(
239         path: String,
240         reverse: Boolean = false,
241         rgbToAlpha: Boolean = false,
242     ) {
243         val generation = ++shapeGeneration
244         textureLoader.loadAsync(
245             path = path,
246             onLoad = { loaded ->
247                 queueEvent {
248                     if (generation != shapeGeneration) return@queueEvent
249                     engineRenderer.applyLoadedShape(loaded, reverse, rgbToAlpha)
250                     requestRender()
251                 }
252             },
253             onError = { error -> post { onTextureError?.invoke(path, error) } },
254         )
255     }
256
257     fun clearShapeTexture() {
258         shapeGeneration++
259         queueEvent {
260             engineRenderer.clearShapeTexture()
261             requestRender()
262         }
263     }
264
265     private fun dispatchPendingThumbnail() {
266         pendingThumbnailCapture.dispatchTo(onThumbnailCaptured)
267     }
268
269     /** Restores persisted WebP bytes off the main and GL threads. */
270     private fun restoreStoredLayerPreviews(layerPreviews: Map<Long, ByteArray>) {
271         if (layerPreviews.isEmpty()) return
272         thumbnailRestoreExecutor.execute {

```

```

273         val files = runCatching {
274             ThumbnailPreviewFiles.restoreLayerPreviews(
275                 outputDirectory = thumbnailOutputDirectory,
276                 previews = layerPreviews,
277             )
278         }.getOrElse { error ->
279             post { onTextureError?.invoke("layer previews", error) }
280             return@execute
281         }
282         queueEvent {
283             engineRendererer.installRestoredLayerThumbnails(files)
284             requestRender()
285         }
286     }
287 }
288
289 override fun onPause() {
290     engineRendererer.cancelActiveStroke()
291     super.onPause()
292 }
293
294 override fun onDetachedFromWindow() {
295     grainGeneration++
296     shapeGeneration++
297     textureLoader.shutdown()
298     thumbnailRestoreExecutor.shutdownNow()
299
300     // Очищаем listeners, чтобы избежать утечек или вызовов в пустоту
301     onTextureError = null
302     onEditorStateChange = null
303     onProjectDocumentChange = null
304     onDocumentSessionLoaded = null
305     onDirtyLayerTiles = null
306     onThumbnailCaptured = null
307     onThumbnailBuildSaved = null
308     pendingThumbnailCapture.clear()
309     engineRendererer.onStateChange = null
310     engineRendererer.onProjectDocumentChange = null
311     engineRendererer.onDocumentSessionLoaded = null
312     engineRendererer.onDirtyLayerTiles = null
313     engineRendererer.onThumbnailCaptured = null
314     engineRendererer.onThumbnailBuildSaved = null
315     engineRendererer.onInputRenderRequested = null
316     engineRendererer.setOnSecondaryPointerDown { }
317
318     // Пытаемся освободить GL-объекты, но не рассчитываем на 100% успех здесь
319     queueEvent {
320         // Отменяем активный штрих перед освобождением
321         engineRendererer.cancelActiveStroke()
322         engineRendererer.releaseGLObjets()
323     }
324     super.onDetachedFromWindow()
325 }
326 }

```

49. app/src/main/java/com/wetinknext/engine/core/StrokeCommitter.kt

```

1  package com.wetinknext.engine.core
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.canvas.PaintLayer
5  import com.wetinknext.engine.canvas.StrokeBlitter
6  import com.wetinknext.engine.gl.CanvasGeometry
7  import com.wetinknext.engine.gl.GlCheck
8  import com.wetinknext.engine.gl.RenderTarget
9  import com.wetinknext.engine.undo.TileSnapshot
10 import com.wetinknext.engine.undo.TileSnapshotCapture
11 import com.wetinknext.engine.undo.UndoManager
12 import com.wetinknext.engine.undo.UndoOperationType
13 import com.wetinknext.engine.undo.RawTileSnapshot
14
15 /**
16  * GL-thread transaction boundary for a prepared stroke.
17  *
18  * The caller renders the active stroke into [strokeTarget]. This class then
19  * snapshots the affected layer tiles, performs the single premultiplied blit,
20  * captures the after-state, and sends the reversible transaction to history.
21  */
22 class StrokeCommitter(
23     private val strokeTarget: RenderTarget,
24     private val canvasToFbo: FloatArray,
25     private val undoPipeline: UndoCompressionPipeline,
26     private val undoManager: UndoManager,
27     private val onTilesCommitted: (PaintLayer, List<RawTileSnapshot>) -> Unit = { _, _ -> },
28 ) {
29     /**
30      * Commits the already-rendered stroke target into [layer]. [dirtyBounds]
31      * is in canvas coordinates and is expanded to a deterministic tile set.
32      */
33     fun commit(
34         layer: PaintLayer,
35         geometry: CanvasGeometry,
36         blitter: StrokeBlitter,
37         dirtyBounds: IntArray,
38         canvasWidth: Int,
39         canvasHeight: Int,
40         opacity: Float,
41         operation: UndoOperationType = UndoOperationType.TILE_EDIT,
42         tag: String = "stroke",
43     ): Boolean {
44         GlCheck.checkOnGltThread()
45         if (!layer.created || layer.isLocked || strokeTarget.textureId == 0) return false
46         if (dirtyBounds.size < 4) return false
47
48         expandBoundsToTiles(dirtyBounds, canvasWidth, canvasHeight)
49         if (dirtyBounds[2] <= dirtyBounds[0] || dirtyBounds[3] <= dirtyBounds[1]) return false
50
51         val beforeRaw = TileSnapshotCapture.capture(layer.target, dirtyBounds)
52         blitter.blit(
53             layer = layer.target,
54             geometry = geometry,
55             strokeTextureId = strokeTarget.textureId,
56             canvasToFbo = canvasToFbo,
57             width = canvasWidth,
58             height = canvasHeight,
59             opacity = opacity.coerceIn(0f, 1f),
60         )
61         GLES30.glFlush()
62         layer.version++
63         val afterRaw = TileSnapshotCapture.capture(layer.target, dirtyBounds)
64         undoPipeline.enqueue(
65             undoManager = undoManager,
66             layerId = layer.id,
67             beforeRaw = beforeRaw,
68             afterRaw = afterRaw,
69             operation = operation,
70             tag = tag,
71         )
72         onTilesCommitted(layer, afterRaw)
73         return true
74     }
75
76     /** Clears one editable layer and records the full canvas as one undo transaction. */
77     fun clearLayer(
78         layer: PaintLayer,
79         canvasWidth: Int,
80         canvasHeight: Int,
81     ): Boolean {
82         GlCheck.checkOnGltThread()
83         if (!layer.created || layer.isLocked) return false
84
85         val fullBounds = intArrayOf(0, 0, canvasWidth, canvasHeight)
86         val beforeRaw = TileSnapshotCapture.capture(layer.target, fullBounds)
87         layer.clear()
88         GLES30.glFlush()
89         layer.version++
90         val afterRaw = TileSnapshotCapture.capture(layer.target, fullBounds)

```

```

91         undoPipeline.enqueue(
92             undoManager = undoManager,
93             layerId = layer.id,
94             beforeRaw = beforeRaw,
95             afterRaw = afterRaw,
96             operation = UndoOperationType.CLEAR_LAYER,
97             tag = "clear_layer",
98         )
99         onTilesCommitted(layer, afterRaw)
100     return true
101 }
102
103 private fun expandBoundsToTiles(
104     bounds: IntArray,
105     canvasWidth: Int,
106     canvasHeight: Int,
107 ) {
108     val tileSize = TileSnapshot.TILE_SIZE
109     bounds[0] = (bounds[0] / tileSize) * tileSize
110     bounds[1] = (bounds[1] / tileSize) * tileSize
111     bounds[2] = ((bounds[2] + tileSize - 1) / tileSize) * tileSize
112     bounds[3] = ((bounds[3] + tileSize - 1) / tileSize) * tileSize
113     bounds[0] = bounds[0].coerceIn(0, canvasWidth)
114     bounds[1] = bounds[1].coerceIn(0, canvasHeight)
115     bounds[2] = bounds[2].coerceIn(0, canvasWidth)
116     bounds[3] = bounds[3].coerceIn(0, canvasHeight)
117 }
118 }

```

50. app/src/main/java/com/wetinknext/engine/core/ThumbnailCapture.kt

```
1 package com.wetinknext.engine.core
2
3 import com.wetinknext.engine.canvas.LayerStack
4 import com.wetinknext.engine.gl.GlCheck
5 import com.wetinknext.engine.thumbnail.ThumbnailRenderer
6
7 /** Captures only the composited canvas; Compose panels and system UI are excluded. */
8 class ThumbnailCapture {
9     data class Rgba(val width: Int, val height: Int, val pixels: ByteArray)
10     private val renderer = ThumbnailRenderer()
11
12     fun capture(layers: LayerStack): Rgba {
13         GlCheck.checkOnGhread()
14         require(layers.canvasWidth > 0 && layers.canvasHeight > 0) { "Cannot capture an empty canvas" }
15         val longestEdge = 384
16         val scale = longestEdge.toFloat() / maxOf(layers.canvasWidth, layers.canvasHeight).toFloat()
17         val captureWidth = (layers.canvasWidth * scale).toInt().coerceAtLeast(1)
18         val captureHeight = (layers.canvasHeight * scale).toInt().coerceAtLeast(1)
19         return Rgba(
20             captureWidth,
21             captureHeight,
22             renderer.renderProject(layers, captureWidth, captureHeight),
23         )
24     }
25
26     fun release() = renderer.release()
27 }
```

51. app/src/main/java/com/wetinknext/engine/core/UndoCompressionPipeline.kt

```

1  package com.wetinknext.engine.core
2
3  import android.util.Log
4  import com.wetinknext.BuildConfig
5  import com.wetinknext.engine.undo.DeflateTileCompressor
6  import com.wetinknext.engine.undo.RawTileSnapshot
7  import com.wetinknext.engine.undo.TileCompressor
8  import com.wetinknext.engine.undo.UndoEntry
9  import com.wetinknext.engine.undo.UndoManager
10 import com.wetinknext.engine.undo.UndoOperationType
11 import java.util.HashMap
12 import java.util.concurrent.ConcurrentLinkedQueue
13 import java.util.concurrent.ExecutorService
14 import java.util.concurrent.Executors
15 import java.util.concurrent.RejectedExecutionException
16 import java.util.concurrent.TimeUnit
17
18 /**
19  * Orders tile-compression work without ever touching GL from a worker thread.
20  *
21  * Call [enqueue], [process], [invalidate], and [shutdown] on the GL thread.
22  * The worker only converts [RawTileSnapshot] objects into compressed [UndoEntry]
23  * instances and requests another frame once a result is ready.
24  */
25 class UndoCompressionPipeline(
26     private val requestRender: () -> Unit,
27     private val compressor: TileCompressor = DeflateTileCompressor(),
28     private val maxPendingJobs: Int = DEFAULT_MAX_PENDING_JOBS,
29     private val executorFactory: () -> ExecutorService = { Executors.newSingleThreadExecutor() },
30 ) {
31     private var executor = executorFactory()
32     private val pendingResults = ConcurrentLinkedQueue<UndoJobResult>()
33     private val completedResults = HashMap<Long, UndoJobResult>()
34
35     private var nextSequence = 0L
36     private var nextSequenceToApply = 0L
37     private var epoch = 0L
38
39     var pendingCount: Int = 0
40     private set
41     var compressionFailureCount: Int = 0
42     private set
43     var staleResultCount: Int = 0
44     private set
45
46     /**
47      * Schedules one ordered undo transaction. The synchronous fallback still
48      * travels through the same result queue, so it cannot overtake older work.
49      */
50     fun enqueue(
51         undoManager: UndoManager,
52         layerId: Long,
53         beforeRaw: List<RawTileSnapshot>,
54         afterRaw: List<RawTileSnapshot>,
55         operation: UndoOperationType = UndoOperationType.TILE_EDIT,
56         tag: String = "stroke",
57     ) {
58         val jobEpoch = epoch
59         val sequence = allocateSequence()
60         pendingCount++
61
62         if (pendingCount >= maxPendingJobs) {
63             compressSynchronously(
64                 epoch = jobEpoch,
65                 sequence = sequence,
66                 layerId = layerId,
67                 beforeRaw = beforeRaw,
68                 afterRaw = afterRaw,
69                 operation = operation,
70                 tag = tag,
71             )
72             process(undoManager)
73             return
74         }
75
76         submit(
77             epoch = jobEpoch,
78             sequence = sequence,
79             layerId = layerId,
80             beforeRaw = beforeRaw,
81             afterRaw = afterRaw,
82             operation = operation,
83             tag = tag,
84         )
85     }
86
87     /** Applies completed work in commit order. Must be called on the GL thread. */
88     fun process(undoManager: UndoManager): Boolean {
89         var changed = false
90

```

```

91         while (true) {
92             val result = pendingResults.poll() ?: break
93             if (result.sequence < nextSequenceToApply) {
94                 discard(result)
95                 decrementPending()
96                 changed = true
97             } else {
98                 completedResults.put(result.sequence, result)?.let(::disposeReady)
99             }
100         }
101
102         while (true) {
103             val result = completedResults.remove(nextSequenceToApply) ?: break
104             apply(result, undoManager)
105             decrementPending()
106             nextSequenceToApply++
107             changed = true
108         }
109
110         return changed
111     }
112
113     /** Rejects compression results that belong to a document state that no longer exists. */
114     fun invalidate() {
115         epoch++
116         nextSequenceToApply = nextSequence
117         val discardedCount = completedResults.size
118         completedResults.values.forEach(::disposeReady)
119         completedResults.clear()
120         pendingCount = (pendingCount - discardedCount).coerceAtLeast(0)
121         staleResultCount += discardedCount
122     }
123
124     /** Reopens the worker after an EGL/context recreation. */
125     fun ensureRunning() {
126         if (executor.isShutdown) executor = executorFactory()
127     }
128
129     /** Releases worker-owned results without changing the caller's [UndoManager]. */
130     fun shutdown() {
131         executor.shutdownNow()
132         try {
133             executor.awaitTermination(SHUTDOWN_TIMEOUT_MS, TimeUnit.MILLISECONDS)
134         } catch (error: InterruptedException) {
135             Thread.currentThread().interrupt()
136         }
137
138         while (true) {
139             val result = pendingResults.poll() ?: break
140             disposeReady(result)
141         }
142         completedResults.values.forEach(::disposeReady)
143         completedResults.clear()
144         pendingCount = 0
145         nextSequenceToApply = nextSequence
146     }
147
148     private fun allocateSequence(): Long = nextSequence++
149
150     private fun submit(
151         epoch: Long,
152         sequence: Long,
153         layerId: Long,
154         beforeRaw: List<RawTileSnapshot>,
155         afterRaw: List<RawTileSnapshot>,
156         operation: UndoOperationType,
157         tag: String,
158     ) {
159         try {
160             executor.execute {
161                 val result = runCatching {
162                     readyResult(
163                         epoch = epoch,
164                         sequence = sequence,
165                         layerId = layerId,
166                         beforeRaw = beforeRaw,
167                         afterRaw = afterRaw,
168                         operation = operation,
169                         tag = tag,
170                     )
171                 }.getOrElse { error ->
172                     UndoJobResult.Failed(epoch = epoch, sequence = sequence, error = error)
173                 }
174                 pendingResults.add(result)
175                 requestRender()
176             }
177         } catch (error: RejectedExecutionException) {
178             pendingResults.add(
179                 UndoJobResult.Failed(epoch = epoch, sequence = sequence, error = error),
180             )
181             requestRender()
182         }

```

```

183     }
184
185     private fun compressSynchronously(
186         epoch: Long,
187         sequence: Long,
188         layerId: Long,
189         beforeRaw: List<RawTileSnapshot>,
190         afterRaw: List<RawTileSnapshot>,
191         operation: UndoOperationType,
192         tag: String,
193     ) {
194         val result = runCatching {
195             readyResult(
196                 epoch = epoch,
197                 sequence = sequence,
198                 layerId = layerId,
199                 beforeRaw = beforeRaw,
200                 afterRaw = afterRaw,
201                 operation = operation,
202                 tag = tag,
203             )
204         }.getOrElse { error ->
205             UndoJobResult.Failed(epoch = epoch, sequence = sequence, error = error)
206         }
207         pendingResults.add(result)
208     }
209
210     private fun readyResult(
211         epoch: Long,
212         sequence: Long,
213         layerId: Long,
214         beforeRaw: List<RawTileSnapshot>,
215         afterRaw: List<RawTileSnapshot>,
216         operation: UndoOperationType,
217         tag: String,
218     ): UndoJobResult.Ready = UndoJobResult.Ready(
219         epoch = epoch,
220         sequence = sequence,
221         entry = UndoEntry(
222             layerId = layerId,
223             beforeTiles = beforeRaw.map { it.compress(compressor) },
224             afterTiles = afterRaw.map { it.compress(compressor) },
225             operation = operation,
226             tag = tag,
227         ),
228     )
229
230     private fun apply(result: UndoJobResult, undoManager: UndoManager) {
231         when (result) {
232             is UndoJobResult.Ready -> {
233                 if (isUndoResultCurrent(result.epoch, epoch)) {
234                     undoManager.push(result.entry)
235                 } else {
236                     result.entry.dispose()
237                     staleResultCount++
238                 }
239             }
240             is UndoJobResult.Failed -> {
241                 compressionFailureCount++
242                 if (BuildConfig.DEBUG) {
243                     Log.e(TAG, "Undo compression failed", result.error)
244                 }
245             }
246         }
247     }
248
249     private fun discard(result: UndoJobResult) {
250         disposeReady(result)
251         staleResultCount++
252     }
253
254     private fun disposeReady(result: UndoJobResult) {
255         if (result is UndoJobResult.Ready) result.entry.dispose()
256     }
257
258     private fun decrementPending() {
259         pendingCount = (pendingCount - 1).coerceAtLeast(0)
260     }
261
262     private companion object {
263         const val DEFAULT_MAX_PENDING_JOBS = 8
264         const val SHUTDOWN_TIMEOUT_MS = 100L
265         const val TAG = "TileUndo"
266     }
267 }
268

```

52. app/src/main/java/com/wetinknext/engine/core/UndoHistory.kt

```
1 package com.wetinknext.engine.core
2
3 /** Pure epoch contract used to reject undo results from a previous document state. */
4 fun isUndoResultCurrent(
5     resultEpoch: Long,
6     currentEpoch: Long,
7 ): Boolean = resultEpoch == currentEpoch
```

53. app/src/main/java/com/wetinknext/engine/core/UndoJobResult.kt

```
1 package com.wetinknext.engine.core
2
3 import com.wetinknext.engine.undo.UndoEntry
4
5 /** Terminal outcome of one tile-compression task. */
6 sealed interface UndoJobResult {
7     val epoch: Long
8     val sequence: Long
9
10     data class Ready(
11         override val epoch: Long,
12         override val sequence: Long,
13         val entry: UndoEntry,
14     ) : UndoJobResult
15
16     data class Failed(
17         override val epoch: Long,
18         override val sequence: Long,
19         val error: Throwable,
20     ) : UndoJobResult
21 }
```

54. app/src/main/java/com/wetinknext/engine/core/ViewTransform.kt

```

1 package com.wetinknext.engine.core
2
3 import kotlin.math.abs
4 import kotlin.math.cos
5 import kotlin.math.sin
6
7 /** Canonical canvas-to-view-local-screen transformation. */
8 data class ViewTransform(
9     val translateX: Float = 0f,
10    val translateY: Float = 0f,
11    val scale: Float = 1f,
12    val rotationRad: Float = 0f,
13    val flipX: Boolean = false,
14 ) {
15     private val flipSign: Float
16     get() = if (flipX) -1f else 1f
17
18     val m00: Float get() = cos(rotationRad) * scale * flipSign
19     val m01: Float get() = -sin(rotationRad) * scale
20     val m10: Float get() = sin(rotationRad) * scale * flipSign
21     val m11: Float get() = cos(rotationRad) * scale
22
23     fun canvasToScreen(canvasX: Float, canvasY: Float, out: FloatArray) {
24         require(out.size >= 2)
25         out[0] = m00 * canvasX + m01 * canvasY + translateX
26         out[1] = m10 * canvasX + m11 * canvasY + translateY
27     }
28
29     fun screenToCanvas(screenX: Float, screenY: Float, out: FloatArray) {
30         require(out.size >= 2)
31         val determinant = m00 * m11 - m01 * m10
32         if (abs(determinant) < 1e-8f) {
33             out[0] = 0f
34             out[1] = 0f
35             return
36         }
37         val inverseDeterminant = 1f / determinant
38         val dx = screenX - translateX
39         val dy = screenY - translateY
40         out[0] = (m11 * dx - m01 * dy) * inverseDeterminant
41         out[1] = (-m10 * dx + m00 * dy) * inverseDeterminant
42     }
43
44     fun zoomAround(anchorX: Float, anchorY: Float, newScale: Float): ViewTransform {
45         val safeScale = newScale.coerceIn(MIN_SCALE, MAX_SCALE)
46         val factor = safeScale / scale
47         return copy(
48             scale = safeScale,
49             translateX = anchorX - (anchorX - translateX) * factor,
50             translateY = anchorY - (anchorY - translateY) * factor,
51         )
52     }
53
54     fun rotateAround(anchorX: Float, anchorY: Float, deltaRad: Float): ViewTransform {
55         val c = cos(deltaRad)
56         val s = sin(deltaRad)
57         val dx = translateX - anchorX
58         val dy = translateY - anchorY
59         return copy(
60             rotationRad = rotationRad + deltaRad,
61             translateX = anchorX + dx * c - dy * s,
62             translateY = anchorY + dx * s + dy * c,
63         )
64     }
65
66     /** Applies one screen-space navigation update around a shared gesture anchor. */
67     fun applyGestureDelta(
68         panX: Float,
69         panY: Float,
70         anchorX: Float,
71         anchorY: Float,
72         zoomFactor: Float,
73         rotationDelta: Float,
74     ): ViewTransform {
75         val translated = copy(
76             translateX = translateX + panX,
77             translateY = translateY + panY,
78         )
79         val zoomed = translated.zoomAround(
80             anchorX = anchorX,
81             anchorY = anchorY,
82             newScale = scale * zoomFactor,
83         )
84         return zoomed.rotateAround(anchorX, anchorY, rotationDelta)
85     }
86
87     /** Builds a column-major matrix for canvas to OpenGL clip coordinates. */
88     fun buildCanvasToClip(viewWidth: Float, viewHeight: Float, out: FloatArray) {
89         require(out.size >= 16)
90         require(viewWidth > 0f)

```

```

91         require(viewHeight > 0f)
92         val sx = 2f / viewWidth
93         val sy = -2f / viewHeight
94         out[0] = m00 * sx; out[1] = m10 * sy; out[2] = 0f; out[3] = 0f
95         out[4] = m01 * sx; out[5] = m11 * sy; out[6] = 0f; out[7] = 0f
96         out[8] = 0f; out[9] = 0f; out[10] = 1f; out[11] = 0f
97         out[12] = translateX * sx - 1f; out[13] = translateY * sy + 1f
98         out[14] = 0f; out[15] = 1f
99     }
100
101     companion object {
102         const val MIN_SCALE = 0.02f
103         const val MAX_SCALE = 64f
104
105         fun buildCanvasToFbo(canvasWidth: Float, canvasHeight: Float, out: FloatArray) {
106             require(out.size >= 16); require(canvasWidth > 0f); require(canvasHeight > 0f)
107             out[0]=2f/canvasWidth;out[1]=0f;out[2]=0f;out[3]=0f;out[4]=0f;out[5]=2f/canvasHeight;out[6]=0f;out[7]=0f;out[
8]=0f;out[9]=0f;out[10]=1f;out[11]=0f;out[12]=-1f;out[13]=-1f;out[14]=0f;out[15]=1f
108         }
109     }
110 }

```

55. app/src/main/java/com/wetinknext/engine/gl/CanvasGeometry.kt

```
1 package com.wetinknext.engine.gl
2
3 import android.opengl.GLES30
4 import java.nio.ByteBuffer
5 import java.nio.ByteOrder
6
7 /** A document-sized quad; GL resources are owned by the render thread. */
8 class CanvasGeometry {
9     var vaoId = 0
10     private set
11     private var vboId = 0
12
13     fun create(width: Int, height: Int) {
14         release()
15         val vertices = floatArrayOf(0f, 0f, width.toFloat(), 0f, 0f, height.toFloat(), width.toFloat(), height.toFloat())
16         val data = ByteBuffer.allocateDirect(vertices.size * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).asFloatBuffer()
17         data.put(vertices).position(0)
18         val ids = IntArray(1)
19         GLES30.glGenVertexArrays(1, ids, 0); vaoId = ids[0]
20         GLES30.glGenBuffers(1, ids, 0); vboId = ids[0]
21         GLES30.glBindVertexArray(vaoId); GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vboId)
22         GLES30.glBufferData(GLES30.GL_ARRAY_BUFFER, vertices.size * Float.SIZE_BYTES, data, GLES30.GL_STATIC_DRAW)
23         GLES30.glEnableVertexAttribArray(0); GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
24         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0); GLES30.glBindVertexArray(0)
25     }
26
27     fun draw() {
28         check(vaoId != 0)
29         GLES30.glBindVertexArray(vaoId); GLES30.glDrawArrays(GLES30.GL_TRIANGLE_STRIP, 0, 4); GLES30.glBindVertexArray(0)
30     }
31
32     fun release() {
33         if (vboId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vboId), 0)
34         if (vaoId != 0) GLES30.glDeleteVertexArrays(1, intArrayOf(vaoId), 0)
35         vaoId = 0; vboId = 0
36     }
37 }
```

56. app/src/main/java/com/wetinknext/engine/gl/GeometryCache.kt

```

1  package com.wetinknext.engine.gl
2
3  import android.opengl.GLES30
4  import java.nio.ByteBuffer
5  import java.nio.ByteOrder
6
7  /** A single cached fullscreen triangle for future present passes. */
8  class GeometryCache {
9      private var vaoId = 0
10     private var vboId = 0
11     fun create() {
12         if (vaoId != 0) return
13         val vertices = floatArrayOf(-1f, -1f, 3f, -1f, -1f, 3f)
14         val data = ByteBuffer.allocateDirect(vertices.size * Float.SIZE_BYTES).order(ByteOrder.nativeOrder()).asFloatBuffer()
15         data.put(vertices).position(0)
16         val ids = IntArray(1); GLES30.glGenVertexArrays(1, ids, 0); vaoId = ids[0]
17         GLES30.glGenBuffers(1, ids, 0); vboId = ids[0]
18         GLES30.glBindVertexArray(vaoId); GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, vboId)
19         GLES30.glBufferData(GLES30.GL_ARRAY_BUFFER, vertices.size * Float.SIZE_BYTES, data, GLES30.GL_STATIC_DRAW)
20         GLES30.glEnableVertexAttribArray(0); GLES30.glVertexAttribPointer(0, 2, GLES30.GL_FLOAT, false, 0, 0)
21         GLES30.glBindBuffer(GLES30.GL_ARRAY_BUFFER, 0); GLES30.glBindVertexArray(0)
22     }
23     fun drawFullscreenTriangle() { check(vaoId != 0); GLES30.glBindVertexArray(vaoId); GLES30.glDrawArrays(GLES30.
GL_TRIANGLES, 0, 3); GLES30.glBindVertexArray(0) }
24     fun release() { if (vboId != 0) GLES30.glDeleteBuffers(1, intArrayOf(vboId), 0); if (vaoId != 0) GLES30.
glDeleteVertexArrays(1, intArrayOf(vaoId), 0); vboId = 0; vaoId = 0 }
25 }

```

57. app/src/main/java/com/wetinknext/engine/gl/GLCaps.kt

```
1 package com.wetinknext.engine.gl
2
3 import android.opengl.GLES30
4
5 /** Capabilities queried only after a GLES 3 context is current. */
6 data class GLCaps(
7     val renderer: String,
8     val version: String,
9     val supportsHalfFloatColorBuffer: Boolean,
10 ) {
11     companion object {
12         fun query(): GLCaps {
13             val extensions = GLES30.glGetString(GLES30.GL_EXTENSIONS).orEmpty()
14             val declared = extensions.contains("EXT_color_buffer_half_float") ||
15                 extensions.contains("EXT_color_buffer_float")
16             // Extension string сам по себе не доказательство: проверяем реальный FBO 4x4.
17             val supported = declared && RenderTarget().probeHalfFloatColorBuffer()
18             return GLCaps(
19                 renderer = GLES30.glGetString(GLES30.GL_RENDERER).orEmpty(),
20                 version = GLES30.glGetString(GLES30.GL_VERSION).orEmpty(),
21                 supportsHalfFloatColorBuffer = supported,
22             )
23         }
24     }
25 }
```

58. app/src/main/java/com/wetinknext/engine/gl/GLCheck.kt

```
1 package com.wetinknext.engine.gl
2
3 import android.opengl.GLES30
4
5 /** GL error checks intended for setup boundaries, never the render hot path. */
6 object GLCheck {
7     @Volatile
8     private var glThread: Thread? = null
9
10    fun setGLThread(thread: Thread) {
11        glThread = thread
12    }
13
14    fun checkOnGLThread() {
15        val current = Thread.currentThread()
16        val expected = glThread
17        check(current === expected) {
18            "GL resource accessed outside GLSurfaceView render thread: current=$current, expected=$expected"
19        }
20    }
21
22    fun noError(label: String) {
23        val error = GLES30.glGetError()
24        check(error == GLES30.GL_NO_ERROR) { "$label: GL error 0x${error.toString(16)}" }
25    }
26
27    fun framebufferComplete(label: String) {
28        val status = GLES30.glCheckFramebufferStatus(GLES30.GL_FRAMEBUFFER)
29        check(status == GLES30.GL_FRAMEBUFFER_COMPLETE) {
30            "$label: framebuffer incomplete (0x${status.toString(16)})"
31        }
32    }
33 }
```

59. app/src/main/java/com/wetinknext/engine/gl/GLProgram.kt

```
1 package com.wetinknext.engine.gl
2
3 import android.opengl.GLES30
4
5 /** Linked GLSL program. Construction and release must run on the GL thread. */
6 class GLProgram(vertexSource: String, fragmentSource: String) {
7     val id: Int = GLES30.glCreateProgram().also { program ->
8         check(program != 0) { "Unable to create GL program" }
9         val vertex = compile(GLES30.GL_VERTEX_SHADER, vertexSource)
10        val fragment = compile(GLES30.GL_FRAGMENT_SHADER, fragmentSource)
11        GLES30.glAttachShader(program, vertex); GLES30.glAttachShader(program, fragment); GLES30.glLinkProgram(program)
12        val linked = IntArray(1); GLES30.glGetProgramiv(program, GLES30.GL_LINK_STATUS, linked, 0)
13        GLES30.glDeleteShader(vertex); GLES30.glDeleteShader(fragment)
14        check(linked[0] == GLES30.GL_TRUE) { "Program link failed: ${GLES30.glGetProgramInfoLog(program)}" }
15    }
16    fun use() = GLES30.glUseProgram(id)
17    fun release() = GLES30.glDeleteProgram(id)
18
19    private fun compile(type: Int, source: String): Int = GLES30.glCreateShader(type).also { shader ->
20        check(shader != 0) { "Unable to create shader" }
21        GLES30.glShaderSource(shader, source); GLES30.glCompileShader(shader)
22        val compiled = IntArray(1); GLES30.glGetShaderiv(shader, GLES30.GL_COMPILE_STATUS, compiled, 0)
23        check(compiled[0] == GLES30.GL_TRUE) { "Shader compile failed: ${GLES30.glGetShaderInfoLog(shader)}" }
24    }
25 }
```

60. app/src/main/java/com/wetinknext/engine/gl/RenderTarget.kt

```

1  package com.wetinknext.engine.gl
2
3  import android.opengl.GLES30
4
5  /** Texture-backed FBO with RGBA16F probing and a guaranteed RGBA8 fallback. */
6  class RenderTarget {
7      var framebufferId: Int = 0
8      private set
9      var textureId: Int = 0
10     private set
11     var width: Int = 0
12     private set
13     var height: Int = 0
14     private set
15     var usesHalfFloat: Boolean = false
16     private set
17
18     fun create(width: Int, height: Int, preferHalfFloat: Boolean) {
19         GLCheck.checkOnGLThread()
20         require(width > 0 && height > 0)
21         release()
22         if (preferHalfFloat && createInternal(width, height, true)) return
23         check(createInternal(width, height, false)) { "RGBA8 render target creation failed" }
24     }
25
26     /** A 4x4 completeness probe used before allocating a document-sized target. */
27     fun probeHalfFloatColorBuffer(): Boolean {
28         release()
29         val result = createInternal(4, 4, true)
30         release()
31         return result
32     }
33
34     fun bind() {
35         check(framebufferId != 0) { "RenderTarget is not created" }
36         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, framebufferId)
37     }
38
39     fun clear(red: Float, green: Float, blue: Float, alpha: Float) {
40         bind()
41         // Clearing is constrained by the current viewport. Always restore the
42         // offscreen target's extent, not the last screen-sized present viewport.
43         GLES30.glViewport(0, 0, width, height)
44         GLES30.glDisable(GLES30.GL_SCISSOR_TEST)
45         GLES30.glDisable(GLES30.GL_BLEND)
46         GLES30.glColorMask(true, true, true, true)
47         GLES30.glClearColor(red, green, blue, alpha)
48         GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT)
49         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
50     }
51
52     fun release() {
53         GLCheck.checkOnGLThread()
54         if (framebufferId != 0) GLES30.glDeleteFramebuffers(1, intArrayOf(framebufferId), 0)
55         if (textureId != 0) GLES30.glDeleteTextures(1, intArrayOf(textureId), 0)
56         framebufferId = 0; textureId = 0; width = 0; height = 0; usesHalfFloat = false
57     }
58
59     private fun createInternal(targetWidth: Int, targetHeight: Int, halfFloat: Boolean): Boolean {
60         val textures = IntArray(1)
61         val framebuffers = IntArray(1)
62         GLES30.glGenTextures(1, textures, 0)
63         GLES30.glGenFramebuffers(1, framebuffers, 0)
64         val texture = textures[0]
65         val framebuffer = framebuffers[0]
66         if (texture == 0 || framebuffer == 0) return false
67         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, texture)
68         GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_MIN_FILTER, GLES30.GL_LINEAR)
69         GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_MAG_FILTER, GLES30.GL_LINEAR)
70         GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_WRAP_S, GLES30.GL_CLAMP_TO_EDGE)
71         GLES30.glTexParameteri(GLES30.GL_TEXTURE_2D, GLES30.GL_TEXTURE_WRAP_T, GLES30.GL_CLAMP_TO_EDGE)
72         GLES30.glTexImage2D(GLES30.GL_TEXTURE_2D, 0, if (halfFloat) GLES30.GL_RGBA16F else GLES30.GL_RGBA8,
73             targetWidth, targetHeight, 0, GLES30.GL_RGBA, if (halfFloat) GLES30.GL_HALF_FLOAT else GLES30.GL_UNSIGNED_BYTE,
74             null)
75         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, framebuffer)
76         GLES30.glFramebufferTexture2D(GLES30.GL_FRAMEBUFFER, GLES30.GL_COLOR_ATTACHMENT0, GLES30.GL_TEXTURE_2D, texture, 0)
77         val complete = GLES30.glCheckFramebufferStatus(GLES30.GL_FRAMEBUFFER) == GLES30.GL_FRAMEBUFFER_COMPLETE
78         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
79         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
80         if (!complete) {
81             GLES30.glDeleteFramebuffers(1, framebuffers, 0); GLES30.glDeleteTextures(1, textures, 0)
82             return false
83         }
84         framebufferId = framebuffer; textureId = texture; width = targetWidth; height = targetHeight; usesHalfFloat =
85         halfFloat
86     }
87 }

```

61. app/src/main/java/com/wetinknext/engine/gl/ShaderLib.kt

```

1  package com.wetinknext.engine.gl
2
3  object ShaderLib {
4      const val compositorVertex = ""#version 300 es
5          layout(location=0) in vec2 aPosition;uniform mat4 uCanvasToClip;uniform vec2 uCanvasSize;out vec2 vUv;void main(){
6      const val compositorFragment = ""#version 300 es
7          precision highp float;
8
9          in vec2 vUv;
10         uniform sampler2D uLayerTex;
11         uniform sampler2D uStrokeTex;
12         uniform int uStrokeActive;
13         uniform float uOpacity;
14         uniform float uStrokeOpacity;
15
16         out vec4 fragColor;
17
18         vec3 linearToSrgb(vec3 color) {
19             vec3 c = clamp(color, 0.0, 1.0);
20             vec3 low = c * 12.92;
21             vec3 high = 1.055 * pow(c, vec3(1.0 / 2.4)) - 0.055;
22             return mix(low, high, step(vec3(0.0031308), c));
23         }
24
25         vec3 unpremultiply(vec4 color) {
26             return color.a <= 0.0001 ? vec3(0.0) : color.rgb / color.a;
27         }
28
29         void main() {
30             vec4 layer = texture(uLayerTex, vUv);
31             if (uStrokeActive == 1) {
32                 vec4 stroke = texture(uStrokeTex, vUv) * uStrokeOpacity;
33                 layer = stroke + layer * (1.0 - stroke.a);
34             }
35
36             float alpha = layer.a * uOpacity;
37             fragColor = vec4(linearToSrgb(unpremultiply(layer)) * alpha, alpha);
38         }
39     ""
40     const val dabVertex = ""#version 300 es
41         layout(location = 0) in vec2 aCorner;
42         layout(location = 1) in vec4 iDab0;
43         layout(location = 2) in vec2 iDab1;
44         uniform mat4 uCanvasToClip;
45         uniform vec2 uCanvasSize;
46         out vec2 vLocal;
47         out vec2 vCanvasUv;
48         flat out float vCoverage;
49         flat out float vFlow;
50         void main(){
51             float c=cos(iDab0.w), s=sin(iDab0.w);
52             vec2 r=vec2(c*aCorner.x-s*aCorner.y,s*aCorner.x+c*aCorner.y);
53             vLocal=aCorner;
54             vCoverage=iDab1.x;
55             vFlow=iDab1.y;
56             vec2 p = iDab0.xy+r*iDab0.z;
57             vCanvasUv = p / uCanvasSize;
58             gl_Position=uCanvasToClip*vec4(p,0.,1.);
59         }
60     ""
61     const val dabFragment = ""#version 300 es
62         precision highp float;
63         in vec2 vLocal;
64         in vec2 vCanvasUv;
65         flat in float vCoverage;
66         flat in float vFlow;
67         uniform vec3 uColorLinear;
68         uniform sampler2D uGrainTex;
69         uniform int uGrainActive;
70         uniform float uGrainScale;
71         uniform int uGrainCanvasLocked;
72         uniform float uTextureDepth;
73         uniform float uTextureContrast;
74         uniform float uStrokeOpacity;
75         uniform sampler2D uShapeTex;
76         uniform int uShapeActive;
77         uniform int uReverseShape;
78         uniform int uRgbToAlpha;
79         out vec4 fragColor;
80         float luminance(vec3 color) {
81             return dot(color, vec3(0.299, 0.587, 0.114));
82         }
83         float applyTextureLevels(float value, float contrast, float depth) {
84             float adjusted = clamp((value - 0.5) * contrast + 0.5, 0.0, 1.0);
85             return mix(1.0 - depth, 1.0, adjusted);
86         }
87         void main(){
88             float r=length(vLocal);
89             float aa=max(fwidth(r),.001);

```

```

90         float cov=1.-smoothstep(1.-aa,1.+aa,r);
91         if (cov <= 0.0) discard;
92
93         float shapeMask = 1.0;
94         if (uShapeActive == 1) {
95             vec2 shapeUv = vLocal * 0.5 + 0.5;
96             vec4 shapeColor = texture(uShapeTex, shapeUv);
97             shapeMask = uRgbToAlpha == 1
98                 ? dot(shapeColor.rgb, vec3(0.299, 0.587, 0.114))
99                 : shapeColor.a;
100             if (uReverseShape == 1) shapeMask = 1.0 - shapeMask;
101         }
102
103         float grainFactor = 1.0;
104         if (uGrainActive == 1) {
105             vec2 uv = uGrainCanvasLocked == 1
106                 ? vCanvasUv * max(uGrainScale, 0.0001)
107                 : (vLocal * 0.5 + 0.5) * max(uGrainScale, 0.0001);
108             vec4 grainColor = texture(uGrainTex, uv);
109             grainFactor = applyTextureLevels(
110                 luminance(grainColor.rgb), uTextureContrast, uTextureDepth
111             );
112         }
113
114         float a = vCoverage * vFlow * cov * shapeMask * grainFactor * uStrokeOpacity;
115         fragColor=vec4(uColorLinear*a,a);
116     }
117     ""
118     const val strokeCompositeFragment = ""#version 300 es
119     precision highp float; in vec2 vUv; uniform sampler2D uCanvasTex; uniform sampler2D uStrokeTex; uniform int
uStrokeActive; out vec4 fragColor;
120     vec3 toSrgb(vec3 c){c=clamp(c,0.,1.);return mix(c*12.92,1.055*pow(c,vec3(1./2.4))-.055,step(vec3(.0031308),c));}
121     vec3 unpre(vec4 c){return c.a<=.0001?vec3(0.):c.rgb/c.a;}
122     void main(){vec4 c=texture(uCanvasTex,vUv);vec4 s=texture(uStrokeTex,vUv);float a=uStrokeActive==1?s.a+c.a*(1.-s.a):c
a;vec3 rgb=uStrokeActive==1&&a>.0001?(unpre(s)*s.a+unpre(c)*c.a*(1.-s.a))/a:unpre(c);fragColor=vec4(toSrgb(rgb),1.);}
123     ""
124     const val canvasPresentVertex = ""#version 300 es
125     layout(location = 0) in vec2 aPosition;
126     uniform mat4 uCanvasToClip;
127     uniform vec2 uCanvasSize;
128     out vec2 vUv;
129     void main() {
130         vUv = aPosition / uCanvasSize;
131         gl_Position = uCanvasToClip * vec4(aPosition, 0.0, 1.0);
132     }
133     ""
134     const val presentFragment = ""#version 300 es
135     precision highp float;
136     in vec2 vUv;
137     uniform sampler2D uTexture;
138     out vec4 fragColor;
139     void main() { fragColor = texture(uTexture, vUv); }
140     ""
141     const val fullscreenVertex = ""#version 300 es
142     layout(location = 0) in vec2 aPosition;
143     out vec2 vUv;
144     void main() { vUv = aPosition * 0.5 + 0.5; gl_Position = vec4(aPosition, 0.0, 1.0); }
145     ""
146     const val solidFragment = ""#version 300 es
147     precision highp float;
148     in vec2 vUv;
149     out vec4 fragColor;
150     void main() { fragColor = vec4(vUv, 0.0, 1.0); }
151     ""
152     const val ribbonVertex = ""#version 300 es
153     layout(location=0) in vec2 aPos; layout(location=1) in float aCoverage; layout(location=2) in float aAlpha;
154     uniform mat4 uCanvasToClip; out float vCoverage; out float vAlpha;
155     void main(){vCoverage=aCoverage;vAlpha=aAlpha;gl_Position=uCanvasToClip*vec4(aPos,0.,1.);}
156     ""
157     const val ribbonFragment = ""#version 300 es
158     precision highp float; in float vCoverage; in float vAlpha; uniform vec3 uColorLinear; uniform float uFlow;
159     uniform int uAntiAliasLevel; uniform int uNoAntialias; out vec4 fragColor;
160     void main(){float cov=vCoverage;if(uNoAntialias==1||uAntiAliasLevel==0)cov=step(.5,cov);else{float
e=uAntiAliasLevel==1?.35:uAntiAliasLevel==2?.5:.7;cov=smoothstep(.5-e,.5+e,cov);}float a=vAlpha*cov*uFlow;fragColor=vec4(
uColorLinear*a,a);}
161     ""
162
163     /**
164     * Один сегмент штриха = капсула (round cone): два круга с линейной интерполяцией радиуса.
165     * Вершинный шейдер строит только AABB сегмента, форма считается во фрагменте.
166     */
167     const val capsuleVertex = ""#version 300 es
168     layout(location = 0) in vec2 aCorner; // -1..1, TRIANGLE_STRIP из 4 вершин
169     layout(location = 1) in vec4 iA; // x, y, radius, coverage (начало сегмента)
170     layout(location = 2) in vec4 iB; // x, y, radius, coverage (конец сегмента)
171     uniform mat4 uCanvasToClip;
172     uniform vec2 uCanvasSize;
173     out vec2 vPos;
174     out vec2 vCanvasUv;
175     flat out vec4 vA;
176     flat out vec4 vB;
177     void main() {

```

```

178         // Запас 2 px на AA-переход, иначе край обрежется по границе квада.
179         vec2 lo = min(iA.xy - iA.z, iB.xy - iB.z) - 2.0;
180         vec2 hi = max(iA.xy + iA.z, iB.xy + iB.z) + 2.0;
181         vec2 p = mix(lo, hi, aCorner * 0.5 + 0.5);
182         vPos = p;
183         vCanvasUv = p / uCanvasSize;
184         vA = iA;
185         vB = iB;
186         gl_Position = uCanvasToClip * vec4(p, 0.0, 1.0);
187     }
188     ""
189
190     /**
191     * Аналитическое покрытие: нет каппов, джойнов, miter и AA-юбки как отдельных сущностей.
192     * Выход строго premultiplied linear: rgb = color * a, a = cov.
193     * Рисовать premultiplied source-over blending:
194     * GL_FUNC_ADD + GL_ONE, GL_ONE_MINUS_SRC_ALPHA.
195     */
196     const val capsuleFragment = ""#version 300 es
197         precision highp float;
198         in vec2 vPos;
199         in vec2 vCanvasUv;
200         flat in vec4 vA;
201         flat in vec4 vB;
202         uniform vec3 uColorLinear;
203         uniform sampler2D uGrainTex;
204         uniform int uGrainActive;
205         uniform float uGrainScale;
206         uniform int uGrainCanvasLocked;
207         uniform float uTextureDepth;
208         uniform float uTextureContrast;
209         uniform float uFlow;
210         out vec4 fragColor;
211
212         float sdRoundCone(vec2 p, vec2 a, vec2 b, float r1, float r2) {
213             vec2 ba = b - a;
214             float l2 = dot(ba, ba);
215             float rr = r1 - r2;
216
217             // Guard 1: вырожденный сегмент (тап, дубль сэмпла) -> обычный круг.
218             if (l2 < 1e-6) return length(p - a) - max(r1, r2);
219             // Guard 2: один круг поглощает другой (скачок давления на коротком шаге).
220             // Без этого a2 < 0 -> sqrt(отрицательного) -> NaN -> чёрные клинья.
221             if (rr * rr >= l2) return (r1 > r2) ? (length(p - a) - r1) : (length(p - b) - r2);
222
223             float a2 = l2 - rr * rr;
224             float il2 = 1.0 / l2;
225             vec2 pa = p - a;
226             float y = dot(pa, ba);
227             float z = y - l2;
228             vec2 xv = pa * l2 - ba * y;
229             float x2 = dot(xv, xv);
230             float y2 = y * y * l2;
231             float z2 = z * z * l2;
232             float k = sign(rr) * rr * rr * x2;
233
234             if (sign(z) * a2 * z2 > k) return sqrt(x2 + z2) * il2 - r2; // круглый конец у B
235             if (sign(y) * a2 * y2 < k) return sqrt(x2 + y2) * il2 - r1; // круглый конец у A
236             return (sqrt(x2 * a2 * il2) + y * rr) * il2 - r1; // касательная боковина
237         }
238
239         void main() {
240             float d = sdRoundCone(vPos, vA.xy, vB.xy, vA.z, vB.z);
241             float w = max(fwidth(d), 1e-4);
242             float cov = 1.0 - smoothstep(-w, w, d);
243             if (cov <= 0.0) discard;
244
245             float grainFactor = 1.0;
246             if (uGrainActive == 1) {
247                 vec2 uv = vCanvasUv * max(uGrainScale, 0.0001);
248                 float val = texture(uGrainTex, uv).r;
249                 val = clamp((val - 0.5) * uTextureContrast + 0.5, 0.0, 1.0);
250                 grainFactor = mix(1.0 - uTextureDepth, 1.0, val);
251             }
252
253             float segmentCoverage = mix(vA.w, vB.w, 0.5);
254             float finalAlpha = cov * segmentCoverage * grainFactor * uFlow;
255             fragColor = vec4(uColorLinear * finalAlpha, finalAlpha);
256         }
257     ""
258
259     const val strokeBlitFragment = ""#version 300 es
260         precision highp float;
261         in vec2 vUv;
262         uniform sampler2D uStrokeTex;
263         uniform float uOpacity;
264         out vec4 fragColor;
265         void main() { fragColor = texture(uStrokeTex, vUv) * uOpacity; }
266     ""
267 }

```

62. app/src/main/java/com/wetinknext/engine/input/GestureRouter.kt

```

1  package com.wetinknext.engine.input
2
3  import android.view.MotionEvent
4  import com.wetinknext.engine.core.Camera
5  import kotlin.math.atan2
6  import kotlin.math.hypot
7
8  /**
9   * Routes view touch input before it reaches the brush pipeline.
10  *
11  * One pointer draws. A second pointer cancels that stroke immediately and
12  * takes ownership of the interaction for canvas navigation. Two- and
13  * three-pointer taps trigger undo and redo respectively.
14  */
15  class GestureRouter(
16      private val camera: Camera,
17      private val drawingInput: StrokeInputCapturer,
18      private val callbacks: Callbacks,
19  ) {
20      interface Callbacks {
21          fun cancelDrawing()
22          fun undoGesture()
23          fun redoGesture()
24          fun resetCamera()
25          fun requestRender()
26      }
27
28      private var mode = InputMode.IDLE
29      private var firstX = 0f
30      private var firstY = 0f
31      private var secondX = 0f
32      private var secondY = 0f
33      private var previousDistance = 0f
34      private var previousAngle = 0f
35      private var gestureStartTime = 0L
36      private var gestureMoved = false
37      private var navigationPointerCount = 0
38      private var drawingStartTime = 0L
39      private var drawingStartX = 0f
40      private var drawingStartY = 0f
41      private var drawingMoved = false
42      private var lastTapTime = 0L
43      private var lastTapX = 0f
44      private var lastTapY = 0f
45
46      fun onTouchEvent(event: MotionEvent): Boolean = when (event.actionMasked) {
47          MotionEvent.ACTION_DOWN -> {
48              mode = InputMode.DRAWING
49              drawingStartTime = event.eventTime
50              drawingStartX = event.x
51              drawingStartY = event.y
52              drawingMoved = false
53              drawingInput.onTouchEvent(event)
54              true
55          }
56
57          MotionEvent.ACTION_POINTER_DOWN -> {
58              if (event.pointerCount >= 2) {
59                  beginNavigation(event)
60                  true
61              } else {
62                  false
63              }
64          }
65
66          MotionEvent.ACTION_MOVE -> when (mode) {
67              InputMode.DRAWING -> {
68                  if (hypot(event.x - drawingStartX, event.y - drawingStartY) > TAP_MOVE_THRESHOLD_PX) {
69                      drawingMoved = true
70                      lastTapTime = 0L
71                  }
72                  drawingInput.onTouchEvent(event)
73                  true
74              }
75
76              InputMode.NAVIGATING -> {
77                  updateNavigation(event)
78                  true
79              }
80
81              else -> false
82          }
83
84          MotionEvent.ACTION_POINTER_UP -> {
85              if (mode == InputMode.NAVIGATING) {
86                  handleNavigationPointerUp(event)
87                  true
88              } else {
89                  drawingInput.onTouchEvent(event)
90                  true

```

```

91     }
92 }
93
94 MotionEvent.ACTION_UP -> {
95     when (mode) {
96         InputMode.DRAWING -> {
97             drawingInput.onTouchEvent(event)
98             val duration = event.eventTime - drawingStartTime
99             if (!drawingMoved && duration <= TAP_MAX_DURATION_MS) {
100                 handleSingleFingerTap(event)
101             }
102         }
103         InputMode.NAVIGATING -> finishNavigation(event)
104         else -> Unit
105     }
106     mode = InputMode.IDLE
107     true
108 }
109
110 MotionEvent.ACTION_CANCEL -> {
111     drawingInput.cancelActiveStroke()
112     callbacks.cancelDrawing()
113     mode = InputMode.CANCELLED
114     callbacks.requestRender()
115     true
116 }
117
118 else -> false
119 }
120
121 private fun beginNavigation(event: MotionEvent) {
122     lastTapTime = 0L
123     if (mode == InputMode.DRAWING) {
124         // The queued CANCEL keeps the brush pipeline coherent; the callback
125         // clears an already-rendered preview on the GL thread.
126         drawingInput.cancelActiveStroke()
127         callbacks.cancelDrawing()
128     }
129
130     mode = InputMode.NAVIGATING
131     navigationPointerCount = event.pointerCount
132     gestureStartTime = event.eventTime
133     gestureMoved = false
134     captureNavigationPointers(event)
135     callbacks.requestRender()
136 }
137
138 private fun updateNavigation(event: MotionEvent) {
139     if (event.pointerCount < 2) return
140
141     val newFirstX = event.getX(0)
142     val newFirstY = event.getY(0)
143     val newSecondX = event.getX(1)
144     val newSecondY = event.getY(1)
145     val oldCenterX = (firstX + secondX) * .5f
146     val oldCenterY = (firstY + secondY) * .5f
147     val newCenterX = (newFirstX + newSecondX) * .5f
148     val newCenterY = (newFirstY + newSecondY) * .5f
149     val newDistance = distance(newFirstX, newFirstY, newSecondX, newSecondY)
150     val zoomFactor = if (previousDistance > MIN_DISTANCE) newDistance / previousDistance else 1f
151     val newAngle = angle(newFirstX, newFirstY, newSecondX, newSecondY)
152     val rotationDelta = normalizeAngle(newAngle - previousAngle)
153     val panX = newCenterX - oldCenterX
154     val panY = newCenterY - oldCenterY
155
156     camera.set(
157         camera.snapshot().applyGestureDelta(
158             panX = panX,
159             panY = panY,
160             anchorX = newCenterX,
161             anchorY = newCenterY,
162             zoomFactor = zoomFactor,
163             rotationDelta = rotationDelta,
164         ),
165     )
166
167     if (hypot(panX, panY) > TAP_MOVE_THRESHOLD_PX ||
168         kotlin.math.abs(zoomFactor - 1f) > TAP_ZOOM_THRESHOLD ||
169         kotlin.math.abs(rotationDelta) > TAP_ROTATION_THRESHOLD_RAD
170     ) {
171         gestureMoved = true
172     }
173
174     firstX = newFirstX
175     firstY = newFirstY
176     secondX = newSecondX
177     secondY = newSecondY
178     previousDistance = newDistance
179     previousAngle = newAngle
180     callbacks.requestRender()
181 }
182

```

```

183 private fun handleNavigationPointerUp(event: MotionEvent) {
184     // Once fewer than two pointers remain, navigation stays active until
185     // ACTION_UP so a two/three-finger tap can be recognised reliably.
186     if (event.pointerCount - 1 < 2) return
187     captureNavigationPointers(event, event.actionIndex)
188 }
189
190 private fun finishNavigation(event: MotionEvent) {
191     val duration = event.eventTime - gestureStartTime
192     if (!gestureMoved && duration <= TAP_MAX_DURATION_MS) {
193         when (navigationPointerCount) {
194             2 -> callbacks.undoGesture()
195             3 -> callbacks.redoGesture()
196         }
197     }
198     callbacks.requestRender()
199 }
200
201 private fun handleSingleFingerTap(event: MotionEvent) {
202     val now = event.eventTime
203     val dx = event.x - lastTapX
204     val dy = event.y - lastTapY
205     val isDoubleTap =
206         now - lastTapTime <= DOUBLE_TAP_TIMEOUT_MS &&
207         hypot(dx, dy) <= DOUBLE_TAP_DISTANCE_PX
208     if (isDoubleTap) {
209         callbacks.resetCamera()
210         lastTapTime = 0L
211     } else {
212         lastTapTime = now
213         lastTapX = event.x
214         lastTapY = event.y
215     }
216 }
217
218 private fun captureNavigationPointers(event: MotionEvent, excludedIndex: Int = -1) {
219     var firstIndex = -1
220     var secondIndex = -1
221     for (index in 0 until event.pointerCount) {
222         if (index == excludedIndex) continue
223         if (firstIndex < 0) firstIndex = index else {
224             secondIndex = index
225             break
226         }
227     }
228     if (firstIndex < 0 || secondIndex < 0) return
229     firstX = event.getX(firstIndex)
230     firstY = event.getY(firstIndex)
231     secondX = event.getX(secondIndex)
232     secondY = event.getY(secondIndex)
233     previousDistance = distance(firstX, firstY, secondX, secondY)
234     previousAngle = angle(firstX, firstY, secondX, secondY)
235 }
236
237 private fun distance(x0: Float, y0: Float, x1: Float, y1: Float): Float =
238     hypot(x1 - x0, y1 - y0)
239
240 private fun angle(x0: Float, y0: Float, x1: Float, y1: Float): Float =
241     atan2(y1 - y0, x1 - x0)
242
243 private fun normalizeAngle(value: Float): Float {
244     var result = value
245     while (result > Math.PI) result -= (Math.PI * 2.0).toFloat()
246     while (result < -Math.PI) result += (Math.PI * 2.0).toFloat()
247     return result
248 }
249
250 private companion object {
251     const val TAP_MAX_DURATION_MS = 220L
252     const val MIN_DISTANCE = .001f
253     const val TAP_MOVE_THRESHOLD_PX = 4f
254     const val TAP_ZOOM_THRESHOLD = .02f
255     const val TAP_ROTATION_THRESHOLD_RAD = .035f
256     const val DOUBLE_TAP_TIMEOUT_MS = 280L
257     const val DOUBLE_TAP_DISTANCE_PX = 48f
258 }
259 }

```

63. app/src/main/java/com/wetinknext/engine/input/InputAction.kt

```
1 package com.wetinknext.engine.input
2
3 enum class InputAction {
4     DOWN,
5     MOVE,
6     UP,
7     CANCEL,
8 }
```

64. app/src/main/java/com/wetinknext/engine/input/InputBatch.kt

```
1 package com.wetinknext.engine.input
2
3 /** Fixed-capacity container for passing input from the UI thread to the GL thread. */
4 class InputBatch(val maxSamples: Int) {
5     val samples = Array(maxSamples) { InputSample() }
6
7     var action: InputAction = InputAction.MOVE
8     private set
9     var sampleCount: Int = 0
10    private set
11    var prediction: Boolean = false
12    private set
13
14    fun begin(action: InputAction, prediction: Boolean = false) {
15        this.action = action
16        this.prediction = prediction
17        sampleCount = 0
18    }
19
20    fun addSample(
21        canvasX: Float, canvasY: Float, pressure: Float, tiltX: Float, tiltY: Float,
22        orientationRad: Float, timestampNanos: Long, pointerId: Int, tool: PointerTool,
23        historical: Boolean,
24    ): Boolean {
25        if (sampleCount >= maxSamples) return false
26        samples[sampleCount].set(
27            canvasX, canvasY, pressure.coerceIn(0f, 1f), tiltX.coerceIn(-1f, 1f),
28            tiltY.coerceIn(-1f, 1f), orientationRad, timestampNanos, pointerId, tool, historical,
29        )
30        sampleCount += 1
31        return true
32    }
33
34    fun isEmpty(): Boolean = sampleCount == 0
35
36    fun clear() {
37        for (index in 0 until sampleCount) samples[index].clear()
38        sampleCount = 0
39        prediction = false
40        action = InputAction.MOVE
41    }
42 }
```

65. app/src/main/java/com/wetinknext/engine/input/InputBatchPool.kt

```

1  package com.wetinknext.engine.input
2
3  import java.util.concurrent.atomic.AtomicInteger
4  import java.util.concurrent.atomic.AtomicReferenceArray
5
6  /** Fixed-size pool that returns null instead of allocating when exhausted. */
7  class InputBatchPool(
8      batchCount: Int = DEFAULT_BATCH_COUNT,
9      maxSamplesPerBatch: Int = DEFAULT_MAX_SAMPLES,
10 ) {
11     private val batches = AtomicReferenceArray<InputBatch>(batchCount)
12     private val available = AtomicInteger(batchCount)
13
14     init {
15         require(batchCount > 0)
16         require(maxSamplesPerBatch > 0)
17         for (index in 0 until batchCount) batches.set(index, InputBatch(maxSamplesPerBatch))
18     }
19
20     fun acquire(): InputBatch? {
21         while (true) {
22             val current = available.get()
23             if (current <= 0) return null
24             if (available.compareAndSet(current, current - 1)) {
25                 for (index in 0 until batches.length()) {
26                     val batch = batches.getAndSet(index, null)
27                     if (batch != null) return batch
28                 }
29                 available.incrementAndGet()
30                 return null
31             }
32         }
33     }
34
35     fun release(batch: InputBatch) {
36         batch.clear()
37         for (index in 0 until batches.length()) {
38             if (batches.compareAndSet(index, null, batch)) {
39                 available.incrementAndGet()
40                 return
41             }
42         }
43         error("InputBatchPool overflow: released batch does not belong to pool")
44     }
45
46     val freeCount: Int get() = available.get()
47
48     companion object {
49         const val DEFAULT_BATCH_COUNT = 96
50         const val DEFAULT_MAX_SAMPLES = 64
51     }
52 }

```

66. app/src/main/java/com/wetinknext/engine/input/InputMode.kt

```
1 package com.wetinknext.engine.input
2
3 /** Current ownership of touch input. Only DRAWING reaches the brush pipeline. */
4 enum class InputMode {
5     IDLE,
6     DRAWING,
7     NAVIGATING,
8     CANCELLED,
9 }
```

67. app/src/main/java/com/wetinknext/engine/input/InputSample.kt

```
1 package com.wetinknext.engine.input
2
3 /** Mutable input sample allocated only while a batch pool is initialized. */
4 class InputSample {
5     var canvasX = 0f
6     var canvasY = 0f
7     var pressure = 0f
8     var tiltX = 0f
9     var tiltY = 0f
10    var orientationRad = 0f
11    var timestampNanos = 0L
12    var pointerId = -1
13    var tool = PointerTool.UNKNOWN
14    var historical = false
15
16    fun set(
17        canvasX: Float, canvasY: Float, pressure: Float, tiltX: Float, tiltY: Float,
18        orientationRad: Float, timestampNanos: Long, pointerId: Int, tool: PointerTool,
19        historical: Boolean,
20    ) {
21        this.canvasX = canvasX; this.canvasY = canvasY; this.pressure = pressure
22        this.tiltX = tiltX; this.tiltY = tiltY; this.orientationRad = orientationRad
23        this.timestampNanos = timestampNanos; this.pointerId = pointerId; this.tool = tool
24        this.historical = historical
25    }
26
27    fun clear() {
28        canvasX = 0f; canvasY = 0f; pressure = 0f; tiltX = 0f; tiltY = 0f
29        orientationRad = 0f; timestampNanos = 0L; pointerId = -1
30        tool = PointerTool.UNKNOWN; historical = false
31    }
32 }
```

68. app/src/main/java/com/wetinknext/engine/input/PointerTool.kt

```
1 package com.wetinknext.engine.input
2
3 enum class PointerTool {
4     FINGER,
5     STYLUS,
6     ERASER,
7     UNKNOWN,
8 }
```

69. app/src/main/java/com/wetinknext/engine/input/StrokeInputCapturer.kt

```

1 package com.wetinknext.engine.input
2
3 import android.view.MotionEvent
4 import com.wetinknext.engine.core.Camera
5 import java.util.concurrent.ArrayBlockingQueue
6
7 /** Converts View-local MotionEvent samples to canvas coordinates on the UI thread. */
8 class StrokeInputCapturer(private val camera: Camera, private val pool: InputBatchPool, private val queue:
ArrayBlockingQueue<InputBatch>) {
9     private var activePointerId = -1
10    private val canvasPoint = FloatArray(2)
11    private val stylusAxes = StylusAxes()
12    private val tiltOut = FloatArray(2)
13
14    var onSecondaryPointerDown: (() -> Unit)? = null
15
16    /** Не скрываем потери: пул кончился => сэмплы утрачены. */
17    var droppedBatches = 0L
18    private set
19
20    /** Cancels the one pointer currently owned by the drawing pipeline. */
21    fun cancelActiveStroke(): Boolean {
22        if (activePointerId < 0) return false
23        activePointerId = -1
24        return enqueueEmpty(InputAction.CANCEL)
25    }
26
27    fun onTouchEvent(event: MotionEvent): Boolean = when (event.actionMasked) {
28        MotionEvent.ACTION_DOWN -> {
29            activePointerId = event.getPointerId(0)
30            enqueue(event, InputAction.DOWN, 0, false)
31        }
32        // Multi-touch is owned by GestureRouter. Drawing input only receives
33        // the one-pointer draw path and explicit cancellation from that router.
34        MotionEvent.ACTION_POINTER_DOWN -> false
35        MotionEvent.ACTION_MOVE -> {
36            val i = event.findPointerIndex(activePointerId)
37            i >= 0 && enqueue(event, InputAction.MOVE, i, true)
38        }
39        MotionEvent.ACTION_UP, MotionEvent.ACTION_POINTER_UP -> {
40            val i = if (event.actionMasked == MotionEvent.ACTION_UP) 0 else event.actionIndex
41            if (event.getPointerId(i) != activePointerId) true else {
42                activePointerId = -1
43                // historical = true: хвост быстрого штриха больше не теряется.
44                enqueue(event, InputAction.UP, i, true)
45            }
46        }
47        MotionEvent.ACTION_CANCEL -> { activePointerId = -1; enqueueEmpty(InputAction.CANCEL) }
48        else -> false
49    }
50
51    private fun enqueue(event: MotionEvent, action: InputAction, index: Int, historical: Boolean): Boolean {
52        val batch = pool.acquire() ?: run { droppedBatches++; return false }
53        batch.begin(action); val t = camera.snapshot(); val id = event.getPointerId(index); val tool = tool(event, index)
54        if (historical) {
55            for (h in 0 until event.historySize) {
56                t.screenToCanvas(event.getHistoricalX(index, h), event.getHistoricalY(index, h), canvasPoint)
57                stylusAxes.readTilt(event, index, h, tool, tiltOut)
58                val orientation = stylusAxes.readOrientation(event, index, h, tool)
59                if (!batch.addSample(
60                    canvasPoint[0], canvasPoint[1], pressure(event, tool, index, h),
61                    tiltOut[0], tiltOut[1], orientation,
62                    event.getHistoricalEventTime(h) * 1_000_000L, id, tool, true
63                )) break
64            }
65        }
66        t.screenToCanvas(event.getX(index), event.getY(index), canvasPoint)
67        stylusAxes.readTilt(event, index, -1, tool, tiltOut)
68        val orientation = stylusAxes.readOrientation(event, index, -1, tool)
69        batch.addSample(
70            canvasPoint[0], canvasPoint[1], pressure(event, tool, index, -1),
71            tiltOut[0], tiltOut[1], orientation,
72            event.eventTime * 1_000_000L, id, tool, false
73        )
74        if (!queue.offer(batch)) {
75            pool.release(batch)
76            droppedBatches++
77            if (action == InputAction.UP) {
78                enqueueEmpty(InputAction.CANCEL)
79            }
80            return false
81        }
82        return true
83    }
84
85    private fun enqueueEmpty(action: InputAction): Boolean {
86        val batch = pool.acquire() ?: run { droppedBatches++; return false }
87        batch.begin(action); if (!queue.offer(batch)) { pool.release(batch); droppedBatches++; return false }; return true }
88    private fun tool(e: MotionEvent, i: Int) = when (e.getToolType(i)) { MotionEvent.TOOL_TYPE_STYLUS -> PointerTool.STYLUS; MotionEvent.
TOOL_TYPE_ERASER -> PointerTool.ERASER; MotionEvent.TOOL_TYPE_FINGER -> PointerTool.FINGER; else -> PointerTool.UNKNOWN }

```

```
89     private fun pressure(e:MotionEvent,t:PointerTool,i:Int,h:Int):Float=if(t==PointerTool.FINGER||t==PointerTool.UNKNOWN)
SYNTHETIC_FINGER_PRESSURE else (if(h>=0)e.getHistoricalPressure(i,h) else e.getPressure(i)).coerceIn(0f,1f)
90
91     companion object {
92         private const val SYNTHETIC_FINGER_PRESSURE = 0.62f
93     }
94 }
```

70. app/src/main/java/com/wetinknext/engine/input/StylusAxes.kt

```

1  package com.wetinknext.engine.input
2
3  import android.view.MotionEvent
4  import kotlin.math.cos
5  import kotlin.math.sin
6
7  class StylusAxes {
8
9      fun readTilt(
10         event: MotionEvent,
11         pointerIndex: Int,
12         historicalIndex: Int,
13         tool: PointerTool,
14         out: FloatArray,
15     ) {
16         if (tool == PointerTool.FINGER ||
17             tool == PointerTool.UNKNOWN) {
18             {
19                 out[0] = 0f
20                 out[1] = 0f
21                 return
22             }
23
24             val tilt = if (historicalIndex >= 0) {
25                 event.getHistoricalAxisValue(
26                     MotionEvent.AXIS_TILT,
27                     pointerIndex,
28                     historicalIndex,
29                 )
30             } else {
31                 event.getAxisValue(
32                     MotionEvent.AXIS_TILT,
33                     pointerIndex,
34                 )
35             }
36
37             val orientation = if (historicalIndex >= 0) {
38                 event.getHistoricalAxisValue(
39                     MotionEvent.AXIS_ORIENTATION,
40                     pointerIndex,
41                     historicalIndex,
42                 )
43             } else {
44                 event.getAxisValue(
45                     MotionEvent.AXIS_ORIENTATION,
46                     pointerIndex,
47                 )
48             }
49
50             val normalizedTilt =
51                 (tilt / (Math.PI.toFloat() * 0.5f))
52                 .coerceIn(0f, 1f)
53
54             out[0] =
55                 (cos(orientation) * normalizedTilt)
56                 .coerceIn(-1f, 1f)
57
58             out[1] =
59                 (sin(orientation) * normalizedTilt)
60                 .coerceIn(-1f, 1f)
61         }
62
63         fun readOrientation(
64             event: MotionEvent,
65             pointerIndex: Int,
66             historicalIndex: Int,
67             tool: PointerTool,
68         ): Float {
69             if (tool == PointerTool.FINGER || tool == PointerTool.UNKNOWN) return 0f
70             return if (historicalIndex >= 0) {
71                 event.getHistoricalAxisValue(MotionEvent.AXIS_ORIENTATION, pointerIndex, historicalIndex)
72             } else {
73                 event.getAxisValue(MotionEvent.AXIS_ORIENTATION, pointerIndex)
74             }
75         }
76     }

```

71. app/src/main/java/com/wetinknext/engine/persistence/LayerTileStore.kt

```

1 package com.wetinknext.engine.persistence
2
3 import com.wetinknext.engine.canvas.PaintLayer
4 import com.wetinknext.engine.undo.RawTileSnapshot
5 import com.wetinknext.engine.undo.TileCoord
6 import java.util.concurrent.ConcurrentHashMap
7
8 /** CPU cache of tiles captured after completed edits. No dab triggers persistence. */
9 class LayerTileStore(initialPayloads: Map<Long, ByteArray> = emptyMap()) {
10     private val tiles = ConcurrentHashMap<Long, ConcurrentHashMap<TileCoord, PersistentLayerTiles.Tile>>()
11     private val dirty = ConcurrentHashMap<Long, MutableSet<TileCoord>>()
12
13     suspend fun writeTile(layerId: Long, tile: RawTileSnapshot) {
14         tiles.getOrPut(layerId) { ConcurrentHashMap() }[tile.coord] = PersistentLayerTiles.fromRaw(tile)
15         dirty.getOrPut(layerId) { ConcurrentHashMap.newKeySet() }.add(tile.coord)
16     }
17
18     suspend fun readTile(layerId: Long, coord: TileCoord): ByteArray =
19         tiles[layerId]?.get(coord)?.bytes?.copyOf() ?: ByteArray(0)
20
21     suspend fun writeLayer(layer: PaintLayer, changedTiles: List<RawTileSnapshot>) {
22         require(layer.created)
23         changedTiles.forEach { writeTile(layer.id, it) }
24     }
25
26     /** GL-thread hand-off: copying one completed tile is bounded and allocation-safe. */
27     fun markDirty(layerId: Long, changedTiles: List<RawTileSnapshot>) {
28         val target = tiles.getOrPut(layerId) { ConcurrentHashMap() }
29         val dirtyCoords = dirty.getOrPut(layerId) { ConcurrentHashMap.newKeySet() }
30         changedTiles.forEach { tile ->
31             target[tile.coord] = PersistentLayerTiles.fromRaw(tile)
32             dirtyCoords.add(tile.coord)
33         }
34     }
35
36     fun takeDirty(): Map<Long, Set<TileCoord>> = dirty.mapValues { it.value.toSet() }
37
38     fun acknowledge(saved: Map<Long, Set<TileCoord>>) {
39         saved.forEach { (layerId, coords) -> dirty[layerId]?.removeAll(coords) }
40     }
41
42     /** Full layer payloads only for layers that have dirty tiles. */
43     fun payloadsForDirty(dirtySnapshot: Map<Long, Set<TileCoord>> = takeDirty()): Map<Long, ByteArray> =
44         dirtySnapshot.keys.associateWith { layerId ->
45             PersistentLayerTiles.encode(tiles[layerId]?.values.orEmpty())
46         }
47
48     init {
49         initialPayloads.forEach { (layerId, payload) ->
50             if (payload.isNotEmpty()) {
51                 val layer = tiles.getOrPut(layerId) { ConcurrentHashMap() }
52                 PersistentLayerTiles.decode(payload).forEach { layer[it.coord] = it }
53             }
54         }
55     }
56 }

```

72. app/src/main/java/com/wetinknext/engine/persistence/PersistentLayerTiles.kt

```

1 package com.wetinknext.engine.persistence
2
3 import com.wetinknext.engine.undo.RawTileSnapshot
4 import com.wetinknext.engine.undo.TileCoord
5 import java.nio.ByteBuffer
6 import java.nio.ByteOrder
7
8 /** Binary, versioned collection of 256px tiles for one persistent paint layer. */
9 object PersistentLayerTiles {
10     private const val MAGIC = 0x57544C53 // WTLS
11     private const val VERSION = 1
12     private const val HEADER_BYTES = 12
13     private const val TILE_HEADER_BYTES = 32
14
15     data class Tile(
16         val coord: TileCoord,
17         val pixelLeft: Int,
18         val pixelTop: Int,
19         val pixelWidth: Int,
20         val pixelHeight: Int,
21         val bytesPerPixel: Int,
22         val bytes: ByteArray,
23     )
24
25     fun decode(payload: ByteArray): List<Tile> {
26         if (payload.isEmpty()) return emptyList()
27         val input = ByteBuffer.wrap(payload).order(ByteOrder.BIG_ENDIAN)
28         require(input.remaining() >= HEADER_BYTES && input.int == MAGIC) { "Unknown persistent layer tile payload" }
29         require(input.int == VERSION) { "Unsupported persistent layer tile version" }
30         val count = input.int
31         require(count >= 0) { "Invalid tile count" }
32         return List(count) {
33             require(input.remaining() >= TILE_HEADER_BYTES) { "Truncated tile header" }
34             val tx = input.int; val ty = input.int
35             val left = input.int; val top = input.int
36             val width = input.int; val height = input.int
37             val bpp = input.int; val size = input.int
38             require(width > 0 && height > 0 && bpp in setOf(4, 8) && size == width * height * bpp && size <= input.remaining()) { "Invalid tile payload" }
39             Tile(TileCoord(tx, ty), left, top, width, height, bpp, ByteArray(size).also(input::get))
40             }.also { require(!input.hasRemaining()) { "Trailing tile payload data" } }
41         }
42
43     fun encode(tiles: Collection<Tile>): ByteArray {
44         val size = HEADER_BYTES + tiles.sumOf { TILE_HEADER_BYTES + it.bytes.size }
45         val output = ByteBuffer.allocate(size).order(ByteOrder.BIG_ENDIAN)
46         output.putInt(MAGIC).putInt(VERSION).putInt(tiles.size)
47         tiles.sortedWith(compareBy<Tile> { it.coord.ty }.thenBy { it.coord.tx }).forEach { tile ->
48             output.putInt(tile.coord.tx).putInt(tile.coord.ty)
49             output.putInt(tile.pixelLeft).putInt(tile.pixelTop)
50             output.putInt(tile.pixelWidth).putInt(tile.pixelHeight)
51             output.putInt(tile.bytesPerPixel).putInt(tile.bytes.size).put(tile.bytes)
52         }
53         return output.array()
54     }
55
56     fun fromRaw(tile: RawTileSnapshot) = Tile(
57         tile.coord, tile.pixelLeft, tile.pixelTop, tile.pixelWidth, tile.pixelHeight,
58         tile.bytesPerPixel, tile.rawBytes.copyOf(),
59     )
60 }

```

73. app/src/main/java/com/wetinknext/engine/thumbnail/ThumbnailBuildScheduler.kt

```

1 package com.wetinknext.engine.thumbnail
2
3 import android.os.Handler
4 import android.os.Looper
5 import com.wetinknext.engine.canvas.LayerStack
6 import com.wetinknext.engine.gl.GlCheck
7 import java.io.File
8 import java.io.IOException
9 import java.nio.file.AtomicMoveNotSupportedException
10 import java.nio.file.Files
11 import java.nio.file.StandardCopyOption
12 import java.util.concurrent.ConcurrentLinkedQueue
13 import java.util.concurrent.ExecutorService
14 import java.util.concurrent.Executors
15 import java.util.concurrent.RejectedExecutionException
16 import java.util.concurrent.atomic.AtomicLong
17
18 /** Result published to the UI only after the current generation reached disk. */
19 data class ThumbnailBuildResult(
20     val generation: Long,
21     val projectPreview: File?,
22     val layerPreviews: Map<Long, File>,
23 )
24
25 /**
26  * Coordinates thumbnail builds across the GL, worker and UI threads.
27  *
28  * Call [markDirty], [buildIfNeeded], [processCompleted] and [shutdown] on the
29  * GL thread. [buildIfNeeded] is the sole point that reads GPU textures. WebP
30  * encoding and file publication happen on [executor]; callbacks are dispatched
31  * to the Android main thread.
32  */
33 class ThumbnailBuildScheduler(
34     private val outputDirectory: File,
35     private val renderer: ThumbnailRenderer,
36     private val requestGlFrame: () -> Unit,
37     private val onSave: (ThumbnailBuildResult) -> Unit,
38     private val onFailure: (Throwable) -> Unit = {},
39     private val executorFactory: () -> ExecutorService = { Executors.newSingleThreadExecutor() },
40     private val uiHandler: Handler = Handler(Looper.getMainLooper()),
41     private val encoder: ThumbnailEncoder = ThumbnailEncoder(),
42     private val previewSizePx: Int = DEFAULT_PREVIEW_SIZE_PX,
43 ) {
44     private var executor = executorFactory()
45     private val pendingResults = ConcurrentLinkedQueue<WorkerResult>()
46     private val generation = AtomicLong(0L)
47     private val dirtyLayerIds = linkedSetOf<Long>()
48
49     private var projectDirty = false
50     private var buildRunning = false
51
52     /**
53      * Records the newest desired preview state. A newer mark makes an ongoing
54      * worker result stale before it is allowed to notify the UI.
55      */
56     fun markDirty(
57         projectDirty: Boolean,
58         dirtyLayerIds: Collection<Long> = emptyList(),
59     ) {
60         GlCheck.checkOnGlThread()
61         if (!projectDirty && dirtyLayerIds.isEmpty()) return
62         this.projectDirty = this.projectDirty || projectDirty
63         this.dirtyLayerIds += dirtyLayerIds
64         generation.incrementAndGet()
65     }
66
67     fun markProjectDirty() = markDirty(projectDirty = true)
68
69     fun markLayerDirty(layerId: Long) = markDirty(
70         projectDirty = false,
71         dirtyLayerIds = listOf(layerId),
72     )
73
74     /** Stops future work for a deleted layer without touching the GL context. */
75     fun removeLayer(layerId: Long) {
76         GlCheck.checkOnGlThread()
77         if (dirtyLayerIds.remove(layerId)) generation.incrementAndGet()
78     }
79
80     /** Recreates the worker after a GL context was deliberately released. */
81     fun ensureRunning() {
82         GlCheck.checkOnGlThread()
83         if (executor.isShutdown) executor = executorFactory()
84     }
85
86     /**
87      * Captures all currently dirty pixels on the GL thread and starts one
88      * worker job. Calling this while another job runs is intentionally a no-op.
89      */
90     fun buildIfNeeded(layers: LayerStack): Boolean {

```

```

91     GLCheck.checkOnGLThread()
92     if (buildRunning || (!projectDirty && dirtyLayerIds.isEmpty())) return false
93
94     val jobGeneration = generation.get()
95     val projectPixels = if (projectDirty) {
96         renderer.renderProject(layers, previewSizePx, previewSizePx)
97     } else {
98         null
99     }
100    val layerPixels = dirtyLayerIds
101        .mapNotNull { layerId ->
102            layers.findLayerById(layerId)?.let { layer ->
103                layerId to renderer.renderLayer(layer, previewSizePx, previewSizePx)
104            }
105        }
106        .toMap()
107
108    val request = BuildRequest(
109        generation = jobGeneration,
110        projectPixels = projectPixels,
111        layerPixels = layerPixels,
112    )
113    buildRunning = true
114    try {
115        executor.execute {
116            pendingResults.add(encodeAndPublish(request))
117            requestGLFrame()
118        }
119    } catch (error: RejectedExecutionException) {
120        pendingResults.add(WorkerResult.Failed(jobGeneration, error))
121        requestGLFrame()
122    }
123    return true
124 }
125
126 /**
127  * Finishes worker jobs on the GL thread and starts no UI callback for stale
128  * generations. The next frame can immediately call [buildIfNeeded] again.
129  */
130 fun processCompleted(): Boolean {
131     GLCheck.checkOnGLThread()
132     var changed = false
133     while (true) {
134         val result = pendingResults.poll() ?: break
135         buildRunning = false
136         changed = true
137
138         if (result.generation != generation.get()) {
139             continue
140         }
141
142         when (result) {
143             is WorkerResult.Saved -> {
144                 if (result.projectPreview != null) projectDirty = false
145                 dirtyLayerIds.removeAll(result.layerPreviews.keys)
146                 uiHandler.post {
147                     onSave(
148                         ThumbnailBuildResult(
149                             generation = result.generation,
150                             projectPreview = result.projectPreview,
151                             layerPreviews = result.layerPreviews,
152                         ),
153                     )
154                 }
155             }
156             is WorkerResult.Failed -> uiHandler.post { onFailure(result.error) }
157             is WorkerResult.Stale -> Unit
158         }
159     }
160     return changed
161 }
162
163 /** Makes in-flight work obsolete and releases the background encoder. */
164 fun shutdown() {
165     GLCheck.checkOnGLThread()
166     generation.incrementAndGet()
167     projectDirty = false
168     dirtyLayerIds.clear()
169     buildRunning = false
170     executor.shutdownNow()
171     pendingResults.clear()
172     renderer.release()
173 }
174
175 private fun encodeAndPublish(request: BuildRequest): WorkerResult = try {
176     val temporaryProject = request.projectPixels?.let {
177         temporaryFileFor(projectPreviewFile, request.generation)
178     }
179     val temporaryLayers = request.layerPixels.keys.associateWith { layerId ->
180         temporaryFileFor(layerPreviewFile(layerId), request.generation)
181     }

```

```

183     }
184
185     try {
186         request.projectPixels?.let { rgba ->
187             // Project preview is small and opaque in the usual case: 88 is
188             // a good size/quality balance.
189             encoder.encodeTopDown(rgba, previewSizePx, previewSizePx, checkNotNull(temporaryProject), lossless = false)
190         }
191         request.layerPixels.forEach { (layerId, rgba) ->
192             // Layer previews must preserve alpha and thin marks.
193             encoder.encodeTopDown(rgba, previewSizePx, previewSizePx, checkNotNull(temporaryLayers[layerId]), lossless =
true)
194         }
195
196         if (request.generation != generation.get()) {
197             temporaryProject?.delete()
198             temporaryLayers.values.forEach(File::delete)
199             WorkerResult.Stale(request.generation)
200         } else {
201             val publishedProject = temporaryProject?.let {
202                 publish(it, projectPreviewFile)
203                 projectPreviewFile
204             }
205             val publishedLayers = temporaryLayers.mapNotNull { (layerId, temporary) ->
206                 val target = layerPreviewFile(layerId)
207                 publish(temporary, target)
208                 layerId to target
209             }.toMap()
210             WorkerResult.Saved(request.generation, publishedProject, publishedLayers)
211         }
212     } catch (error: Throwable) {
213         temporaryProject?.delete()
214         temporaryLayers.values.forEach(File::delete)
215         throw error
216     }
217 } catch (error: Throwable) {
218     WorkerResult.Failed(request.generation, error)
219 }
220
221 private fun publish(temporary: File, destination: File) {
222     destination.parentFile?.mkdirs()
223     try {
224         Files.move(
225             temporary.toPath(),
226             destination.toPath(),
227             StandardCopyOption.ATOMIC_MOVE,
228             StandardCopyOption.REPLACE_EXISTING,
229         )
230     } catch (_: AtomicMoveNotSupportedException) {
231         Files.move(
232             temporary.toPath(),
233             destination.toPath(),
234             StandardCopyOption.REPLACE_EXISTING,
235         )
236     } catch (error: IOException) {
237         throw IOException("Unable to publish thumbnail ${destination.path}", error)
238     }
239 }
240
241 private fun temporaryFileFor(destination: File, jobGeneration: Long): File {
242     val parent = destination.parentFile ?: error("Thumbnail output requires a parent directory")
243     if (!parent.exists() && !parent.mkdirs()) {
244         throw IOException("Cannot create thumbnail directory: ${parent.path}")
245     }
246     return File(parent, ".$${destination.name}.${jobGeneration.tmp}")
247 }
248
249 private val projectPreviewFile: File
250     get() = File(outputDirectory, PROJECT_PREVIEW_FILE_NAME)
251
252 private fun layerPreviewFile(layerId: Long): File =
253     File(File(outputDirectory, LAYER_PREVIEW_DIRECTORY), "$layerId.webp")
254
255 private data class BuildRequest(
256     val generation: Long,
257     val projectPixels: ByteArray?,
258     val layerPixels: Map<Long, ByteArray>,
259 )
260
261 private sealed interface WorkerResult {
262     val generation: Long
263
264     data class Saved(
265         override val generation: Long,
266         val projectPreview: File?,
267         val layerPreviews: Map<Long, File>,
268     ) : WorkerResult
269
270     data class Failed(
271         override val generation: Long,
272         val error: Throwable,
273     ) : WorkerResult

```

```
274
275     data class Stale(
276         override val generation: Long,
277     ) : WorkerResult
278
279 }
280
281 private companion object {
282     const val DEFAULT_PREVIEW_SIZE_PX = 256
283     const val PROJECT_PREVIEW_FILE_NAME = "thumbnail.webp"
284     const val LAYER_PREVIEW_DIRECTORY = "previews"
285 }
286 }
```

74. app/src/main/java/com/wetinknext/engine/thumbnail/ThumbnailEncoder.kt

```

1 package com.wetinknext.engine.thumbnail
2
3 import android.graphics.Bitmap
4 import android.os.Build
5 import java.io.File
6
7 /**
8  * Persists a raw OpenGL thumbnail as WebP.
9  *
10  * [rgba] follows [android.opengl.GLES30.glReadPixels] row ordering: the first
11  * row is the bottom row. The encoder converts it to Android Bitmap's top-down
12  * order while preserving RGBA alpha exactly before WebP compression.
13  */
14 class ThumbnailEncoder {
15     fun encode(
16         rgba: ByteArray,
17         width: Int,
18         height: Int,
19         output: File,
20         lossless: Boolean,
21     ) = encodeInternal(
22         rgba = rgba,
23         width = width,
24         height = height,
25         output = output,
26         lossless = lossless,
27         inputRowsBottomUp = true,
28     )
29
30     /**
31      * Writes an image returned by [ThumbnailRenderer], whose rows have already
32      * been normalised into Android's top-to-bottom order.
33      */
34     fun encodeTopDown(
35         rgba: ByteArray,
36         width: Int,
37         height: Int,
38         output: File,
39         lossless: Boolean,
40     ) = encodeInternal(
41         rgba = rgba,
42         width = width,
43         height = height,
44         output = output,
45         lossless = lossless,
46         inputRowsBottomUp = false,
47     )
48
49     private fun encodeInternal(
50         rgba: ByteArray,
51         width: Int,
52         height: Int,
53         output: File,
54         lossless: Boolean,
55         inputRowsBottomUp: Boolean,
56     ) {
57         require(width > 0 && height > 0) { "Thumbnail dimensions must be positive" }
58         val expectedByteCount = width * height * RGBA_BYTES_PER_PIXEL
59         require(rgba.size == expectedByteCount) {
60             "RGBA byte count ${rgba.size} does not match ${width}x$height"
61         }
62
63         val pixels = IntArray(width * height)
64         for (bitmapY in 0 until height) {
65             // Bitmap begins at the top; glReadPixels begins at the bottom.
66             val sourceY = if (inputRowsBottomUp) height - 1 - bitmapY else bitmapY
67             var source = sourceY * width * RGBA_BYTES_PER_PIXEL
68             val destination = bitmapY * width
69             for (x in 0 until width) {
70                 val red = rgba[source++].toInt() and 0xFF
71                 val green = rgba[source++].toInt() and 0xFF
72                 val blue = rgba[source++].toInt() and 0xFF
73                 val alpha = rgba[source++].toInt() and 0xFF
74                 pixels[destination + x] =
75                     (alpha shl 24) or (red shl 16) or (green shl 8) or blue
76             }
77         }
78
79         output.parentFile?.mkdirs()
80         val bitmap = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888)
81         try {
82             bitmap.setPixels(pixels, 0, width, 0, 0, width, height)
83             output.outputStream().use { stream ->
84                 check(bitmap.compress(webpFormat(lossless), quality(lossless), stream)) {
85                     "WebP encoding failed for ${output.absolutePath}"
86                 }
87             }
88         } finally {
89             bitmap.recycle()
90         }
91     }
92 }

```

```

91     }
92
93     private fun webpFormat(lossless: Boolean): Bitmap.CompressFormat = when {
94         lossless && Build.VERSION.SDK_INT >= Build.VERSION_CODES.R ->
95             Bitmap.CompressFormat.WEBP_LOSSLESS
96         !lossless && Build.VERSION.SDK_INT >= Build.VERSION_CODES.R ->
97             Bitmap.CompressFormat.WEBP_LOSSY
98         else -> Bitmap.CompressFormat.WEBP
99     }
100
101     private fun quality(lossless: Boolean): Int = if (lossless) 100 else PROJECT_PREVIEW_QUALITY
102
103     private companion object {
104         const val RGBA_BYTES_PER_PIXEL = 4
105         const val PROJECT_PREVIEW_QUALITY = 88
106     }
107 }

```

75. app/src/main/java/com/wetinknext/engine/thumbnail/ThumbnailPreviewFiles.kt

```

1  package com.wetinknext.engine.thumbnail
2
3  import java.io.File
4  import java.io.IOException
5  import java.nio.file.AtomicMoveNotSupportedException
6  import java.nio.file.Files
7  import java.nio.file.StandardCopyOption
8
9  /** Cache-file contract shared by preview restoration and the GL build scheduler. */
10 object ThumbnailPreviewFiles {
11     fun layerPreviewFile(outputDirectory: File, layerId: Long): File =
12         File(File(outputDirectory, LAYER_PREVIEW_DIRECTORY), "$layerId.webp")
13
14     /**
15      * Restores previews unpacked from a .wetink container to cache atomically.
16      * It deliberately performs no bitmap decoding, so this work remains cheap
17      * and safe to execute before the GL session is ready.
18      */
19     fun restoreLayerPreviews(
20         outputDirectory: File,
21         previews: Map<Long, ByteArray>,
22     ): Map<Long, File> = previews.mapNotNull { (layerId, bytes) ->
23         if (bytes.isEmpty()) return@mapNotNull null
24         val target = layerPreviewFile(outputDirectory, layerId)
25         val parent = target.parentFile ?: return@mapNotNull null
26         if (!parent.exists() && !parent.mkdirs()) {
27             throw IOException("Cannot create preview directory: ${parent.path}")
28         }
29         val temporary = File(parent, ".${target.name}.restore.tmp")
30         try {
31             temporary.outputStream().use { it.write(bytes) }
32             publish(temporary, target)
33             layerId to target
34         } finally {
35             if (temporary.exists()) temporary.delete()
36         }
37     }.toMap()
38
39     private fun publish(temporary: File, destination: File) {
40         try {
41             Files.move(
42                 temporary.toPath(),
43                 destination.toPath(),
44                 StandardCopyOption.ATOMIC_MOVE,
45                 StandardCopyOption.REPLACE_EXISTING,
46             )
47         } catch (_: AtomicMoveNotSupportedException) {
48             Files.move(
49                 temporary.toPath(),
50                 destination.toPath(),
51                 StandardCopyOption.REPLACE_EXISTING,
52             )
53         }
54     }
55
56     private const val LAYER_PREVIEW_DIRECTORY = "previews"
57 }

```

76. app/src/main/java/com/wetinknext/engine/thumbnail/ThumbnailRenderer.kt

```

1  package com.wetinknext.engine.thumbnail
2
3  import android.opengl.GLES30
4  import com.wetinknext.engine.canvas.Compositor
5  import com.wetinknext.engine.canvas.LayerStack
6  import com.wetinknext.engine.canvas.PaintLayer
7  import com.wetinknext.engine.gl.CanvasGeometry
8  import com.wetinknext.engine.gl.GlCheck
9  import com.wetinknext.engine.gl.RenderTarget
10 import java.nio.ByteBuffer
11 import java.nio.ByteOrder
12
13 /**
14  * GL-thread renderer for document thumbnails.
15  *
16  * It renders directly from document render targets, never from the surface
17  * framebuffer. Camera position, zoom, editor chrome and Android system UI can
18  * therefore never leak into a preview.
19  */
20 class ThumbnailRenderer {
21     private val target = RenderTarget()
22     private val compositor = Compositor()
23     private var geometry: CanvasGeometry? = null
24     private var geometryWidth = 0
25     private var geometryHeight = 0
26     private val canvasToClip = FloatArray(16)
27     private var readBuffer: ByteBuffer? = null
28     private var created = false
29
30     /**
31      * Returns top-to-bottom RGBA8 bytes for one layer.
32      *
33      * A hidden layer still receives a preview of its own pixels; the visibility
34      * flag belongs to project composition, whereas a layer thumbnail is used to
35      * identify that layer in the layers panel. Layer opacity is preserved.
36      */
37     fun renderLayer(
38         layer: PaintLayer,
39         outputWidth: Int,
40         outputHeight: Int,
41     ): ByteArray {
42         GlCheck.checkOnGUIThread()
43         require(outputWidth > 0 && outputHeight > 0) { "Thumbnail size must be positive" }
44         if (!layer.created || layer.target.width <= 0 || layer.target.height <= 0) {
45             return ByteArray(outputWidth * outputHeight * RGBA_BYTES_PER_PIXEL)
46         }
47
48         prepare(
49             canvasWidth = layer.target.width,
50             canvasHeight = layer.target.height,
51             outputWidth = outputWidth,
52             outputHeight = outputHeight,
53         )
54         target.clear(0f, 0f, 0f, 0f)
55         target.bind()
56         GLES30.glViewport(0, 0, outputWidth, outputHeight)
57         try {
58             compositor.renderLayer(
59                 geometry = checkNotNull(geometry),
60                 layer = layer,
61                 canvasWidth = layer.target.width,
62                 canvasHeight = layer.target.height,
63                 canvasToClip = canvasToClip,
64             )
65             return readTopDownRgba(outputWidth, outputHeight)
66         } finally {
67             GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
68         }
69     }
70
71     /**
72      * Returns a top-to-bottom RGBA8 image of all visible layers, from the
73      * bottom of [layers] to the top. The compositor applies every layer's
74      * opacity and follows the same blend-policy path as the on-canvas view.
75      */
76     fun renderProject(
77         layers: LayerStack,
78         outputWidth: Int,
79         outputHeight: Int,
80     ): ByteArray {
81         GlCheck.checkOnGUIThread()
82         require(outputWidth > 0 && outputHeight > 0) { "Thumbnail size must be positive" }
83         require(layers.canvasWidth > 0 && layers.canvasHeight > 0) {
84             "Cannot render a thumbnail for an empty canvas"
85         }
86
87         prepare(
88             canvasWidth = layers.canvasWidth,
89             canvasHeight = layers.canvasHeight,
90             outputWidth = outputWidth,

```

```

91         outputHeight = outputHeight,
92     )
93     target.clear(0f, 0f, 0f, 0f)
94     target.bind()
95     GLES30.glViewport(0, 0, outputWidth, outputHeight)
96     try {
97         compositor.render(
98             geometry = checkNotNull(geometry),
99             layers = layers,
100             activeLayerId = NO_ACTIVE_LAYER,
101             strokeTextureId = 0,
102             canvasToClip = canvasToClip,
103             strokeOpacity = 1f,
104         )
105         return readTopDownRgba(outputWidth, outputHeight)
106     } finally {
107         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
108     }
109 }
110
111 /** Releases GL objects. Call while the context is current. */
112 fun release() {
113     GLCheck.checkOnGThread()
114     readBuffer = null
115     target.release()
116     geometry?.release()
117     geometry = null
118     geometryWidth = 0
119     geometryHeight = 0
120     if (created) compositor.release()
121     created = false
122 }
123
124 private fun prepare(
125     canvasWidth: Int,
126     canvasHeight: Int,
127     outputWidth: Int,
128     outputHeight: Int,
129 ) {
130     if (!created) {
131         compositor.create()
132         created = true
133     }
134     if (geometry == null || geometryWidth != canvasWidth || geometryHeight != canvasHeight) {
135         geometry?.release()
136         geometry = CanvasGeometry().also { it.create(canvasWidth, canvasHeight) }
137         geometryWidth = canvasWidth
138         geometryHeight = canvasHeight
139     }
140     if (target.width != outputWidth || target.height != outputHeight) {
141         target.create(outputWidth, outputHeight, preferHalfFloat = false)
142     }
143     buildAspectFitMatrix(canvasWidth, canvasHeight, outputWidth, outputHeight, canvasToClip)
144 }
145
146 private fun readTopDownRgba(width: Int, height: Int): ByteArray {
147     val rowBytes = width * RGBA_BYTES_PER_PIXEL
148     val byteCount = rowBytes * height
149     val buffer = obtainReadBuffer(byteCount)
150     buffer.clear()
151     GLES30.glPixelStorei(GLES30.GL_PACK_ALIGNMENT, 1)
152     try {
153         GLES30.glReadPixels(
154             0,
155             0,
156             width,
157             height,
158             GLES30.GL_RGBA,
159             GLES30.GL_UNSIGNED_BYTE,
160             buffer,
161         )
162         GLCheck.noError("ThumbnailRenderer glReadPixels")
163
164         val pixels = ByteArray(byteCount)
165         for (destinationRow in 0 until height) {
166             val sourceOffset = (height - 1 - destinationRow) * rowBytes
167             buffer.position(sourceOffset)
168             buffer.get(pixels, destinationRow * rowBytes, rowBytes)
169         }
170         return pixels
171     } finally {
172         GLES30.glPixelStorei(GLES30.GL_PACK_ALIGNMENT, 4)
173     }
174 }
175
176 private fun obtainReadBuffer(size: Int): ByteBuffer {
177     val current = readBuffer
178     if (current != null && current.capacity() >= size) return current
179     return ByteBuffer.allocateDirect(size)
180         .order(ByteOrder.nativeOrder())
181         .also { readBuffer = it }
182 }

```

```

183
184     private fun buildAspectFitMatrix(
185         canvasWidth: Int,
186         canvasHeight: Int,
187         outputWidth: Int,
188         outputHeight: Int,
189         out: FloatArray,
190     ) {
191         val canvasAspect = canvasWidth.toFloat() / canvasHeight.toFloat()
192         val outputAspect = outputWidth.toFloat() / outputHeight.toFloat()
193         val scaleX = if (canvasAspect < outputAspect) canvasAspect / outputAspect else 1f
194         val scaleY = if (canvasAspect > outputAspect) outputAspect / canvasAspect else 1f
195
196         out.fill(0f)
197         out[0] = 2f * scaleX / canvasWidth.toFloat()
198         out[5] = 2f * scaleY / canvasHeight.toFloat()
199         out[10] = 1f
200         out[12] = -scaleX
201         out[13] = -scaleY
202         out[15] = 1f
203     }
204
205     private companion object {
206         const val RGBA_BYTES_PER_PIXEL = 4
207         const val NO_ACTIVE_LAYER = -1L
208     }
209 }

```

77. app/src/main/java/com/wetinknext/engine/undo/DeflateTileCompressor.kt

```

1  package com.wetinknext.engine.undo
2
3  import java.io.ByteArrayOutputStream
4  import java.util.zip.Deflater
5  import java.util.zip.Inflater
6
7  class DeflateTileCompressor(
8      private val level: Int = Deflater.BEST_SPEED,
9  ) : TileCompressor {
10
11      override fun compress(data: ByteArray): ByteArray {
12          val deflater = Deflater(level)
13          return try {
14              deflater.setInput(data)
15              deflater.finish()
16
17              val output = ByteArrayOutputStream(data.size)
18              val buffer = ByteArray(16 * 1024)
19
20              while (!deflater.finished()) {
21                  val count = deflater.deflate(buffer)
22                  output.write(buffer, 0, count)
23              }
24
25              output.toByteArray()
26          } finally {
27              deflater.end()
28          }
29      }
30
31      override fun decompress(
32          data: ByteArray,
33          expectedSize: Int,
34      ): ByteArray {
35          val inflater = Inflater()
36          return try {
37              inflater.setInput(data)
38
39              val output = ByteArrayOutputStream(expectedSize)
40              val buffer = ByteArray(16 * 1024)
41
42              while (!inflater.finished()) {
43                  val count = inflater.inflate(buffer)
44                  if (count == 0 && inflater.finished()) break
45                  check(count > 0) { "Decompression stalled or data corrupted" }
46                  output.write(buffer, 0, count)
47              }
48
49              output.toByteArray()
50          } finally {
51              inflater.end()
52          }
53      }
54  }

```

78. app/src/main/java/com/wetinknext/engine/undo/RawTileSnapshot.kt

```
1 package com.wetinknext.engine.undo
2
3 /** Uncompressed tile data captured on the GL thread. */
4 class RawTileSnapshot(
5     val coord: TileCoord,
6     val pixelLeft: Int,
7     val pixelTop: Int,
8     val pixelWidth: Int,
9     val pixelHeight: Int,
10    val bytesPerPixel: Int,
11    val rawBytes: ByteArray,
12 ) {
13     val rawSize: Int get() = rawBytes.size
14
15     fun compress(compressor: TileCompressor): TileSnapshot {
16         return TileSnapshot(
17             coord = coord,
18             pixelLeft = pixelLeft,
19             pixelTop = pixelTop,
20             pixelWidth = pixelWidth,
21             pixelHeight = pixelHeight,
22             bytesPerPixel = bytesPerPixel,
23             storedData = compressor.compress(rawBytes),
24             compressor = compressor,
25         )
26     }
27 }
```

79. app/src/main/java/com/wetinknext/engine/undo/TileCompressor.kt

```
1 package com.wetinknext.engine.undo
2
3 /**
4  * Extension point for P6.5 compression. P6 uses the identity implementation,
5  * so the 96 MB budget reflects the exact bytes that are currently retained.
6  */
7 interface TileCompressor {
8     fun compress(data: ByteArray): ByteArray
9
10    fun decompress(data: ByteArray, expectedSize: Int): ByteArray
11 }
12
13 /** P6 storage policy: no compression and no extra byte-array allocation. */
14 object IdentityTileCompressor : TileCompressor {
15     override fun compress(data: ByteArray): ByteArray = data
16
17     override fun decompress(data: ByteArray, expectedSize: Int): ByteArray {
18         check(data.size == expectedSize) {
19             "Raw tile has ${data.size} bytes, expected $expectedSize"
20         }
21         return data
22     }
23 }
```

80. app/src/main/java/com/wetinknext/engine/undo/TileSnapshot.kt

```
1 package com.wetinknext.engine.undo
2
3 data class TileCoord(val tx: Int, val ty: Int)
4
5 /** One full 256 px tile (or an edge tile) in the source layer's native format. */
6 class TileSnapshot(
7     val coord: TileCoord,
8     val pixelLeft: Int,
9     val pixelTop: Int,
10    val pixelWidth: Int,
11    val pixelHeight: Int,
12    val bytesPerPixel: Int,
13    private val storedData: ByteArray,
14    private val compressor: TileCompressor = IdentityTileCompressor,
15 ) {
16     val memorySize: Int get() = storedData.size
17     val rawSize: Int get() = pixelWidth * pixelHeight * bytesPerPixel
18
19     /** Returns raw bytes. */
20     fun decompress(): ByteArray = compressor.decompress(storedData, rawSize)
21
22     fun dispose() {
23         // P6 keeps heap ByteArrays; a later off-heap implementation can release here.
24     }
25
26     companion object {
27         const val TILE_SIZE = 256
28         const val BYTES_PER_PIXEL_RGBA16F = 8
29         const val BYTES_PER_PIXEL_RGBA8 = 4
30     }
31 }
```

81. app/src/main/java/com/wetinknext/engine/undo/TileSnapshotCapture.kt

```

1 package com.wetinknext.engine.undo
2 import android.opengl.GLES30
3 import android.util.Log
4 import com.wetinknext.BuildConfig
5 import com.wetinknext.engine.gl.GlCheck
6 import com.wetinknext.engine.gl.RenderTarget
7 import java.nio.ByteBuffer
8 import java.nio.ByteOrder
9 object TileSnapshotCapture {
10     private var reusable: ByteBuffer? = null
11
12     /** Captures every 256 px tile intersecting [bounds] = left, top, right, bottom. */
13     fun capture(target: RenderTarget, bounds: IntArray): List<RawTileSnapshot> {
14         GlCheck.checkOnGThread()
15         require(target.framebufferId != 0)
16         require(target.textureId != 0)
17         require(bounds.size >= 4)
18         val left = bounds[0].coerceIn(0, target.width)
19         val top = bounds[1].coerceIn(0, target.height)
20         val right = bounds[2].coerceIn(0, target.width)
21         val bottom = bounds[3].coerceIn(0, target.height)
22         if (right <= left || bottom <= top) return emptyList()
23
24         if (BuildConfig.DEBUG) {
25             Log.d(
26                 TAG,
27                 "capture target=${target.textureId} fbo=${target.framebufferId} " +
28                 "canvasBounds=[$left,$top,$right,$bottom] " +
29                 "target=${target.width}x${target.height}",
30             )
31         }
32
33         val bytesPerPixel = if (target.usesHalfFloat) {
34             TileSnapshot.BYTES_PER_PIXEL_RGBA16F
35         } else {
36             TileSnapshot.BYTES_PER_PIXEL_RGBA8
37         }
38         val glType = if (target.usesHalfFloat) GLES30.GL_HALF_FLOAT else GLES30.GL_UNSIGNED_BYTE
39         val result = ArrayList<RawTileSnapshot>()
40         target.bind()
41         GLES30.glPixelStorei(GLES30.GL_PACK_ALIGNMENT, 1)
42         try {
43             for (tileY in top / TileSnapshot.TILE_SIZE..(bottom - 1) / TileSnapshot.TILE_SIZE) {
44                 for (tileX in left / TileSnapshot.TILE_SIZE..(right - 1) / TileSnapshot.TILE_SIZE) {
45                     val pixelLeft = tileX * TileSnapshot.TILE_SIZE
46                     // Tile coordinates are document/canvas coordinates (Y grows down).
47                     val canvasTop = tileY * TileSnapshot.TILE_SIZE
48                     val pixelWidth = minOf(TileSnapshot.TILE_SIZE, target.width - pixelLeft)
49                     val pixelHeight = minOf(TileSnapshot.TILE_SIZE, target.height - canvasTop)
50                     // buildCanvasToFbo maps canvas y=0 to the FBO's bottom row.
51                     // Canvas tile coordinates therefore already equal GL's bottom Y.
52                     val glBottom = canvasTop
53                     if (BuildConfig.DEBUG) {
54                         Log.d(
55                             TAG,
56                             "capture tile=($tileX,$tileY) " +
57                             "canvas=[$pixelLeft,$canvasTop,$pixelWidth,$pixelHeight] " +
58                             "gl=[$pixelLeft,$glBottom,$pixelWidth,$pixelHeight]",
59                         )
60                     }
61                     val byteCount = pixelWidth * pixelHeight * bytesPerPixel
62                     val buffer = obtainBuffer(byteCount)
63                     buffer.clear()
64                     GLES30.glReadPixels(
65                         pixelLeft, glBottom, pixelWidth, pixelHeight,
66                         GLES30.GL_RGBA, glType, buffer,
67                     )
68                     if (BuildConfig.DEBUG) {
69                         val error = GLES30.glGetError()
70                         check(error == GLES30.GL_NO_ERROR) {
71                             "TileSnapshotCapture.glReadPixels failed: 0x${error.toString(16)}"
72                         }
73                     }
74                     buffer.rewind()
75                     val rawBytes = ByteArray(byteCount)
76                     buffer.get(rawBytes)
77                     result += RawTileSnapshot(
78                         coord = TileCoord(tileX, tileY),
79                         pixelLeft = pixelLeft,
80                         pixelTop = canvasTop,
81                         pixelWidth = pixelWidth,
82                         pixelHeight = pixelHeight,
83                         bytesPerPixel = bytesPerPixel,
84                         rawBytes = rawBytes,
85                     )
86                 }
87             }
88             return result
89         } finally {
90             GLES30.glPixelStorei(GLES30.GL_PACK_ALIGNMENT, 4)

```

```
91         GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
92     }
93 }
94
95 private fun obtainBuffer(size: Int): ByteBuffer {
96     val current = reusable
97     if (current != null && current.capacity() >= size) return current
98     return ByteBuffer.allocateDirect(size).order(ByteOrder.nativeOrder()).also { reusable = it }
99 }
100
101 private const val TAG = "TileUndo"
102 }
```

82. app/src/main/java/com/wetinknext/engine/undo/TileSnapshotRestore.kt

```

1  package com.wetinknext.engine.undo
2
3  import android.opengl.GLES30
4  import android.util.Log
5  import com.wetinknext.BuildConfig
6  import com.wetinknext.engine.gl.GlCheck
7  import com.wetinknext.engine.gl.RenderTarget
8  import java.nio.ByteBuffer
9  import java.nio.ByteOrder
10
11 /** Restores snapshot tiles into an already-created layer texture on the GL thread. */
12 object TileSnapshotRestore {
13     private var reusableBuffer: ByteBuffer? = null
14
15     fun restore(target: RenderTarget, tiles: List<TileSnapshot>): Boolean {
16         GlCheck.checkOnGLThread()
17         if (target.textureId == 0) return false
18         if (tiles.isEmpty()) return true
19
20         // Discard an unrelated previous error, then verify every upload below.
21         GLES30.glGetError()
22         GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
23         GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, target.textureId)
24         GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 1)
25         try {
26             for (tile in tiles) {
27                 if (!isValidTile(target, tile)) return false
28                 val data = tile.decompress()
29                 if (data.size != tile.rawSize) return false
30
31                 val glType = when (tile.bytesPerPixel) {
32                     TileSnapshot.BYTES_PER_PIXEL_RGBA16F -> GLES30.GL_HALF_FLOAT
33                     TileSnapshot.BYTES_PER_PIXEL_RGBA8 -> GLES30.GL_UNSIGNED_BYTE
34                     else -> return false
35                 }
36                 val buffer = obtainBuffer(data.size)
37                 buffer.clear()
38                 buffer.put(data)
39                 buffer.position(0)
40
41                 // buildCanvasToFbo maps canvas y=0 to the FBO's bottom row.
42                 val glBottom = tile.pixelTop
43                 if (BuildConfig.DEBUG) {
44                     Log.d(TAG, "restore tex=${target.textureId} tile=(${tile.coord.tx},${tile.coord.ty}) " +
45                         "canvas=(${tile.pixelLeft},${tile.pixelTop},${tile.pixelWidth},${tile.pixelHeight})")
46                 }
47                 GLES30.glTexSubImage2D(
48                     GLES30.GL_TEXTURE_2D, 0, tile.pixelLeft, glBottom,
49                     tile.pixelWidth, tile.pixelHeight, GLES30.GL_RGBA, glType, buffer,
50                 )
51                 if (GLES30.glGetError() != GLES30.GL_NO_ERROR) return false
52             }
53             return true
54         } catch (error: Throwable) {
55             if (BuildConfig.DEBUG) Log.e(TAG, "Restore failed", error)
56             return false
57         } finally {
58             GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 4)
59             GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, 0)
60             GLES30.glActiveTexture(GLES30.GL_TEXTURE0)
61             GLES30.glBindFramebuffer(GLES30.GL_FRAMEBUFFER, 0)
62         }
63     }
64
65     private fun isValidTile(target: RenderTarget, tile: TileSnapshot): Boolean {
66         if (tile.pixelWidth <= 0 || tile.pixelHeight <= 0) return false
67         if (tile.pixelLeft < 0 || tile.pixelTop < 0) return false
68         if (tile.pixelLeft + tile.pixelWidth > target.width ||
69             tile.pixelTop + tile.pixelHeight > target.height) return false
70         val expectedBytesPerPixel = if (target.usesHalfFloat) {
71             TileSnapshot.BYTES_PER_PIXEL_RGBA16F
72         } else {
73             TileSnapshot.BYTES_PER_PIXEL_RGBA8
74         }
75         return tile.bytesPerPixel == expectedBytesPerPixel
76     }
77
78     private fun obtainBuffer(size: Int): ByteBuffer {
79         val current = reusableBuffer
80         if (current != null && current.capacity() >= size) return current
81         return ByteBuffer.allocateDirect(size).order(ByteOrder.nativeOrder()).also { reusableBuffer = it }
82     }
83
84     private const val TAG = "TileUndo"
85 }

```

83. app/src/main/java/com/wetinknext/engine/undo/UndoEntry.kt

```
1 package com.wetinknext.engine.undo
2
3 /** One reversible history operation, retaining exact states from before and after its merge. */
4 data class UndoEntry(
5     val layerId: Long,
6     val beforeTiles: List<TileSnapshot>,
7     val afterTiles: List<TileSnapshot>,
8     val operation: UndoOperationType = UndoOperationType.TILE_EDIT,
9     val tag: String = "stroke",
10 ) {
11     val memorySize: Int
12     get() = beforeTiles.sumOf(TileSnapshot::memorySize) + afterTiles.sumOf(TileSnapshot::memorySize)
13
14     fun dispose() {
15         beforeTiles.forEach(TileSnapshot::dispose)
16         afterTiles.forEach(TileSnapshot::dispose)
17     }
18 }
```

84. app/src/main/java/com/wetinknext/engine/undo/UndoManager.kt

```

1 package com.wetinknext.engine.undo
2
3 /**
4  * Pure memory-bounded history. GPU reads and texture restores deliberately live
5  * in EngineRenderer, keeping this class deterministic and unit-testable.
6  */
7 class UndoManager(
8     private val maxMemoryBytes: Long = DEFAULT_MAX_MEMORY_BYTES,
9     private val maxSteps: Int = DEFAULT_MAX_STEPS,
10 ) {
11     private val undoStack = ArrayDeque<UndoEntry>()
12     private val redoStack = ArrayDeque<UndoEntry>()
13     private var undoMemoryBytes = 0L
14     private var redoMemoryBytes = 0L
15
16     val canUndo: Boolean get() = undoStack.isNotEmpty()
17     val canRedo: Boolean get() = redoStack.isNotEmpty()
18     val memoryBytes: Long get() = undoMemoryBytes + redoMemoryBytes
19     val undoCount: Int get() = undoStack.size
20     val redoCount: Int get() = redoStack.size
21
22     /** Records a new operation and invalidates the alternate redo history. */
23     fun push(entry: UndoEntry) {
24         undoStack.addLast(entry)
25         undoMemoryBytes += entry.memorySize
26         clearRedo()
27         trimToLimits()
28     }
29
30     /** Returns the candidate without changing history; restore it before committing. */
31     fun peekUndo(): UndoEntry? = undoStack.lastOrNull()
32
33     /** Moves the exact current undo candidate to redo after a successful restore. */
34     fun commitUndo(entry: UndoEntry): Boolean {
35         if (undoStack.lastOrNull() != entry) return false
36         undoStack.removeLast()
37         undoMemoryBytes -= entry.memorySize
38         redoStack.addLast(entry)
39         redoMemoryBytes += entry.memorySize
40         trimToLimits()
41         return true
42     }
43
44     /** Discards a no-longer-restorable undo record without moving it to redo. */
45     fun dropUndo(entry: UndoEntry): Boolean {
46         if (undoStack.lastOrNull() != entry) return false
47         undoStack.removeLast()
48         undoMemoryBytes -= entry.memorySize
49         entry.dispose()
50         return true
51     }
52
53     /** Returns the candidate without changing history; restore it before committing. */
54     fun peekRedo(): UndoEntry? = redoStack.lastOrNull()
55
56     /** Moves the exact current redo candidate to undo after a successful restore. */
57     fun commitRedo(entry: UndoEntry): Boolean {
58         if (redoStack.lastOrNull() != entry) return false
59         redoStack.removeLast()
60         redoMemoryBytes -= entry.memorySize
61         undoStack.addLast(entry)
62         undoMemoryBytes += entry.memorySize
63         trimToLimits()
64         return true
65     }
66
67     /** Discards a no-longer-restorable redo record without moving it to undo. */
68     fun dropRedo(entry: UndoEntry): Boolean {
69         if (redoStack.lastOrNull() != entry) return false
70         redoStack.removeLast()
71         redoMemoryBytes -= entry.memorySize
72         entry.dispose()
73         return true
74     }
75
76     fun clearRedo() {
77         while (redoStack.isNotEmpty()) {
78             val entry = redoStack.removeLast()
79             redoMemoryBytes -= entry.memorySize
80             entry.dispose()
81         }
82         redoMemoryBytes = 0L
83     }
84
85     fun clear() {
86         while (undoStack.isNotEmpty()) undoStack.removeLast().dispose()
87         undoMemoryBytes = 0L
88         clearRedo()
89     }
90

```

```

91  /** Removes history that can no longer be restored after deleting a layer. */
92  fun removeEntriesForLayer(layerId: Long) {
93      removeEntriesForLayer(undoStack, layerId) { entry -> undoMemoryBytes -= entry.memorySize }
94      removeEntriesForLayer(redoStack, layerId) { entry -> redoMemoryBytes -= entry.memorySize }
95      undoMemoryBytes = undoMemoryBytes.coerceAtLeast(0L)
96      redoMemoryBytes = redoMemoryBytes.coerceAtLeast(0L)
97  }
98
99  private fun removeEntriesForLayer(
100      stack: ArrayDeque<UndoEntry>,
101      layerId: Long,
102      subtractMemory: (UndoEntry) -> Unit,
103  ) {
104      if (stack.isEmpty()) return
105      val retained = ArrayDeque<UndoEntry>(stack.size)
106      while (stack.isNotEmpty()) {
107          val entry = stack.removeFirst()
108          if (entry.layerId == layerId) {
109              subtractMemory(entry)
110              entry.dispose()
111          } else {
112              retained.addLast(entry)
113          }
114      }
115      stack.addAll(retained)
116  }
117
118  private fun trimToLimits() {
119      while (undoStack.size > maxSteps) evictOldestUndo()
120      while (redoStack.size > maxSteps) evictOldestRedo()
121      while (memoryBytes > maxMemoryBytes && undoStack.size + redoStack.size > 1) {
122          when {
123              redoStack.size > 1 && redoMemoryBytes >= undoMemoryBytes -> evictOldestRedo()
124              undoStack.size > 1 -> evictOldestUndo()
125              redoStack.size > 1 -> evictOldestRedo()
126              else -> return
127          }
128      }
129  }
130
131  private fun evictOldestUndo() {
132      val entry = undoStack.removeFirstOrNull() ?: return
133      undoMemoryBytes -= entry.memorySize
134      entry.dispose()
135  }
136
137  private fun evictOldestRedo() {
138      val entry = redoStack.removeFirstOrNull() ?: return
139      redoMemoryBytes -= entry.memorySize
140      entry.dispose()
141  }
142
143  companion object {
144      const val DEFAULT_MAX_MEMORY_BYTES = 96L * 1024L * 1024L
145      const val DEFAULT_MAX_STEPS = 50
146  }
147 }

```

85. app/src/main/java/com/wetinknext/engine/undo/UndoOperationType.kt

```
1 package com.wetinknext.engine.undo
2
3 /**
4  * A lightweight operation discriminator for the current tile-snapshot history.
5  * Future history entries can replace this with full typed operations without
6  * changing existing stored tile edits.
7  */
8 enum class UndoOperationType {
9     TILE_EDIT,
10    CLEAR_LAYER,
11    LAYER_PROPERTIES,
12 }
```

86. app/src/main/java/com/wetinknext/ui/animation/AnimationSettingsMenu.kt

```

1  package com.wetinknext.ui.animation
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.*
7  import androidx.compose.foundation.rememberScrollState
8  import androidx.compose.foundation.shape.CircleShape
9  import androidx.compose.foundation.verticalScroll
10 import androidx.compose.foundation.shape.RoundedCornerShape
11 import androidx.compose.material3.*
12 import androidx.compose.runtime.*
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.draw.clip
16 import androidx.compose.ui.graphics.Color
17 import androidx.compose.ui.graphics.toArgb
18 import androidx.compose.ui.text.font.FontWeight
19 import androidx.compose.ui.unit.dp
20 import androidx.compose.ui.unit.sp
21 import com.wetinknext.domain.animation.AnimationDocument
22 import com.wetinknext.domain.animation.AnimationPlaybackMode
23 import com.wetinknext.domain.animation.OnionSkinSettings
24 import com.wetinknext.ui.theme.AppTheme
25
26 @Composable
27 fun AnimationSettingsMenu(
28     theme: AppTheme,
29     document: AnimationDocument,
30     onDocumentChange: (AnimationDocument) -> Unit,
31     onDismiss: () -> Unit,
32     modifier: Modifier = Modifier
33 ) {
34     Box(
35         modifier = modifier
36             .width(280.dp)
37             .heightIn(max = 360.dp)
38             .clip(RoundedCornerShape(24.dp))
39             .background(theme.panelBg.copy(alpha = 0.98f))
40             .border(1.dp, theme.panelStroke, RoundedCornerShape(24.dp))
41             .padding(16.dp)
42     ) {
43         Column(
44             modifier = Modifier.verticalScroll(rememberScrollState()),
45             verticalArrangement = Arrangement.spacedBy(10.dp),
46         ) {
47             Text("Настройки анимации", color = theme.textPrimary, fontSize = 16.sp, fontWeight = FontWeight.Bold)
48
49             // Playback Mode
50             Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
51                 Text("Режим", color = theme.textSecondary, fontSize = 12.sp)
52                 Row(
53                     modifier = Modifier.fillMaxWidth(),
54                     horizontalArrangement = Arrangement.spacedBy(8.dp)
55                 ) {
56                     PlaybackModeButton("Цикл", AnimationPlaybackMode.LOOP, document.playbackMode, theme) {
57                         onDocumentChange(document.copy(playbackMode = it))
58                     }
59                     PlaybackModeButton("Туда-сюда", AnimationPlaybackMode.PING_PONG, document.playbackMode, theme) {
60                         onDocumentChange(document.copy(playbackMode = it))
61                     }
62                     PlaybackModeButton("Один раз", AnimationPlaybackMode.ONE_SHOT, document.playbackMode, theme) {
63                         onDocumentChange(document.copy(playbackMode = it))
64                     }
65                 }
66             }
67
68             // FPS
69             Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
70                 Row(horizontalArrangement = Arrangement.SpaceBetween, modifier = Modifier.fillMaxWidth()) {
71                     Text("Скорость (FPS)", color = theme.textSecondary, fontSize = 12.sp)
72                     Text("${document.framesPerSecond}", color = theme.accent, fontSize = 12.sp, fontWeight = FontWeight.Bold)
73                 }
74                 Slider(
75                     value = document.framesPerSecond.toFloat(),
76                     onValueChange = { onDocumentChange(document.copy(framesPerSecond = it.toInt())) },
77                     valueRange = 1f..60f,
78                     colors = SliderDefaults.colors(thumbColor = theme.accent, activeTrackColor = theme.accent)
79                 )
80             }
81
82             HorizontalDivider(color = theme.panelStroke)
83
84             // Onion Skin Settings
85             Row(
86                 modifier = Modifier.fillMaxWidth(),
87                 horizontalArrangement = Arrangement.SpaceBetween,
88                 verticalAlignment = Alignment.CenterVertically
89             ) {
90                 Text("Калька (Onion Skin)", color = theme.textPrimary, fontSize = 14.sp)

```

```

91         Switch(
92             checked = document.onionSkin.enabled,
93             onCheckedChange = { onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(enabled = it))) },
94             colors = SwitchDefaults.colors(checkedThumbColor = theme.accent, checkedTrackColor = theme.accent.copy(
alpha = 0.5f))
95         )
96     }
97
98     if (document.onionSkin.enabled) {
99         OnionCountSlider("Предыдущие", document.onionSkin.previousFrames, theme) {
100             onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(previousFrames = it)))
101         }
102         OnionColorPicker(
103             label = "Цвет предыдущих",
104             selectedArgb = document.onionSkin.previousColorArgb,
105             theme = theme,
106             ) { color ->
107             onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(previousColorArgb = color.toArgb())))
108         }
109         OnionCountSlider("Следующие", document.onionSkin.nextFrames, theme) {
110             onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(nextFrames = it)))
111         }
112         OnionColorPicker(
113             label = "Цвет следующих",
114             selectedArgb = document.onionSkin.nextColorArgb,
115             theme = theme,
116             ) { color ->
117             onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(nextColorArgb = color.toArgb())))
118         }
119
120         Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
121             Text("Прозрачность", color = theme.textSecondary, fontSize = 12.sp)
122             Slider(
123                 value = document.onionSkin.opacity,
124                 onValueChange = { onDocumentChange(document.copy(onionSkin = document.onionSkin.copy(opacity = it)))
125             },
126                 valueRange = 0f..1f,
127                 colors = SliderDefaults.colors(thumbColor = theme.accent, activeTrackColor = theme.accent)
128             )
129         }
130     }
131 }
132 }
133
134 @Composable
135 private fun RowScope.PlaybackModeButton(
136     label: String,
137     mode: AnimationPlaybackMode,
138     current: AnimationPlaybackMode,
139     theme: AppTheme,
140     onClick: (AnimationPlaybackMode) -> Unit
141 ) {
142     val selected = mode == current
143     Box(
144         modifier = Modifier
145             .weight(1f)
146             .height(32.dp)
147             .clip(RoundedCornerShape(8.dp))
148             .background(if (selected) theme.accent else theme.panelInsetSoft)
149             .clickable { onClick(mode) },
150         contentAlignment = Alignment.Center
151     ) {
152         Text(label, color = if (selected) Color.White else theme.textSecondary, fontSize = 10.sp)
153     }
154 }
155
156 @Composable
157 private fun OnionCountSlider(
158     label: String,
159     value: Int,
160     theme: AppTheme,
161     onValueChange: (Int) -> Unit
162 ) {
163     Column(verticalArrangement = Arrangement.spacedBy(2.dp)) {
164         Row(horizontalArrangement = Arrangement.SpaceBetween, modifier = Modifier.fillMaxWidth()) {
165             Text(label, color = theme.textSecondary, fontSize = 11.sp)
166             Text("$value", color = theme.textPrimary, fontSize = 11.sp)
167         }
168         Slider(
169             value = value.toFloat(),
170             onValueChange = { onValueChange(it.toInt()) },
171             valueRange = 0f..12f,
172             colors = SliderDefaults.colors(thumbColor = theme.accent, activeTrackColor = theme.accent)
173         )
174     }
175 }
176
177 @Composable
178 private fun OnionColorPicker(
179     label: String,
180     selectedArgb: Int,

```

```

181     theme: AppTheme,
182     onColorChange: (Color) -> Unit,
183 ) {
184     val palette = listOf(
185         Color(0xFFFFF4FA3),
186         Color(0xFFFFF7A59),
187         Color(0xFFFFFC857),
188         Color(0xFF57D68D),
189         Color(0xFF4A8DFF),
190         Color(0xFFB079FF),
191     )
192     Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
193         Text(label, color = theme.textSecondary, fontSize = 12.sp)
194         Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
195             palette.forEach { color ->
196                 val selected = color.toArgb() == selectedArgb
197                 Box(
198                     modifier = Modifier
199                         .size(24.dp)
200                         .clip(CircleShape)
201                         .background(color)
202                         .border(
203                             width = if (selected) 2.dp else 1.dp,
204                             color = if (selected) Color.White else theme.panelStroke,
205                             shape = CircleShape,
206                         )
207                     .clickable { onColorChange(color) }
208                 )
209             }
210         }
211     }
212 }

```

87. app/src/main/java/com/wetinknext/ui/animation/AnimationTimelineToolbar.kt

```

1 package com.wetinknext.ui.animation
2
3 import androidx.compose.foundation.Image
4 import androidx.compose.foundation.background
5 import androidx.compose.foundation.border
6 import androidx.compose.foundation.clickable
7 import androidx.compose.foundation.gestures.detectDragGesturesAfterLongPress
8 import androidx.compose.foundation.layout.*
9 import androidx.compose.foundation.lazy.LazyRow
10 import androidx.compose.foundation.lazy.itemsIndexed
11 import androidx.compose.foundation.lazy.rememberLazyListState
12 import androidx.compose.foundation.shape.CircleShape
13 import androidx.compose.foundation.shape.RoundedCornerShape
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.filled.*
16 import androidx.compose.material3.*
17 import androidx.compose.runtime.*
18 import androidx.compose.ui.Alignment
19 import androidx.compose.ui.Modifier
20 import androidx.compose.ui.draw.clip
21 import androidx.compose.ui.graphics.Color
22 import androidx.compose.ui.graphics.ImageBitmap
23 import androidx.compose.ui.graphics.graphicsLayer
24 import androidx.compose.ui.graphics.vector.ImageVector
25 import androidx.compose.ui.input.pointer.pointerInput
26 import androidx.compose.ui.layout.ContentScale
27 import androidx.compose.ui.platform.LocalDensity
28 import androidx.compose.ui.text.font.FontWeight
29 import androidx.compose.ui.unit.IntOffset
30 import androidx.compose.ui.unit.dp
31 import androidx.compose.ui.unit.sp
32 import androidx.compose.ui.window.Popup
33 import com.wetinknext.domain.animation.AnimationDocument
34 import com.wetinknext.domain.animation.AnimationFrame
35 import com.wetinknext.domain.animation.AnimationFrameRole
36 import com.wetinknext.ui.theme.AppTheme
37 import kotlin.math.roundToInt
38
39 data class FrameThumbnailLayer(
40     val image: ImageBitmap,
41     val opacity: Float,
42 )
43
44 @Composable
45 fun AnimationTimelineToolbar(
46     theme: AppTheme,
47     document: AnimationDocument,
48     currentFrameId: Long,
49     frameThumbnails: Map<Long, List<FrameThumbnailLayer>> = emptyMap(),
50     onFrameSelect: (Long) -> Unit,
51     onFrameMove: (frameId: Long, direction: Int) -> Unit,
52     onAddFrame: () -> Unit,
53     onDuplicateFrame: (Long) -> Unit,
54     onDeleteFrame: (Long) -> Unit,
55     onHoldChange: (Long, Int) -> Unit,
56     onRoleToggle: (Long, AnimationFrameRole) -> Unit,
57     onAssembleLayers: () -> Unit,
58     onLayersClick: () -> Unit,
59     onClose: () -> Unit,
60     onSettingsClick: () -> Unit = {},
61     isPlaying: Boolean = false,
62     onPlayToggle: () -> Unit = {},
63     modifier: Modifier = Modifier
64 ) {
65     val listState = rememberLazyListState()
66     val frames = document.frames
67     var showContextMenuForId by remember { mutableStateOf<Long?>(null) }
68
69     // Auto-scroll to current frame
70     LaunchedEffect(currentFrameId) {
71         val index = frames.indexOfFirst { it.id == currentFrameId }
72         if (index >= 0) {
73             listState.animateScrollToItem(index)
74         }
75     }
76
77     Column(
78         modifier = modifier
79             .fillMaxWidth()
80             .padding(bottom = 16.dp),
81         horizontalAlignment = Alignment.CenterHorizontally
82     ) {
83         Row(
84             modifier = Modifier
85                 .widthIn(min = 400.dp, max = 820.dp)
86                 .height(86.dp)
87                 .clip(RoundedCornerShape(28.dp))
88                 .background(theme.panelBg.copy(alpha = 0.94f))
89                 .border(1.2.dp, theme.panelStroke, RoundedCornerShape(28.dp))
90                 .padding(horizontal = 14.dp, vertical = 8.dp),

```

```

91         verticalAlignment = Alignment.CenterVertically
92     ) {
93         // Play toggle
94         Column(
95             horizontalAlignment = Alignment.CenterHorizontally,
96             modifier = Modifier.padding(end = 4.dp)
97         ) {
98             Box(
99                 modifier = Modifier
100                     .size(44.dp)
101                     .clip(CircleShape)
102                     .background(if (isPlaying) theme.accent.copy(alpha = 0.4f) else theme.accent)
103                     .clickable { onPlayToggle() },
104                 contentAlignment = Alignment.Center
105             ) {
106                 Icon(
107                     if (isPlaying) Icons.Default.Pause else Icons.Default.PlayArrow,
108                     null,
109                     tint = Color.White
110                 )
111             }
112             Text(if (isPlaying) "Пауза" else "Пуск", color = theme.textSecondary, fontSize = 9.sp)
113         }
114
115         Spacer(Modifier.width(10.dp))
116         Box(Modifier.width(1.dp).fillMaxHeight(0.6f).background(theme.panelStroke))
117         Spacer(Modifier.width(10.dp))
118
119         // Frames timeline
120         LazyRow(
121             state = listState,
122             modifier = Modifier.weight(1f),
123             horizontalArrangement = Arrangement.spacedBy(8.dp),
124             verticalAlignment = Alignment.CenterVertically
125         ) {
126             itemsIndexed(frames, key = { _, f -> f.id }) { _, frame ->
127                 TimelineFrameItem(
128                     frame = frame,
129                     isSelected = frame.id == currentFrameId,
130                     thumbnailLayers = frameThumbnails[frame.id].orEmpty(),
131                     theme = theme,
132                     canMoveLeft = frames.indexOf(frame) > 0,
133                     canMoveRight = frames.indexOf(frame) < frames.lastIndex,
134                     onMove = { direction -> onFrameMove(frame.id, direction) },
135                     onClick = {
136                         if (currentFrameId == frame.id) {
137                             showContextMenuForId = frame.id
138                         } else {
139                             onFrameSelect(frame.id)
140                         }
141                     }
142                 )
143             }
144         }
145
146         Spacer(Modifier.width(10.dp))
147         Box(Modifier.width(1.dp).fillMaxHeight(0.6f).background(theme.panelStroke))
148         Spacer(Modifier.width(10.dp))
149
150         // Actions
151         Row(
152             verticalAlignment = Alignment.CenterVertically,
153             horizontalArrangement = Arrangement.spacedBy(12.dp)
154         ) {
155             TimelineActionIcon(Icons.Default.Add, "Кадр", theme, onClick = onAddFrame)
156             TimelineActionIcon(Icons.Default.Layers, "Собрать", theme, onClick = onAssembleLayers)
157             TimelineActionIcon(Icons.Default.ContentCopy, "Копия", theme, onClick = { onDuplicateFrame(currentFrameId) })
158             TimelineActionIcon(Icons.Default.Delete, "Удалить", theme, enabled = frames.size > 1, onClick = {
onDeleteFrame(currentFrameId) })
159             TimelineActionIcon(Icons.Default.Settings, "Настр.", theme, onClick = onSettingsClick)
160             TimelineActionIcon(Icons.Default.Layers, "Слой", theme, onClick = onLayersClick)
161             TimelineActionIcon(Icons.Default.Close, "Закрыть", theme, onClick = onClose)
162         }
163     }
164
165     // Context Menu
166     showContextMenuForId?.let { frameId ->
167         val frame = frames.find { it.id == frameId }
168         if (frame != null) {
169             Popup(
170                 alignment = Alignment.TopCenter,
171                 offset = IntOffset(0, -220),
172                 onDismissRequest = { showContextMenuForId = null }
173             ) {
174                 FrameContextMenu(
175                     frame = frame,
176                     theme = theme,
177                     onDuplicate = { onDuplicateFrame(frameId); showContextMenuForId = null },
178                     onDelete = { onDeleteFrame(frameId); showContextMenuForId = null },
179                     onHoldChange = { onHoldChange(frameId, it) },
180                     onRoleToggle = { onRoleToggle(frameId, it); showContextMenuForId = null },
181                     canDelete = frames.size > 1

```

```

182     )
183     }
184     }
185     }
186     }
187 }
188
189 @Composable
190 private fun FrameContextMenu(
191     frame: AnimationFrame,
192     theme: AppTheme,
193     onDuplicate: () -> Unit,
194     onDelete: () -> Unit,
195     onHoldChange: (Int) -> Unit,
196     onRoleToggle: (AnimationFrameRole) -> Unit,
197     canDelete: Boolean
198 ) {
199     Surface(
200         modifier = Modifier
201             .width(220.dp)
202             .clip(RoundedCornerShape(16.dp))
203             .border(1.dp, theme.panelStroke, RoundedCornerShape(16.dp)),
204         color = theme.panelBg,
205         tonalElevation = 8.dp
206     ) {
207         Column(modifier = Modifier.padding(12.dp)) {
208             ContextMenuItem("Дублировать", Icons.Default.ContentCopy, theme, onClick = onDuplicate)
209             ContextMenuItem("Удалить", Icons.Default.Delete, theme, enabled = canDelete, onClick = onDelete)
210
211             HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp), color = theme.panelStroke)
212
213             Text("Задержка: ${frame.holdFrames}", color = theme.textSecondary, fontSize = 11.sp, modifier = Modifier.padding(
horizontal = 8.dp))
214             Slider(
215                 value = frame.holdFrames.toFloat(),
216                 onValueChange = { onHoldChange(it.toInt()) },
217                 valueRange = 1f..120f,
218                 colors = SliderDefaults.colors(thumbColor = theme.accent, activeTrackColor = theme.accent)
219             )
220
221             HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp), color = theme.panelStroke)
222
223             val isBg = frame.role == AnimationFrameRole.BACKGROUND
224             val isFg = frame.role == AnimationFrameRole.FOREGROUND
225
226             Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.fillMaxWidth().clickable { onRoleToggle(
if (isBg) AnimationFrameRole.NORMAL else AnimationFrameRole.BACKGROUND) }.padding(8.dp)) {
227                 Checkbox(checked = isBg, onCheckedChange = null, colors = CheckboxDefaults.colors(checkedColor = theme.
accent))
228                 Spacer(Modifier.width(8.dp))
229                 Text("Фоновый слой", color = theme.textPrimary, fontSize = 13.sp)
230             }
231             Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.fillMaxWidth().clickable { onRoleToggle(
if (isFg) AnimationFrameRole.NORMAL else AnimationFrameRole.FOREGROUND) }.padding(8.dp)) {
232                 Checkbox(checked = isFg, onCheckedChange = null, colors = CheckboxDefaults.colors(checkedColor = theme.
accent))
233                 Spacer(Modifier.width(8.dp))
234                 Text("Передний план", color = theme.textPrimary, fontSize = 13.sp)
235             }
236         }
237     }
238 }
239
240 @Composable
241 private fun ContextMenuItem(
242     label: String,
243     icon: ImageVector,
244     theme: AppTheme,
245     enabled: Boolean = true,
246     onClick: () -> Unit
247 ) {
248     Row(
249         modifier = Modifier
250             .fillMaxWidth()
251             .clickable(enabled = enabled) { onClick() }
252             .padding(8.dp)
253             .alphaIfDisabled(!enabled),
254         verticalAlignment = Alignment.CenterVertically
255     ) {
256         Icon(icon, null, tint = if (enabled) theme.textPrimary else theme.textSecondary.copy(alpha = 0.5f), modifier =
Modifier.size(18.dp))
257         Spacer(Modifier.width(12.dp))
258         Text(label, color = if (enabled) theme.textPrimary else theme.textSecondary.copy(alpha = 0.5f), fontSize = 14.sp)
259     }
260 }
261
262 @Composable
263 private fun TimelineFrameItem(
264     frame: AnimationFrame,
265     isSelected: Boolean,
266     thumbnailLayers: List<FrameThumbnailLayer>,
267     theme: AppTheme,

```

```

268     canMoveLeft: Boolean,
269     canMoveRight: Boolean,
270     onMove: (Int) -> Unit,
271     onClick: () -> Unit
272 ) {
273     val moveThresholdPx = with(LocalDensity.current) { 44.dp.toPx() }
274     var accumulatedDragX by remember(frame.id) { mutableFloatStateOf(0f) }
275     Column(
276         horizontalAlignment = Alignment.CenterHorizontally,
277         modifier = Modifier
278             .pointerInput(frame.id, canMoveLeft, canMoveRight) {
279                 detectDragGesturesAfterLongPress(
280                     onDragStart = { accumulatedDragX = 0f },
281                     onDrag = { change, dragAmount ->
282                         accumulatedDragX += dragAmount.x
283                         val direction = when {
284                             accumulatedDragX <= -moveThresholdPx && canMoveLeft -> -1
285                             accumulatedDragX >= moveThresholdPx && canMoveRight -> 1
286                             else -> 0
287                         }
288                         if (direction != 0) {
289                             change.consume()
290                             onMove(direction)
291                             accumulatedDragX = 0f
292                         }
293                     }
294             )
295     )
296     .clickable { onClick() }
297 ) {
298     Box(
299         modifier = Modifier
300             .size(54.dp, 40.dp)
301             .clip(RoundedCornerShape(4.dp))
302             .background(if (isSelected) theme.accent.copy(alpha = 0.2f) else theme.panelInsetSoft)
303             .border(
304                 width = if (isSelected) 2.dp else 1.dp,
305                 color = if (isSelected) theme.accent else theme.panelStroke,
306                 shape = RoundedCornerShape(4.dp)
307             ),
308         contentAlignment = Alignment.Center
309     ) {
310         if (thumbnailLayers.isNotEmpty()) {
311             thumbnailLayers.forEach { layer ->
312                 Image(
313                     bitmap = layer.image,
314                     contentDescription = null,
315                     contentScale = ContentScale.Fit,
316                     modifier = Modifier
317                         .fillMaxSize()
318                         .padding(2.dp)
319                         .graphicsLayer(alpha = layer.opacity.coerceIn(0f, 1f)),
320                 )
321             }
322         } else {
323             Icon(Icons.Default.Image, null, tint = theme.textSecondary.copy(alpha = 0.3f), modifier = Modifier.size(20.
324 dp))
325         }
326         if (frame.holdFrames > 1) {
327             Text("+${frame.holdFrames - 1}", color = theme.accent, fontSize = 9.sp, fontWeight = FontWeight.Bold)
328         } else {
329             Spacer(Modifier.height(11.dp))
330         }
331     }
332 }
333
334 @Composable
335 private fun TimelineActionIcon(
336     icon: ImageVector,
337     label: String,
338     theme: AppTheme,
339     enabled: Boolean = true,
340     onClick: () -> Unit
341 ) {
342     Column(
343         horizontalAlignment = Alignment.CenterHorizontally,
344         modifier = Modifier
345             .alphaIfDisabled(!enabled)
346             .clickable(enabled = enabled) { onClick() }
347     ) {
348         Icon(icon, null, tint = if (enabled) theme.textPrimary else theme.textSecondary.copy(alpha = 0.5f), modifier =
349 Modifier.size(22.dp))
350         Text(label, color = theme.textSecondary, fontSize = 9.sp)
351     }
352
353     private fun Modifier.alphaIfDisabled(disabled: Boolean): Modifier = if (disabled) this.then(Modifier.graphicsLayer { alpha =
0.5f }) else this

```

88. app/src/main/java/com/wetinknext/ui/color/ColorCollection.kt

```

1 package com.wetinknext.ui.color
2
3 import androidx.compose.ui.graphics.Color
4
5 data class ColorCollection(
6     val id: Long,
7     val name: String,
8     val colors: List<Int>,
9     val builtin: Boolean = false,
10 )
11
12 fun ColorCollection.composeColors(): List<Color> = colors.map { Color(it) }
13
14 private fun hex(s: String): Int {
15     val v = s.removePrefix("#").trim()
16     val r = v.substring(0, 2).toInt(16)
17     val g = v.substring(2, 4).toInt(16)
18     val b = v.substring(4, 6).toInt(16)
19     return (0xFF shl 24) or (r shl 16) or (g shl 8) or b
20 }
21
22 private fun palette(vararg hex: String): List<Int> = hex.map { hex(it) }
23
24 object BuiltinCollections {
25     val all: List<ColorCollection> = listOf(
26         ColorCollection(
27             id = -1L,
28             name = "Стандарт",
29             colors = palette(
30                 "#FFF3B0", "#FFD6A5", "#FFC2B0", "#FFCCD5", "#F1C6F2",
31                 "#D8C7F5", "#C5D7FF", "#B8E5FF", "#B6F0EE", "#C9F2D8",
32                 "#FFE066", "#FFB87A", "#FF9F8B", "#FF9CB0", "#E59FE6",
33                 "#B89BE8", "#9CB6F5", "#86CFF2", "#7FD9D5", "#8AD9A8",
34                 "#F7C548", "#F09870", "#E07A6E", "#D9748E", "#C383C6",
35                 "#9678D0", "#7798E0", "#5DB1E6", "#5CB8B5", "#65BD8A",
36             ),
37             builtin = true,
38         ),
39         ColorCollection(
40             id = -2L,
41             name = "Пастель",
42             colors = palette(
43                 "#F9E79F", "#F7D08A", "#F5B187", "#F0998A", "#EC9090",
44                 "#E68F9C", "#D88BAE", "#C58CC0", "#B391CD", "#9C95D6",
45                 "#A8D5B6", "#9FD2C9", "#9CCFD8", "#A0C9E0", "#A8C0E3",
46                 "#B5BBE3", "#C0B8DE", "#CCB5D3", "#D6B3C5", "#DDB3B6",
47                 "#C7E1A1", "#B7DAA3", "#A8D2AE", "#9CCABD", "#94C0CB",
48                 "#94B5D2", "#9DA9D1", "#B0A0CB", "#C19BBE", "#CC9AAB",
49             ),
50             builtin = true,
51         ),
52         ColorCollection(
53             id = -3L,
54             name = "Персонажи",
55             colors = palette(
56                 "#F4C9B5", "#F0D4BC", "#E8B89A", "#D89C7E", "#C0805F",
57                 "#9C5F3F", "#74442B", "#4F2C1C", "#311A11", "#1B0F0A",
58                 "#FBE3D4", "#F8DCC9", "#F2C5A5", "#E2A77F", "#C58557",
59                 "#A36739", "#7A4824", "#552E14", "#321A0A", "#FFFAF0",
60                 "#E8C7C1", "#D49C9C", "#B97070", "#8C4F4F", "#5C3030",
61                 "#C7BF81", "#9C9484", "#6E6757", "#A0B0C4", "#5A6878",
62             ),
63             builtin = true,
64         ),
65         ColorCollection(
66             id = -4L,
67             name = "Пейзаж",
68             colors = palette(
69                 "#AEDCF0", "#7FBFE1", "#56A0CB", "#3D80B0", "#2E608E",
70                 "#244670", "#1B335A", "#142545", "#0D1830", "#070D1E",
71                 "#CFE3B0", "#A8CF8C", "#7FB36A", "#5C9651", "#3F7A3F",
72                 "#2E5E33", "#214728", "#16321C", "#5C7B4A", "#3F5732",
73                 "#F4D58D", "#E3B26E", "#D08A4A", "#B5642E", "#923F1C",
74                 "#6E2A14", "#8B5A2B", "#5C3A1A", "#A38560", "#6F5B3D",
75             ),
76             builtin = true,
77         ),
78         ColorCollection(
79             id = -5L,
80             name = "Анимация",
81             colors = palette(
82                 "#111318", "#3B414A", "#7B8491", "#B8C0CC", "#FFFFFF",
83                 "#4A90E2", "#35B9D5", "#42B883", "#A5D646", "#F4D44D",
84                 "#F5A23D", "#F26B4F", "#E6536F", "#E66FAD", "#A368DB",
85                 "#6D8CFF", "#64D4E8", "#76D7A6", "#D8EA72", "#FFE58A",
86                 "#245EA8", "#167E98", "#20794F", "#6B8F20", "#B58B08",
87                 "#B65A12", "#B53832", "#A62E58", "#A23E86", "#633B9B",
88             ),
89             builtin = true,
90         ),
91     )
92 }

```

```
91      )  
92  }
```

89. app/src/main/java/com/wetinknext/ui/color/ColorMath.kt

```

1 package com.wetinknext.ui.color
2
3 import androidx.compose.ui.graphics.Color
4 import kotlin.math.abs
5
6 data class Hsl(val h: Float, val s: Float, val l: Float)
7
8 fun rgbToHsl(c: Color): Hsl {
9     val r = c.red
10    val g = c.green
11    val b = c.blue
12    val max = maxOf(r, g, b)
13    val min = minOf(r, g, b)
14    val l = (max + min) / 2f
15    val d = max - min
16    if (d == 0f) return Hsl(0f, 0f, l)
17    val s = if (l > 0.5f) d / (2f - max - min) else d / (max + min)
18    val h = when (max) {
19        r -> 60f * (((g - b) / d) % 6f)
20        g -> 60f * ((b - r) / d + 2f)
21        else -> 60f * ((r - g) / d + 4f)
22    }
23    return Hsl(((h % 360f) + 360f) % 360f, s, l)
24 }
25
26 fun hslToColor(h: Float, s: Float, l: Float): Color {
27     val hh = ((h % 360f) + 360f) % 360f
28     val c = (1f - abs(2f * l - 1f)) * s
29     val x = c * (1f - abs((hh / 60f) % 2f - 1f))
30     val m = l - c / 2f
31     val (r1, g1, b1) = when ((hh / 60f).toInt()) {
32         0 -> Triple(c, x, 0f)
33         1 -> Triple(x, c, 0f)
34         2 -> Triple(0f, c, x)
35         3 -> Triple(0f, x, c)
36         4 -> Triple(x, 0f, c)
37         else -> Triple(c, 0f, x)
38     }
39     return Color(
40         (r1 + m).coerceIn(0f, 1f),
41         (g1 + m).coerceIn(0f, 1f),
42         (b1 + m).coerceIn(0f, 1f),
43         1f,
44     )
45 }
46
47 fun hslToColor(hsl: Hsl): Color = hslToColor(hsl.h, hsl.s, hsl.l)
48
49 fun Color.toHex6(): String {
50     val r = (red * 255f).toInt().coerceIn(0, 255)
51     val g = (green * 255f).toInt().coerceIn(0, 255)
52     val b = (blue * 255f).toInt().coerceIn(0, 255)
53     return "#%02X%02X%02X".format(r, g, b)
54 }
55
56 enum class HarmonyType(
57     val displayName: String,
58     val offsets: FloatArray,
59 ) {
60     Complementary("Комплементарная", floatArrayOf(0f, 180f)),
61     Analogous("Аналоговая", floatArrayOf(-30f, 0f, 30f)),
62     Triadic("Триадная", floatArrayOf(0f, 120f, 240f)),
63     SplitComplementary(
64         "Сплит-комплементарная",
65         floatArrayOf(0f, 150f, 210f),
66     ),
67     Tetradic("Тетрадная", floatArrayOf(0f, 60f, 180f, 240f)),
68     Square("Квадратная", floatArrayOf(0f, 90f, 180f, 270f)),
69     ;
70
71     fun derive(base: Hsl): List<Hsl> {
72         val h = base.h
73         return offsets.map { off ->
74             Hsl(((h + off) % 360f + 360f) % 360f, base.s, base.l)
75         }
76     }
77 }

```

90. app/src/main/java/com/wetinknext/ui/color/ColorPanel.kt

```

1  package com.wetinknext.ui.color
2
3  import androidx.compose.animation.core.animateFloatAsState
4  import androidx.compose.animation.core.tween
5  import androidx.compose.foundation.Canvas
6  import androidx.compose.foundation.ExperimentalFoundationApi
7  import androidx.compose.foundation.background
8  import androidx.compose.foundation.border
9  import androidx.compose.foundation.clickable
10 import androidx.compose.foundation.combinedClickable
11 import androidx.compose.foundation.gestures.detectDragGestures
12 import androidx.compose.foundation.gestures.detectHorizontalDragGestures
13 import androidx.compose.foundation.gestures.detectTapGestures
14 import androidx.compose.foundation.layout.*
15 import androidx.compose.foundation.rememberScrollState
16 import androidx.compose.foundation.shape.CircleShape
17 import androidx.compose.foundation.shape.RoundedCornerShape
18 import androidx.compose.foundation.text.BasicTextField
19 import androidx.compose.foundation.verticalScroll
20 import androidx.compose.material3.*
21 import androidx.compose.runtime.*
22 import androidx.compose.ui.Alignment
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.draw.clip
25 import androidx.compose.ui.draw.shadow
26 import androidx.compose.ui.geometry.Offset
27 import androidx.compose.ui.geometry.Size
28 import androidx.compose.ui.graphics.Brush
29 import androidx.compose.ui.graphics.Color
30 import androidx.compose.ui.graphics.SolidColor
31 import androidx.compose.ui.graphics.drawscope.DrawScope
32 import androidx.compose.ui.graphics.drawscope.Stroke
33 import androidx.compose.ui.input.pointer.pointerInput
34 import androidx.compose.ui.platform.LocalDensity
35 import androidx.compose.ui.text.TextStyle
36 import androidx.compose.ui.text.font.FontWeight
37 import androidx.compose.ui.text.style.TextAlign
38 import androidx.compose.ui.unit.dp
39 import androidx.compose.ui.unit.IntOffset
40 import androidx.compose.ui.unit.sp
41 import androidx.compose.ui.window.Dialog
42 import com.wetinknext.ui.components.PanelSurface
43 import com.wetinknext.ui.theme.AppTheme
44 import kotlinx.coroutines.delay
45 import kotlin.math.atan2
46 import kotlin.math.cos
47 import kotlin.math.hypot
48 import kotlin.math.sin
49 import kotlin.math.roundToInt
50
51 private enum class ColorTab { Wheel, Map }
52
53 @Composable
54 fun ColorPanel(
55     state: GlesColorState,
56     color: Color,
57     theme: AppTheme,
58     onColorChange: (Color) -> Unit,
59     onAddFromPhoto: () -> Unit,
60     title: String = "Ғберә",
61     recordRecent: Boolean = true,
62     allowCollectionEditing: Boolean = true,
63     onPinCollection: (ColorCollection) -> Unit = {},
64     modifier: Modifier = Modifier,
65     onDismiss: () -> Unit,
66 ) {
67     var tab by remember { mutableStateOf(ColorTab.Wheel) }
68     var hsl by remember { mutableStateOf(rgbToHsl(color)) }
69     var harmonyType by remember { mutableStateOf(HarmonyType.Triadic) }
70     val latestHsl by rememberUpdatedState(hsl)
71
72     val wheelScrollState = rememberScrollState()
73     val mapScrollState = rememberScrollState()
74     val tabScrollState = when (tab) {
75         ColorTab.Wheel -> wheelScrollState
76         ColorTab.Map -> mapScrollState
77     }
78
79     LaunchedEffect(hsl, recordRecent) {
80         val selectedColor = hslToColor(hsl)
81         onColorChange(selectedColor)
82         if (recordRecent) {
83             delay(350)
84             state.pushRecentColor(selectedColor)
85         }
86     }
87
88     val dismissAndSave = {
89         if (recordRecent) state.pushRecentColor(hslToColor(hsl))
90         onDismiss()
91     }

```

```

91     }
92
93     DisposableEffect(Unit) {
94         onDispose {
95             if (recordRecent) state.pushRecentColor(hslToColor(latestHsl))
96         }
97     }
98
99     PanelSurface(
100         modifier = modifier.width(320.dp),
101     ) {
102         Column(
103             modifier = Modifier.padding(14.dp),
104         ) {
105             ColorHeader(
106                 preview = hslToColor(hsl),
107                 title = title,
108                 theme = theme,
109                 onConfirm = dismissAndSave,
110             )
111             Spacer(Modifier.height(10.dp))
112             TabBar(tab, theme) { tab = it }
113             Spacer(Modifier.height(12.dp))
114             Box(
115                 modifier = Modifier
116                     .height(TabContentHeight)
117                     .fillMaxWidth(),
118             ) {
119                 Column(
120                     modifier = Modifier
121                         .fillMaxSize()
122                         .verticalScroll(tabScrollState),
123                 ) {
124                     when (tab) {
125                         ColorTab.Wheel -> WheelTab(
126                             hsl = hsl,
127                             theme = theme,
128                             onHslChange = { hsl = it },
129                             harmonyType = harmonyType,
130                             onHarmonyTypeChange = { harmonyType = it },
131                             recent = state.recentColors.toList(),
132                             onPickRecent = { picked ->
133                                 val p = rgbToHsl(picked)
134                                 hsl = hsl.copy(h = p.h, s = p.s)
135                                 if (recordRecent) state.pushRecentColor(hslToColor(hsl))
136                             },
137                         )
138                         ColorTab.Map -> MapTab(
139                             userCollections = state.userCollections.toList(),
140                             currentColor = hslToColor(hsl),
141                             theme = theme,
142                             onPick = { picked ->
143                                 hsl = rgbToHsl(picked)
144                                 if (recordRecent) state.pushRecentColor(picked)
145                             },
146                             onDeleteUserCollection = { id ->
147                                 state.deleteUserCollection(id)
148                             },
149                             onAddFromPhoto = onAddFromPhoto,
150                             allowCollectionEditing = allowCollectionEditing,
151                             onCreatePalette = { name ->
152                                 state.addUserCollection(
153                                     ColorCollection(
154                                         id = System.currentTimeMillis(),
155                                         name = name.ifBlank { "Новая палитра" },
156                                         colors = emptyList(),
157                                         builtin = false,
158                                     ),
159                                 )
160                             },
161                             onAddColorToCollection = { id, color ->
162                                 state.addColorToCollection(id, color)
163                             },
164                             onReplaceColorInCollection = { id, slot, color ->
165                                 state.replaceColorInCollection(id, slot, color)
166                             },
167                             onRemoveColorFromCollection = { id, slot ->
168                                 state.removeColorFromCollection(id, slot)
169                             },
170                             onPinCollection = onPinCollection,
171                         )
172                     }
173                 }
174             }
175         }
176     }
177 }
178
179 private val TabContentHeight = 440.dp
180
181 @Composable
182 private fun ColorHeader(preview: Color, onConfirm: () -> Unit, title: String, theme: AppTheme) {

```

```

183     Row(verticalAlignment = Alignment.CenterVertically) {
184         Text(
185             title,
186             color = theme.textPrimary,
187             fontWeight = FontWeight.Bold,
188             fontSize = 18.sp,
189         )
190         Spacer(Modifier.weight(1f))
191         Box(
192             modifier = Modifier
193                 .size(28.dp)
194                 .clip(CircleShape)
195                 .background(preview)
196                 .border(1.dp, theme.panelStroke, CircleShape)
197                 .clickable(onClick = onConfirm),
198         )
199     }
200 }
201
202 @Composable
203 private fun TabBar(current: ColorTab, theme: AppTheme, onChange: (ColorTab) -> Unit) {
204     Row(
205         modifier = Modifier
206             .fillMaxWidth()
207             .clip(RoundedCornerShape(10.dp))
208             .background(theme.panelInset)
209             .padding(3.dp),
210         horizontalArrangement = Arrangement.spacedBy(3.dp),
211     ) {
212         TabChip(label = "Кольцо", selected = current == ColorTab.Wheel, theme = theme) { onChange(ColorTab.Wheel) }
213         TabChip(label = "Карта", selected = current == ColorTab.Map, theme = theme) { onChange(ColorTab.Map) }
214     }
215 }
216
217 @Composable
218 private fun RowScope.TabChip(
219     label: String,
220     selected: Boolean,
221     theme: AppTheme,
222     onClick: () -> Unit,
223 ) {
224     Box(
225         modifier = Modifier
226             .weight(1f)
227             .clip(RoundedCornerShape(8.dp))
228             .background(if (selected) theme.accent.copy(alpha = 0.25f) else Color.Transparent)
229             .clickable(onClick = onClick)
230             .padding(vertical = 7.dp),
231         contentAlignment = Alignment.Center,
232     ) {
233         Text(
234             label,
235             color = if (selected) theme.textPrimary else theme.textSecondary,
236             fontSize = 13.sp,
237             fontWeight = if (selected) FontWeight.SemiBold else FontWeight.Normal,
238         )
239     }
240 }
241
242 @Composable
243 private fun WheelTab(
244     hsl: Hsl,
245     theme: AppTheme,
246     onHslChange: (Hsl) -> Unit,
247     harmonyType: HarmonyType,
248     onHarmonyTypeChange: (HarmonyType) -> Unit,
249     recent: List<Color>,
250     onPickRecent: (Color) -> Unit,
251 ) {
252     HueSatWheel(
253         hsl = hsl,
254         harmonyType = harmonyType,
255         onHsChange = { newH, newS -> onHslChange(hsl.copy(h = newH, s = newS)) },
256         modifier = Modifier
257             .fillMaxWidth()
258             .height(160.dp),
259     )
260     Spacer(Modifier.height(8.dp))
261     LightnessSlider(
262         hsl = hsl,
263         theme = theme,
264         onChange = { newL -> onHslChange(hsl.copy(l = newL)) },
265     )
266     Spacer(Modifier.height(8.dp))
267     HexRow(color = hslToColor(hsl), theme = theme)
268     Spacer(Modifier.height(10.dp))
269     HarmonySection(
270         hsl = hsl,
271         theme = theme,
272         harmonyType = harmonyType,
273         onHarmonyTypeChange = onHarmonyTypeChange,
274         onPick = onPickRecent,

```

```

275     )
276     Spacer(Modifier.height(10.dp))
277     SectionTitle("Недавние", theme)
278     Spacer(Modifier.height(6.dp))
279     RecentRow(colors = recent.take(GRID_COLUMNS), theme = theme, onPick = onPickRecent)
280 }
281
282 @Composable
283 private fun HueSatWheel(
284     hsl: Hsl,
285     harmonyType: HarmonyType,
286     onHsChange: (Float, Float) -> Unit,
287     modifier: Modifier = Modifier,
288 ) {
289     val latestOnHsChange by rememberUpdatedState(onHsChange)
290     Canvas(
291         modifier = modifier
292             .clip(RoundedCornerShape(16.dp))
293             .pointerInput(Unit) {
294                 detectDragGestures { change, _ ->
295                     val (nh, ns) = pickHueSat(change.position, size.width.toFloat(), size.height.toFloat())
296                     latestOnHsChange(nh, ns)
297                 }
298             }
299             .pointerInput(Unit) {
300                 detectTapGestures(onTap = { offset ->
301                     val (nh, ns) = pickHueSat(offset, size.width.toFloat(), size.height.toFloat())
302                     latestOnHsChange(nh, ns)
303                 })
304             },
305     ) {
306         val w = size.width
307         val hgt = size.height
308         val cx = w / 2f
309         val cy = hgt / 2f
310         val outer = minOf(w, hgt) / 2f - 4f
311         if (outer <= 0f) return@Canvas
312
313         val steps = 120
314         for (i in 0 until steps) {
315             val a0 = i * 360f / steps
316             val a1 = (i + 1) * 360f / steps
317             val midA = (a0 + a1) / 2f
318             val hueColor = hslToColor(midA, 1f, 0.5f)
319             drawArc(
320                 color = hueColor,
321                 startAngle = a0 - 90f,
322                 sweepAngle = (a1 - a0) + 0.6f,
323                 useCenter = true,
324                 topLeft = Offset(cx - outer, cy - outer),
325                 size = Size(outer * 2, outer * 2),
326             )
327         }
328         drawCircle(
329             brush = Brush.radialGradient(
330                 colors = listOf(Color.White, Color.White.copy(alpha = 0f)),
331                 center = Offset(cx, cy),
332                 radius = outer,
333             ),
334             radius = outer,
335             center = Offset(cx, cy),
336         )
337
338         val derived = harmonyType.derive(hsl)
339         val verts = derived.map { d -> hslToWheelPoint(d, cx, cy, outer) }
340         drawHarmonyShape(harmonyType, verts)
341
342         val current = hslToWheelPoint(hsl, cx, cy, outer)
343         drawCircle(color = Color.White, radius = 7f, center = current, style = Stroke(width = 2f))
344         drawCircle(color = Color.Black, radius = 7f, center = current, style = Stroke(width = 1f))
345     }
346 }
347
348 private fun hslToWheelPoint(h: Hsl, cx: Float, cy: Float, outer: Float): Offset {
349     val a = Math.toRadians((h.h - 90.0))
350     val r = h.s.coerceIn(0f, 1f) * outer
351     return Offset(cx + (r * cos(a)).toFloat(), cy + (r * sin(a)).toFloat())
352 }
353
354 private fun DrawScope.drawHarmonyShape(type: HarmonyType, verts: List<Offset>) {
355     if (verts.size < 2) return
356     val edges: List<Pair<Int, Int>> = when (type) {
357         HarmonyType.Complementary -> listOf(0 to 1)
358         HarmonyType.Analogous -> listOf(0 to 1, 1 to 2)
359         HarmonyType.Triadic -> listOf(0 to 1, 1 to 2, 2 to 0)
360         HarmonyType.SplitComplementary -> listOf(0 to 1, 0 to 2, 1 to 2)
361         HarmonyType.Tetradic,
362         HarmonyType.Square -> listOf(0 to 1, 1 to 2, 2 to 3, 3 to 0)
363     }
364     edges.forEach { (a, b) ->
365         drawLine(color = Color.White.copy(alpha = 0.85f), start = verts[a], end = verts[b], strokeWidth = 2.5f)
366         drawLine(color = Color.Black.copy(alpha = 0.55f), start = verts[a], end = verts[b], strokeWidth = 1f)
367     }
368 }

```

```

367     }
368     verts.forEach { v ->
369         drawCircle(color = Color.White, radius = 4f, center = v)
370         drawCircle(color = Color.Black, radius = 4f, center = v, style = Stroke(width = 1f))
371     }
372 }
373
374 private fun pickHueSat(p: Offset, w: Float, h: Float): Pair<Float, Float> {
375     val cx = w / 2f
376     val cy = h / 2f
377     val outer = minOf(w, h) / 2f - 4f
378     val dx = p.x - cx
379     val dy = p.y - cy
380     val r = hypot(dx, dy)
381     val ang = Math.toDegrees(atan2(dy, dx).toDouble()) + 90.0
382     val nh = ((ang + 360.0) % 360.0).toFloat()
383     val ns = (r / outer).coerceIn(0f, 1f)
384     return nh to ns
385 }
386
387 @Composable
388 private fun LightnessSlider(hsl: Hsl, theme: AppTheme, onChange: (Float) -> Unit) {
389     val hue = hsl.h
390     val sat = hsl.s
391     val gradient = remember(hue, sat) {
392         Brush.horizontalGradient(
393             colors = listOf(
394                 hslToColor(hue, sat, 0f),
395                 hslToColor(hue, sat, 0.5f),
396                 hslToColor(hue, sat, 1f),
397             ),
398         )
399     }
400     val latestOnChange by rememberUpdatedState(onChange)
401     Row(verticalAlignment = Alignment.CenterVertically) {
402         Text("L", color = theme.textSecondary, fontSize = 12.sp, modifier = Modifier.width(16.dp))
403         Spacer(Modifier.width(8.dp))
404         Box(
405             modifier = Modifier
406                 .height(20.dp)
407                 .fillMaxWidth()
408                 .clip(RoundedCornerShape(10.dp))
409                 .background(gradient)
410                 .border(1.5.dp, theme.panelStroke, RoundedCornerShape(10.dp))
411                 .pointerInput(Unit) {
412                     detectDragGestures { change, _ ->
413                         val f = (change.position.x / size.width).coerceIn(0f, 1f)
414                         latestOnChange(f)
415                     }
416                 }
417                 .pointerInput(Unit) {
418                     detectTapGestures(onTap = { offset ->
419                         val f = (offset.x / size.width).coerceIn(0f, 1f)
420                         latestOnChange(f)
421                     })
422                 },
423         ) {
424             Canvas(modifier = Modifier.fillMaxSize()) {
425                 val knobR = (size.height / 2f - 2f).coerceAtMost(size.width / 2f).coerceAtLeast(0f)
426                 if (knobR <= 0f) return@Canvas
427                 val knobX = (hsl.l.coerceIn(0f, 1f) * size.width).coerceIn(knobR + 2f, size.width - knobR - 2f)
428
429                 drawCircle(color = Color.White, radius = knobR, center = Offset(knobX, size.height / 2f), style = Stroke(
width = 2.5f))
430                 drawCircle(color = theme.panelStroke, radius = knobR, center = Offset(knobX, size.height / 2f), style =
Stroke(width = 1f))
431             }
432         }
433     }
434 }
435
436 @Composable
437 private fun HexRow(color: Color, theme: AppTheme) {
438     Row(verticalAlignment = Alignment.CenterVertically) {
439         Text("Hex", color = theme.textSecondary, fontSize = 11.sp)
440         Spacer(Modifier.weight(1f))
441         Box(
442             modifier = Modifier
443                 .clip(RoundedCornerShape(8.dp))
444                 .background(theme.panelInset)
445                 .border(1.dp, theme.panelStroke, RoundedCornerShape(8.dp))
446                 .padding(horizontal = 10.dp, vertical = 4.dp),
447         ) {
448             Text(color.toHexString(), color = theme.textPrimary, fontSize = 12.sp, fontWeight = FontWeight.Medium)
449         }
450     }
451 }
452
453 @Composable
454 private fun HarmonySection(
455     hsl: Hsl,
456     theme: AppTheme,

```

```

457     harmonyType: HarmonyType,
458     onHarmonyTypeChange: (HarmonyType) -> Unit,
459     onPick: (Hsl) -> Unit,
460 ) {
461     SectionTitle("Тип гармонии", theme)
462     Spacer(Modifier.height(6.dp))
463     Row(
464         horizontalArrangement = Arrangement.spacedBy(6.dp),
465         modifier = Modifier.fillMaxWidth(),
466     ) {
467         HarmonyType.entries.forEach { type ->
468             HarmonyIcon(
469                 type = type,
470                 theme = theme,
471                 selected = type == harmonyType,
472                 modifier = Modifier.weight(1f),
473                 ) { onHarmonyTypeChange(type) }
474         }
475     }
476     Spacer(Modifier.height(6.dp))
477     Text(harmonyType.displayName, color = theme.textSecondary, fontSize = 11.sp, textAlign = TextAlign.Center, modifier =
478     Modifier.fillMaxWidth())
479     Spacer(Modifier.height(8.dp))
480     val derived = remember(harmonyType, hsl) { harmonyType.derive(hsl) }
481     Row(
482         horizontalArrangement = Arrangement.spacedBy(6.dp),
483         verticalAlignment = Alignment.CenterVertically,
484         modifier = Modifier.fillMaxWidth(),
485     ) {
486         derived.forEach { d ->
487             Box(
488                 modifier = Modifier
489                     .weight(1f)
490                     .height(32.dp)
491                     .clip(RoundedCornerShape(8.dp))
492                     .background(hslToColor(d))
493                     .border(1.dp, theme.panelInset, RoundedCornerShape(8.dp))
494                     .clickable { onPick(d) },
495             )
496         }
497         repeat(4 - derived.size) { Box(modifier = Modifier.weight(1f)) }
498     }
499 }
500 @Composable
501 private fun HarmonyIcon(
502     type: HarmonyType,
503     theme: AppTheme,
504     selected: Boolean,
505     modifier: Modifier = Modifier,
506     onClick: () -> Unit,
507 ) {
508     Box(
509         modifier = modifier
510             .aspectRatio(1f)
511             .clip(RoundedCornerShape(8.dp))
512             .background(if (selected) theme.accent.copy(alpha = 0.25f) else theme.panelInset)
513             .border(1.dp, if (selected) theme.accent else theme.panelInset, RoundedCornerShape(8.dp))
514             .clickable(onClick = onClick)
515             .padding(6.dp),
516         contentAlignment = Alignment.Center,
517     ) {
518         Canvas(modifier = Modifier.fillMaxSize()) {
519             val cx = size.width / 2f
520             val cy = size.height / 2f
521             val r = minOf(size.width, size.height) / 2f - 1f
522             val ringColor = (if (selected) Color.White else theme.textSecondary).copy(alpha = 0.6f)
523             val shapeColor = if (selected) Color.White else theme.textSecondary
524             drawCircle(color = ringColor, radius = r, center = Offset(cx, cy), style = Stroke(width = 1.2f))
525             val verts = type.offsets.map { off ->
526                 val a = Math.toRadians((off - 90.0).toDouble())
527                 Offset(cx + (r * 0.78f * cos(a)).toFloat(), cy + (r * 0.78f * sin(a)).toFloat())
528             }
529             val edges: List<Pair<Int, Int>> = when (type) {
530                 HarmonyType.Complementary -> listOf(0 to 1)
531                 HarmonyType.Analogous -> listOf(0 to 1, 1 to 2)
532                 HarmonyType.Triadic -> listOf(0 to 1, 1 to 2, 2 to 0)
533                 HarmonyType.SplitComplementary -> listOf(0 to 1, 0 to 2, 1 to 2)
534                 HarmonyType.Tetradic,
535                 HarmonyType.Square -> listOf(0 to 1, 1 to 2, 2 to 3, 3 to 0)
536             }
537             edges.forEach { (a, b) -> drawLine(color = shapeColor, start = verts[a], end = verts[b], strokeWidth = 1.8f) }
538             verts.forEach { v -> drawCircle(color = shapeColor, radius = 2.2f, center = v) }
539         }
540     }
541 }
542
543 private const val GRID_COLUMNS = 10
544
545 @Composable
546 private fun MapTab(
547     userCollections: List<ColorCollection>,

```

```

548     currentColor: Color,
549     theme: AppTheme,
550     onPick: (Color) -> Unit,
551     onDeleteUserCollection: (Long) -> Unit,
552     onAddFromPhoto: () -> Unit,
553     allowCollectionEditing: Boolean,
554     onCreatePalette: (String) -> Unit,
555     onAddColorToCollection: (Long, Color) -> Unit,
556     onReplaceColorInCollection: (Long, Int, Color) -> Unit,
557     onRemoveColorFromCollection: (Long, Int) -> Unit,
558     onPinCollection: (ColorCollection) -> Unit,
559 ) {
560     var showCreate by remember { mutableStateOf(false) }
561     Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
562         if (allowCollectionEditing) {
563             Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
564                 MapActionButton(label = "+ Создать", theme = theme, modifier = Modifier.weight(1f), onClick = { showCreate =
true })
565                 MapActionButton(label = "+ Из фото", theme = theme, modifier = Modifier.weight(1f), onClick = onAddFromPhoto)
566             }
567         }
568         userCollections.forEach { collection ->
569             CollectionCard(collection = collection, theme = theme, onPick = onPick, onDelete = { onDeleteUserCollection(
collection.id) },
570                 onAddCurrentColor = { onAddColorToCollection(collection.id, currentColor) },
571                 onReplaceAt = { slot -> onReplaceColorInCollection(collection.id, slot, currentColor) },
572                 onRemoveAt = { slot -> onRemoveColorFromCollection(collection.id, slot) },
573                 onPin = { onPinCollection(collection) },
574             )
575         }
576         BuiltinCollections.all.forEach { collection ->
577             CollectionCard(collection = collection, theme = theme, onPick = onPick, onDelete = null, onAddCurrentColor = null,
onReplaceAt = null, onRemoveAt = null, onPin = { onPinCollection(collection) })
578         }
579     }
580     if (showCreate && allowCollectionEditing) {
581         CreatePaletteDialog(theme = theme, onConfirm = { name -> onCreatePalette(name); showCreate = false }, onDismiss = {
showCreate = false })
582     }
583 }
584
585 @Composable
586 private fun MapActionButton(label: String, theme: AppTheme, modifier: Modifier = Modifier, onClick: () -> Unit) {
587     Box(modifier = modifier.fillMaxWidth().clip(RoundedCornerShape(10.dp)).background(theme.accent.copy(alpha = 0.15f)).
border(1.2.dp, theme.accentSoft, RoundedCornerShape(10.dp)).clickable(onClick = onClick).padding(vertical = 10.dp, horizontal = 10.
dp), contentAlignment = Alignment.Center) {
588         Text(label, color = theme.textPrimary, fontSize = 13.sp, fontWeight = FontWeight.SemiBold)
589     }
590 }
591
592 @Composable
593 private fun CollectionCard(collection: ColorCollection, theme: AppTheme, onPick: (Color) -> Unit, onDelete: (() -> Unit)?,
onAddCurrentColor: (() -> Unit)?, onReplaceAt: ((Int) -> Unit)?, onRemoveAt: ((Int) -> Unit)?, onPin: () -> Unit) {
594     var editingIdx by remember { mutableStateOf<Int?>(null) }
595     val revealPx = with(LocalDensity.current) { 96.dp.toPx() }
596     var rawOffset by remember(collection.id) { mutableFloatStateOf(0f) }
597     val animatedOffset by animateFloatAsState(rawOffset, animationSpec = tween(140), label = "paletteSwipe")
598     val cardShape = RoundedCornerShape(10.dp)
599     Box(modifier = Modifier.fillMaxWidth().clip(cardShape)) {
600         Box(
601             modifier = Modifier.matchParentSize().background(theme.accent.copy(alpha = 0.9f)),
602             contentAlignment = Alignment.CenterEnd,
603         ) {
604             Text(
605                 "Закрепить",
606                 color = Color.White,
607                 fontSize = 12.sp,
608                 fontWeight = FontWeight.SemiBold,
609                 modifier = Modifier.width(96.dp).fillMaxHeight().clickable { onPin(); rawOffset = 0f }.wrapContentHeight(
Alignment.CenterVertically),
610                 textAlign = TextAlign.Center,
611             )
612         }
613         Column(
614             modifier = Modifier
615                 .fillMaxWidth()
616                 .offset { IntOffset(animatedOffset.roundToInt(), 0) }
617                 .background(theme.panelInset)
618                 .border(1.dp, theme.panelStroke, cardShape)
619                 .pointerInput(collection.id, revealPx) {
620                     detectHorizontalDragGestures(
621                         onHorizontalDrag = { change, dx ->
622                             if (dx < 0f || rawOffset < 0f) {
623                                 change.consume()
624                                 rawOffset = (rawOffset + dx).coerceIn(-revealPx, 0f)
625                             }
626                     },
627                     onDragEnd = { rawOffset = if (rawOffset < -revealPx * 0.4f) -revealPx else 0f },
628                     onDragCancel = { rawOffset = 0f },
629                 )
630             )
631         .padding(horizontal = 8.dp, vertical = 8.dp),

```

```

632         verticalArrangement = Arrangement.spacedBy(6.dp),
633     ) {
634         Row(verticalAlignment = Alignment.CenterVertically) {
635             Text(collection.name, color = theme.textPrimary, fontSize = 13.sp, fontWeight = FontWeight.SemiBold)
636             Spacer(Modifier.weight(1f))
637             if (onDelete != null) {
638                 Box(modifier = Modifier.size(22.dp).clip(CircleShape).background(theme.panelBg.copy(alpha = 0.7f)).border(1.
dp, theme.panelStroke, CircleShape).clickable(onClick = onDelete), contentAlignment = Alignment.Center) {
639                     Text("x", color = theme.textSecondary, fontSize = 14.sp, fontWeight = FontWeight.Bold)
640                 }
641             }
642         }
643         SwatchGrid(colors = collection.composeColors(), theme = theme, onPick = onPick, trailingPlusOnClick =
onAddCurrentColor, onLongPressIndex = if (onReplaceAt != null && onRemoveAt != null) { { idx -> editingIdx = idx } } else null)
644     }
645     }
646     val activeIdx = editingIdx
647     if (activeIdx != null && onReplaceAt != null && onRemoveAt != null) {
648         val swatchColor = collection.composeColors().getOrNull(activeIdx)
649         if (swatchColor != null) {
650             EditSwatchSheet(swatchColor = swatchColor, theme = theme, onReplace = { onReplaceAt(activeIdx); editingIdx = null
}, onRemove = { onRemoveAt(activeIdx); editingIdx = null }, onDismiss = { editingIdx = null })
651         } else editingIdx = null
652     }
653 }
654
655 @OptIn(ExperimentalFoundationApi::class)
656 @Composable
657 private fun SwatchGrid(colors: List<Color>, theme: AppTheme, onPick: (Color) -> Unit, trailingPlusOnClick: (() -> Unit)? =
null, onLongPressIndex: ((Int) -> Unit)? = null) {
658     val total = colors.size + if (trailingPlusOnClick != null) 1 else 0
659     if (total == 0) return
660     val itemRows = (0 until total).toList().chunked(GRID_COLUMNS)
661     Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
662         itemRows.forEach { row ->
663             Row(horizontalArrangement = Arrangement.spacedBy(4.dp), modifier = Modifier.fillMaxWidth()) {
664                 row.forEach { i ->
665                     if (i < colors.size) {
666                         val c = colors[i]
667                         val cellModifier = Modifier.weight(1f).aspectRatio(1f).clip(RoundedCornerShape(5.dp)).background(c).
border(1.dp, theme.panelInset, RoundedCornerShape(5.dp))
668                         if (onLongPressIndex != null) Box(modifier = cellModifier.combinedClickable(onClick = { onPick(c) },
onLongClick = { onLongPressIndex(i) }))
669                         else Box(modifier = cellModifier.clickable { onPick(c) })
670                     } else PlusTile(theme = theme, modifier = Modifier.weight(1f).aspectRatio(1f), onClick =
trailingPlusOnClick!!)
671                 }
672                 repeat(GRID_COLUMNS - row.size) { Box(modifier = Modifier.weight(1f)) }
673             }
674         }
675     }
676 }
677
678 @Composable
679 private fun EditSwatchSheet(swatchColor: Color, theme: AppTheme, onReplace: () -> Unit, onRemove: () -> Unit, onDismiss: () -
> Unit) {
680     Dialog(onDismissRequest = onDismiss) {
681         Column(modifier = Modifier.width(280.dp).clip(RoundedCornerShape(16.dp)).background(theme.panelBg).padding(16.dp),
verticalArrangement = Arrangement.spacedBy(12.dp)) {
682             Row(verticalAlignment = Alignment.CenterVertically) {
683                 Box(modifier = Modifier.size(28.dp).clip(RoundedCornerShape(6.dp)).background(swatchColor).border(1.dp, theme.
panelInset, RoundedCornerShape(6.dp)))
684                 Spacer(Modifier.width(10.dp)); Text("Цвет", color = theme.textPrimary, fontSize = 16.sp, fontWeight =
FontWeight.SemiBold)
685             }
686             DialogButton(label = "Заменить текущим", theme = theme, primary = true, modifier = Modifier.fillMaxWidth(),
onClick = onReplace)
687             DialogButton(label = "Удалить", theme = theme, primary = false, modifier = Modifier.fillMaxWidth(), onClick =
onRemove)
688             DialogButton(label = "Отмена", theme = theme, primary = false, modifier = Modifier.fillMaxWidth(), onClick =
onDismiss)
689         }
690     }
691 }
692
693 @Composable
694 private fun PlusTile(theme: AppTheme, modifier: Modifier = Modifier, onClick: () -> Unit) {
695     Box(modifier = modifier.clip(RoundedCornerShape(5.dp)).background(theme.panelBg.copy(alpha = 0.5f)).border(1.dp, theme.
accent.copy(alpha = 0.7f), RoundedCornerShape(5.dp)).clickable(onClick = onClick), contentAlignment = Alignment.Center) {
696         Text("+", color = theme.textPrimary, fontSize = 16.sp, fontWeight = FontWeight.Bold)
697     }
698 }
699
700 @Composable
701 private fun CreatePaletteDialog(theme: AppTheme, onConfirm: (String) -> Unit, onDismiss: () -> Unit) {
702     var name by remember { mutableStateOf("") }
703     Dialog(onDismissRequest = onDismiss) {
704         Column(modifier = Modifier.width(280.dp).clip(RoundedCornerShape(16.dp)).background(theme.panelBg).padding(16.dp),
verticalArrangement = Arrangement.spacedBy(12.dp)) {
705             Text("Новая палитра", color = theme.textPrimary, fontSize = 16.sp, fontWeight = FontWeight.SemiBold)
706             Box(modifier = Modifier.fillMaxWidth().clip(RoundedCornerShape(8.dp)).background(theme.panelInset).border(1.dp,
theme.panelInset, RoundedCornerShape(8.dp)).padding(horizontal = 10.dp, vertical = 8.dp)) {

```

```

707         BasicTextField(value = name, onValueChange = { name = it.take(40) }, singleLine = true, textStyle = TextStyle(
color = theme.textPrimary, fontSize = 14.sp), cursorBrush = SolidColor(theme.accent), decorationBox = { inner -> if (name.isEmpty(
)) Text("Название", color = theme.textSecondary, fontSize = 14.sp); inner() }, modifier = Modifier.fillMaxWidth())
708     }
709     Row(horizontalArrangement = Arrangement.spacedBy(8.dp), modifier = Modifier.fillMaxWidth()) {
710         DialogButton(label = "Отмена", theme = theme, primary = false, modifier = Modifier.weight(1f), onClick =
onDismiss)
711         DialogButton(label = "Создать", theme = theme, primary = true, modifier = Modifier.weight(1f), onClick = {
onConfirm(name) })
712     }
713 }
714 }
715 }
716 }
717 @Composable
718 private fun DialogButton(label: String, theme: AppTheme, primary: Boolean, modifier: Modifier = Modifier, onClick: () ->
Unit) {
719     Box(modifier = modifier.clip(RoundedCornerShape(8.dp)).background(if (primary) theme.accent.copy(alpha = 0.35f) else
theme.panelInset).border(1.dp, if (primary) theme.accent else theme.panelInset, RoundedCornerShape(8.dp)).clickable(onClick =
onClick).padding(vertical = 10.dp), contentAlignment = Alignment.Center) {
720         Text(label, color = theme.textPrimary, fontSize = 13.sp, fontWeight = FontWeight.SemiBold)
721     }
722 }
723 }
724 @Composable
725 private fun SectionTitle(text: String, theme: AppTheme) {
726     Text(text, color = theme.textSecondary, fontSize = 11.sp, fontWeight = FontWeight.Medium)
727 }
728 }
729 @Composable
730 private fun RecentRow(colors: List<Color>, theme: AppTheme, onPick: (Color) -> Unit) {
731     Row(horizontalArrangement = Arrangement.spacedBy(4.dp), verticalAlignment = Alignment.CenterVertically, modifier =
Modifier.fillMaxWidth()) {
732         repeat(GRID_COLUMNS) { i ->
733             val c = colors.getOrNull(i)
734             if (c != null) Box(modifier = Modifier.weight(1f).aspectRatio(1f).clip(RoundedCornerShape(5.dp)).background(c).
border(1.dp, theme.panelInset, RoundedCornerShape(5.dp)).clickable { onPick(c) })
735             else Box(modifier = Modifier.weight(1f).aspectRatio(1f).clip(RoundedCornerShape(5.dp)).background(theme.
panelInset).border(1.dp, theme.panelInset, RoundedCornerShape(5.dp)))
736         }
737     }
738 }

```

91. app/src/main/java/com/wetinknext/ui/color/GlesColorState.kt

```

1  package com.wetinknext.ui.color
2
3  import android.content.Context
4  import androidx.compose.runtime.mutableStateListOf
5  import androidx.compose.runtime.mutableLongStateOf
6  import androidx.compose.runtime.mutableStateOf
7  import androidx.compose.runtime.getValue
8  import androidx.compose.runtime.setValue
9  import androidx.compose.ui.graphics.Color
10 import androidx.compose.ui.graphics.toArgb
11 import androidx.compose.ui.geometry.Offset
12 import org.json.JSONArray
13 import org.json.JSONObject
14
15 class GlesColorState(context: Context? = null) {
16     private val prefs = context?.applicationContext?.getSharedPreferences(PrefsName, Context.MODE_PRIVATE)
17
18     val recentColors = mutableStateListOf<Color>()
19     val userCollections = mutableStateListOf<ColorCollection>()
20     var pinnedCollectionId by mutableLongStateOf(prefs?.getLong(PinnedCollectionKey, NoPinnedCollection) ?:
NoPinnedCollection)
21     private set
22     var pinnedPaletteOffset by mutableStateOf(
23         Offset(
24             prefs?.getFloat(PinnedPaletteOffsetXKey, DefaultPinnedOffsetX) ?: DefaultPinnedOffsetX,
25             prefs?.getFloat(PinnedPaletteOffsetYKey, DefaultPinnedOffsetY) ?: DefaultPinnedOffsetY,
26         ),
27     )
28     private set
29
30     init {
31         loadRecentColors()
32         loadUserCollections()
33     }
34
35     fun pushRecentColor(color: Color) {
36         val argb = color.toArgb()
37         val existingIndex = recentColors.indexOfFirst { it.toArgb() == argb }
38         if (existingIndex == 0) return
39         if (existingIndex > 0) recentColors.removeAt(existingIndex)
40         recentColors.add(0, color)
41         while (recentColors.size > MaxRecentColors) {
42             recentColors.removeAt(recentColors.lastIndex)
43         }
44         saveRecentColors()
45     }
46
47     private fun loadRecentColors() {
48         val encoded = prefs?.getString(RecentColorsKey, null) ?: return
49         encoded.split(',')
50             .mapNotNull { it.toIntOrNull() }
51             .take(MaxRecentColors)
52             .forEach { recentColors.add(Color(it)) }
53     }
54
55     private fun saveRecentColors() {
56         prefs?.edit()
57             ?.putString(RecentColorsKey, recentColors.joinToString(",") { it.toArgb().toString() })
58             ?.apply()
59     }
60
61     fun addUserCollection(collection: ColorCollection) {
62         userCollections.add(0, collection.copy(builtin = false))
63         saveUserCollections()
64     }
65
66     fun deleteUserCollection(id: Long) {
67         userCollections.removeAll { it.id == id }
68         if (pinnedCollectionId == id) unpinCollection()
69         saveUserCollections()
70     }
71
72     fun pinCollection(id: Long) {
73         pinnedCollectionId = id
74         prefs?.edit()?.putLong(PinnedCollectionKey, id)?.apply()
75     }
76
77     fun unpinCollection() {
78         pinnedCollectionId = NoPinnedCollection
79         prefs?.edit()?.remove(PinnedCollectionKey)?.apply()
80     }
81
82     fun persistPinnedPaletteOffset(offset: Offset) {
83         pinnedPaletteOffset = offset
84         prefs?.edit()
85             ?.putFloat(PinnedPaletteOffsetXKey, offset.x)
86             ?.putFloat(PinnedPaletteOffsetYKey, offset.y)
87             ?.apply()
88     }
89

```

```

90 fun addColorToCollection(id: Long, color: Color) {
91     updateUserCollection(id) { collection ->
92         collection.copy(colors = collection.colors + color.toArgb())
93     }
94 }
95
96 fun replaceColorInCollection(id: Long, slot: Int, color: Color) {
97     updateUserCollection(id) { collection ->
98         if (slot !in collection.colors.indices) return@updateUserCollection collection
99         val colors = collection.colors.toMutableList()
100         colors[slot] = color.toArgb()
101         collection.copy(colors = colors)
102     }
103 }
104
105 fun removeColorFromCollection(id: Long, slot: Int) {
106     updateUserCollection(id) { collection ->
107         if (slot !in collection.colors.indices) return@updateUserCollection collection
108         val colors = collection.colors.toMutableList()
109         colors.removeAt(slot)
110         collection.copy(colors = colors)
111     }
112 }
113
114 private fun updateUserCollection(id: Long, transform: (ColorCollection) -> ColorCollection) {
115     val index = userCollections.indexOfFirst { it.id == id }
116     if (index < 0) return
117     userCollections[index] = transform(userCollections[index]).copy(builtin = false)
118     saveUserCollections()
119 }
120
121 private fun loadUserCollections() {
122     val encoded = prefs?.getString(UserCollectionsKey, null) ?: return
123     runCatching {
124         val array = JSONArray(encoded)
125         for (i in 0 until array.length()) {
126             val item = array.optJSONObject(i) ?: continue
127             val colorsJson = item.optJSONArray("colors") ?: JSONArray()
128             val colors = buildList {
129                 for (j in 0 until colorsJson.length()) {
130                     add(colorsJson.getInt(j))
131                 }
132             }
133             userCollections.add(
134                 ColorCollection(
135                     id = item.optLong("id", System.currentTimeMillis() + i),
136                     name = item.optString("name", "Новая палитра"),
137                     colors = colors,
138                     builtin = false,
139                 ),
140             )
141         }
142     }
143 }
144
145 private fun saveUserCollections() {
146     val array = JSONArray()
147     userCollections.filterNot { it.builtin }.forEach { collection ->
148         val colors = JSONArray()
149         collection.colors.forEach { colors.put(it) }
150         array.put(
151             JSONObject()
152                 .put("id", collection.id)
153                 .put("name", collection.name)
154                 .put("colors", colors),
155         )
156     }
157     prefs?.edit()
158         ?.putString(UserCollectionsKey, array.toString())
159         ?.apply()
160 }
161
162 private companion object {
163     const val PrefsName = "wetinknext_color_state"
164     const val RecentColorsKey = "recent_colors"
165     const val UserCollectionsKey = "user_collections"
166     const val PinnedCollectionKey = "pinned_collection"
167     const val PinnedPaletteOffsetXKey = "pinned_palette_offset_x"
168     const val PinnedPaletteOffsetYKey = "pinned_palette_offset_y"
169     const val NoPinnedCollection = Long.MIN_VALUE
170     const val DefaultPinnedOffsetX = 84f
171     const val DefaultPinnedOffsetY = 128f
172     const val MaxRecentColors = 14
173 }
174 }

```

92. app/src/main/java/com/wetinknext/ui/components/AdjustmentsPanel.kt

```

1  package com.wetinknext.ui.components
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.*
7  import androidx.compose.foundation.rememberScrollState
8  import androidx.compose.foundation.shape.RoundedCornerShape
9  import androidx.compose.foundation.verticalScroll
10 import androidx.compose.material3.HorizontalDivider
11 import androidx.compose.material3.Text
12 import androidx.compose.runtime.Composable
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.draw.clip
16 import androidx.compose.ui.draw.shadow
17 import androidx.compose.ui.graphics.Color
18 import androidx.compose.ui.text.font.FontWeight
19 import androidx.compose.ui.unit.dp
20 import androidx.compose.ui.unit.sp
21 import com.wetinknext.ui.theme.AppTheme
22
23 enum class AdjustmentAction(val title: String) {
24     BRIGHTNESS_CONTRAST("Яркость и контраст"),
25     THRESHOLD("Бинаризация"),
26     BRIGHTNESS_TO_OPACITY("Преобразовать яркость в непрозрачность"),
27 }
28
29 @Composable
30 fun AdjustmentsPanel(
31     theme: AppTheme,
32     onClose: () -> Unit,
33     onAction: (AdjustmentAction) -> Unit,
34     modifier: Modifier = Modifier
35 ) {
36     Column(
37         modifier = modifier
38         .width(280.dp)
39         .shadow(8.dp, RoundedCornerShape(16.dp))
40         .clip(RoundedCornerShape(16.dp))
41         .background(theme.panelBg.copy(alpha = 0.98f))
42         .border(1.dp, theme.panelStroke, RoundedCornerShape(16.dp))
43         .padding(vertical = 8.dp)
44     ) {
45         Text(
46             text = "Коррекция",
47             color = theme.textPrimary,
48             fontSize = 16.sp,
49             fontWeight = FontWeight.Bold,
50             modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp)
51         )
52
53         Column(
54             modifier = Modifier
55                 .fillMaxWidth()
56                 .verticalScroll(rememberScrollState())
57         ) {
58             AdjustmentAction.entries.forEachIndexed { index, action ->
59                 AdjustmentItem(
60                     title = action.title,
61                     theme = theme,
62                     onClick = {
63                         onClose()
64                         onAction(action)
65                     }
66                 )
67                 if (index < AdjustmentAction.entries.size - 1) {
68                     HorizontalDivider(
69                         modifier = Modifier.padding(horizontal = 16.dp),
70                         thickness = 0.5.dp,
71                         color = theme.panelStroke.copy(alpha = 0.5f)
72                     )
73                 }
74             }
75         }
76     }
77 }
78
79 @Composable
80 private fun AdjustmentItem(
81     title: String,
82     theme: AppTheme,
83     onClick: () -> Unit
84 ) {
85     Row(
86         modifier = Modifier
87             .fillMaxWidth()
88             .clickable { onClick() }
89             .padding(horizontal = 16.dp, vertical = 12.dp),
90         verticalAlignment = Alignment.CenterVertically,

```

```
91         horizontalArrangement = Arrangement.SpaceBetween
92     ) {
93         Text(
94             text = title,
95             color = theme.textPrimary,
96             fontSize = 14.sp,
97             modifier = Modifier.weight(1f)
98         )
99     }
100 }
```

93. app/src/main/java/com/wetinknext/ui/components/BrightnessContrastPanel.kt

```

1  package com.wetinknext.ui.components
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.*
7  import androidx.compose.foundation.shape.CircleShape
8  import androidx.compose.foundation.shape.RoundedCornerShape
9  import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.filled.Check
11 import androidx.compose.material.icons.filled.Close
12 import androidx.compose.material3.Icon
13 import androidx.compose.material3.Slider
14 import androidx.compose.material3.SliderDefaults
15 import androidx.compose.material3.Text
16 import androidx.compose.runtime.Composable
17 import androidx.compose.ui.Alignment
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.draw.clip
20 import androidx.compose.ui.graphics.Color
21 import androidx.compose.ui.text.font.FontWeight
22 import androidx.compose.ui.unit.dp
23 import androidx.compose.ui.unit.sp
24 import com.wetinknext.ui.theme.AppTheme
25 import kotlin.math.roundToInt
26
27 @Composable
28 fun BrightnessContrastPanel(
29     theme: AppTheme,
30     brightness: Float, // -1..1
31     contrast: Float,   // -1..1
32     onBrightnessChange: (Float) -> Unit,
33     onContrastChange: (Float) -> Unit,
34     onApply: () -> Unit,
35     onCancel: () -> Unit,
36     isApplyEnabled: Boolean = true,
37     modifier: Modifier = Modifier
38 ) {
39     Column(
40         modifier = modifier
41             .width(360.dp)
42             .clip(RoundedCornerShape(28.dp))
43             .background(theme.panelBg.copy(alpha = 0.95f))
44             .border(1.2.dp, theme.panelStroke, RoundedCornerShape(28.dp))
45             .padding(horizontal = 20.dp, vertical = 16.dp),
46         verticalArrangement = Arrangement.spacedBy(16.dp)
47     ) {
48         Row(
49             modifier = Modifier.fillMaxWidth(),
50             horizontalArrangement = Arrangement.SpaceBetween,
51             verticalAlignment = Alignment.CenterVertically
52         ) {
53             Text(
54                 text = "Яркость и контраст",
55                 color = theme.textPrimary,
56                 fontSize = 15.sp,
57                 fontWeight = FontWeight.Bold
58             )
59
60             Row(
61                 verticalAlignment = Alignment.CenterVertically,
62                 horizontalArrangement = Arrangement.spacedBy(12.dp)
63             ) {
64                 Box(
65                     modifier = Modifier
66                         .size(36.dp)
67                         .clip(CircleShape)
68                         .background(theme.panelInset)
69                         .clickable { onCancel() },
70                     contentAlignment = Alignment.Center
71                 ) {
72                     Icon(Icons.Default.Close, null, tint = theme.textPrimary, modifier = Modifier.size(20.dp))
73                 }
74
75                 Box(
76                     modifier = Modifier
77                         .size(36.dp)
78                         .clip(CircleShape)
79                         .background(if (isApplyEnabled) theme.accent else theme.panelInset)
80                         .clickable(enabled = isApplyEnabled) { onApply() },
81                     contentAlignment = Alignment.Center
82                 ) {
83                     Icon(Icons.Default.Check, null, tint = Color.White, modifier = Modifier.size(20.dp))
84                 }
85             }
86         }
87
88         // Brightness
89         Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
90             Row(

```

```

91         modifier = Modifier.fillMaxWidth(),
92         horizontalArrangement = Arrangement.SpaceBetween
93     ) {
94         Text("Яркость", color = theme.textSecondary, fontSize = 13.sp)
95         Text("${(brightness * 100).roundToInt()}", color = theme.accent, fontSize = 13.sp, fontWeight = FontWeight.
Bold)
96     }
97     Slider(
98         value = brightness,
99         onValueChange = onBrightnessChange,
100        valueRange = -1f..1f,
101        colors = SliderDefaults.colors(
102            thumbColor = theme.accent,
103            activeTrackColor = theme.accent,
104            inactiveTrackColor = theme.panelInset
105        )
106    )
107 }
108
109 // Contrast
110 Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
111     Row(
112         modifier = Modifier.fillMaxWidth(),
113         horizontalArrangement = Arrangement.SpaceBetween
114     ) {
115         Text("Контраст", color = theme.textSecondary, fontSize = 13.sp)
116         Text("${(contrast * 100).roundToInt()}", color = theme.accent, fontSize = 13.sp, fontWeight = FontWeight.Bold)
117     }
118     Slider(
119         value = contrast,
120         onValueChange = onContrastChange,
121         valueRange = -1f..1f,
122         colors = SliderDefaults.colors(
123             thumbColor = theme.accent,
124             activeTrackColor = theme.accent,
125             inactiveTrackColor = theme.panelInset
126         )
127     )
128 }
129 }
130 }

```

94. app/src/main/java/com/wetinknext/ui/components/BrightnessToOpacityPanel.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.border
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.shape.CircleShape
8 import androidx.compose.foundation.shape.RoundedCornerShape
9 import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.filled.Check
11 import androidx.compose.material.icons.filled.Close
12 import androidx.compose.material3.Icon
13 import androidx.compose.material3.Switch
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.draw.clip
19 import androidx.compose.ui.graphics.Color
20 import androidx.compose.ui.text.font.FontWeight
21 import androidx.compose.ui.unit.dp
22 import androidx.compose.ui.unit.sp
23 import com.wetinknext.ui.theme.AppTheme
24
25 @Composable
26 fun BrightnessToOpacityPanel(
27     theme: AppTheme,
28     isInverted: Boolean,
29     onInvertChange: (Boolean) -> Unit,
30     onApply: () -> Unit,
31     onCancel: () -> Unit,
32     isApplyEnabled: Boolean = true,
33     modifier: Modifier = Modifier
34 ) {
35     Column(
36         modifier = modifier
37         .width(360.dp)
38         .clip(RoundedCornerShape(28.dp))
39         .background(theme.panelBg.copy(alpha = 0.95f))
40         .border(1.2.dp, theme.panelStroke, RoundedCornerShape(28.dp))
41         .padding(horizontal = 20.dp, vertical = 16.dp),
42         verticalArrangement = Arrangement.spacedBy(16.dp)
43     ) {
44         Row(
45             modifier = Modifier.fillMaxWidth(),
46             horizontalArrangement = Arrangement.SpaceBetween,
47             verticalAlignment = Alignment.CenterVertically
48         ) {
49             Text(
50                 text = "Яркость в непрозрачность",
51                 color = theme.textPrimary,
52                 fontSize = 15.sp,
53                 fontWeight = FontWeight.Bold
54             )
55
56             Row(
57                 verticalAlignment = Alignment.CenterVertically,
58                 horizontalArrangement = Arrangement.spacedBy(12.dp)
59             ) {
60                 Box(
61                     modifier = Modifier
62                     .size(36.dp)
63                     .clip(CircleShape)
64                     .background(theme.panelInset)
65                     .clickable { onCancel() },
66                     contentAlignment = Alignment.Center
67                 ) {
68                     Icon(Icons.Default.Close, null, tint = theme.textPrimary, modifier = Modifier.size(20.dp))
69                 }
70
71                 Box(
72                     modifier = Modifier
73                     .size(36.dp)
74                     .clip(CircleShape)
75                     .background(if (isApplyEnabled) theme.accent else theme.panelInset)
76                     .clickable(enabled = isApplyEnabled) { onApply() },
77                     contentAlignment = Alignment.Center
78                 ) {
79                     Icon(Icons.Default.Check, null, tint = Color.White, modifier = Modifier.size(20.dp))
80                 }
81             }
82         }
83
84         Text(
85             text = "Светлые области станут прозрачными, тёмные – непрозрачными",
86             color = theme.textSecondary,
87             fontSize = 12.sp,
88             lineHeight = 16.sp
89         )
90     }

```

```

91         Row(
92             modifier = Modifier.fillMaxWidth(),
93             verticalAlignment = Alignment.CenterVertically,
94             horizontalArrangement = Arrangement.SpaceBetween
95         ) {
96             Text(
97                 text = "Инвертировать",
98                 color = theme.textPrimary,
99                 fontSize = 14.sp
100            )
101            Switch(
102                checked = isInverted,
103                onCheckedChange = onInvertChange
104            )
105        }
106    }
107 }

```

95. app/src/main/java/com/wetinknext/ui/components/BrushPanel.kt

```

1  package com.wetinknext.ui.components
2
3  import androidx.compose.foundation.Canvas
4  import androidx.compose.foundation.Image
5  import androidx.compose.foundation.background
6  import androidx.compose.foundation.border
7  import androidx.compose.foundation.clickable
8  import androidx.compose.foundation.gestures.detectTapGestures
9  import androidx.compose.foundation.layout.*
10 import androidx.compose.foundation.lazy.LazyColumn
11 import androidx.compose.foundation.lazy.items
12 import androidx.compose.foundation.shape.CircleShape
13 import androidx.compose.foundation.shape.RoundedCornerShape
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.filled.*
16 import androidx.compose.material3.Icon
17 import androidx.compose.material3.Text
18 import androidx.compose.runtime.*
19 import androidx.compose.ui.Alignment
20 import androidx.compose.ui.Modifier
21 import androidx.compose.ui.draw.clip
22 import androidx.compose.ui.draw.drawBehind
23 import androidx.compose.ui.geometry.Offset
24 import androidx.compose.ui.geometry.Size
25 import androidx.compose.ui.graphics.Color
26 import androidx.compose.ui.graphics.ImageBitmap
27 import androidx.compose.ui.graphics.vector.ImageVector
28 import androidx.compose.ui.input.pointer.pointerInput
29 import androidx.compose.ui.layout.ContentScale
30 import androidx.compose.ui.text.font.FontWeight
31 import androidx.compose.ui.unit.dp
32 import androidx.compose.ui.unit.sp
33 import com.wetinknext.engine.brush.BrushLibrary
34 import com.wetinknext.engine.brush.BrushPreset
35 import com.wetinknext.engine.brush.BrushUiCategory
36 import com.wetinknext.ui.theme.AppTheme
37
38 private val BRUSH_SWIPE_REVEAL_WIDTH = 88.dp
39
40 @Composable
41 fun BrushPanel(
42     currentBrush: BrushPreset,
43     onBrushSelect: (BrushPreset) -> Unit,
44     onBrushStudioOpen: (BrushPreset) -> Unit,
45     theme: AppTheme,
46     onDismiss: () -> Unit,
47     onDuplicate: (BrushPreset) -> Unit,
48     onDelete: (BrushPreset) -> Unit,
49     modifier: Modifier = Modifier,
50     categories: List<BrushUiCategory> = BrushLibrary.allCategories,
51     previews: Map<String, ImageBitmap> = emptyMap(),
52     previewKey: (BrushPreset) -> String = { it.id },
53     onRequestPreview: (BrushPreset) -> Unit = {},
54     title: String = "Кисти",
55 ) {
56     fun categoryIndexForCurrentBrush(): Int {
57         return categories.indexOfFirst { category -> category.brushes.any { it.id == currentBrush.id } }
58     }
59     ?.takeIf { it >= 0 }
60     ?: 0
61
62     var selectedCategoryIndex by remember(categories, currentBrush.id) {
63         mutableIntStateOf(categoryIndexForCurrentBrush())
64     }
65     LaunchedEffect(categories, currentBrush.id) {
66         selectedCategoryIndex = categoryIndexForCurrentBrush()
67     }
68     val selectedCategory = categories.getOrNull(selectedCategoryIndex)
69
70     PanelSurface(
71         modifier = modifier
72             .width(420.dp)
73             .height(560.dp),
74     ) {
75         Row(Modifier.fillMaxSize()) {
76             Column(
77                 modifier = Modifier
78                     .width(70.dp)
79                     .fillMaxHeight()
80                     .clip(RoundedCornerShape(topStart = 22.dp, bottomStart = 22.dp))
81                     .background(theme.panelInset)
82                     .drawBehind {
83                         drawLine(
84                             color = theme.panelStroke,
85                             start = Offset(size.width, 0f),
86                             end = Offset(size.width, size.height),
87                             strokeWidth = 1.dp.toPx(),
88                         )
89                     }
90             )
91             .padding(vertical = 16.dp),
92             horizontalAlignment = Alignment.CenterHorizontally,

```

```

91         verticalArrangement = Arrangement.spacedBy(20.dp),
92     ) {
93         categories.forEachIndexed { index, category ->
94             val isSelected = index == selectedCategoryId
95             Box(
96                 modifier = Modifier
97                     .size(44.dp)
98                     .clip(CircleShape)
99                     .background(if (isSelected) theme.accent.copy(alpha = 0.3f) else Color.Transparent)
100                     .border(
101                         if (isSelected) 1.dp else 0.dp,
102                         if (isSelected) theme.accentSoft else Color.Transparent,
103                         CircleShape,
104                     )
105                 .clickable { selectedCategoryId = index },
106                 contentAlignment = Alignment.Center,
107             ) {
108                 Icon(
109                     category.icon,
110                     contentDescription = category.name,
111                     tint = if (isSelected) theme.accent else theme.iconInactive,
112                     modifier = Modifier.size(24.dp),
113                 )
114             }
115         }
116     }
117
118     Column(
119         modifier = Modifier
120             .weight(1f)
121             .fillMaxHeight()
122             .padding(14.dp),
123     ) {
124         Row(
125             verticalAlignment = Alignment.CenterVertically,
126             horizontalArrangement = Arrangement.spacedBy(12.dp)
127         ) {
128             Column(verticalArrangement = Arrangement.spacedBy(1.dp)) {
129                 Text(
130                     text = title,
131                     color = theme.textPrimary,
132                     fontSize = 18.sp,
133                     fontWeight = FontWeight.Bold,
134                 )
135                 selectedCategory?.let { category ->
136                     Text(
137                         text = category.name,
138                         color = theme.accent,
139                         fontSize = 11.sp,
140                         fontWeight = FontWeight.SemiBold,
141                     )
142                 }
143             }
144             Spacer(Modifier.weight(1f))
145
146             Icon(
147                 imageVector = Icons.Default.Close,
148                 contentDescription = "Закреть",
149                 tint = theme.textSecondary,
150                 modifier = Modifier.size(24.dp).clickable { onDismiss() }
151             )
152         }
153
154         Spacer(Modifier.height(14.dp))
155
156         LazyColumn(
157             verticalArrangement = Arrangement.spacedBy(8.dp),
158             modifier = Modifier.fillMaxSize(),
159         ) {
160             items(selectedCategory?.brushes.orEmpty(), key = { it.id }) { brush ->
161                 val isSelected = brush.id == currentBrush.id
162                 BrushItem(
163                     brush = brush,
164                     isSelected = isSelected,
165                     theme = theme,
166                     preview = previews[previewKey(brush)],
167                     onRequestPreview = { onRequestPreview(brush) },
168                     onStudioOpen = { onBrushStudioOpen(brush) },
169                     onDuplicate = { onDuplicate(brush) },
170                     onDelete = { onDelete(brush) },
171                     onClick = {
172                         if (isSelected) onBrushStudioOpen(brush)
173                         else onBrushSelect(brush)
174                     },
175                 )
176             }
177
178             if (selectedCategory?.brushes.isNullOrEmpty()) {
179                 item {
180                     Text(
181                         text = "Кисти отсутствуют",
182                         color = theme.textSecondary,

```

```

183                                     fontSize = 14.sp,
184                                     modifier = Modifier.padding(top = 20.dp),
185                                 )
186                             }
187                         }
188                     }
189                 }
190             }
191         }
192     }
193
194     @Composable
195     private fun BrushItem(
196         brush: BrushPreset,
197         isSelected: Boolean,
198         theme: AppTheme,
199         preview: ImageBitmap?,
200         onRequestPreview: () -> Unit,
201         onStudioOpen: () -> Unit,
202         onDuplicate: () -> Unit,
203         onDelete: () -> Unit,
204         onClick: () -> Unit,
205     ) {
206         val contentBg = if (isSelected) theme.panelBgSolid else theme.panelInset
207         LaunchedEffect(brush.id, brush.settings, preview) {
208             if (preview == null) {
209                 onRequestPreview()
210             }
211         }
212
213         SwipeRevealRow(
214             revealWidth = BRUSH_SWIPE_REVEAL_WIDTH,
215             onSwipeRight = onStudioOpen,
216             modifier = Modifier
217                 .fillMaxWidth()
218                 .clip(RoundedCornerShape(8.dp))
219                 .background(theme.panelInset),
220             actions = {
221                 BrushSwipeActions(
222                     theme = theme,
223                     onDuplicate = onDuplicate,
224                     onDelete = onDelete,
225                     canDelete = true // Simplified for now
226                 )
227             },
228         ) {
229             Box(
230                 modifier = Modifier
231                     .fillMaxWidth()
232                     .background(contentBg)
233                     .border(1.dp, if (isSelected) theme.accentSoft else theme.panelStroke, RoundedCornerShape(8.dp))
234                     .pointerInput(brush.id) {
235                         detectTapGestures(
236                             onTap = { onClick() },
237                         )
238                     }
239                     .padding(10.dp),
240             ) {
241                 Row(
242                     verticalAlignment = Alignment.CenterVertically,
243                     horizontalArrangement = Arrangement.spacedBy(12.dp),
244                 ) {
245                     BrushPreviewStrip(
246                         preview = preview,
247                         brush = brush,
248                         theme = theme,
249                         modifier = Modifier
250                             .width(142.dp)
251                             .height(46.dp),
252                     )
253                     Column(Modifier.weight(1f)) {
254                         Text(
255                             text = brush.settings.name,
256                             color = if (isSelected) theme.accent else theme.textPrimary,
257                             fontSize = 15.sp,
258                             fontWeight = if (isSelected) FontWeight.SemiBold else FontWeight.Medium,
259                             maxLines = 1,
260                         )
261                         Text(
262                             text = "Размер: ${((brush.settings.baseRadiusPx * 2f).toInt())} px",
263                             color = theme.textSecondary,
264                             fontSize = 12.sp,
265                             maxLines = 1,
266                         )
267                     }
268                     if (isSelected) {
269                         Box(
270                             Modifier
271                                 .size(8.dp)
272                                 .clip(CircleShape)
273                                 .background(theme.accent),
274                         )
275                     }
276                 }
277             }
278         }
279     }

```

```

275         }
276     }
277 }
278 }
279 }
280
281 @Composable
282 private fun BrushSwipeActions(
283     theme: AppTheme,
284     onDuplicate: () -> Unit,
285     onDelete: () -> Unit,
286     canDelete: Boolean
287 ) {
288     Row(
289         modifier = Modifier
290             .fillMaxWidth()
291             .fillMaxHeight(),
292         horizontalArrangement = Arrangement.End,
293         verticalAlignment = Alignment.CenterVertically,
294     ) {
295         BrushSwipeButton(
296             icon = Icons.Default.ContentCopy,
297             label = "Дубль",
298             color = theme.accent,
299             onClick = onDuplicate
300         )
301         BrushSwipeButton(
302             icon = Icons.Default.Delete,
303             label = "Удал.",
304             color = theme.danger,
305             enabled = canDelete,
306             onClick = onDelete
307         )
308     }
309 }
310
311 @Composable
312 private fun BrushSwipeButton(
313     icon: ImageVector,
314     label: String,
315     color: Color,
316     enabled: Boolean = true,
317     onClick: () -> Unit,
318 ) {
319     Column(
320         modifier = Modifier
321             .width(44.dp)
322             .fillMaxHeight()
323             .background(if (enabled) color else color.copy(alpha = 0.3f))
324             .clickable(enabled = enabled, onClick = onClick)
325             .padding(vertical = 4.dp),
326         horizontalAlignment = Alignment.CenterHorizontally,
327         verticalArrangement = Arrangement.Center,
328     ) {
329         Icon(icon, contentDescription = label, tint = Color.White, modifier = Modifier.size(18.dp))
330         Text(label, color = Color.White, fontSize = 8.sp, fontWeight = FontWeight.Medium)
331     }
332 }
333
334 @Composable
335 private fun BrushPreviewStrip(
336     preview: ImageBitmap?,
337     brush: BrushPreset,
338     theme: AppTheme,
339     modifier: Modifier = Modifier,
340 ) {
341     Box(
342         modifier = modifier
343             .clip(RoundedCornerShape(4.dp))
344             .background(Color.White)
345             .border(1.dp, theme.panelStroke, RoundedCornerShape(4.dp)),
346         contentAlignment = Alignment.Center,
347     ) {
348         PreviewCheckerboard(Modifier.fillMaxSize(), cellSize = 7f)
349         if (preview != null) {
350             Image(
351                 bitmap = preview,
352                 contentDescription = null,
353                 contentScale = ContentScale.Fit,
354                 modifier = Modifier.fillMaxSize(),
355             )
356         } else {
357             Canvas(Modifier.fillMaxSize()) {
358                 drawLine(
359                     color = theme.accent.copy(alpha = brush.settings.opacity.coerceIn(0.25f, 1f)),
360                     start = Offset(size.width * 0.12f, size.height * 0.5f),
361                     end = Offset(size.width * 0.88f, size.height * 0.5f),
362                     strokeWidth = (brush.settings.baseRadiusPx * 2f).coerceIn(2f, 12f),
363                 )
364             }
365         }
366     }

```

```

367 }
368
369 @Composable
370 private fun PreviewCheckerboard(
371     modifier: Modifier = Modifier,
372     cellSize: Float,
373 ) {
374     Canvas(modifier) {
375         val light = Color(0xFFE9E9E9)
376         val dark = Color(0xFFD0D0D0)
377         var y = 0f
378         var row = 0
379         while (y < size.height) {
380             var x = 0f
381             var col = 0
382             while (x < size.width) {
383                 drawRect(
384                     color = if ((row + col) % 2 == 0) light else dark,
385                     topLeft = Offset(x, y),
386                     size = Size(cellSize, cellSize),
387                 )
388                 x += cellSize
389                 col += 1
390             }
391             y += cellSize
392             row += 1
393         }
394     }
395 }

```

96. app/src/main/java/com/wetinknext/ui/components/LayersPanel.kt

```

1  package com.wetinknext.ui.components
2
3  import android.graphics.BitmapFactory
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.border
6  import androidx.compose.foundation.clickable
7  import androidx.compose.foundation.layout.*
8  import androidx.compose.foundation.lazy.LazyColumn
9  import androidx.compose.foundation.lazy.items
10 import androidx.compose.foundation.shape.CircleShape
11 import androidx.compose.foundation.shape.RoundedCornerShape
12 import androidx.compose.material.icons.Icons
13 import androidx.compose.material.icons.filled.*
14 import androidx.compose.material3.*
15 import androidx.compose.runtime.*
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.draw.clip
19 import androidx.compose.ui.graphics.Color
20 import androidx.compose.ui.graphics.asImageBitmap
21 import androidx.compose.ui.graphics.vector.ImageVector
22 import androidx.compose.ui.layout.ContentScale
23 import androidx.compose.ui.unit.Dp
24 import androidx.compose.ui.text.font.FontWeight
25 import androidx.compose.ui.text.style.TextOverflow
26 import androidx.compose.ui.unit.dp
27 import androidx.compose.ui.unit.sp
28 import com.wetinknext.ui.state.LayerItem
29 import com.wetinknext.ui.state.LayerState
30 import com.wetinknext.ui.theme.AppTheme
31 import com.wetinknext.ui.thumbnail.LayerThumbnail
32 import java.io.File
33 import kotlinx.coroutines.Dispatchers
34 import kotlinx.coroutines.withContext
35 import kotlin.math.roundToInt
36
37 private val PANEL_WIDTH = 320.dp
38
39 @Composable
40 fun LayersPanel(
41     state: LayerState,
42     theme: AppTheme,
43     modifier: Modifier = Modifier,
44     onDismiss: () -> Unit,
45     onAddLayer: () -> Unit,
46     onSelectLayer: (Long) -> Unit,
47     onVisibleChange: (Long, Boolean) -> Unit,
48     onOpacityChange: (Long, Float) -> Unit,
49     onRemoveLayer: (Long) -> Unit,
50 ) {
51     PanelSurface(
52         modifier = modifier.width(PANEL_WIDTH),
53     ) {
54         Column(
55             modifier = Modifier.padding(10.dp),
56             verticalArrangement = Arrangement.spacedBy(8.dp),
57         ) {
58             Row(
59                 verticalAlignment = Alignment.CenterVertically,
60                 horizontalArrangement = Arrangement.spacedBy(6.dp),
61             ) {
62                 Text(
63                     "Слой",
64                     color = theme.textPrimary,
65                     fontWeight = FontWeight.Bold,
66                     fontSize = 18.sp,
67                 )
68                 Spacer(Modifier.weight(1f))
69                 HeaderIconButton(
70                     theme = theme,
71                     icon = Icons.Default.Add,
72                     description = "Добавить слой",
73                     onClick = onAddLayer,
74                 )
75                 HeaderIconButton(
76                     theme = theme,
77                     icon = Icons.Default.Close,
78                     description = "Закрыть",
79                     onClick = onDismiss,
80                 )
81             }
82
83             LazyColumn(
84                 modifier = Modifier
85                     .fillMaxWidth()
86                     .heightIn(max = 400.dp),
87                 verticalArrangement = Arrangement.spacedBy(4.dp),
88             ) {
89                 items(
90                     items = state.layers.asReversed(),

```

```

91         key = { it.id },
92     ) { layerItem ->
93         LayerRow(
94             layer = layerItem,
95             theme = theme,
96             active = layerItem.id == state.activeLayerId,
97             onSelect = { onSelectLayer(layerItem.id) },
98             onVisibleChange = { onVisibleChange(layerItem.id, it) },
99             onOpacityChange = { onOpacityChange(layerItem.id, it) },
100             onDelete = { onRemoveLayer(layerItem.id) },
101             canDelete = state.layers.size > 1 && !layerItem.locked
102         )
103     }
104 }
105 }
106 }
107 }
108
109 @Composable
110 private fun LayerRow(
111     layer: LayerItem,
112     theme: AppTheme,
113     active: Boolean,
114     onSelect: () -> Unit,
115     onVisibleChange: (Boolean) -> Unit,
116     onOpacityChange: (Float) -> Unit,
117     onDelete: () -> Unit,
118     canDelete: Boolean,
119 ) {
120     val cardBg = if (active) theme.panelBgSolid else theme.panelBg
121
122     Column(
123         modifier = Modifier
124             .fillMaxWidth()
125             .clip(RoundedCornerShape(12.dp))
126             .background(cardBg)
127             .border(
128                 width = if (active) 1.2.dp else 0.dp,
129                 color = if (active) theme.accent else Color.Transparent,
130                 shape = RoundedCornerShape(12.dp),
131             )
132         .clickable { onSelect() }
133         .padding(8.dp),
134         verticalArrangement = Arrangement.spacedBy(4.dp)
135     ) {
136         Row(
137             modifier = Modifier.fillMaxWidth(),
138             verticalAlignment = Alignment.CenterVertically,
139         ) {
140             HeaderIconButton(
141                 theme = theme,
142                 icon = if (layer.visible) Icons.Default.Visibility else Icons.Default.VisibilityOff,
143                 description = "Видимость",
144                 onClick = { onVisibleChange(!layer.visible) },
145                 tint = if (layer.visible) (if (active) theme.accent else theme.textPrimary) else theme.iconInactive,
146             )
147
148             Spacer(Modifier.width(8.dp))
149
150             LayerThumbnail(
151                 layerId = layer.id,
152                 path = layer.thumbnailPath,
153                 thumbnailVersion = layer.thumbnailVersion,
154                 modifier = Modifier.size(48.dp, 36.dp),
155             )
156
157             Spacer(Modifier.width(8.dp))
158
159             Text(
160                 layer.name,
161                 modifier = Modifier.weight(1f),
162                 color = theme.textPrimary,
163                 fontSize = 14.sp,
164                 fontWeight = if (active) FontWeight.SemiBold else FontWeight.Normal,
165                 maxLines = 1,
166                 overflow = TextOverflow.Ellipsis,
167             )
168
169             if (canDelete) {
170                 HeaderIconButton(
171                     theme = theme,
172                     icon = Icons.Default.Delete,
173                     description = "Удалить",
174                     onClick = onDelete,
175                     tint = theme.danger
176                 )
177             }
178         }
179
180         if (active) {
181             Column(modifier = Modifier.fillMaxWidth().padding(horizontal = 4.dp)) {
182                 Row(verticalAlignment = Alignment.CenterVertically) {

```

```

183         Text("Непрозрачность", color = theme.textSecondary, fontSize = 11.sp)
184         Spacer(Modifier.weight(1f))
185         Text(
186             "${(layer.opacity * 100f).roundToInt()}%",
187             color = theme.textSecondary,
188             fontSize = 11.sp,
189         )
190     }
191     Slider(
192         value = layer.opacity.coerceIn(0f, 1f),
193         onValueChange = onOpacityChange,
194         valueRange = 0f..1f,
195         colors = SliderDefaults.colors(
196             thumbColor = theme.accent,
197             activeTrackColor = theme.accent,
198             inactiveTrackColor = theme.accentMuted,
199         ),
200     )
201 }
202 }
203 }
204 }
205
206 /** Small, non-blocking thumbnail loader for the persisted layer WebP preview. */
207 @Composable
208 private fun LegacyLayerThumbnail(
209     layer: LayerItem,
210     theme: AppTheme,
211     size: Dp = 38.dp,
212 ) {
213     var preview by remember(layer.thumbnailPath, layer.thumbnailVersion) {
214         mutableStateOf<androidx.compose.ui.graphics.ImageBitmap?>(null)
215     }
216     LaunchedEffect(layer.thumbnailPath, layer.thumbnailVersion) {
217         preview = withContext(Dispatchers.IO) {
218             layer.thumbnailPath
219                 ?.let(::File)
220                 ?.takeIf(File::isFile)
221                 ?.let { BitmapFactory.decodeFile(it.absolutePath) }
222                 ?.asImageBitmap()
223         }
224     }
225
226     Box(
227         modifier = Modifier
228             .size(size)
229             .clip(RoundedCornerShape(7.dp))
230             .background(theme.panelInset)
231             .border(1.dp, theme.panelStroke, RoundedCornerShape(7.dp)),
232         contentAlignment = Alignment.Center,
233     ) {
234         if (preview != null) {
235             androidx.compose.foundation.Image(
236                 bitmap = checkNotNull(preview),
237                 contentDescription = "Превью слоя ${layer.name}",
238                 modifier = Modifier.fillMaxSize(),
239                 contentScale = ContentScale.Fit,
240             )
241         } else {
242             Text(
243                 text = layer.name.take(1).uppercase(),
244                 color = theme.iconInactive,
245                 fontSize = 12.sp,
246                 fontWeight = FontWeight.Bold,
247             )
248         }
249     }
250 }
251
252 @Composable
253 private fun HeaderIconButton(
254     theme: AppTheme,
255     icon: ImageVector,
256     description: String,
257     onClick: () -> Unit,
258     enabled: Boolean = true,
259     tint: Color = theme.textPrimary,
260 ) {
261     Box(
262         modifier = Modifier
263             .size(32.dp)
264             .clip(CircleShape)
265             .background(if (enabled) theme.panelInsetSoft else theme.panelInset)
266             .clickable(enabled = enabled, onClick = onClick),
267         contentAlignment = Alignment.Center,
268     ) {
269         Icon(
270             icon,
271             contentDescription = description,
272             tint = if (enabled) tint else theme.iconInactive,
273             modifier = Modifier.size(18.dp),
274         )
275     }
276 }

```

```
275     }  
276 }
```

97. app/src/main/java/com/wetinknext/ui/components/PanelSurface.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.border
5 import androidx.compose.foundation.layout.Box
6 import androidx.compose.foundation.layout.BoxScope
7 import androidx.compose.foundation.shape.RoundedCornerShape
8 import androidx.compose.runtime.Composable
9 import androidx.compose.ui.Modifier
10 import androidx.compose.ui.draw.clip
11 import androidx.compose.ui.draw.shadow
12 import androidx.compose.ui.input.pointer.PointerEventPass
13 import androidx.compose.ui.input.pointer.pointerInput
14 import androidx.compose.ui.unit.dp
15 import com.wetinknext.ui.theme.LocalAppTheme
16
17 @Composable
18 fun PanelSurface(
19     modifier: Modifier = Modifier,
20     content: @Composable BoxScope.() -> Unit,
21 ) {
22     val theme = LocalAppTheme.current
23     val shape = RoundedCornerShape(22.dp)
24     Box(
25         modifier = modifier
26             .shadow(12.dp, shape, ambientColor = theme.panelInset.copy(alpha = 0.5f), spotColor = theme.panelInset.copy(alpha
= 0.5f))
27             .clip(shape)
28             .background(theme.panelBg)
29             .border(1.2.dp, theme.panelStroke, shape)
30     ) {
31         Box(modifier = Modifier.matchParentSize().swallowChrome())
32         content()
33     }
34 }
35
36 private fun Modifier.swallowChrome(): Modifier = pointerInput(Unit) {
37     awaitPointerEventScope {
38         while (true) {
39             val event = awaitPointerEvent(PointerEventPass.Initial)
40             for (change in event.changes) {
41                 if (change.pressed && !change.previousPressed) {
42                     change.consume()
43                 }
44             }
45         }
46     }
47 }

```

98. app/src/main/java/com/wetinknext/ui/components/SelectionToolbar.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.foundation.BorderStroke
4 import androidx.compose.foundation.background
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.shape.CircleShape
8 import androidx.compose.foundation.shape.RoundedCornerShape
9 import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.automirrored.filled.CallMerge
11 import androidx.compose.material.icons.filled.*
12 import androidx.compose.material3.Icon
13 import androidx.compose.material3.Surface
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.draw.clip
19 import androidx.compose.ui.graphics.Color
20 import androidx.compose.ui.graphics.vector.ImageVector
21 import androidx.compose.ui.text.font.FontWeight
22 import androidx.compose.ui.unit.dp
23 import androidx.compose.ui.unit.sp
24 import com.wetinknext.ui.theme.AppTheme
25
26 enum class SelectionShapeUi {
27     FREEHAND,
28     RECTANGLE,
29     ELLIPSE,
30 }
31
32 @Composable
33 fun SelectionToolbar(
34     theme: AppTheme,
35     currentShape: SelectionShapeUi,
36     onShapeChange: (SelectionShapeUi) -> Unit,
37     onDone: () -> Unit,
38     modifier: Modifier = Modifier,
39 ) {
40     val panelBorder = BorderStroke(1.dp, theme.panelStroke)
41
42     Column(
43         modifier = modifier
44             .fillMaxWidth()
45             .navigationBarsPadding()
46             .padding(horizontal = 16.dp, vertical = 20.dp),
47         horizontalAlignment = Alignment.CenterHorizontally,
48         verticalArrangement = Arrangement.spacedBy(12.dp),
49     ) {
50         // Row 1 – shape modes
51         Surface(
52             color = theme.panelBg.copy(alpha = 0.94f),
53             shape = RoundedCornerShape(24.dp),
54             border = panelBorder,
55         ) {
56             Row(
57                 modifier = Modifier
58                     .width(340.dp)
59                     .padding(4.dp)
60                     .height(32.dp)
61                     .clip(RoundedCornerShape(20.dp))
62                     .background(theme.panelInsetSoft),
63                 verticalAlignment = Alignment.CenterVertically,
64             ) {
65                 val shapes = listOf(
66                     SelectionShapeUi.FREEHAND to "От руки",
67                     SelectionShapeUi.RECTANGLE to "Прямоугольник",
68                     SelectionShapeUi.ELLIPSE to "Эллипс",
69                 )
70                 shapes.forEach { (shape, label) ->
71                     val isSelected = currentShape == shape
72                     Box(
73                         modifier = Modifier
74                             .weight(1f)
75                             .fillMaxHeight()
76                             .clip(RoundedCornerShape(20.dp))
77                             .background(if (isSelected) theme.accent else Color.Transparent)
78                             .clickable { onShapeChange(shape) },
79                         contentAlignment = Alignment.Center,
80                     ) {
81                         Text(
82                             text = label,
83                             color = if (isSelected) Color.White else theme.textPrimary,
84                             fontSize = 13.sp,
85                             fontWeight = if (isSelected) FontWeight.Bold else FontWeight.Normal,
86                             maxLines = 1,
87                         )
88                     }
89                 }
90             }
91         }
92     }
93 }

```

```

91     }
92
93     // Row 2 – Action icons (mostly disabled for now)
94     Surface(
95         color = theme.panelBg.copy(alpha = 0.94f),
96         shape = RoundedCornerShape(24.dp),
97         border = panelBorder,
98     ) {
99         Row(
100             modifier = Modifier.padding(horizontal = 4.dp, vertical = 4.dp),
101             horizontalArrangement = Arrangement.spacedBy(4.dp),
102             verticalAlignment = Alignment.CenterVertically,
103         ) {
104             SelectionIconButton(Icons.Default.Add, "Добавить", theme, isSelected = true, enabled = false)
105             SelectionIconButton(Icons.Default.Delete, "Удалить", theme, enabled = false)
106             SelectionIconButton(Icons.AutoMirrored.Filled.CallMerge, "Пересечь", theme, enabled = false)
107             SelectionIconButton(Icons.Default.Flip, "Инверсия", theme, enabled = false)
108             SelectionIconButton(Icons.Default.FormatColorFill, "Залить", theme, enabled = false)
109             SelectionIconButton(Icons.Default.ContentCopy, "Копировать", theme, enabled = false)
110             SelectionIconButton(Icons.Default.ContentCut, "Вырезать", theme, enabled = false)
111             SelectionIconButton(Icons.Default.AddToPhotos, "Вставить", theme, enabled = false)
112         }
113     }
114
115     // Row 3 – Reset / Delete / Done
116     Row(
117         horizontalArrangement = Arrangement.spacedBy(16.dp),
118         verticalAlignment = Alignment.CenterVertically,
119     ) {
120         SelectionRoundButton(Icons.Default.Refresh, "Сбросить", theme, enabled = false)
121         SelectionRoundButton(Icons.Default.Delete, "Удалить", theme, enabled = false)
122         Surface(
123             color = theme.accent,
124             shape = CircleShape,
125             modifier = Modifier.size(48.dp),
126         ) {
127             Box(
128                 modifier = Modifier
129                     .fillMaxSize()
130                     .clickable { onDone() },
131                 contentAlignment = Alignment.Center,
132             ) {
133                 Icon(Icons.Default.Check, "Готово", tint = Color.White)
134             }
135         }
136     }
137 }
138
139 @Composable
140 private fun SelectionIconButton(
141     icon: ImageVector,
142     description: String,
143     theme: AppTheme,
144     isSelected: Boolean = false,
145     enabled: Boolean = true,
146     onClick: () -> Unit = {},
147 ) {
148     Box(
149         modifier = Modifier
150             .size(48.dp)
151             .clip(RoundedCornerShape(8.dp))
152             .background(if (isSelected && enabled) theme.accent.copy(alpha = 0.30f) else Color.Transparent)
153             .clickable(enabled = enabled) { onClick() },
154         contentAlignment = Alignment.Center,
155     ) {
156         Icon(
157             imageVector = icon,
158             contentDescription = description,
159             tint = if (!enabled) theme.iconInactive.copy(alpha = 0.5f) else if (isSelected) theme.accent else theme.
160 textPrimary,
161             modifier = Modifier.size(24.dp),
162         )
163     }
164 }
165
166 @Composable
167 private fun SelectionRoundButton(
168     icon: ImageVector,
169     description: String,
170     theme: AppTheme,
171     enabled: Boolean = true,
172     onClick: () -> Unit = {},
173 ) {
174     Surface(
175         color = if (enabled) theme.panelBg.copy(alpha = 0.9f) else theme.panelBg.copy(alpha = 0.4f),
176         shape = CircleShape,
177         border = BorderStroke(1.dp, if (enabled) theme.panelStroke else theme.panelStroke.copy(alpha = 0.4f)),
178         modifier = Modifier.size(48.dp),
179     ) {
180         Box(
181             modifier = Modifier

```

```
182         .fillMaxSize()
183         .clickable(enabled = enabled) { onClick() },
184         contentAlignment = Alignment.Center,
185     ) {
186         Icon(
187             icon,
188             description,
189             tint = if (enabled) theme.textPrimary else theme.textPrimary.copy(alpha = 0.4f)
190         )
191     }
192 }
193 }
```

99. app/src/main/java/com/wetinknext/ui/components/SwipeRevealRow.kt

```

1  package com.wetinknext.ui.components
2
3  import androidx.compose.animation.core.animateFloatAsState
4  import androidx.compose.foundation.gestures.detectHorizontalDragGestures
5  import androidx.compose.foundation.layout.Arrangement
6  import androidx.compose.foundation.layout.Box
7  import androidx.compose.foundation.layout.Row
8  import androidx.compose.foundation.layout.RowScope
9  import androidx.compose.foundation.layout.fillMaxWidth
10 import androidx.compose.foundation.layout.offset
11 import androidx.compose.runtime.Composable
12 import androidx.compose.runtime.getValue
13 import androidx.compose.runtime.mutableStateOf
14 import androidx.compose.runtime.remember
15 import androidx.compose.runtime.setValue
16 import androidx.compose.ui.Modifier
17 import androidx.compose.ui.input.pointer.pointerInput
18 import androidx.compose.ui.platform.LocalDensity
19 import androidx.compose.ui.unit.Dp
20 import androidx.compose.ui.unit.IntOffset
21 import androidx.compose.ui.unit.dp
22 import kotlin.math.roundToInt
23
24 @Composable
25 fun SwipeRevealRow(
26     revealWidth: Dp,
27     actions: @Composable RowScope() -> Unit,
28     modifier: Modifier = Modifier,
29     rightSwipeThreshold: Dp = 56.dp,
30     onSwipeRight: (() -> Unit)? = null,
31     content: @Composable () -> Unit,
32 ) {
33     var rawOffset by remember { mutableStateOf(0f) }
34     var rightDragOffset by remember { mutableStateOf(0f) }
35     val revealPx = with(LocalDensity.current) { revealWidth.toPx() }
36     val rightSwipeThresholdPx = with(LocalDensity.current) { rightSwipeThreshold.toPx() }
37     val animatedOffset by animateFloatAsState(
38         targetValue = rawOffset.coerceIn(-revealPx, 0f),
39         label = "swipeRevealRow",
40     )
41
42     Box(modifier = modifier.fillMaxWidth()) {
43         Row(
44             modifier = Modifier.fillMaxWidth(),
45             horizontalArrangement = Arrangement.End,
46             content = actions,
47         )
48
49         Box(
50             modifier = Modifier
51                 .fillMaxWidth()
52                 .offset { IntOffset(animatedOffset.roundToInt(), 0) }
53                 .pointerInput(revealWidth) {
54                     detectHorizontalDragGestures(
55                         onHorizontalDrag = { change, dx ->
56                             if (dx < 0f || rawOffset < 0f) {
57                                 change.consume()
58                                 rawOffset += dx
59                                 rightDragOffset = 0f
60                             } else if (dx > 0f && rawOffset == 0f && onSwipeRight != null) {
61                                 change.consume()
62                                 rightDragOffset += dx
63                             }
64                         },
65                         onDragEnd = {
66                             if (rightDragOffset > rightSwipeThresholdPx) {
67                                 onSwipeRight?.invoke()
68                             }
69                             rawOffset = if (rawOffset < -revealPx * 0.4f) -revealPx else 0f
70                             rightDragOffset = 0f
71                         },
72                         onDragCancel = {
73                             rawOffset = 0f
74                             rightDragOffset = 0f
75                         },
76                     )
77                 },
78     ) {
79         content()
80     }
81 }
82 }
```

100. app/src/main/java/com/wetinknext/ui/components/ThresholdPanel.kt

```

1  package com.wetinknext.ui.components
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.layout.*
7  import androidx.compose.foundation.shape.CircleShape
8  import androidx.compose.foundation.shape.RoundedCornerShape
9  import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.filled.Check
11 import androidx.compose.material.icons.filled.Close
12 import androidx.compose.material3.Icon
13 import androidx.compose.material3.Slider
14 import androidx.compose.material3.SliderDefaults
15 import androidx.compose.material3.Switch
16 import androidx.compose.material3.Text
17 import androidx.compose.runtime.Composable
18 import androidx.compose.ui.Alignment
19 import androidx.compose.ui.Modifier
20 import androidx.compose.ui.draw.clip
21 import androidx.compose.ui.graphics.Color
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.unit.dp
24 import androidx.compose.ui.unit.sp
25 import com.wetinknext.ui.theme.AppTheme
26 import kotlin.math.roundToInt
27
28 @Composable
29 fun ThresholdPanel(
30     theme: AppTheme,
31     threshold: Float, // 0..1
32     preserveTransparency: Boolean,
33     onThresholdChange: (Float) -> Unit,
34     onPreserveTransparencyChange: (Boolean) -> Unit,
35     onApply: () -> Unit,
36     onCancel: () -> Unit,
37     isApplyEnabled: Boolean = true,
38     modifier: Modifier = Modifier
39 ) {
40     Column(
41         modifier = modifier
42             .width(360.dp)
43             .clip(RoundedCornerShape(28.dp))
44             .background(theme.panelBg.copy(alpha = 0.95f))
45             .border(1.2.dp, theme.panelStroke, RoundedCornerShape(28.dp))
46             .padding(horizontal = 20.dp, vertical = 16.dp),
47         verticalArrangement = Arrangement.spacedBy(16.dp)
48     ) {
49         Row(
50             modifier = Modifier.fillMaxWidth(),
51             horizontalArrangement = Arrangement.SpaceBetween,
52             verticalAlignment = Alignment.CenterVertically
53         ) {
54             Text(
55                 text = "Бинаризация",
56                 color = theme.textPrimary,
57                 fontSize = 15.sp,
58                 fontWeight = FontWeight.Bold
59             )
60
61             Row(
62                 verticalAlignment = Alignment.CenterVertically,
63                 horizontalArrangement = Arrangement.spacedBy(12.dp)
64             ) {
65                 Box(
66                     modifier = Modifier
67                         .size(36.dp)
68                         .clip(CircleShape)
69                         .background(theme.panelInset)
70                         .clickable { onCancel() },
71                     contentAlignment = Alignment.Center
72                 ) {
73                     Icon(Icons.Default.Close, null, tint = theme.textPrimary, modifier = Modifier.size(20.dp))
74                 }
75
76                 Box(
77                     modifier = Modifier
78                         .size(36.dp)
79                         .clip(CircleShape)
80                         .background(if (isApplyEnabled) theme.accent else theme.panelInset)
81                         .clickable(enabled = isApplyEnabled) { onApply() },
82                     contentAlignment = Alignment.Center
83                 ) {
84                     Icon(Icons.Default.Check, null, tint = Color.White, modifier = Modifier.size(20.dp))
85                 }
86             }
87         }
88
89         Text(
90             text = "Пиксели светлее порога станут белыми, остальные – чёрными",

```

```

91         color = theme.textSecondary,
92         fontSize = 12.sp,
93         lineHeight = 16.sp
94     )
95
96     // Threshold
97     Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
98         Row(
99             modifier = Modifier.fillMaxWidth(),
100             horizontalArrangement = Arrangement.SpaceBetween
101         ) {
102             Text("Нопор", color = theme.textSecondary, fontSize = 13.sp)
103             Text("${(threshold * 255).roundToInt()}", color = theme.accent, fontSize = 13.sp, fontWeight = FontWeight.
Bold)
104         }
105         Slider(
106             value = threshold,
107             onValueChange = onThresholdChange,
108             valueRange = 0f..1f,
109             colors = SliderDefaults.colors(
110                 thumbColor = theme.accent,
111                 activeTrackColor = theme.accent,
112                 inactiveTrackColor = theme.panelInset
113             )
114         )
115     }
116
117     Row(
118         modifier = Modifier.fillMaxWidth(),
119         verticalAlignment = Alignment.CenterVertically,
120         horizontalArrangement = Arrangement.SpaceBetween
121     ) {
122         Text(
123             text = "Сохранять прозрачность",
124             color = theme.textPrimary,
125             fontSize = 14.sp
126         )
127         Switch(
128             checked = preserveTransparency,
129             onCheckedChange = onPreserveTransparencyChange
130         )
131     }
132 }
133 }

```

101. app/src/main/java/com/wetinknext/ui/components/Toolbars.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.animation.core.Spring
4 import androidx.compose.animation.core.animateFloatAsState
5 import androidx.compose.animation.core.spring
6 import androidx.compose.foundation.background
7 import androidx.compose.foundation.border
8 import androidx.compose.foundation.clickable
9 import androidx.compose.foundation.gestures.detectDragGestures
10 import androidx.compose.foundation.gestures.detectTapGestures
11 import androidx.compose.foundation.layout.*
12 import androidx.compose.foundation.shape.CircleShape
13 import androidx.compose.foundation.shape.RoundedCornerShape
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.automirrored.filled.Redo
16 import androidx.compose.material.icons.automirrored.filled.Undo
17 import androidx.compose.material.icons.filled.*
18 import androidx.compose.material3.Icon
19 import androidx.compose.runtime.*
20 import androidx.compose.ui.Alignment
21 import androidx.compose.ui.Modifier
22 import androidx.compose.ui.draw.clip
23 import androidx.compose.ui.graphics.Brush
24 import androidx.compose.ui.graphics.Color
25 import androidx.compose.ui.graphics.graphicsLayer
26 import androidx.compose.ui.graphics.vector.ImageVector
27 import androidx.compose.ui.input.pointer.pointerInput
28 import androidx.compose.ui.layout.onSizeChanged
29 import androidx.compose.ui.platform.LocalDensity
30 import androidx.compose.ui.res.vectorResource
31 import androidx.compose.ui.unit.IntOffset
32 import androidx.compose.ui.unit.dp
33 import com.wetinknext.R
34 import com.wetinknext.ui.theme.AppTheme
35
36 @Composable
37 fun TopToolbar(
38     theme: AppTheme,
39     currentColor: Color,
40     canUndo: Boolean,
41     canRedo: Boolean,
42     isSelectionActive: Boolean,
43     isTransformActive: Boolean,
44     isAnimationActive: Boolean,
45     isAdjustmentsActive: Boolean = false,
46     onUndoClick: () -> Unit,
47     onRedoClick: () -> Unit,
48     onSelectionClick: () -> Unit,
49     onTransformClick: () -> Unit,
50     onAnimationClick: () -> Unit,
51     onAdjustmentsClick: () -> Unit = {},
52     onSettingsClick: () -> Unit,
53     onLayersClick: () -> Unit,
54     onColorClick: () -> Unit,
55 ) {
56     Row(
57         modifier = Modifier
58             .padding(top = 12.dp, end = 12.dp)
59             .clip(RoundedCornerShape(25.dp))
60             .background(theme.panelBg.copy(alpha = 0.9f))
61             .border(1.2.dp, theme.panelStroke, RoundedCornerShape(25.dp))
62             .padding(horizontal = 8.dp, vertical = 6.dp),
63         verticalAlignment = Alignment.CenterVertically,
64         horizontalArrangement = Arrangement.spacedBy(10.dp)
65     ) {
66         ToolbarIcon(Icons.AutoMirrored.Filled.Undo, theme, isSelected = false, enabled = canUndo, onClick = onUndoClick)
67         ToolbarIcon(Icons.AutoMirrored.Filled.Redo, theme, isSelected = false, enabled = canRedo, onClick = onRedoClick)
68
69         // SelectAll has a visually denser glyph than Fullscreen; its 18 dp drawing size
70         // matches Transform's perceived 20 dp footprint while both buttons stay 34 dp.
71         ToolbarIcon(Icons.Default.SelectAll, theme, isSelected = isSelectionActive, iconSize = 18.dp, onClick =
onSelectionClick)
72         ToolbarIcon(Icons.Default.Movie, theme, isSelected = isAnimationActive, onClick = onAnimationClick)
73         ToolbarIcon(Icons.Default.Fullscreen, theme, isSelected = isTransformActive, onClick = onTransformClick)
74         ToolbarIcon(Icons.Default.Tune, theme, isSelected = isAdjustmentsActive, onClick = onAdjustmentsClick)
75         ToolbarIcon(Icons.Default.Layers, theme, onClick = onLayersClick)
76         ToolbarIcon(Icons.Default.Settings, theme, contentDescription = "Настройки", onClick = onSettingsClick)
77
78         ColorPickerDot(
79             color = currentColor,
80             theme = theme,
81             onClick = onColorClick
82         )
83     }
84 }
85
86 /** Compact actions placed directly under the left brush toolbar. */
87 @Composable
88 fun LeftCanvasActions(
89     theme: AppTheme,

```

```

90     onClearLayerClick: () -> Unit,
91     onMirrorClick: () -> Unit,
92 ) {
93     Column(
94         modifier = Modifier.padding(start = 12.dp, top = 10.dp),
95         verticalArrangement = Arrangement.spacedBy(10.dp),
96         horizontalAlignment = Alignment.CenterHorizontally,
97     ) {
98         CanvasActionButton(
99             icon = Icons.Default.DeleteSweep,
100            description = "Очистить активный слой",
101            theme = theme,
102            onClick = onClearLayerClick,
103        )
104        CanvasActionButton(
105            icon = Icons.Default.Flip,
106            description = "Зеркальный просмотр холста",
107            theme = theme,
108            onClick = onMirrorClick,
109        )
110    }
111 }
112
113 @Composable
114 fun CanvasActionButton(
115     icon: ImageVector,
116     description: String,
117     theme: AppTheme,
118     onClick: () -> Unit,
119 ) {
120     Box(
121         modifier = Modifier
122             .size(44.dp)
123             .clip(CircleShape)
124             .background(theme.panelBg.copy(alpha = 0.9f))
125             .border(1.2.dp, theme.panelStroke, CircleShape)
126             .clickable(onClick = onClick),
127         contentAlignment = Alignment.Center,
128     ) {
129         Icon(icon, contentDescription = description, tint = theme.textPrimary, modifier = Modifier.size(20.dp))
130     }
131 }
132
133 @Composable
134 fun ColorPickerDot(
135     color: Color,
136     theme: AppTheme,
137     onClick: () -> Unit
138 ) {
139     var isPressed by remember { mutableStateOf(false) }
140
141     val scale by animateFloatAsState(
142         targetValue = if (isPressed) 1.15f else 1f,
143         animationSpec = spring(dampingRatio = Spring.DampingRatioMediumBouncy, stiffness = Spring.StiffnessLow),
144         label = "scale"
145     )
146
147     Box(
148         modifier = Modifier
149             .graphicsLayer {
150                 scaleX = scale
151                 scaleY = scale
152             }
153             .size(30.dp)
154             .clip(CircleShape)
155             .background(color)
156             .border(1.5.dp, theme.panelStroke, CircleShape)
157             .clickable { onClick() }
158     )
159 }
160
161 @Composable
162 fun SideToolbar(
163     theme: AppTheme,
164     brushSize: Float,
165     brushOpacity: Float,
166     isEraser: Boolean,
167     onBrushSizeChange: (Float) -> Unit,
168     onBrushOpacityChange: (Float) -> Unit,
169     onBrushClick: () -> Unit,
170     onEraserClick: (Boolean) -> Unit,
171 ) {
172     Column(
173         modifier = Modifier
174             .padding(start = 12.dp)
175             .width(44.dp)
176             .clip(RoundedCornerShape(22.dp))
177             .background(theme.panelBg.copy(alpha = 0.9f))
178             .border(1.2.dp, theme.panelStroke, RoundedCornerShape(22.dp))
179             .padding(vertical = 10.dp),
180         horizontalAlignment = Alignment.CenterHorizontally,
181         verticalArrangement = Arrangement.spacedBy(12.dp)

```

```

182     ) {
183         ToolbarIcon(Icons.Default.Brush, theme, isSelected = !isEraser, onClick = {
184             onEraserClick(false)
185             onBrushClick()
186         })
187         ToolbarIcon(ImageVector.vectorResource(R.drawable.ic_eraser), theme, isSelected = isEraser, onClick = { onEraserClick(
true) })
188
189         Spacer(Modifier.height(4.dp))
190
191         VerticalSlider(
192             value = brushSize,
193             onValueChange = onBrushSizeChange,
194             valueRange = 2f..400f,
195             theme = theme
196         )
197
198         VerticalSlider(
199             value = brushOpacity,
200             onValueChange = onBrushOpacityChange,
201             valueRange = 0f..1f,
202             theme = theme
203         )
204     }
205 }
206
207 @Composable
208 private fun VerticalSlider(
209     value: Float,
210     onValueChange: (Float) -> Unit,
211     valueRange: ClosedFloatingPointRange<Float> = 0f..1f,
212     theme: AppTheme,
213     modifier: Modifier = Modifier
214 ) {
215     val range = valueRange.endInclusive - valueRange.start
216     var trackHeightPx by remember { mutableFloatStateOf(1f) }
217     val density = LocalDensity.current
218
219     val norm = ((value - valueRange.start) / range).coerceIn(0f, 1f)
220
221     fun updateFromY(y: Float) {
222         val n = (1f - (y / trackHeightPx)).coerceIn(0f, 1f)
223         onValueChange(valueRange.start + n * range)
224     }
225
226     Box(
227         modifier = modifier
228             .width(28.dp)
229             .height(120.dp)
230             .clip(RoundedCornerShape(14.dp))
231             .background(theme.accentMuted.copy(alpha = 0.15f))
232             .border(1.dp, theme.panelStroke, RoundedCornerShape(14.dp))
233             .onSizeChanged { trackHeightPx = it.height.toFloat() }
234             .pointerInput(valueRange) {
235                 detectTapGestures { offset -> updateFromY(offset.y) }
236             }
237             .pointerInput(valueRange) {
238                 detectDragGestures { change, _ ->
239                     change.consume()
240                     updateFromY(change.position.y)
241                 }
242             }
243     ) {
244         Box(
245             Modifier
246                 .align(Alignment.BottomCenter)
247                 .fillMaxWidth()
248                 .fillMaxHeight(norm)
249                 .background(Brush.verticalGradient(listOf(theme.accentSoft, theme.accent)))
250         )
251         Box(
252             Modifier
253                 .align(Alignment.TopCenter)
254                 .offset {
255                     val thumbSizePx = with(density) { 22.dp.toPx() }
256                     IntOffset(0, ((1f - norm) * (trackHeightPx - thumbSizePx)).toInt())
257                 }
258                 .padding(2.dp)
259                 .size(22.dp)
260                 .clip(CircleShape)
261                 .background(theme.textPrimary)
262                 .border(1.dp, theme.panelStroke, CircleShape)
263         )
264     }
265 }
266
267 @Composable
268 private fun ToolbarIcon(
269     icon: ImageVector,
270     theme: AppTheme,
271     isSelected: Boolean = false,
272     enabled: Boolean = true,

```

```

273     contentDescription: String? = null,
274     iconSize: androidx.compose.ui.unit.Dp = 20.dp,
275     onClick: () -> Unit = {}
276 ) {
277     Box(
278         modifier = Modifier
279             .size(34.dp)
280             .clip(CircleShape)
281             .background(if (isSelected) theme.accentMuted.copy(alpha = 0.3f) else Color.Transparent)
282             .border(if (isSelected) 1.dp else 0.dp, if (isSelected) theme.accentSoft else Color.Transparent, CircleShape)
283             .clickable(enabled = enabled) { onClick() },
284         contentAlignment = Alignment.Center
285     ) {
286         Icon(
287             icon,
288             contentDescription = contentDescription,
289             tint = if (!enabled) theme.iconInactive.copy(alpha = 0.5f) else if (isSelected) theme.accent else theme.
textPrimary,
290             modifier = Modifier.size(iconSize)
291         )
292     }
293 }

```

102. app/src/main/java/com/wetinknext/ui/components/TransformMenuView.kt

```

1 package com.wetinknext.ui.components
2
3 import androidx.compose.foundation.BorderStroke
4 import androidx.compose.foundation.background
5 import androidx.compose.foundation.clickable
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.shape.CircleShape
8 import androidx.compose.foundation.shape.RoundedCornerShape
9 import androidx.compose.material.icons.Icons
10 import androidx.compose.material.icons.filled.*
11 import androidx.compose.material3.*
12 import androidx.compose.runtime.Composable
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.draw.clip
16 import androidx.compose.ui.draw.rotate
17 import androidx.compose.ui.graphics.Color
18 import androidx.compose.ui.graphics.vector.ImageVector
19 import androidx.compose.ui.text.font.FontWeight
20 import androidx.compose.ui.unit.dp
21 import androidx.compose.ui.unit.sp
22 import com.wetinknext.ui.theme.AppTheme
23
24 enum class TransformModeUi {
25     FREEFORM,
26     UNIFORM,
27     DISTORT,
28     WARP
29 }
30
31 interface TransformActions {
32     fun reset()
33     fun cancel()
34     fun apply()
35     fun flipHorizontal()
36     fun flipVertical()
37     fun setMode(mode: TransformModeUi)
38 }
39
40 @Composable
41 fun TransformMenuView(
42     theme: AppTheme,
43     actions: TransformActions,
44     currentMode: TransformModeUi,
45     modifier: Modifier = Modifier
46 ) {
47     val panelBorder = BorderStroke(1.dp, theme.panelStroke)
48
49     Column(
50         modifier = modifier
51             .fillMaxWidth()
52             .navigationBarPadding()
53             .padding(horizontal = 16.dp, vertical = 20.dp),
54         horizontalAlignment = Alignment.CenterHorizontally,
55         verticalArrangement = Arrangement.spacedBy(12.dp)
56     ) {
57         // Row 1: Modes
58         Surface(
59             color = theme.panelBg.copy(alpha = 0.94f),
60             shape = RoundedCornerShape(24.dp),
61             border = panelBorder
62         ) {
63             Row(
64                 modifier = Modifier
65                     .width(340.dp)
66                     .height(32.dp)
67                     .padding(4.dp),
68                 horizontalArrangement = Arrangement.spacedBy(4.dp)
69             ) {
70                 TransformModeButton("Свободно", TransformModeUi.FREEFORM, currentMode, theme, actions::setMode)
71                 TransformModeButton("Пропорц.", TransformModeUi.UNIFORM, currentMode, theme, actions::setMode)
72                 TransformModeButton("Исказить", TransformModeUi.DISTORT, currentMode, theme, actions::setMode)
73                 TransformModeButton("Деформ.", TransformModeUi.WARP, currentMode, theme, actions::setMode)
74             }
75         }
76
77         // Row 2: Actions
78         Surface(
79             color = theme.panelBg.copy(alpha = 0.94f),
80             shape = RoundedCornerShape(24.dp),
81             border = panelBorder
82         ) {
83             Row(
84                 modifier = Modifier.padding(horizontal = 4.dp, vertical = 4.dp),
85                 horizontalArrangement = Arrangement.spacedBy(4.dp),
86                 verticalAlignment = Alignment.CenterVertically
87             ) {
88                 ActionIconButton(Icons.Default.Flip, "Отразить Г", theme, enabled = false) { actions.flipHorizontal() }
89                 ActionIconButton(Icons.Default.Flip, "Отразить В", theme, iconModifier = Modifier.rotate(90f), enabled =
false) { actions.flipVertical() }

```

```

90         IconButton(Icons.Default.RotateRight, "Поворот 45", theme, enabled = false) { /* ... */ }
91         IconButton(Icons.Default.FitScreen, "Вписать", theme, enabled = false) { actions.reset() }
92
93         VerticalDivider(modifier = Modifier.height(24.dp).padding(horizontal = 4.dp), color = theme.panelStroke)
94
95         // Placeholder for interpolation/snapping
96         Box(modifier = Modifier.size(48.dp), contentAlignment = Alignment.Center) {
97             Icon(Icons.Default.LinkOff, null, tint = theme.textSecondary.copy(alpha = 0.4f), modifier = Modifier.size(
24.dp))
98         }
99     }
100 }
101
102 // Row 3: Main Controls
103 Row(
104     modifier = Modifier.fillMaxWidth(),
105     horizontalArrangement = Arrangement.spacedBy(16.dp, Alignment.CenterHorizontally),
106     verticalAlignment = Alignment.CenterVertically
107 ) {
108     // Reset
109     TransformRoundButton(Icons.Default.Refresh, "Сбросить", theme, enabled = false) { actions.reset() }
110
111     // Cancel
112     TransformRoundButton(Icons.Default.Close, "Отмена", theme) { actions.cancel() }
113
114     // Apply
115     Surface(
116         color = theme.accent.copy(alpha = 0.4f),
117         shape = CircleShape,
118         modifier = Modifier.size(48.dp)
119     ) {
120         Box(
121             modifier = Modifier.fillMaxSize(),
122             contentAlignment = Alignment.Center
123         ) {
124             Icon(Icons.Default.Check, "Применить", tint = Color.White.copy(alpha = 0.4f))
125         }
126     }
127 }
128 }
129 }
130
131 @Composable
132 private fun RowScope.TransformModeButton(
133     label: String,
134     mode: TransformModeUi,
135     currentMode: TransformModeUi,
136     theme: AppTheme,
137     onClick: (TransformModeUi) -> Unit
138 ) {
139     val isSelected = mode == currentMode
140     val enabled = false // Transform engine not ready
141     Box(
142         modifier = Modifier
143             .weight(1f)
144             .fillMaxHeight()
145             .clip(RoundedCornerShape(20.dp))
146             .background(if (isSelected) theme.accent.copy(alpha = 0.4f) else Color.Transparent)
147             .clickable(enabled = enabled) { onClick(mode) },
148         contentAlignment = Alignment.Center
149     ) {
150         Text(
151             text = label,
152             color = if (isSelected) Color.White.copy(alpha = 0.4f) else theme.textPrimary.copy(alpha = 0.4f),
153             fontSize = 13.sp,
154             fontWeight = if (isSelected) FontWeight.Bold else FontWeight.Normal
155         )
156     }
157 }
158
159 @Composable
160 private fun ActionIconButton(
161     icon: ImageVector,
162     description: String,
163     theme: AppTheme,
164     iconModifier: Modifier = Modifier,
165     enabled: Boolean = true,
166     onClick: () -> Unit
167 ) {
168     Box(
169         modifier = Modifier
170             .size(48.dp)
171             .clickable(enabled = enabled) { onClick() },
172         contentAlignment = Alignment.Center
173     ) {
174         Icon(
175             imageVector = icon,
176             contentDescription = description,
177             tint = if (enabled) theme.textPrimary else theme.textPrimary.copy(alpha = 0.4f),
178             modifier = iconModifier.size(24.dp)
179         )
180     }

```

```

181 }
182
183 @Composable
184 private fun TransformRoundButton(
185     icon: ImageVector,
186     description: String,
187     theme: AppTheme,
188     enabled: Boolean = true,
189     onClick: () -> Unit
190 ) {
191     Surface(
192         color = if (enabled) theme.panelBg.copy(alpha = 0.9f) else theme.panelBg.copy(alpha = 0.4f),
193         shape = CircleShape,
194         border = BorderStroke(1.dp, if (enabled) theme.panelStroke else theme.panelStroke.copy(alpha = 0.4f)),
195         modifier = Modifier.size(48.dp)
196     ) {
197         Box(
198             modifier = Modifier.fillMaxSize().clickable(enabled = enabled) { onClick() },
199             contentAlignment = Alignment.Center
200         ) {
201             Icon(icon, description, tint = if (enabled) theme.textPrimary else theme.textPrimary.copy(alpha = 0.4f))
202         }
203     }
204 }

```

103. app/src/main/java/com/wetinknext/ui/home/HomeScreen.kt

```

1  package com.wetinknext.ui.home
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.Image
7  import androidx.compose.foundation.layout.*
8  import androidx.compose.foundation.lazy.grid.GridCells
9  import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
10 import androidx.compose.foundation.lazy.grid.items
11 import androidx.compose.foundation.rememberScrollState
12 import androidx.compose.foundation.shape.RoundedCornerShape
13 import androidx.compose.foundation.verticalScroll
14 import androidx.compose.material.icons.Icons
15 import androidx.compose.material.icons.outlined.Add
16 import androidx.compose.material.icons.outlined.Delete
17 import androidx.compose.material.icons.outlined.ContentCopy
18 import androidx.compose.material.icons.outlined.Edit
19 import androidx.compose.material.icons.outlined.MoreVert
20 import androidx.compose.material3.*
21 import androidx.compose.runtime.*
22 import androidx.compose.ui.Alignment
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.draw.clip
25 import androidx.compose.ui.graphics.Color
26 import androidx.compose.ui.graphics.ImageBitmap
27 import androidx.compose.ui.graphics.asImageBitmap
28 import android.graphics.BitmapFactory
29 import android.util.LruCache
30 import androidx.compose.ui.graphics.graphicsLayer
31 import androidx.compose.ui.text.font.FontWeight
32 import androidx.compose.ui.text.style.TextAlign
33 import androidx.compose.ui.text.style.TextOverflow
34 import androidx.compose.ui.unit.dp
35 import androidx.compose.ui.unit.sp
36 import com.wetinknext.ui.theme.AppTheme
37 import java.text.SimpleDateFormat
38 import java.util.Date
39 import java.util.Locale
40 import kotlinx.coroutines.Dispatchers
41 import kotlinx.coroutines.withContext
42
43 // Minimal models for Home UI shell
44 data class HomeProject(
45     val id: String,
46     val name: String,
47     val canvasWidth: Int,
48     val canvasHeight: Int,
49     val updatedAtMs: Long,
50     val thumbnailBytes: ByteArray? = null,
51     val thumbnailVersion: Long = 0L,
52 )
53
54 data class DeletedHomeProject(
55     val project: HomeProject,
56     val deletedAtMs: Long,
57 )
58
59 private data class ProjectThumbnailKey(
60     val projectId: String,
61     val version: Long,
62 )
63
64 /** Bytes are primed from ProjectSummary; decoded bitmaps survive grid row recycling. */
65 private object ProjectThumbnailBitmapCache {
66     private val encoded = LruCache<ProjectThumbnailKey, ByteArray>(32)
67     private val decoded = LruCache<ProjectThumbnailKey, ImageBitmap>(32)
68
69     fun prime(key: ProjectThumbnailKey, bytes: ByteArray?) {
70         if (bytes != null) encoded.put(key, bytes)
71     }
72
73     fun bitmap(key: ProjectThumbnailKey): ImageBitmap? = decoded.get(key)
74
75     fun decode(key: ProjectThumbnailKey): ImageBitmap? {
76         decoded.get(key)?.let { return it }
77         val bytes = encoded.get(key) ?: return null
78         return BitmapFactory.decodeByteArray(bytes, 0, bytes.size)
79             ?.asImageBitmap()
80             ?.also { decoded.put(key, it) }
81     }
82 }
83
84 @Composable
85 private fun rememberProjectThumbnail(
86     projectId: String,
87     version: Long,
88 ): ImageBitmap? {
89     val key = remember(projectId, version) { ProjectThumbnailKey(projectId, version) }
90     // Preserve the old preview for this project until a newly saved WebP is decoded.

```

```

91     var bitmap by remember(projectId) { mutableStateOf(ProjectThumbnailBitmapCache.bitmap(key)) }
92
93     LaunchedEffect(key) {
94         withContext(Dispatchers.Default) {
95             ProjectThumbnailBitmapCache.decode(key)
96         }.let { bitmap = it }
97     }
98     return bitmap
99 }
100
101 @Composable
102 fun HomeScreen(
103     theme: AppTheme,
104     projects: List<HomeProject>,
105     deletedProjects: List<DeletedHomeProject> = emptyList(),
106     onOpenProject: (HomeProject) -> Unit,
107     onCreateProject: (name: String, width: Int, height: Int, dpi: Int, colorProfile: String) -> Unit,
108     onRenameProject: (HomeProject, String) -> Unit,
109     onDuplicateProject: (HomeProject) -> Unit,
110     onDeleteProject: (HomeProject) -> Unit,
111     onRestoreProject: (DeletedHomeProject) -> Unit,
112     onDeleteForever: (DeletedHomeProject) -> Unit,
113 ) {
114     var showNewDialog by remember { mutableStateOf(false) }
115     var renaming by remember { mutableStateOf<HomeProject?>(null) }
116     var deleting by remember { mutableStateOf<HomeProject?>(null) }
117     var showTrash by remember { mutableStateOf(false) }
118
119     Box(
120         modifier = Modifier
121             .fillMaxSize()
122             .background(theme.panelBgSolid),
123     ) {
124         Column(modifier = Modifier.fillMaxSize()) {
125             HomeTopBar(
126                 theme = theme,
127                 total = projects.size,
128                 onCreate = { showNewDialog = true },
129                 onTrash = { showTrash = true },
130             )
131             LazyVerticalGrid(
132                 columns = GridCells.Adaptive(minSize = 180.dp),
133                 contentPadding = PaddingValues(16.dp),
134                 horizontalArrangement = Arrangement.spacedBy(12.dp),
135                 verticalArrangement = Arrangement.spacedBy(12.dp),
136                 modifier = Modifier.fillMaxSize(),
137             ) {
138                 items(projects, key = { it.id }) { project ->
139                     ProjectTile(
140                         project = project,
141                         theme = theme,
142                         onOpen = { onOpenProject(project) },
143                         onRename = { renaming = project },
144                         onDuplicate = { onDuplicateProject(project) },
145                         onDelete = { deleting = project },
146                     )
147                 }
148             }
149         }
150     }
151
152     if (showNewDialog) {
153         NewCanvasDialog(
154             theme = theme,
155             defaultWidth = 2800,
156             defaultHeight = 1840,
157             onDismiss = { showNewDialog = false },
158             onCreate = { name, width, height, dpi, colorProfile ->
159                 showNewDialog = false
160                 onCreateProject(name, width, height, dpi, colorProfile)
161             },
162         )
163     }
164
165     renaming?.let { project ->
166         RenameDialog(
167             theme = theme,
168             initial = project.name,
169             onDismiss = { renaming = null },
170             onConfirm = { newName ->
171                 onRenameProject(project, newName)
172                 renaming = null
173             },
174         )
175     }
176
177     deleting?.let { project ->
178         AlertDialog(
179             onDismissRequest = { deleting = null },
180             confirmButton = {
181                 TextButton(
182                     onClick = {

```

```

183                 onDeleteProject(project)
184                 deleting = null
185             },
186         ) {
187             Text("Удалить", color = theme.danger, fontWeight = FontWeight.SemiBold)
188         }
189     },
190     dismissButton = {
191         TextButton(onClick = { deleting = null }) {
192             Text("Отмена", color = theme.textSecondary)
193         }
194     },
195     title = { Text("Удалить холст?", color = theme.textPrimary) },
196     text = { Text(project.name, color = theme.textSecondary) },
197     containerColor = theme.panelBgSolid,
198 )
199 }
200
201 if (showTrash) {
202     TrashDialog(
203         theme = theme,
204         deletedProjects = deletedProjects,
205         onRestore = { deleted ->
206             onRestoreProject(deleted)
207         },
208         onDeleteForever = { deleted ->
209             onDeleteForever(deleted)
210         },
211         onDismiss = { showTrash = false },
212     )
213 }
214 }
215
216 @Composable
217 private fun HomeTopBar(
218     theme: AppTheme,
219     total: Int,
220     onCreate: () -> Unit,
221     onTrash: () -> Unit,
222 ) {
223     Row(
224         modifier = Modifier
225             .fillMaxWidth()
226             .padding(horizontal = 16.dp, vertical = 14.dp),
227         verticalAlignment = Alignment.CenterVertically,
228     ) {
229         Column(modifier = Modifier.weight(1f)) {
230             Text(
231                 "Мои работы",
232                 color = theme.textPrimary,
233                 fontSize = 22.sp,
234                 fontWeight = FontWeight.SemiBold,
235             )
236             Spacer(Modifier.height(3.dp))
237             Text(
238                 "Всего: $total",
239                 color = theme.textSecondary,
240                 fontSize = 13.sp,
241                 fontWeight = FontWeight.Medium,
242             )
243         }
244         TopIconButton(theme = theme, onClick = onCreate) {
245             Icon(Icons.Outlined.Add, null, tint = theme.textPrimary, modifier = Modifier.size(22.dp))
246         }
247         Spacer(Modifier.size(10.dp))
248         TopIconButton(theme = theme, onClick = onTrash) {
249             Icon(Icons.Outlined.Delete, null, tint = theme.textPrimary, modifier = Modifier.size(22.dp))
250         }
251     }
252 }
253
254 @Composable
255 private fun TopIconButton(
256     theme: AppTheme,
257     onClick: () -> Unit,
258     content: @Composable () -> Unit,
259 ) {
260     Box(
261         modifier = Modifier
262             .size(44.dp)
263             .clip(RoundedCornerShape(14.dp))
264             .background(theme.panelBg)
265             .border(1.dp, theme.panelStroke, RoundedCornerShape(14.dp))
266             .clickable(onClick = onClick),
267         contentAlignment = Alignment.Center,
268     ) {
269         content()
270     }
271 }
272
273 @Composable
274 private fun ProjectTile(

```

```

275     project: HomeProject,
276     theme: AppTheme,
277     onOpen: () -> Unit,
278     onRename: () -> Unit,
279     onDuplicate: () -> Unit,
280     onDelete: () -> Unit,
281 ) {
282     var menuExpanded by remember { mutableStateOf(false) }
283
284     Column(
285         modifier = Modifier
286             .fillMaxWidth()
287             .clip(RoundedCornerShape(8.dp))
288             .background(theme.panelBg)
289             .border(1.2.dp, theme.panelStroke, RoundedCornerShape(8.dp))
290             .clickable(onClick = onOpen)
291             .padding(10.dp),
292     ) {
293         ProjectPreview(project = project, theme = theme)
294         Spacer(Modifier.height(10.dp))
295         Row(verticalAlignment = Alignment.CenterVertically) {
296             Column(modifier = Modifier.weight(1f)) {
297                 Text(
298                     project.name,
299                     color = theme.textPrimary,
300                     fontSize = 14.sp,
301                     fontWeight = FontWeight.SemiBold,
302                     maxLines = 1,
303                     overflow = TextOverflow.Ellipsis,
304                 )
305                 Text(
306                     "${project.canvasWidth} x ${project.canvasHeight} px · ${formatDate(project.updatedAtMs)}",
307                     color = theme.textSecondary,
308                     fontSize = 11.sp,
309                     maxLines = 1,
310                     overflow = TextOverflow.Ellipsis,
311                 )
312             }
313             Box {
314                 Box(
315                     modifier = Modifier
316                         .size(34.dp)
317                         .clip(RoundedCornerShape(12.dp))
318                         .clickable { menuExpanded = true },
319                     contentAlignment = Alignment.Center,
320                 ) {
321                     Icon(Icons.Outlined.MoreVert, null, tint = theme.textSecondary, modifier = Modifier.size(20.dp))
322                 }
323                 DropdownMenu(
324                     expanded = menuExpanded,
325                     onDismissRequest = { menuExpanded = false },
326                     modifier = Modifier.background(theme.panelBg)
327                 ) {
328                     DropdownMenuItem(
329                         text = { Text("Переименовать", color = theme.textPrimary) },
330                         leadingIcon = { Icon(Icons.Outlined.Edit, null, tint = theme.textSecondary) },
331                         onClick = {
332                             menuExpanded = false
333                             onRename()
334                         },
335                     )
336                     DropdownMenuItem(
337                         text = { Text("Дублировать", color = theme.textPrimary) },
338                         leadingIcon = { Icon(Icons.Outlined.ContentCopy, null, tint = theme.textSecondary) },
339                         onClick = {
340                             menuExpanded = false
341                             onDuplicate()
342                         },
343                     )
344                     DropdownMenuItem(
345                         text = { Text("Удалить", color = theme.danger) },
346                         leadingIcon = { Icon(Icons.Outlined.Delete, null, tint = theme.danger) },
347                         onClick = {
348                             menuExpanded = false
349                             onDelete()
350                         },
351                     )
352                 }
353             }
354         }
355     }
356 }
357
358 @Composable
359 private fun ProjectPreview(
360     project: HomeProject,
361     theme: AppTheme,
362 ) {
363     // Placeholder for actual preview loading
364     Box(
365         modifier = Modifier
366             .fillMaxWidth()

```

```

367         .aspectRatio(
368             project.canvasWidth.toFloat() /
369             project.canvasHeight.coerceAtLeast(1).toFloat(),
370         )
371         .clip(RoundedCornerShape(6.dp))
372         .background(theme.panelInset)
373         .border(1.dp, theme.panelStroke, RoundedCornerShape(6.dp)),
374         contentAlignment = Alignment.Center,
375     ) {
376         ProjectThumbnailBitmapCache.prime(
377             key = ProjectThumbnailKey(project.id, project.thumbnailVersion),
378             bytes = project.thumbnailBytes,
379         )
380         val preview = rememberProjectThumbnail(
381             projectId = project.id,
382             version = project.thumbnailVersion,
383         )
384         if (preview != null) {
385             Image(bitmap = preview, contentDescription = "Превью ${project.name}", modifier = Modifier.fillMaxSize())
386             return@Box
387         }
388         EmptyProjectPlaceholder(theme)
389     }
390 }
391
392 @Composable
393 private fun EmptyProjectPlaceholder(theme: AppTheme) {
394     Box(
395         modifier = Modifier
396             .fillMaxSize()
397             .clip(RoundedCornerShape(4.dp))
398             .background(Color(0xFFFAFAFA))
399             .border(1.dp, theme.panelStroke, RoundedCornerShape(4.dp)),
400     )
401 }
402
403 @Composable
404 private fun RenameDialog(
405     theme: AppTheme,
406     initial: String,
407     onDismiss: () -> Unit,
408     onConfirm: (String) -> Unit,
409 ) {
410     var name by remember(initial) { mutableStateOf(initial) }
411     AlertDialog(
412         onDismissRequest = onDismiss,
413         confirmButton = {
414             TextButton(
415                 onClick = {
416                     val cleaned = name.trim()
417                     if (cleaned.isNotEmpty()) onConfirm(cleaned)
418                 },
419             ) {
420                 Text("Готово", color = theme.accent, fontWeight = FontWeight.SemiBold)
421             }
422         },
423         dismissButton = {
424             TextButton(onClick = onDismiss) {
425                 Text("Отмена", color = theme.textSecondary)
426             }
427         },
428         title = { Text("Название", color = theme.textPrimary) },
429         text = {
430             OutlinedTextField(
431                 value = name,
432                 onValueChange = { name = it.take(48) },
433                 singleLine = true,
434                 colors = OutlinedTextFieldDefaults.colors(
435                     focusedBorderColor = theme.accent,
436                     unfocusedBorderColor = theme.panelStroke,
437                     focusedTextColor = theme.textPrimary,
438                     unfocusedTextColor = theme.textPrimary
439                 )
440             )
441         },
442         containerColor = theme.panelBgSolid,
443     )
444 }
445
446 @Composable
447 private fun TrashDialog(
448     theme: AppTheme,
449     deletedProjects: List<DeletedHomeProject>,
450     onRestore: (DeletedHomeProject) -> Unit,
451     onDeleteForever: (DeletedHomeProject) -> Unit,
452     onDismiss: () -> Unit,
453 ) {
454     AlertDialog(
455         onDismissRequest = onDismiss,
456         confirmButton = {
457             TextButton(onClick = onDismiss) {
458                 Text("Готово", color = theme.accent, fontWeight = FontWeight.SemiBold)

```

```

459     }
460 },
461 title = { Text("Корзина", color = theme.textPrimary) },
462 text = {
463     if (deletedProjects.isEmpty()) {
464         Text("Удалённые работы хранятся здесь 3 дня.", color = theme.textSecondary, fontSize = 13.sp)
465     } else {
466         Column(
467             modifier = Modifier
468                 .fillMaxWidth()
469                 .height(360.dp)
470                 .verticalScroll(rememberScrollState()),
471             verticalArrangement = Arrangement.spacedBy(10.dp),
472         ) {
473             Text(
474                 "Работы хранятся 3 дня.",
475                 color = theme.textSecondary,
476                 fontSize = 12.sp,
477             )
478             deletedProjects.sortedByDescending { it.deletedAtMs }.forEach { deleted ->
479                 DeletedProjectRow(
480                     deleted = deleted,
481                     theme = theme,
482                     onRestore = { onRestore(deleted) },
483                     onDeleteForever = { onDeleteForever(deleted) },
484                 )
485             }
486         }
487     }
488 },
489 containerColor = theme.panelBgSolid,
490 )
491 }
492
493 @Composable
494 private fun DeletedProjectRow(
495     deleted: DeletedHomeProject,
496     theme: AppTheme,
497     onRestore: () -> Unit,
498     onDeleteForever: () -> Unit,
499 ) {
500     val project = deleted.project
501     Row(
502         modifier = Modifier
503             .fillMaxWidth()
504             .clip(RoundedCornerShape(12.dp))
505             .background(theme.panelBg)
506             .border(1.dp, theme.panelStroke, RoundedCornerShape(12.dp))
507             .padding(10.dp),
508         verticalAlignment = Alignment.CenterVertically,
509     ) {
510         Box(
511             modifier = Modifier
512                 .size(54.dp)
513                 .clip(RoundedCornerShape(6.dp))
514                 .background(theme.panelInset)
515                 .border(1.dp, theme.panelStroke, RoundedCornerShape(6.dp)),
516             contentAlignment = Alignment.Center,
517         ) {
518             Box(
519                 modifier = Modifier
520                     .fillMaxWidth(0.5f)
521                     .aspectRatio(project.canvasWidth.toFloat() / project.canvasHeight.coerceAtLeast(1).toFloat())
522                     .background(Color(0xFFFAFAFA)),
523             )
524         }
525         Column(
526             modifier = Modifier
527                 .weight(1f)
528                 .padding(start = 10.dp),
529         ) {
530             Text(
531                 project.name,
532                 color = theme.textPrimary,
533                 fontSize = 13.sp,
534                 fontWeight = FontWeight.SemiBold,
535                 maxLines = 1,
536                 overflow = TextOverflow.Ellipsis,
537             )
538             Text(
539                 "Осталось: ${trashDaysLeft(deleted.deletedAtMs)} дн.",
540                 color = theme.textSecondary,
541                 fontSize = 11.sp,
542             )
543             Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
544                 TextButton(onClick = onRestore) {
545                     Text("Восстановить", color = theme.accent, fontSize = 12.sp)
546                 }
547                 TextButton(onClick = onDeleteForever) {
548                     Text("Удалить", color = theme.textSecondary, fontSize = 12.sp)
549                 }
550             }
551         }
552     }
553 }

```

```

551     }
552 }
553 }
554
555 private fun formatDate(timeMs: Long): String =
556     SimpleDateFormat("d MMM, HH:mm", Locale.forLanguageTag("ru")).format(Date(timeMs))
557
558 private fun trashDaysLeft(deletedAtMs: Long): Int {
559     val retentionMs = 3L * 24L * 60L * 60L * 1000L
560     val remainingMs = (retentionMs - (System.currentTimeMillis() - deletedAtMs)).coerceAtLeast(0L)
561     return ((remainingMs + 24L * 60L * 60L * 1000L - 1L) / (24L * 60L * 60L * 1000L)).toInt().coerceAtLeast(1)
562 }

```

104. app/src/main/java/com/wetinknext/ui/home/NewCanvasDialog.kt

```

1  package com.wetinknext.ui.home
2
3  import androidx.compose.foundation.background
4  import androidx.compose.foundation.border
5  import androidx.compose.foundation.clickable
6  import androidx.compose.foundation.horizontalScroll
7  import androidx.compose.foundation.layout.*
8  import androidx.compose.foundation.rememberScrollState
9  import androidx.compose.foundation.shape.RoundedCornerShape
10 import androidx.compose.foundation.text.KeyboardOptions
11 import androidx.compose.material3.*
12 import androidx.compose.runtime.*
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.draw.clip
16 import androidx.compose.ui.graphics.Color
17 import androidx.compose.ui.text.font.FontWeight
18 import androidx.compose.ui.text.input.KeyboardType
19 import androidx.compose.ui.text.style.TextAlign
20 import androidx.compose.ui.unit.dp
21 import androidx.compose.ui.unit.sp
22 import com.wetinknext.ui.theme.AppTheme
23 import com.wetinknext.domain.document.ProjectDocument
24 import kotlin.math.max
25
26 private data class CanvasPreset(
27     val label: String,
28     val width: Int,
29     val height: Int,
30     val description: String,
31 )
32
33 private val QuickPresets = listOf(
34     CanvasPreset("Экран", 2800, 1840, "2800 x 1840 px"),
35     CanvasPreset("Квадрат", 2048, 2048, "2048 x 2048 px"),
36     CanvasPreset("4K", 4096, 2160, "4096 x 2160 px"),
37     CanvasPreset("A4", 2480, 3508, "210 x 297 мм"),
38     CanvasPreset("1080P", 1920, 1080, "1920 x 1080 px"),
39     CanvasPreset("9:16", 1080, 1920, "1080 x 1920 px"),
40 )
41
42 private val DpiOptions = listOf(72, 144, 300, 350, 600, 1200)
43 private val ColorProfiles = listOf(
44     ProjectDocument.SRGB_PROFILE,
45     ProjectDocument.DISPLAY_P3_PROFILE,
46 )
47
48 @Composable
49 fun NewCanvasDialog(
50     theme: AppTheme,
51     defaultWidth: Int,
52     defaultHeight: Int,
53     onDismiss: () -> Unit,
54     onConfirm: (name: String, width: Int, height: Int, dpi: Int, colorProfile: String) -> Unit,
55 ) {
56     val initialIndex = QuickPresets.indexOfFirst {
57         it.width == defaultWidth && it.height == defaultHeight
58     }
59     var selected by remember { mutableIntStateOf(initialIndex) }
60     var customW by remember { mutableStateOf(defaultWidth.toString()) }
61     var customH by remember { mutableStateOf(defaultHeight.toString()) }
62     var name by remember { mutableStateOf("Новый холст") }
63     var dpi by remember { mutableIntStateOf(300) }
64     var colorProfile by remember { mutableStateOf(ProjectDocument.SRGB_PROFILE) }
65     var dpiExpanded by remember { mutableStateOf(false) }
66     var profileExpanded by remember { mutableStateOf(false) }
67     var validationError by remember { mutableStateOf<String?>(null) }
68     val isCustom = selected !in QuickPresets.indices
69
70     AlertDialog(
71         onDismissRequest = onDismiss,
72         title = {
73             Text("Новый холст", color = theme.textPrimary, fontWeight = FontWeight.SemiBold)
74         },
75         text = {
76             Column {
77                 OutlinedTextField(
78                     value = name,
79                     onValueChange = { name = it.take(48) },
80                     label = { Text("Имя") },
81                     singleLine = true,
82                     modifier = Modifier.fillMaxWidth(),
83                     colors = OutlinedTextFieldDefaults.colors(
84                         focusedBorderColor = theme.accent,
85                         unfocusedBorderColor = theme.panelStroke,
86                         focusedLabelColor = theme.accent,
87                         unfocusedLabelColor = theme.textSecondary
88                     )
89                 )
90                 Spacer(Modifier.height(12.dp))

```

```

91 Text("Размеры", color = theme.textSecondary, fontSize = 13.sp)
92 Spacer(Modifier.height(8.dp))
93 Row(
94     horizontalArrangement = Arrangement.spacedBy(8.dp),
95     modifier = Modifier.horizontalScroll(rememberScrollState()),
96 ) {
97     QuickPresets.forEachIndexed { index, preset ->
98         QuickPresetCard(
99             preset = preset,
100             selected = selected == index,
101             theme = theme,
102             onClick = {
103                 selected = index
104                 customW = preset.width.toString()
105                 customH = preset.height.toString()
106             },
107         )
108     }
109 }
110
111 Spacer(Modifier.height(12.dp))
112 TextButton(onClick = { selected = -1 }) {
113     Text(
114         "Свой размер",
115         color = if (isCustom) theme.accent else theme.textSecondary,
116         fontWeight = if (isCustom) FontWeight.SemiBold else FontWeight.Normal,
117         fontSize = 13.sp,
118     )
119 }
120
121 if (isCustom) {
122     Spacer(Modifier.height(4.dp))
123     Row(verticalAlignment = Alignment.CenterVertically) {
124         OutlinedTextField(
125             value = customW,
126             onValueChange = { v -> customW = v.filter { it.isDigit() }.take(5) },
127             label = { Text("Ширина") },
128             singleLine = true,
129             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
130             modifier = Modifier.weight(1f),
131             colors = OutlinedTextFieldDefaults.colors(
132                 focusedBorderColor = theme.accent,
133                 unfocusedBorderColor = theme.panelStroke,
134                 focusedLabelColor = theme.accent,
135                 unfocusedLabelColor = theme.textSecondary
136             )
137         )
138         Spacer(Modifier.width(8.dp))
139         Text("x", color = theme.textSecondary)
140         Spacer(Modifier.width(8.dp))
141         OutlinedTextField(
142             value = customH,
143             onValueChange = { v -> customH = v.filter { it.isDigit() }.take(5) },
144             label = { Text("Высота") },
145             singleLine = true,
146             keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
147             modifier = Modifier.weight(1f),
148             colors = OutlinedTextFieldDefaults.colors(
149                 focusedBorderColor = theme.accent,
150                 unfocusedBorderColor = theme.panelStroke,
151                 focusedLabelColor = theme.accent,
152                 unfocusedLabelColor = theme.textSecondary
153             )
154         )
155     }
156 }
157
158 Spacer(Modifier.height(14.dp))
159 OptionRow(label = "DPI", value = "$dpi dpi", theme = theme, expanded = dpiExpanded, onExpand = { dpiExpanded
= true }) {
160     DpiOptions.forEach { option ->
161         DropdownMenuItem(
162             text = { Text("$option dpi") },
163             onClick = {
164                 dpi = option
165                 dpiExpanded = false
166             },
167         )
168     }
169 }
170 OptionRow(label = "Цветовой профиль", value = colorProfile, theme = theme, expanded = profileExpanded,
onExpand = { profileExpanded = true }) {
171     ColorProfiles.forEach { option ->
172         DropdownMenuItem(
173             text = { Text(option) },
174             onClick = {
175                 colorProfile = option
176                 profileExpanded = false
177             },
178         )
179     }
180 }

```

```

181         validationError?.let { message ->
182             Spacer(Modifier.height(8.dp))
183             Text(message, color = MaterialTheme.colorScheme.error, fontSize = 12.sp)
184         }
185     },
186 },
187     confirmButton = {
188         TextButton(
189             onClick = {
190                 val (w, h) = if (isCustom) {
191                     val cw = customW.toIntOrNull()?.coerceIn(MinDimension, MaxDimension) ?: return@TextButton
192                     val ch = customH.toIntOrNull()?.coerceIn(MinDimension, MaxDimension) ?: return@TextButton
193                     cw to ch
194                 } else {
195                     val preset = QuickPresets[selected]
196                     preset.width to preset.height
197                 }
198                 validationError = runCatching {
199                     ProjectDocument.validateNewCanvas(w, h, dpi, colorProfile)
200                 }.exceptionOrNull()?.message
201                 if (validationError == null) {
202                     onConfirm(name.trim().ifBlank { "Новый холст" }, w, h, dpi, colorProfile)
203                 }
204             },
205         ) {
206             Text("Создать", color = theme.accent, fontWeight = FontWeight.SemiBold)
207         }
208     },
209     dismissButton = {
210         TextButton(onClick = onDismiss) {
211             Text("Отмена", color = theme.textSecondary)
212         }
213     },
214     containerColor = theme.panelBgSolid,
215 )
216 }
217
218 @Composable
219 private fun OptionRow(
220     label: String,
221     value: String,
222     theme: AppTheme,
223     expanded: Boolean,
224     onExpand: () -> Unit,
225     menu: @Composable () -> Unit,
226 ) {
227     Row(verticalAlignment = Alignment.CenterVertically, modifier = Modifier.fillMaxWidth()) {
228         Text(label, color = theme.textPrimary, fontSize = 13.sp)
229         Spacer(Modifier.weight(1f))
230         Box {
231             TextButton(onClick = onExpand) {
232                 Text(value, color = theme.accent, fontWeight = FontWeight.SemiBold)
233             }
234             DropdownMenu(
235                 expanded = expanded,
236                 onDismissRequest = { /* onExpand is handled by parent */ },
237                 modifier = Modifier.background(theme.panelBg)
238             ) {
239                 menu()
240             }
241         }
242     }
243 }
244
245 @Composable
246 private fun QuickPresetCard(
247     preset: CanvasPreset,
248     selected: Boolean,
249     theme: AppTheme,
250     onClick: () -> Unit,
251 ) {
252     Column(
253         horizontalAlignment = Alignment.CenterHorizontally,
254         modifier = Modifier
255             .width(100.dp)
256             .clip(RoundedCornerShape(12.dp))
257             .background(if (selected) theme.accent.copy(alpha = 0.15f) else theme.panelBg)
258             .border(1.5.dp, if (selected) theme.accent else theme.panelStroke, RoundedCornerShape(12.dp))
259             .clickable(onClick = onClick)
260             .padding(vertical = 12.dp, horizontal = 8.dp),
261     ) {
262         PresetSwatch(preset.width, preset.height, selected, theme)
263         Spacer(Modifier.height(8.dp))
264         Text(preset.label, color = theme.textPrimary, fontWeight = FontWeight.Medium, fontSize = 13.sp, textAlign = TextAlign.Center)
265         Text(preset.description, color = theme.textSecondary, fontSize = 10.sp, textAlign = TextAlign.Center, maxLines = 1)
266     }
267 }
268
269 @Composable
270 private fun PresetSwatch(width: Int, height: Int, selected: Boolean, theme: AppTheme) {
271     val scale = 40f / max(width.coerceAtLeast(1), height.coerceAtLeast(1))

```

```

272     val previewW = (width * scale).coerceAtLeast(8f)
273     val previewH = (height * scale).coerceAtLeast(8f)
274     Box(modifier = Modifier.size(48.dp), contentAlignment = Alignment.Center) {
275         Box(
276             modifier = Modifier
277                 .width(previewW.dp)
278                 .height(previewH.dp)
279                 .clip(RoundedCornerShape(4.dp))
280                 .background(if (selected) theme.accent else Color(0xFF5A5A60))
281                 .border(1.dp, if (selected) Color.White else theme.panelStroke, RoundedCornerShape(4.dp)),
282             )
283     }
284 }
285
286 private const val MinDimension = 256
287 private const val MaxDimension = 8192

```

105. app/src/main/java/com/wetinknext/ui/state/EditorActions.kt

```
1 package com.wetinknext.ui.state
2
3 /**
4  * Interface defining all possible user actions from the UI that affect the paint engine state.
5  */
6 interface EditorActions {
7     fun undo()
8     fun redo()
9     fun setBrushSize(px: Float)
10    fun setBrushOpacity(opacity: Float)
11
12    fun addLayer()
13    fun removeLayer(id: Long)
14    fun setActiveLayer(id: Long)
15    fun setLayerVisible(id: Long, visible: Boolean)
16    fun setLayerOpacity(id: Long, opacity: Float)
17 }
```

106. app/src/main/java/com/wetinknext/ui/state/LayerState.kt

```

1 package com.wetinknext.ui.state
2
3 import androidx.compose.runtime.*
4 import com.wetinknext.engine.core.EditorUiState
5
6 class LayerItem(
7     val id: Long,
8     name: String,
9     visible: Boolean,
10    opacity: Float,
11    locked: Boolean,
12    thumbnailPath: String? = null,
13    thumbnailVersion: Long = 0L,
14 ) {
15     var name by mutableStateOf(name)
16     var visible by mutableStateOf(visible)
17     var opacity by mutableFloatStateOf(opacity)
18     var locked by mutableStateOf(locked)
19     var thumbnailPath by mutableStateOf(thumbnailPath)
20     var thumbnailVersion by mutableLongStateOf(thumbnailVersion)
21 }
22
23 class LayerState {
24     val layers = mutableStateListOf<LayerItem>()
25     var activeLayerId by mutableStateOf(-1L)
26
27     fun syncFromEditor(state: EditorUiState) {
28         val ids = state.layers.map { it.id }.toSet()
29         layers.removeAll { it.id !in ids }
30
31         state.layers.forEachIndexed { index, model ->
32             val item = layers.firstOrNull { it.id == model.id }
33
34             if (item == null) {
35                 layers.add(
36                     index.coerceAtMost(layers.size),
37                     LayerItem(
38                         id = model.id,
39                         name = model.name,
40                         visible = model.isVisible,
41                         opacity = model.opacity,
42                         locked = model.isLocked,
43                         thumbnailPath = model.thumbnailPath,
44                         thumbnailVersion = model.thumbnailVersion,
45                     ),
46                 )
47             } else {
48                 item.name = model.name
49                 item.visible = model.isVisible
50                 item.opacity = model.opacity
51                 item.locked = model.isLocked
52                 item.thumbnailPath = model.thumbnailPath
53                 item.thumbnailVersion = model.thumbnailVersion
54
55                 // If the item needs to be moved to match the index in state
56                 val currentIndex = layers.indexOf(item)
57                 if (currentIndex != index && index < layers.size) {
58                     layers.removeAt(currentIndex)
59                     layers.add(index, item)
60                 }
61             }
62         }
63
64         activeLayerId = state.activeLayerId
65     }
66 }

```

107. app/src/main/java/com/wetinknext/ui/theme/AppThemes.kt

```

1  package com.wetinknext.ui.theme
2
3  import androidx.compose.runtime.Immutable
4  import androidx.compose.ui.graphics.Color
5
6  @Immutable
7  data class AppTheme(
8      val id: String,
9      val displayName: String,
10     val isLight: Boolean,
11
12     // Core Layout
13     val appBg: Color,
14     val gridLine: Color,
15     val panelBg: Color,
16     val panelBgSolid: Color,
17     val panelStroke: Color,
18     val panelInset: Color,
19     val panelInsetSoft: Color,
20
21     // Canvas
22     val canvasBackdrop: Color,
23     val canvasGrid: Color,
24
25     // Typography & Icons
26     val textPrimary: Color,
27     val textSecondary: Color,
28     val iconInactive: Color,
29
30     // Accents
31     val accent: Color,
32     val accentSoft: Color,
33     val accentMuted: Color,
34     val danger: Color
35 ) {
36     val panelBgVariant: Color get() = panelInset
37 }
38
39 object AppThemes {
40     val Dark = AppTheme(
41         id = "dark", displayName = "Dark", isLight = false,
42         appBg = Color(0xFF1A0C12), gridLine = Color(0xFF2A1418),
43         panelBg = Color(0xFF2A141C), panelBgSolid = Color(0xFF2D1620),
44         panelStroke = Color(0xFF3D1F2A), panelInset = Color(0xFF1F1015), panelInsetSoft = Color(0xFF2A1820),
45         canvasBackdrop = Color(0xFF1A0C12), canvasGrid = Color(0xFF3A1C24),
46         textPrimary = Color(0xFFFF5E8E), textSecondary = Color(0xFF9A7F8A), iconInactive = Color(0xFFC8B0B8),
47         accent = Color(0xFFFF4D7A), accentSoft = Color(0xFFFF7AA0), accentMuted = Color(0x55FF4D7A), danger = Color(
0xFFFF3B5C)
48     )
49
50     val Light = AppTheme(
51         id = "light", displayName = "Light", isLight = true,
52         appBg = Color(0xFFFFE4E9), gridLine = Color(0xFF5D0D7),
53         panelBg = Color(0xFFFFF1F4), panelBgSolid = Color(0xFFFFFFFF),
54         panelStroke = Color(0xFFF0C8D0), panelInset = Color(0xFFFFE0E6), panelInsetSoft = Color(0xFFFFD0DA),
55         canvasBackdrop = Color(0xFFFFE4E9), canvasGrid = Color(0xFF5D0D7),
56         textPrimary = Color(0xFF3D1B26), textSecondary = Color(0xFF8A5A6A), iconInactive = Color(0xFFB07A88),
57         accent = Color(0xFFFF4D7A), accentSoft = Color(0xFFFFA0B7), accentMuted = Color(0x33FF4D7A), danger = Color(
0xFFFF3B5C)
58     )
59
60     val Red = AppTheme(
61         id = "red", displayName = "Red", isLight = false,
62         appBg = Color(0xFF1A0606), gridLine = Color(0xFF2A0A0A),
63         panelBg = Color(0xFF2A0C0C), panelBgSolid = Color(0xFF2D0E0E),
64         panelStroke = Color(0xFF3D1414), panelInset = Color(0xFF1F0808), panelInsetSoft = Color(0xFF2A1010),
65         canvasBackdrop = Color(0xFF1A0606), canvasGrid = Color(0xFF3A1414),
66         textPrimary = Color(0xFFFF5E8E), textSecondary = Color(0xFF9A7F7F), iconInactive = Color(0xFFC8B0B0),
67         accent = Color(0xFFFF3D3D), accentSoft = Color(0xFFFF7A7A), accentMuted = Color(0x55FF3D3D), danger = Color(
0xFFFF8888)
68     )
69
70     val Vampire = AppTheme(
71         id = "vampire", displayName = "Vampire", isLight = false,
72         appBg = Color(0xFF1B1C1B), gridLine = Color(0xFF252525),
73         panelBg = Color(0xFF262626), panelBgSolid = Color(0xFF2D2D2D),
74         panelStroke = Color(0xFF4B2529), panelInset = Color(0xFF171717), panelInsetSoft = Color(0xFF2A1B1E),
75         canvasBackdrop = Color(0xFF1B1C1B), canvasGrid = Color(0xFF2B2B2B),
76         textPrimary = Color(0xFFF2ECC), textSecondary = Color(0xFFCDA2A6), iconInactive = Color(0xFFE15761),
77         accent = Color(0xFFFF3547), accentSoft = Color(0xFFFFA3A8), accentMuted = Color(0xA AFF3547), danger = Color(
0xFFFF4A5F)
78     )
79
80     val Olive = AppTheme(
81         id = "olive", displayName = "Olive", isLight = false,
82         appBg = Color(0xFF171B0D), gridLine = Color(0xFF252A16),
83         panelBg = Color(0xFF202711), panelBgSolid = Color(0xFF263014),
84         panelStroke = Color(0xFF566032), panelInset = Color(0xFF121806), panelInsetSoft = Color(0xFF2B3418),
85         canvasBackdrop = Color(0xFF171B0D), canvasGrid = Color(0xFF2F31C),
86         textPrimary = Color(0xFF1E7D0), textSecondary = Color(0xFFC7A77E), iconInactive = Color(0xFFE3A15E),

```

```

87         accent = Color(0xFFFFF982F), accentSoft = Color(0xFFFFC46A), accentMuted = Color(0x88FF982F), danger = Color(
0xFFFF624D)
88     )
89
90     val Blue = AppTheme(
91         id = "blue", displayName = "Blue", isLight = false,
92         appBg = Color(0xFF06101A), gridLine = Color(0xFF0A1828),
93         panelBg = Color(0xFF0C1A2A), panelBgSolid = Color(0xFF0E1D2E),
94         panelStroke = Color(0xFF14283D), panelInset = Color(0xFF08141F), panelInsetSoft = Color(0xFF101F2A),
95         canvasBackdrop = Color(0xFF06101A), canvasGrid = Color(0xFF14283A),
96         textPrimary = Color(0xFFE8EEF5), textSecondary = Color(0xFF7F8C9A), iconInactive = Color(0xFFB0BBC8),
97         accent = Color(0xFF4D7AFF), accentSoft = Color(0xFF7AA0FF), accentMuted = Color(0x554D7AFF), danger = Color(
0xFFFF3B5C)
98     )
99
100    val Studio = AppTheme(
101        id = "studio", displayName = "Studio", isLight = true,
102        appBg = Color(0xFFE1E1E1), gridLine = Color(0xFFD1D6DA),
103        panelBg = Color(0xFFFFF0F2F4), panelBgSolid = Color(0xFFFFBFCFD),
104        panelStroke = Color(0xFFD4DAE0), panelInset = Color(0xFFE7EBEF), panelInsetSoft = Color(0xFFDCE8F3),
105        canvasBackdrop = Color(0xFFDADADA), canvasGrid = Color(0xFFCFCFCF),
106        textPrimary = Color(0xFF20252A), textSecondary = Color(0xFF65737F), iconInactive = Color(0xFF5C8DB3),
107        accent = Color(0xFF4A9AD8), accentSoft = Color(0xFF7FBCEB), accentMuted = Color(0x334A9AD8), danger = Color(
0xFFE65C73)
108    )
109
110    val Neon = AppTheme(
111        id = "neon", displayName = "Neon", isLight = false,
112        appBg = Color(0xFF2B123E), gridLine = Color(0xFF32184A),
113        panelBg = Color(0xFF4A2366), panelBgSolid = Color(0xFF522A70),
114        panelStroke = Color(0xFF6C3A8D), panelInset = Color(0xFF321344), panelInsetSoft = Color(0xFF3C1853),
115        canvasBackdrop = Color(0xFF2B123E), canvasGrid = Color(0xFF3C1D55),
116        textPrimary = Color(0xFFFF3EAFE), textSecondary = Color(0xFFC6A8D8), iconInactive = Color(0xFF2FE0EA),
117        accent = Color(0xFF25DDE8), accentSoft = Color(0xFF73F3FA), accentMuted = Color(0x5525DDE8), danger = Color(
0xFFFF6E9C)
118    )
119
120    val Cyan = AppTheme(
121        id = "cyan", displayName = "Cyan", isLight = false,
122        appBg = Color(0xFF06181A), gridLine = Color(0xFF0A2628),
123        panelBg = Color(0xFF0C282A), panelBgSolid = Color(0xFF0E2C2E),
124        panelStroke = Color(0xFF143C3D), panelInset = Color(0xFF081F20), panelInsetSoft = Color(0xFF102A2A),
125        canvasBackdrop = Color(0xFF06181A), canvasGrid = Color(0xFF143A3A),
126        textPrimary = Color(0xFFE8F5F5), textSecondary = Color(0xFF7F9A9A), iconInactive = Color(0xFFB0C8C8),
127        accent = Color(0xFF26D9D9), accentSoft = Color(0xFF7AE0E0), accentMuted = Color(0x5526D9D9), danger = Color(
0xFFFF3B5C)
128    )
129
130    val Green = AppTheme(
131        id = "green", displayName = "Green", isLight = false,
132        appBg = Color(0xFF071A0C), gridLine = Color(0xFF0A2814),
133        panelBg = Color(0xFF0C2A14), panelBgSolid = Color(0xFF0E2D16),
134        panelStroke = Color(0xFF143D1F), panelInset = Color(0xFF081F0C), panelInsetSoft = Color(0xFF102A14),
135        canvasBackdrop = Color(0xFF071A0C), canvasGrid = Color(0xFF143A1F),
136        textPrimary = Color(0xFFE8F5EC), textSecondary = Color(0xFF7F9A88), iconInactive = Color(0xFFB0C8B8),
137        accent = Color(0xFF3DDB6E), accentSoft = Color(0xFF7AE0A0), accentMuted = Color(0x553DDB6E), danger = Color(
0xFFFF3B5C)
138    )
139
140    val Yellow = AppTheme(
141        id = "yellow", displayName = "Yellow", isLight = true,
142        appBg = Color(0xFFFFF8E0), gridLine = Color(0xFFFF5E5A0),
143        panelBg = Color(0xFFFFF4D0), panelBgSolid = Color(0xFFFFF0CF0),
144        panelStroke = Color(0xFFE6CC60), panelInset = Color(0xFFFFF0B8), panelInsetSoft = Color(0xFFFFE890),
145        canvasBackdrop = Color(0xFFFFF8E0), canvasGrid = Color(0xFFFF0DA80),
146        textPrimary = Color(0xFF3D2E08), textSecondary = Color(0xFF8A7A40), iconInactive = Color(0xFFB09860),
147        accent = Color(0xFFFF5B800), accentSoft = Color(0xFFFFD050), accentMuted = Color(0x555B800), danger = Color(
0xFFFF3B5C)
148    )
149
150    val Orange = AppTheme(
151        id = "orange", displayName = "Orange", isLight = true,
152        appBg = Color(0xFFFFF8D5), gridLine = Color(0xFFFF5C898),
153        panelBg = Color(0xFFFFF0C5), panelBgSolid = Color(0xFFFFF1E5),
154        panelStroke = Color(0xFFE69C58), panelInset = Color(0xFFFFD8B5), panelInsetSoft = Color(0xFFFFC890),
155        canvasBackdrop = Color(0xFFFFF8D5), canvasGrid = Color(0xFFFF0BC80),
156        textPrimary = Color(0xFF3D1E08), textSecondary = Color(0xFF8A5A30), iconInactive = Color(0xFFB07A50),
157        accent = Color(0xFFFFF8800), accentSoft = Color(0xFFFFD050), accentMuted = Color(0x55FF8800), danger = Color(
0xFFFF3B5C)
158    )
159
160    val Gray = AppTheme(
161        id = "gray", displayName = "Gray", isLight = false,
162        appBg = Color(0xFF1A1A1A), gridLine = Color(0xFF2A2A2A),
163        panelBg = Color(0xFF2A2A2A), panelBgSolid = Color(0xFF303030),
164        panelStroke = Color(0xFF404040), panelInset = Color(0xFF1F1F1F), panelInsetSoft = Color(0xFF2B2B2B),
165        canvasBackdrop = Color(0xFF1A1A1A), canvasGrid = Color(0xFF3A3A3A),
166        textPrimary = Color(0xFFFFF0F0), textSecondary = Color(0xFF909090), iconInactive = Color(0xFFB8B8B8),
167        accent = Color(0xFFB0B0B0), accentSoft = Color(0xFFD0D0D0), accentMuted = Color(0x55B0B0B0), danger = Color(
0xFFFF3B5C)
168    )
169

```

```

170     val White = AppTheme(
171         id = "white", displayName = "White", isLight = true,
172         appBg = Color(0xFFFFFFFF), gridLine = Color(0xFFE8E8E8),
173         panelBg = Color(0xFFF5F5F5), panelBgSolid = Color(0xFFFFFFFF),
174         panelStroke = Color(0xFFD8D8D8), panelInset = Color(0xFFEEEEEE), panelInsetSoft = Color(0xFFE0E0E0),
175         canvasBackdrop = Color(0xFFFFFFFF), canvasGrid = Color(0xFFE0E0E0),
176         textPrimary = Color(0xFF202020), textSecondary = Color(0xFF707070), iconInactive = Color(0xFF989898),
177         accent = Color(0xFF606060), accentSoft = Color(0xFF888888), accentMuted = Color(0x55606060), danger = Color(
0xFFFF3B5C)
178     )
179
180     val all = listOf(
181         Dark, Light, Red, Vampire, Olive, Blue, Studio, Neon, Cyan, Green, Yellow, Orange, Gray, White
182     )
183
184     val default = Gray
185 }
186
187 enum class UIFont { Classic, Italic }

```

108. app/src/main/java/com/wetinknext/ui/theme/Theme.kt

```

1 package com.wetinknext.ui.theme
2
3 import androidx.compose.material3.MaterialTheme
4 import androidx.compose.material3.Typography
5 import androidx.compose.material3.darkColorScheme
6 import androidx.compose.material3.lightColorScheme
7 import androidx.compose.runtime.Composable
8 import androidx.compose.runtime.CompositionLocalProvider
9 import androidx.compose.ui.graphics.Color
10 import androidx.compose.ui.text.TextStyle
11 import androidx.compose.ui.text.font.FontFamily
12 import androidx.compose.ui.text.font.FontStyle
13
14 @Composable
15 fun WetInkTheme(
16     theme: AppTheme,
17     fontMode: UiFont,
18     content: @Composable () -> Unit
19 ) {
20     val fontStyle = if (fontMode == UiFont.Italic) FontStyle.Italic else FontStyle.Normal
21     val fontFamily = if (fontMode == UiFont.Italic) FontFamily.Serif else FontFamily.SansSerif
22
23     val typography = Typography(
24         bodyLarge = TextStyle(fontStyle = fontStyle, fontFamily = fontFamily),
25         bodyMedium = TextStyle(fontStyle = fontStyle, fontFamily = fontFamily),
26         bodySmall = TextStyle(fontStyle = fontStyle, fontFamily = fontFamily),
27         titleLarge = TextStyle(fontStyle = fontStyle, fontFamily = fontFamily),
28         titleMedium = TextStyle(fontStyle = fontStyle, fontFamily = fontFamily),
29         titleSmall = TextStyle(fontStyle = fontStyle, fontFamily = fontFamily),
30         labelLarge = TextStyle(fontStyle = fontStyle, fontFamily = fontFamily),
31         labelMedium = TextStyle(fontStyle = fontStyle, fontFamily = fontFamily),
32         labelSmall = TextStyle(fontStyle = fontStyle, fontFamily = fontFamily),
33     )
34
35     val scheme = if (!theme.isLight) {
36         darkColorScheme(
37             primary = theme.accent,
38             onPrimary = theme.textPrimary,
39             background = theme.appBg, // Цвет стола
40             onBackground = theme.textPrimary,
41             surface = theme.panelBg, // Цвет панелей
42             onSurface = theme.textPrimary,
43             surfaceVariant = theme.panelBgVariant,
44             onSurfaceVariant = theme.textSecondary,
45         )
46     } else {
47         lightColorScheme(
48             primary = theme.accent,
49             onPrimary = Color.White,
50             background = theme.appBg,
51             onBackground = theme.textPrimary,
52             surface = theme.panelBg,
53             onSurface = theme.textPrimary,
54             surfaceVariant = theme.panelBgVariant,
55             onSurfaceVariant = theme.textSecondary,
56         )
57     }
58
59     CompositionLocalProvider(LocalAppTheme provides theme) {
60         MaterialTheme(
61             colorScheme = scheme,
62             typography = typography,
63             content = content,
64         )
65     }
66 }

```

109. app/src/main/java/com/wetinknext/ui/theme/ThemeController.kt

```

1  package com.wetinknext.ui.theme
2
3  import android.content.Context
4  import androidx.compose.runtime.*
5  import androidx.compose.runtime.saveable.Saver
6  import androidx.compose.runtime.saveable.rememberSaveable
7  import androidx.compose.ui.graphics.Color
8  import androidx.compose.ui.platform.LocalContext
9  import kotlin.math.pow
10
11  val LocalAppTheme = compositionLocalOf { AppThemes.Gray }
12
13  data class CustomThemeColors(
14      val appBackground: Int = 0xFF101826.toInt(),
15      val grid: Int = 0xFF38424C.toInt(),
16      val panel: Int = 0xFF172033.toInt(),
17      val accent: Int = 0xFF58A6FF.toInt(),
18      val text: Int = 0xFFFF4F7FB.toInt(),
19  )
20
21  class ThemeController(
22      initialThemeId: String = "gray",
23      initialFontMode: String = "Classic",
24      initialCustomColors: CustomThemeColors = CustomThemeColors()
25  ) {
26      var current by mutableStateOf(resolveTheme(initialThemeId, initialCustomColors))
27      private set
28      var font by mutableStateOf(if (initialFontMode == "Italic") UiFont.Italic else UiFont.Classic)
29      private set
30      var customColors by mutableStateOf(initialCustomColors)
31      private set
32      val customTheme: AppTheme
33      get() = createCustomTheme(customColors)
34
35      fun select(theme: AppTheme) {
36          current = if (theme.id == "custom") createCustomTheme(customColors) else theme
37      }
38
39      fun selectCustom() {
40          current = createCustomTheme(customColors)
41      }
42
43      fun selectFont(f: UiFont) { font = f }
44
45      fun updateCustomColors(newColors: CustomThemeColors) {
46          customColors = newColors
47          if (current.id == "custom") current = createCustomTheme(newColors)
48      }
49
50      private fun resolveTheme(id: String, colors: CustomThemeColors): AppTheme {
51          if (id == "custom") return createCustomTheme(colors)
52          return AppThemes.all.find { it.id == id } ?: AppThemes.Gray
53      }
54
55      private fun createCustomTheme(colors: CustomThemeColors): AppTheme {
56          val appBg = Color(colors.appBackground)
57          val panel = Color(colors.panel)
58          val accent = Color(colors.accent)
59          val text = Color(colors.text)
60          val grid = Color(colors.grid)
61          val isDark = luminanceOf(appBg) < 0.5f
62
63          return AppTheme(
64              id = "custom", displayName = "Своя", isLight = !isDark,
65              appBg = appBg, gridLine = grid,
66              panelBg = panel, panelBgSolid = panel.mix(if (isDark) Color.White else Color.Black, 0.08f),
67              panelStroke = panel.mix(accent, 0.3f),
68              panelInset = panel.mix(if (isDark) Color.Black else Color.White, 0.2f),
69              panelInsetSoft = panel.mix(accent, 0.1f),
70              canvasBackdrop = appBg, canvasGrid = grid,
71              textPrimary = text, textSecondary = text.copy(alpha = 0.6f),
72              iconInactive = text.copy(alpha = 0.4f),
73              accent = accent, accentSoft = accent.copy(alpha = 0.7f),
74              accentMuted = accent.copy(alpha = 0.3f),
75              danger = Color(0xFFFF4A5F)
76          )
77      }
78
79      private fun Color.mix(target: Color, amount: Float): Color {
80          return Color(
81              red = red * (1f - amount) + target.red * amount,
82              green = green * (1f - amount) + target.green * amount,
83              blue = blue * (1f - amount) + target.blue * amount,
84              alpha = alpha
85          )
86      }
87
88      private fun luminanceOf(color: Color): Float {
89          fun channel(value: Float): Float =
90              if (value <= 0.03928f) value / 12.92f else ((value + 0.055f) / 1.055f).pow(2.4f)

```

```

91         return 0.2126f * channel(color.red) + 0.7152f * channel(color.green) + 0.0722f * channel(color.blue)
92     }
93
94     companion object {
95         val Saver: Saver<ThemeController, *> = Saver(
96             save = { listOf(it.current.id, it.font.name, it.customColors.appBackground, it.customColors.grid, it.customColors.
panel, it.customColors.accent, it.customColors.text) },
97             restore = { list ->
98                 val l = list as List<*>
99                 ThemeController(
100                     initialThemeId = l[0] as String,
101                     initialFontMode = l[1] as String,
102                     initialCustomColors = CustomThemeColors(
103                         l[2] as Int, l[3] as Int, l[4] as Int, l[5] as Int, l[6] as Int
104                     )
105                 )
106             }
107         )
108     }
109 }
110
111 @Composable
112 fun rememberThemeController(): ThemeController {
113     val context = LocalContext.current
114     val prefs = remember { context.getSharedPreferences("wetinknext_theme_prefs", Context.MODE_PRIVATE) }
115
116     val controller = rememberSaveable(saver = ThemeController.Saver) {
117         val themeId = prefs.getString("theme_id", "gray") ?: "gray"
118         val fontMode = prefs.getString("font_mode", "Classic") ?: "Classic"
119         val customColors = CustomThemeColors(
120             appBackground = prefs.getInt("custom_bg", 0xFF101826.toInt()),
121             grid = prefs.getInt("custom_grid", 0xFF38424C.toInt()),
122             panel = prefs.getInt("custom_panel", 0xFF172033.toInt()),
123             accent = prefs.getInt("custom_accent", 0xFF58A6FF.toInt()),
124             text = prefs.getInt("custom_text", 0xFFFF4F7FB.toInt())
125         )
126         ThemeController(themeId, fontMode, customColors)
127     }
128
129     LaunchedEffect(controller.current.id, controller.font, controller.customColors) {
130         prefs.edit()
131             .putString("theme_id", controller.current.id)
132             .putString("font_mode", controller.font.name)
133             .putInt("custom_bg", controller.customColors.appBackground)
134             .putInt("custom_grid", controller.customColors.grid)
135             .putInt("custom_panel", controller.customColors.panel)
136             .putInt("custom_accent", controller.customColors.accent)
137             .putInt("custom_text", controller.customColors.text)
138             .apply()
139     }
140
141     return controller
142 }

```

110. app/src/main/java/com/wetinknext/ui/theme/WetInkNextPalette.kt

```
1 package com.wetinknext.ui.theme
2
3 import androidx.compose.ui.graphics.Color
4
5 data class WetInkNextPalette(
6     val appBg: Color,
7     val panelBg: Color,
8     val panelStroke: Color,
9     val textPrimary: Color,
10    val textSecondary: Color,
11    val accent: Color,
12 ) {
13     companion object {
14         val default = WetInkNextPalette(
15             appBg = Color(0xFF17181C),
16             panelBg = Color(0xFF25272D),
17             panelStroke = Color(0xFF3A3D46),
18             textPrimary = Color(0xFFF3F4F6),
19             textSecondary = Color(0xFFA4A8B3),
20             accent = Color(0xFF4D7AFF),
21         )
22     }
23 }
```

111. app/src/main/java/com/wetinknext/ui/thumbnaill/LayerThumbnail.kt

```

1  package com.wetinknext.ui.thumbnaill
2
3  import android.graphics.BitmapFactory
4  import android.util.LruCache
5  import androidx.compose.foundation.Canvas
6  import androidx.compose.foundation.Image
7  import androidx.compose.foundation.background
8  import androidx.compose.foundation.border
9  import androidx.compose.foundation.layout.Box
10 import androidx.compose.foundation.layout.fillMaxSize
11 import androidx.compose.foundation.shape.RoundedCornerShape
12 import androidx.compose.runtime.Composable
13 import androidx.compose.runtime.LaunchedEffect
14 import androidx.compose.runtime.getValue
15 import androidx.compose.runtime.mutableStateOf
16 import androidx.compose.runtime.remember
17 import androidx.compose.runtime.setValue
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.draw.clip
20 import androidx.compose.ui.geometry.Offset
21 import androidx.compose.ui.geometry.Size
22 import androidx.compose.ui.graphics.Color
23 import androidx.compose.ui.graphics.ImageBitmap
24 import androidx.compose.ui.graphics.asImageBitmap
25 import androidx.compose.ui.layout.ContentScale
26 import androidx.compose.ui.unit.dp
27 import java.io.File
28 import kotlinx.coroutines.Dispatchers
29 import kotlinx.coroutines.withContext
30
31 private data class LayerThumbnailKey(
32     val layerId: Long,
33     val version: Long,
34 )
35
36 /** Process-local cache; the layer version prevents recycled rows showing stale previews. */
37 private object LayerThumbnailBitmapCache {
38     private val bitmaps = LruCache<LayerThumbnailKey, ImageBitmap>(48)
39
40     fun get(key: LayerThumbnailKey): ImageBitmap? = bitmaps.get(key)
41
42     fun put(key: LayerThumbnailKey, bitmap: ImageBitmap) {
43         bitmaps.put(key, bitmap)
44     }
45 }
46
47 /**
48  * A persisted WebP preview with alpha. Transparent pixels reveal the checkerboard;
49  * the last decoded bitmap remains visible while a newer version is read from disk.
50  */
51 @Composable
52 fun LayerThumbnail(
53     layerId: Long,
54     path: String?,
55     thumbnailVersion: Long,
56     modifier: Modifier = Modifier,
57 ) {
58     val key = remember(layerId, thumbnailVersion) {
59         LayerThumbnailKey(layerId, thumbnailVersion)
60     }
61     var displayedBitmap by remember(layerId) {
62         mutableStateOf(LayerThumbnailBitmapCache.get(key))
63     }
64
65     LaunchedEffect(key, path) {
66         val bitmap = LayerThumbnailBitmapCache.get(key) ?: withContext(Dispatchers.IO) {
67             path
68                 ?.let(::File)
69                 ?.takeIf(File::isFile)
70                 ?.let { BitmapFactory.decodeFile(it.absolutePath) }
71                 ?.asImageBitmap()
72                 ?.also { LayerThumbnailBitmapCache.put(key, it) }
73         }
74         if (bitmap != null) {
75             displayedBitmap = bitmap
76         }
77     }
78
79     Box(
80         modifier = modifier
81             .clip(RoundedCornerShape(7.dp))
82             .background(CheckerboardLight)
83             .border(1.dp, CheckerboardBorder, RoundedCornerShape(7.dp)),
84     ) {
85         Checkerboard(modifier = Modifier.fillMaxSize())
86         displayedBitmap?.let { bitmap ->
87             Image(
88                 bitmap = bitmap,
89                 contentDescription = "Layer preview",
90                 modifier = Modifier.fillMaxSize(),

```

```

91         contentScale = ContentScale.Fit,
92     )
93 }
94 }
95 }
96
97 @Composable
98 private fun Checkerboard(modifier: Modifier = Modifier) {
99     Canvas(modifier = modifier) {
100         val cell = 6.dp.toPx()
101         var row = 0
102         var top = 0f
103         while (top < size.height) {
104             var column = 0
105             var left = 0f
106             while (left < size.width) {
107                 drawRect(
108                     color = if ((row + column) % 2 == 0) CheckerboardLight else CheckerboardDark,
109                     topLeft = Offset(left, top),
110                     size = Size(cell, cell),
111                 )
112                 left += cell
113                 column++
114             }
115             top += cell
116             row++
117         }
118     }
119 }
120
121 private val CheckerboardLight = Color(0xFFF5F3EF)
122 private val CheckerboardDark = Color(0xFFE2DED8)
123 private val CheckerboardBorder = Color(0xFFCEC7BF)

```

112. app/src/main/res/drawable/ic_eraser.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <vector xmlns:android="http://schemas.android.com/apk/res/android"
3      android:width="24dp"
4      android:height="24dp"
5      android:viewportWidth="24"
6      android:viewportHeight="24">
7      <path
8          android:fillColor="#FFFFFF"
9          android:pathData="M14.59,3.41a2,2 0,0 1,2.83 0l3.17,3.17a2,2 0,0 1,0 2.83l11.41,18.59a2,2 0,0 1,-2.83 0l4.41,14.41a2,
2 0,0 1,0 -2.83l10.18,-8.17zM3,20h18v2H3z" />
10     <path
11         android:fillColor="#FFFFFF"
12         android:fillAlpha="0.55"
13         android:pathData="M7.05,11l4.95,4.95l3.18,-3.18l-4.95,-4.95z" />
14 </vector>
```

113. app/src/main/res/drawable/ic_flip_horizontal.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <vector xmlns:android="http://schemas.android.com/apk/res/android"
3      android:width="24dp"
4      android:height="24dp"
5      android:viewportWidth="24"
6      android:viewportHeight="24">
7      <!-- Central Axis (Centered at 12) -->
8      <path
9          android:fillColor="#FFFFFF"
10         android:pathData="M11.25,4h1.5v16h-1.5V4z"/>
11
12     <!-- Left solid bracket (Symmetrical) -->
13     <path
14         android:fillColor="#FFFFFF"
15         android:pathData="M9,6.5H5v11h4V16H6.5V8H9V6.5z"/>
16
17     <!-- Right dashed bracket (Mirror of left, perfectly aligned) -->
18     <!-- Top dash -->
19     <path android:fillColor="#FFFFFF" android:pathData="M15,6.5h2.5V8H15V6.5z"/>
20     <path android:fillColor="#FFFFFF" android:pathData="M18.5,6.5h0.5V8h-0.5V6.5z"/>
21     <!-- Side dashes -->
22     <path android:fillColor="#FFFFFF" android:pathData="M17.5,9.5h1.5v2h-1.5V9.5z"/>
23     <path android:fillColor="#FFFFFF" android:pathData="M17.5,12.5h1.5v2h-1.5V12.5z"/>
24     <!-- Bottom dash -->
25     <path android:fillColor="#FFFFFF" android:pathData="M15,16h2.5v1.5H15V16z"/>
26     <path android:fillColor="#FFFFFF" android:pathData="M18.5,16h0.5v1.5h-0.5V16z"/>
27 </vector>
```

114. app/src/main/res/drawable/ic_launcher_background.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <vector xmlns:android="http://schemas.android.com/apk/res/android"
3     android:width="108dp"
4     android:height="108dp"
5     android:viewportWidth="108"
6     android:viewportHeight="108">
7     <path
8         android:fillColor="#FF211019"
9         android:pathData="M0,0h108v108h-108z" />
10 </vector>
```

115. app/src/main/res/drawable/ic_launcher_foreground.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <vector xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:aapt="http://schemas.android.com/aapt"
4      android:width="108dp"
5      android:height="108dp"
6      android:viewportWidth="108"
7      android:viewportHeight="108">
8      <group
9          android:pivotX="54"
10         android:pivotY="54"
11         android:scaleX="0.6"
12         android:scaleY="0.6">
13         <path
14             android:pathData="M14,88h80"
15             android:strokeWidth="5"
16             android:strokeLineCap="round">
17             <aapt:attr name="android:strokeColor">
18                 <gradient
19                     android:startX="78"
20                     android:startY="20"
21                     android:endX="30"
22                     android:endY="80"
23                     android:type="linear">
24                     <item android:offset="0" android:color="#FFFFB1C5" />
25                     <item android:offset="1" android:color="#FFFF3D70" />
26                 </gradient>
27             </aapt:attr>
28         </path>
29         <path
30             android:pathData="M66,16c3.3,-3.3 8.7,-3.3 12,0l4,4c3.3,3.3 3.3,8.7 0,12l34,80l-16,4l4,-16l66,16z"
31             android:fillType="nonZero">
32             <aapt:attr name="android:fillColor">
33                 <gradient
34                     android:startX="78"
35                     android:startY="20"
36                     android:endX="30"
37                     android:endY="80"
38                     android:type="linear">
39                     <item android:offset="0" android:color="#FFFFB1C5" />
40                     <item android:offset="1" android:color="#FFFF3D70" />
41                 </gradient>
42             </aapt:attr>
43         </path>
44     </group>
45 </vector>
```

116. app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
3     <background android:drawable="@drawable/ic_launcher_background" />
4     <foreground android:drawable="@drawable/ic_launcher_foreground" />
5 </adaptive-icon>
```

117. app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
3     <background android:drawable="@drawable/ic_launcher_background" />
4     <foreground android:drawable="@drawable/ic_launcher_foreground" />
5 </adaptive-icon>
```

118. app/src/main/res/mipmap-anydpi/ic_launcher.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <layer-list xmlns:android="http://schemas.android.com/apk/res/android">
3     <item android:drawable="@drawable/ic_launcher_background" />
4     <item android:drawable="@drawable/ic_launcher_foreground" />
5 </layer-list>
```

119. app/src/main/res/mipmap-anydpi/ic_launcher_round.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <layer-list xmlns:android="http://schemas.android.com/apk/res/android">
3     <item android:drawable="@drawable/ic_launcher_background" />
4     <item android:drawable="@drawable/ic_launcher_foreground" />
5 </layer-list>
```

120. app/src/main/res/values/strings.xml

```
1 <resources>
2     <string name="app_name">WetInk Next</string>
3 </resources>
```

121. app/src/main/res/values/themes.xml

```
1 <resources>
2     <style name="Theme.WetInkNext" parent="android:style/Theme.Material.NoActionBar" />
3 </resources>
```

122. app/src/test/java/com/wetinknext/data/project/AutosaveCoordinatorTest.kt

```
1 package com.wetinknext.data.project
2
3 import kotlinx.coroutines.CoroutineScope
4 import kotlinx.coroutines.Dispatchers
5 import kotlinx.coroutines.SupervisorJob
6 import kotlinx.coroutines.cancel
7 import org.junit.Assert.assertEquals
8 import org.junit.Assert.assertTrue
9 import org.junit.Test
10 import java.util.concurrent.CopyOnWriteArrayList
11 import java.util.concurrent.CountDownLatch
12 import java.util.concurrent.TimeUnit
13
14 class AutosaveCoordinatorTest {
15     @Test
16     fun laterCompletedEditReplacesEarlierPendingSave() {
17         val scope = CoroutineScope(SupervisorJob() + Dispatchers.Default)
18         val coordinator = AutosaveCoordinator(scope, debounceMillis = 30L)
19         val saved = CopyOnWriteArrayList<String>()
20         val latch = CountDownLatch(1)
21
22         coordinator.schedule { saved += "first" }
23         coordinator.schedule {
24             saved += "second"
25             latch.countDown()
26         }
27
28         assertTrue(latch.await(2, TimeUnit.SECONDS))
29         assertEquals(listOf("second"), saved)
30         scope.cancel()
31     }
32 }
```

123. app/src/test/java/com/wetinknext/data/project/FileProjectRepositoryTest.kt

```

1 package com.wetinknext.data.project
2
3 import com.wetinknext.domain.document.ProjectAssets
4 import com.wetinknext.domain.document.ProjectDocument
5 import com.wetinknext.domain.document.WetInkProjectContainer
6 import org.junit.Assert.assertArrayEquals
7 import org.junit.Assert.assertEquals
8 import org.junit.Assert.assertFalse
9 import org.junit.Assert.assertTrue
10 import org.junit.Test
11 import java.io.File
12 import java.nio.file.Files
13 import java.util.concurrent.atomic.AtomicLong
14 import kotlin.coroutines.Continuation
15 import kotlin.coroutines.EmptyCoroutineContext
16 import kotlin.coroutines.startCoroutine
17
18 class FileProjectRepositoryTest {
19     @Test
20     fun projectLifecycleKeepsGalleryMetadataCompact() {
21         val root = Files.createTempDirectory("wetink-repository").toFile()
22         val time = AtomicLong(100L)
23         val repository = FileProjectRepository(
24             projectsDirectory = root,
25             container = WetInkProjectContainer(clockMillis = { time.incrementAndGet() }),
26         )
27
28         val first = runSuspend {
29             repository.create(CreateProjectRequest(name = "First", width = 800, height = 600))
30         }
31         val second = runSuspend {
32             repository.create(CreateProjectRequest(name = "Second", width = 1200, height = 900))
33         }
34
35         val listed = runSuspend { repository.list() }
36         assertEquals(listOf(second.id, first.id), listed.map(ProjectSummary::id))
37         assertEquals("Second", listed.first().name)
38         assertEquals(1200, listed.first().width)
39         assertEquals(900, listed.first().height)
40
41         val opened = runSuspend { repository.open(first.id) }
42         assertEquals("First", opened.name)
43         assertEquals(800, opened.width)
44
45         runSuspend { repository.rename(first.id, "Renamed") }
46         assertEquals("Renamed", runSuspend { repository.open(first.id) }.name)
47
48         runSuspend { repository.moveToTrash(first.id) }
49         assertFalse(runSuspend { repository.list() }.any { it.id == first.id })
50         assertEquals(first.id, runSuspend { repository.listTrash() }.single().project.id)
51         runSuspend { repository.restore(first.id) }
52         assertTrue(runSuspend { repository.list() }.any { it.id == first.id })
53         runSuspend { repository.moveToTrash(first.id) }
54         runSuspend { repository.deleteForever(first.id) }
55         assertFalse(File(root, ".trash/${first.id}.wetink").exists())
56     }
57
58     @Test
59     fun metadataSavePreservesExistingTilePayloads() {
60         val root = Files.createTempDirectory("wetink-repository-save").toFile()
61         val container = WetInkProjectContainer(clockMillis = { 500L })
62         val repository = FileProjectRepository(root, container)
63         val document = ProjectDocument.newUntitled(id = "project-1", nowMillis = 100L)
64         val file = File(root, "project-1.wetink")
65         container.save(
66             file,
67             document,
68             ProjectAssets(mapOf(1L to byteArrayOf(1, 2), 2L to byteArrayOf(3, 4))),
69         )
70
71         runSuspend { repository.save(document.copy(name = "Saved name")) }
72
73         // Saving a legacy ZIP is the migration point: the same project is now
74         // represented by projects/project-1/{manifest,document,layers}.
75         assertTrue(File(root, "project-1").isDirectory)
76         val loaded = runSuspend { repository.openForEditor("project-1") }
77         assertEquals("Saved name", loaded.document.name)
78         assertArrayEquals(byteArrayOf(1, 2), loaded.assets.layerTiles.getValue(1L))
79         assertArrayEquals(byteArrayOf(3, 4), loaded.assets.layerTiles.getValue(2L))
80     }
81
82     @Test
83     fun recoverySnapshotPreservesNewestTilePayloadUntilPrimarySave() {
84         val root = Files.createTempDirectory("wetink-recovery").toFile()
85         val repository = FileProjectRepository(root, WetInkProjectContainer(clockMillis = { 700L }))
86         val summary = runSuspend { repository.create(CreateProjectRequest(name = "Recovery")) }
87         val document = runSuspend { repository.open(summary.id) }
88
89         runSuspend { repository.saveRecovery(document, mapOf(2L to byteArrayOf(9, 8, 7))) }
90         val recovered = runSuspend { repository.openRecovery(summary.id) }

```

```

91         assertEquals(byteArrayOf(9, 8, 7), checkNotNull(recovered).assets.layerTiles.getValue(2L))
92     }
93     runSuspend { repository.save(recovered.document, recovered.assets.layerTiles) }
94     runSuspend { repository.clearRecovery(summary.id) }
95     assertTrue(runSuspend { repository.openRecovery(summary.id) } == null)
96     assertEquals(byteArrayOf(9, 8, 7), runSuspend { repository.openForEditor(summary.id) }.assets.layerTiles.
getValue(2L))
97     }
98 }
99 @Test
100 fun thumbnailSaveIsAtomicAndGalleryReadsThePublishedPreview() {
101     val root = Files.createTempDirectory("wetink-thumbnail").toFile()
102     val repository = FileProjectRepository(root, WetInkProjectContainer(clockMillis = { 800L }))
103     val summary = runSuspend { repository.create(CreateProjectRequest(name = "Preview")) }
104     val document = runSuspend { repository.open(summary.id) }
105     val webp = byteArrayOf(0x52, 0x49, 0x46, 0x46, 0x57, 0x45, 0x42, 0x50)
106
107     runSuspend { repository.save(document, emptyMap(), webp) }
108
109     val galleryItem = runSuspend { repository.list() }.single()
110     assertEquals(webp, galleryItem.thumbnailBytes)
111     assertEquals(webp, runSuspend { repository.openForEditor(summary.id) }.assets.thumbnailWebp)
112 }
113
114 @Test
115 fun duplicateCreatesIndependentDirectoryProject() {
116     val root = Files.createTempDirectory("wetink-duplicate").toFile()
117     val repository = FileProjectRepository(root, WetInkProjectContainer(clockMillis = { 900L }))
118     val original = runSuspend { repository.create(CreateProjectRequest(name = "Original")) }
119
120     val duplicate = runSuspend { repository.duplicate(original.id) }
121
122     assertFalse(original.id == duplicate.id)
123     assertEquals("Original copy", duplicate.name)
124     assertTrue(File(root, duplicate.id).isDirectory)
125     val originalTiles = runSuspend { repository.openForEditor(original.id) }.assets.layerTiles
126     val duplicateTiles = runSuspend { repository.openForEditor(duplicate.id) }.assets.layerTiles
127     assertEquals(originalTiles.keys, duplicateTiles.keys)
128     originalTiles.forEach { (layerId, bytes) ->
129         assertEquals(bytes, duplicateTiles.getValue(layerId))
130     }
131 }
132
133 private fun <T> runSuspend(block: suspend () -> T): T {
134     var result: Result<T>? = null
135     block.startCoroutine(object : Continuation<T> {
136         override val context = EmptyCoroutineContext
137
138         override fun resumeWith(value: Result<T>) {
139             result = value
140         }
141     })
142     return checkNotNull(result).getOrNull()
143 }
144 }

```

124. app/src/test/java/com/wetinknext/domain/document/ProjectDocumentTest.kt

```

1  package com.wetinknext.domain.document
2
3  import kotlinx.serialization.encodeToString
4  import kotlinx.serialization.json.Json
5  import org.junit.Assert.assertEquals
6  import org.junit.Assert.assertFalse
7  import org.junit.Assert.assertTrue
8  import org.junit.Assert.fail
9  import org.junit.Test
10
11 class ProjectDocumentTest {
12     private val json = Json {
13         encodeDefaults = true
14         ignoreUnknownKeys = true
15     }
16
17     @Test
18     fun untitledProjectRoundTripsWithoutEmbeddingLayerPixels() {
19         val document = ProjectDocument.newUntitled(
20             id = "project-1",
21             name = "Sketch",
22             width = 2048,
23             height = 1536,
24             dpi = 300,
25             nowMillis = 1234L,
26         )
27
28         val encoded = json.encodeToString(document)
29         val decoded = json.decodeFromString<ProjectDocument>(encoded)
30
31         assertEquals(document, decoded)
32         assertTrue(encoded.contains("\pixelFile\":"layers/1.tiles\"))
33         assertFalse(encoded.contains("rawBytes"))
34         assertFalse(encoded.contains("textureId"))
35     }
36
37     @Test(expected = IllegalArgumentException::class)
38     fun projectRejectsAnUnknownActiveLayer() {
39         val document = ProjectDocument.newUntitled(id = "project-1", nowMillis = 1234L)
40         document.copy(activeLayerId = 999L)
41     }
42
43     @Test
44     fun newCanvasRejectsAnUnsafeInitialGpuAllocation() {
45         try {
46             ProjectDocument.newUntitled(width = 8192, height = 8192)
47             fail("Expected memory-budget validation to reject an 8192px canvas")
48         } catch (_: IllegalArgumentException) {
49             // Expected: two initial layers and the stroke target would exceed the budget.
50         }
51     }
52 }

```

125. app/src/test/java/com/wetinknext/domain/document/WetInkProjectContainerTest.kt

```

1 package com.wetinknext.domain.document
2
3 import org.junit.Assert.assertArrayEquals
4 import org.junit.Assert.assertEquals
5 import org.junit.Assert.assertFalse
6 import org.junit.Assert.fail
7 import org.junit.Test
8 import java.io.File
9 import java.nio.file.Files
10 import java.util.zip.ZipFile
11
12 class WetInkProjectContainerTest {
13     @Test
14     fun savePublishesOneContainerWithSeparateLayerPayloads() {
15         val root = Files.createTempDirectory("wetink-project-test").toFile()
16         val projectFile = File(root, "sketch.wetink")
17         val document = ProjectDocument.newUntitled(
18             id = "project-1",
19             nowMillis = 100L,
20         )
21         val assets = ProjectAssets(
22             layerTiles = mapOf(
23                 1L to byteArrayOf(1, 2, 3),
24                 2L to byteArrayOf(4, 5, 6),
25             ),
26             thumbnailWebp = byteArrayOf(7, 8),
27             layerPreviewsWebp = mapOf(2L to byteArrayOf(9)),
28         )
29         val container = WetInkProjectContainer(clockMillis = { 200L })
30
31         val saved = container.save(projectFile, document, assets)
32         val loaded = container.load(projectFile)
33
34         assertEquals(100L, document.updatedAt)
35         assertEquals(200L, saved.document.updatedAt)
36         assertEquals(saved.document, loaded.document)
37         assertArrayEquals(byteArrayOf(1, 2, 3), loaded.assets.layerTiles.getValue(1L))
38         assertArrayEquals(byteArrayOf(4, 5, 6), loaded.assets.layerTiles.getValue(2L))
39         assertArrayEquals(byteArrayOf(7, 8), loaded.assets.thumbnailWebp)
40         assertArrayEquals(byteArrayOf(9), loaded.assets.layerPreviewsWebp.getValue(2L))
41         assertFalse(File(root, "sketch.wetink.tmp").exists())
42
43         ZipFile(projectFile).use { zip ->
44             assertEquals(true, zip.getEntry("manifest.json") != null)
45             assertEquals(true, zip.getEntry("document.json") != null)
46             assertEquals(true, zip.getEntry("layers/1.tiles") != null)
47             assertEquals(true, zip.getEntry("layers/2.tiles") != null)
48             assertEquals(true, zip.getEntry("thumbnail.webp") != null)
49             assertEquals(true, zip.getEntry("previews/2.webp") != null)
50         }
51     }
52
53     @Test
54     fun invalidSecondSaveLeavesPublishedProjectUntouched() {
55         val root = Files.createTempDirectory("wetink-project-atomic").toFile()
56         val projectFile = File(root, "sketch.wetink")
57         val document = ProjectDocument.newUntitled(
58             id = "project-1",
59             nowMillis = 100L,
60         )
61         val container = WetInkProjectContainer(clockMillis = { 200L })
62         container.save(
63             projectFile,
64             document,
65             ProjectAssets(mapOf(1L to byteArrayOf(1), 2L to byteArrayOf(2))),
66         )
67
68         try {
69             container.save(
70                 projectFile,
71                 document,
72                 ProjectAssets(mapOf(1L to byteArrayOf(99))),
73             )
74             fail("Missing layer payload must reject the save")
75         } catch (_: IllegalArgumentException) {
76             // Expected: validation happens before replacing the published file.
77         }
78
79         val loaded = container.load(projectFile)
80         assertArrayEquals(byteArrayOf(1), loaded.assets.layerTiles.getValue(1L))
81         assertArrayEquals(byteArrayOf(2), loaded.assets.layerTiles.getValue(2L))
82     }
83 }

```

126. app/src/test/java/com/wetinknext/engine/brush/BrushDynamicsTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class BrushDynamicsTest {
7     @Test
8     fun resolve_separatesLocalCoverageFromGlobalOpacity() {
9         val settings = BrushSettings(
10             baseRadiusPx = 10f,
11             opacity = 0.2f,
12             pressureGamma = 2f,
13             minSizeRatio = 0.1f,
14             tiltToSize = 0.5f,
15             tiltToOpacity = 1f,
16         )
17         val result = ResolvedDab()
18
19         BrushDynamics.resolve(
20             settings = settings,
21             pressure = 0.5f,
22             tiltX = 1f,
23             tiltY = 0f,
24             out = result,
25         )
26
27         //  $p = 0.5^2 = 0.25$ ; pressure size = .325; tilt size = 1.5.
28         assertEquals(4.875f, result.radius, 0.0001f)
29         // Tilt attenuates, rather than amplifies, local coverage.
30         assertEquals(0.1375f, result.coverage, 0.0001f)
31         assertEquals(result.coverage, result.opacity, 0.0001f)
32     }
33 }
```

127. app/src/test/java/com/wetinknext/engine/brush/BrushSerializationTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import kotlinx.serialization.encodeToString
4 import org.junit.Assert.assertEquals
5 import org.junit.Test
6
7 class BrushSerializationTest {
8
9     @Test
10     fun settingsRoundTripKeepsRenderFields() {
11         val json = BrushRepository.defaultJson
12         val original = BrushSettings(
13             name = "Pencil 6B",
14             grainAssetPath = "asset:brush/pencil_6b.png",
15             grainScale = 5.5f,
16             textureDepth = .65f,
17             tiltToSize = .55f,
18             blendPolicy = BlendPolicy.NORMAL_BUILDUP,
19         )
20
21         val encoded = json.encodeToString(original)
22         val restored = json.decodeFromString<BrushSettings>(encoded)
23
24         assertEquals(original, restored)
25     }
26
27     @Test
28     fun presetRoundTrip() {
29         val repo = BrushRepository()
30         val original = BrushPreset(
31             id = "test-preset",
32             settings = BrushSettings(name = "Test Brush")
33         )
34
35         val encoded = repo.encode(original)
36         val restored = repo.decode(encoded)
37
38         assertEquals(original, restored)
39     }
40 }
```

128. app/src/test/java/com/wetinknext/engine/brush/BrushSettingsTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class BrushSettingsTest {
7     @Test
8     fun resolved_clampsBrushValues() {
9         val result = BrushSettings(
10             baseRadiusPx = -10f,
11             opacity = 3f,
12             flow = -1f,
13             spacing = 100f,
14             pressureGamma = 0f,
15             minSizeRatio = 4f,
16         ).resolved()
17
18         assertEquals(0.25f, result.baseRadiusPx, 0.001f)
19         assertEquals(1f, result.opacity, 0.001f)
20         assertEquals(0f, result.flow, 0.001f)
21         assertEquals(4f, result.spacing, 0.001f)
22         assertEquals(0.05f, result.pressureGamma, 0.001f)
23         assertEquals(1f, result.minSizeRatio, 0.001f)
24     }
25 }
```

129. app/src/test/java/com/wetinknext/engine/brush/CapsuleEmitterTest.kt

```

1  package com.wetinknext.engine.brush
2
3  import com.wetinknext.engine.input.InputAction
4  import com.wetinknext.engine.input.InputBatch
5  import com.wetinknext.engine.input.PointerTool
6  import org.junit.Assert.assertTrue
7  import org.junit.Test
8
9  class CapsuleEmitterTest {
10
11     @Test
12     fun finishFlushesTheLastInterpolatedSection() {
13         val settings = BrushSettings(
14             renderMode = BrushRenderMode.RIBBON,
15             baseRadiusPx = 10f,
16             smoothing = 0.5f,
17             streamline = 0f,
18         )
19         val emitter = CapsuleEmitter(settings)
20         // We need a dummy renderer to receive segments
21         // Since we can't easily mock GL context here, we just use a small maxSegments
22         val renderer = CapsuleStrokeRenderer(maxSegments = 128)
23
24         emitter.begin(batch(InputAction.DOWN, 0f, 0f, 0L), renderer)
25         emitter.append(batch(InputAction.MOVE, 100f, 0f, 16_000_000L), renderer)
26         emitter.append(batch(InputAction.MOVE, 200f, 30f, 32_000_000L), renderer)
27
28         val segmentsBeforeFinish = renderer.segmentCount
29         emitter.finish(renderer, cancel = false)
30         val segmentsAfterFinish = renderer.segmentCount
31
32         assertTrue("Should have some segments after append", segmentsBeforeFinish > 0)
33         assertTrue("Finish should have flushed more segments", segmentsAfterFinish > segmentsBeforeFinish)
34         assertTrue(emitter.hasStroke)
35     }
36
37     private fun batch(action: InputAction, x: Float, y: Float, t: Long) = InputBatch(8).apply {
38         begin(action)
39         addSample(x, y, .5f, 0f, 0f, 0f, t, 0, PointerTool.STYLUS, false)
40     }
41 }

```

130. app/src/test/java/com/wetinknext/engine/brush/CapsuleStrokeRendererTest.kt

```

1  package com.wetinknext.engine.brush
2
3  import org.junit.Assert.assertEquals
4  import org.junit.Assert.assertFalse
5  import org.junit.Assert.assertTrue
6  import org.junit.Test
7  import com.wetinknext.engine.gl.ShaderLib
8
9  class CapsuleStrokeRendererTest {
10
11     @Test
12     fun capsuleShaderAppliesFlowInsideTheStrokeTarget() {
13         assertTrue(ShaderLib.capsuleFragment.contains("uniform float uFlow"))
14         assertTrue(ShaderLib.capsuleFragment.contains("grainFactor * uFlow"))
15     }
16
17     @Test
18     fun initiallyEmpty() {
19         val renderer = CapsuleStrokeRenderer(maxSegments = 100)
20         assertTrue(renderer.isEmpty)
21         assertEquals(0, renderer.segmentCount)
22     }
23
24     @Test
25     fun addsSegmentsWithinCapacity() {
26         val renderer = CapsuleStrokeRenderer(maxSegments = 10)
27         renderer.beginStroke()
28
29         for (i in 0 until 10) {
30             val added = renderer.addSegment(0f, 0f, 5f, 1f, 10f, 10f, 5f, 1f)
31             assertTrue("Segment $i should be added", added)
32         }
33
34         assertEquals(10, renderer.segmentCount)
35         assertFalse(renderer.isEmpty)
36
37         val addedOverflow = renderer.addSegment(0f, 0f, 5f, 1f, 10f, 10f, 5f, 1f)
38         assertFalse("Overflow segment should not be added", addedOverflow)
39         assertEquals(1, renderer.overflowCount)
40     }
41
42     @Test
43     fun clearResetsStateButNotOverflow() {
44         val renderer = CapsuleStrokeRenderer(maxSegments = 10)
45         renderer.beginStroke()
46         renderer.addSegment(0f, 0f, 5f, 1f, 10f, 10f, 5f, 1f)
47
48         // Overflow it
49         for (i in 0 until 10) renderer.addSegment(0f, 0f, 1f, 1f, 1f, 1f, 1f, 1f)
50         assertEquals(1, renderer.overflowCount)
51
52         renderer.clearStrokeData()
53         assertTrue(renderer.isEmpty)
54         assertEquals(0, renderer.segmentCount)
55         assertEquals(1, renderer.overflowCount)
56     }
57 }

```

131. app/src/test/java/com/wetinknext/engine/brush/ColorSpacesTest.kt

```
1 package com.wetinknext.engine.brush
2 import org.junit.Assert.*
3 import org.junit.Test
4 class ColorSpacesTest { @Test fun blackAndWhiteAreStable(){val b=FloatArray(3);val w=FloatArray(3);ColorSpaces.srgb8ToLinear(
0xFF000000L,b);ColorSpaces.srgb8ToLinear(0xFFFFFFFFL,w);assertEquals(0f,b[0],.0001f);assertEquals(1f,w[0],.0001f)} @Test fun
roundTrip(){assertEquals(.25f,ColorSpaces.srgbToLinear(ColorSpaces.linearToSrgb(.25f)),.0001f)} }
```

132. app/src/test/java/com/wetinknext/engine/brush/DabBufferTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import com.wetinknext.engine.core.DirtyRect
4 import org.junit.Assert.*
5 import org.junit.Test
6
7 class DabBufferTest {
8     @Test fun overflowIsCounted(){val b=DabBuffer(2);assertTrue(b.add(0f,0f,1f,0f,1f,1f));assertTrue(b.add(1f,1f,1f,0f,1f,
9 1f));assertFalse(b.add(2f,2f,1f,0f,1f,1f));assertEquals(1L,b.overflowCount)}
10    @Test fun clearResets(){val b=DabBuffer(1);b.add(0f,0f,1f,0f,1f,1f);b.clear();assertEquals(0,b.count)}
11    @Test fun dirtyRectIncludesTextureMargin(){
12        val buffer = DabBuffer(1)
13        buffer.add(10f, 20f, 5f, 0f, 1f, 1f)
14        val rect = DirtyRect()
15        buffer.includeDirtyRect(rect, extraMargin = 4f)
16        assertEquals(1f, rect.left, 0.001f)
17        assertEquals(19f, rect.right, 0.001f)
18        assertEquals(11f, rect.top, 0.001f)
19        assertEquals(29f, rect.bottom, 0.001f)
20    }
```

133. app/src/test/java/com/wetinknext/engine/brush/OneEuroFilterTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertFalse
5 import org.junit.Test
6
7 class OneEuroFilterTest {
8     @Test fun firstSamplePassesThrough() { val f=OneEuroFilter();val out=FloatArray(2);f.filter(0,10f,20f,out);assertEquals(
10f,out[0],.0001f);assertEquals(20f,out[1],.0001f) }
9     @Test fun samplesRemainFinite() { val f=OneEuroFilter();val out=FloatArray(2);f.filter(0,0f,0f,out);f.filter(8_000_000,
10f,5f,out);assertFalse(out[0].isNaN());assertFalse(out[1].isInfinite()) }
10 }
```

134. app/src/test/java/com/wetinknext/engine/brush/PressureFilterTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertTrue
4 import org.junit.Test
5
6 class PressureFilterTest {
7     @Test fun singleZeroPressureGlitchDoesNotCollapseWidth() {
8         val filter = PressureFilter()
9         filter.filter(0L, .6f)
10        val filtered = filter.filter(16_000_000L, 0f)
11        assertTrue("zero glitch must be ignored while stroke is active", filtered > .55f)
12    }
13
14    @Test fun realPressureChangeIsSmoothed() {
15        val filter = PressureFilter()
16        filter.filter(0L, .8f)
17        val filtered = filter.filter(16_000_000L, .2f)
18        assertTrue(filtered in .2f.. .8f)
19    }
20 }
```

135. app/src/test/java/com/wetinknext/engine/brush/StabilizerTest.kt

```
1 package com.wetinknext.engine.brush
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertTrue
5 import org.junit.Test
6
7 class StabilizerTest {
8     @Test fun strengthZeroPassesRawThrough(){val s=Stabilizer();s.process(0,100f,200f);assertEquals(100f,s.x,.0001f);
assertEquals(200f,s.y,.0001f)}
9     @Test fun velocityIsPositiveAfterMovement(){val s=Stabilizer();s.process(0,0f,0f);s.process(1_000_000_000,100f,0f);
assertTrue(s.velocity>0f)}
10 }
```

136. app/src/test/java/com/wetinknext/engine/brush/StampEmitterTest.kt

```

1  package com.wetinknext.engine.brush
2
3  import com.wetinknext.engine.input.*
4  import org.junit.Assert.assertEquals
5  import org.junit.Test
6
7  class StampEmitterTest {
8      @Test fun movingStrokeHasNoExtraDownDab(){val e=StampEmitter(BrushSettings(baseRadiusPx=10f,spacing=1f,
spacingUsesDiameter=false,pressureToSize=false,pressureToOpacity=false));val d=DabBuffer(16);val down=InputBatch(1).apply{begin(
InputAction.DOWN);addSample(0f,0f,1f,0f,0f,0f,0,0,PointerTool.STYLUS,false));val move=InputBatch(1).apply{begin(InputAction.MOVE);
addSample(20f,0f,1f,0f,0f,0f,1,0,PointerTool.STYLUS,false));e.begin(down,d);e.append(move,d);e.finish(d,false);assertEquals(2,d.
count)}}
9
10     @Test fun tapProducesExactlyOneDab(){val e=StampEmitter(BrushSettings());val d=DabBuffer(16);val down=InputBatch(1).apply{
begin(InputAction.DOWN);addSample(5f,5f,1f,0f,0f,0f,0,0,PointerTool.STYLUS,false));e.begin(down,d);e.finish(d,false);assertEquals(1,
d.count)}}
11
12     @Test
13     fun activeStrokeKeepsItsSettingsAfterUiBrushChanges() {
14         val strokeSettings = BrushSettings(
15             baseRadiusPx = 5f,
16             spacing = 1f,
17             spacingUsesDiameter = false,
18             smoothing = 0f,
19             pressureToSize = false,
20             pressureToOpacity = false,
21         )
22         val emitter = StampEmitter(strokeSettings)
23         val dabs = DabBuffer(16)
24         val down = InputBatch(1).apply {
25             begin(InputAction.DOWN)
26             addSample(0f, 0f, 1f, 0f, 0f, 0f, 0L, 0, PointerTool.STYLUS, false)
27         }
28         val move = InputBatch(1).apply {
29             begin(InputAction.MOVE)
30             addSample(20f, 0f, 1f, 0f, 0f, 0f, 1L, 0, PointerTool.STYLUS, false)
31         }
32
33         emitter.begin(down, dabs, strokeSettings)
34         emitter.updateSettings(strokeSettings.copy(baseRadiusPx = 20f))
35         emitter.append(move, dabs)
36
37         assertEquals(5f, dabs.floats.get(2), 0.001f)
38     }
39 }

```

137. app/src/test/java/com/wetinknext/engine/core/DirtyRectTest.kt

```
1 package com.wetinknext.engine.core
2
3 import org.junit.Assert.assertArrayEquals
4 import org.junit.Assert.assertTrue
5 import org.junit.Test
6
7 class DirtyRectTest {
8     @Test fun includesDabsAndConvertsToPixelBounds() {
9         val rect = DirtyRect()
10        rect.include(100f, 120f, 10f)
11        rect.include(150f, 140f, 5f)
12        val pixels = IntArray(4)
13        rect.toPixelBounds(pixels)
14        assertArrayEquals(intArrayOf(90, 110, 155, 145), pixels)
15    }
16
17    @Test fun clampCanMakeRectEmpty() {
18        val rect = DirtyRect()
19        rect.include(-100f, -100f, -10f)
20        rect.clamp(100f, 100f)
21        assertTrue(rect.isEmpty)
22    }
23
24    @Test fun clearResetsRect() {
25        val rect = DirtyRect()
26        rect.include(0f, 0f, 10f)
27        rect.clear()
28        assertTrue(rect.isEmpty)
29    }
30 }
```

138. app/src/test/java/com/wetinknext/engine/core/DocumentSessionTest.kt

```
1 package com.wetinknext.engine.core
2
3 import com.wetinknext.domain.document.ProjectDocument
4 import com.wetinknext.engine.canvas.LayerStack
5 import com.wetinknext.engine.undo.UndoManager
6 import org.junit.Assert.assertEquals
7 import org.junit.Assert.assertFalse
8 import org.junit.Assert.assertTrue
9 import org.junit.Test
10
11 class DocumentSessionTest {
12     @Test
13     fun committedLayersRemainDirtyUntilTheirSaveIsAcknowledged() {
14         val session = DocumentSession(
15             ProjectDocument.newUntitled(id = "session-test"),
16             LayerStack(),
17             UndoManager(),
18             emptyMap(),
19         )
20
21         session.markLayerDirty(2L)
22         session.markLayerDirty(2L)
23
24         assertEquals(setOf(2L), session.dirtyLayerIds)
25         assertTrue(session.projectDirty)
26
27         session.markSaved(setOf(2L))
28
29         assertTrue(session.dirtyLayerIds.isEmpty())
30         assertFalse(session.projectDirty)
31     }
32 }
```

139. app/src/test/java/com/wetinknext/engine/core/LatestValueDeliveryTest.kt

```
1 package com.wetinknext.engine.core
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class LatestValueDeliveryTest {
7     @Test
8     fun captureBeforeListenerIsDeliveredAfterListenerIsInstalled() {
9         val delivery = LatestValueDelivery<String>()
10        val delivered = mutableListOf<String>()
11
12        delivery.offer("first-thumbnail")
13        delivery.dispatchTo(null)
14        delivery.dispatchTo(delivered::add)
15
16        assertEquals(listOf("first-thumbnail"), delivered)
17    }
18
19    @Test
20    fun newestCaptureReplacesAnUndeliveredOlderCapture() {
21        val delivery = LatestValueDelivery<String>()
22        val delivered = mutableListOf<String>()
23
24        delivery.offer("outdated")
25        delivery.offer("current")
26        delivery.dispatchTo(delivered::add)
27
28        assertEquals(listOf("current"), delivered)
29    }
30 }
```

140. app/src/test/java/com/wetinknext/engine/core/LayerTileCodecTest.kt

```
1 package com.wetinknext.engine.core
2
3 import org.junit.Assert.assertArrayEquals
4 import org.junit.Assert.assertEquals
5 import org.junit.Assert.assertNull
6 import org.junit.Test
7
8 class LayerTileCodecTest {
9     @Test
10     fun roundTripPreservesRgba8LayerPayload() {
11         val pixels = byteArrayOf(1, 2, 3, 4, 5, 6, 7, 8)
12         val decoded = LayerTileCodec.decode(LayerTileCodec.encode(2, 1, 4, pixels))
13
14         checkNotNull(decoded)
15         assertEquals(2, decoded.width)
16         assertEquals(1, decoded.height)
17         assertEquals(4, decoded.bytesPerPixel)
18         assertArrayEquals(pixels, decoded.pixels)
19     }
20
21     @Test
22     fun emptyPayloadMeansNewTransparentLayer() {
23         assertNull(LayerTileCodec.decode(ByteArray(0)))
24     }
25 }
```

141. app/src/test/java/com/wetinknext/engine/core/UndoCompressionPipelineTest.kt

```

1 package com.wetinknext.engine.core
2
3 import com.wetinknext.engine.undo.IdentityTileCompressor
4 import com.wetinknext.engine.undo.RawTileSnapshot
5 import com.wetinknext.engine.undo.TileCompressor
6 import com.wetinknext.engine.undo.TileCoord
7 import com.wetinknext.engine.undo.UndoManager
8 import org.junit.Assert.assertEquals
9 import org.junit.Assert.assertFalse
10 import org.junit.Assert.assertTrue
11 import org.junit.Test
12 import java.util.concurrent.CountDownLatch
13 import java.util.concurrent.Executors
14 import java.util.concurrent.TimeUnit
15 import java.util.concurrent.atomic.AtomicInteger
16
17 class UndoCompressionPipelineTest {
18     @Test
19     fun synchronousFallbackUsesTheSameOrderedHistoryPath() {
20         val manager = UndoManager()
21         val pipeline = UndoCompressionPipeline(
22             requestRender = {},
23             compressor = IdentityTileCompressor,
24             maxPendingJobs = 1,
25         )
26
27         pipeline.enqueue(
28             undoManager = manager,
29             layerId = 7L,
30             beforeRaw = listOf(raw(byteArrayOf(1, 2, 3, 4))),
31             afterRaw = listOf(raw(byteArrayOf(5, 6, 7, 8))),
32         )
33
34         assertEquals(0, pipeline.pendingCount)
35         assertEquals(1, manager.undoCount)
36         assertEquals(0, manager.redoCount)
37         assertTrue(manager.canUndo)
38         pipeline.shutdown()
39     }
40
41     @Test
42     fun laterWorkerResultWaitsForTheEarlierSequence() {
43         val firstCompressionStarted = CountDownLatch(1)
44         val allowFirstCompression = CountDownLatch(1)
45         val secondResultReady = CountDownLatch(1)
46         val allResultsReady = CountDownLatch(2)
47         val resultCount = AtomicInteger(0)
48         val executor = Executors.newFixedThreadPool(2)
49         val compressor = object : TileCompressor {
50             override fun compress(data: ByteArray): ByteArray {
51                 if (data.first() == 0.toByte()) {
52                     firstCompressionStarted.countDown()
53                     check(allowFirstCompression.await(2, TimeUnit.SECONDS))
54                 }
55                 return data
56             }
57
58             override fun decompress(data: ByteArray, expectedSize: Int): ByteArray = data
59         }
60         val manager = UndoManager()
61         val pipeline = UndoCompressionPipeline(
62             requestRender = {
63                 if (resultCount.incrementAndGet() == 1) secondResultReady.countDown()
64                 allResultsReady.countDown()
65             },
66             compressor = compressor,
67             executorFactory = { executor },
68         )
69
70         pipeline.enqueue(
71             undoManager = manager,
72             layerId = 1L,
73             beforeRaw = listOf(raw(byteArrayOf(0, 0, 0, 0))),
74             afterRaw = listOf(raw(byteArrayOf(2, 0, 0, 0))),
75         )
76         assertTrue(firstCompressionStarted.await(1, TimeUnit.SECONDS))
77
78         pipeline.enqueue(
79             undoManager = manager,
80             layerId = 2L,
81             beforeRaw = listOf(raw(byteArrayOf(1, 0, 0, 0))),
82             afterRaw = listOf(raw(byteArrayOf(3, 0, 0, 0))),
83         )
84         assertTrue(secondResultReady.await(1, TimeUnit.SECONDS))
85
86         assertFalse(pipeline.process(manager))
87         assertEquals(0, manager.undoCount)
88         assertEquals(2, pipeline.pendingCount)
89
90         allowFirstCompression.countDown()

```

```

91         assertTrue(allResultsReady.await(1, TimeUnit.SECONDS))
92
93         assertTrue(pipeline.process(manager))
94         assertEquals(2, manager.undoCount)
95         assertEquals(0, pipeline.pendingCount)
96         pipeline.shutdown()
97     }
98
99     private fun raw(bytes: ByteArray) = RawTileSnapshot(
100         coord = TileCoord(0, 0),
101         pixelLeft = 0,
102         pixelTop = 0,
103         pixelWidth = 1,
104         pixelHeight = 1,
105         bytesPerPixel = 4,
106         rawBytes = bytes,
107     )
108 }

```

142. app/src/test/java/com/wetinknext/engine/core/UndoHistoryTest.kt

```
1 package com.wetinknext.engine.core
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertFalse
5 import org.junit.Assert.assertTrue
6 import org.junit.Test
7
8 class UndoHistoryTest {
9     @Test
10     fun staleResultIsRejected() {
11         assertFalse(isUndoResultCurrent(resultEpoch = 3L, currentEpoch = 4L))
12     }
13
14     @Test
15     fun failedCompressionDoesNotLeavePendingCountLocked() {
16         var pending = 1
17         val result: UndoJobResult =
18             UndoJobResult.Failed(epoch = 1L, sequence = 0L, error = IllegalStateException())
19
20         pending = (pending - 1).coerceAtLeast(0)
21
22         assertEquals(0, pending)
23         assertTrue(result is UndoJobResult.Failed)
24     }
25 }
```

143. app/src/test/java/com/wetinknext/engine/core/ViewTransformFboTest.kt

```
1 package com.wetinknext.engine.core
2 import org.junit.Assert.assertEquals
3 import org.junit.Test
4 class ViewTransformFboTest { @Test fun orthoValues(){val m=FloatArray(16);ViewTransform.buildCanvasToFbo(100f,200f,m);
assertEquals(.02f,m[0],.0001f);assertEquals(.01f,m[5],.0001f);assertEquals(-1f,m[12],.0001f);assertEquals(-1f,m[13],.0001f)} }
```

144. app/src/test/java/com/wetinknext/engine/core/ViewTransformTest.kt

```

1 package com.wetinknext.engine.core
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class ViewTransformTest {
7     @Test fun canvasAndScreenCoordinatesAreInverse() {
8         val transform = ViewTransform(120f, 80f, 2.5f, 0.35f, true)
9         val screen = FloatArray(2)
10        val canvas = FloatArray(2)
11        transform.canvasToScreen(240f, 310f, screen)
12        transform.screenToCanvas(screen[0], screen[1], canvas)
13        assertEquals(240f, canvas[0], 0.001f)
14        assertEquals(310f, canvas[1], 0.001f)
15    }
16
17    @Test fun zoomAroundAnchorKeepsAnchorStable() {
18        val initial = ViewTransform(translateX = 20f, translateY = 30f)
19        val before = FloatArray(2)
20        val after = FloatArray(2)
21        initial.screenToCanvas(400f, 300f, before)
22        initial.zoomAround(400f, 300f, 3f).screenToCanvas(400f, 300f, after)
23        assertEquals(before[0], after[0], 0.001f)
24        assertEquals(before[1], after[1], 0.001f)
25    }
26
27    @Test fun gestureDeltaKeepsTheAnchorStableWithoutPan() {
28        val initial = ViewTransform(translateX = 30f, translateY = -10f, scale = 1.4f, rotationRad = .2f)
29        val before = FloatArray(2)
30        val after = FloatArray(2)
31        initial.screenToCanvas(400f, 300f, before)
32        initial.applyGestureDelta(
33            panX = 0f,
34            panY = 0f,
35            anchorX = 400f,
36            anchorY = 300f,
37            zoomFactor = 1.8f,
38            rotationDelta = .3f,
39        ).screenToCanvas(400f, 300f, after)
40        assertEquals(before[0], after[0], .001f)
41        assertEquals(before[1], after[1], .001f)
42    }
43 }

```

145. app/src/test/java/com/wetinknext/engine/thumbnail/ThumbnailPreviewFilesTest.kt

```
1 package com.wetinknext.engine.thumbnail
2
3 import java.nio.file.Files
4 import org.junit.Assert.assertArrayEquals
5 import org.junit.Assert.assertEquals
6 import org.junit.Test
7
8 class ThumbnailPreviewFilesTest {
9     @Test
10     fun restoredLayerPreviewKeepsItsBytesAndStablePath() {
11         val directory = Files.createTempDirectory("wetink-preview-test").toFile()
12         try {
13             val bytes = byteArrayOf(0x52, 0x49, 0x46, 0x46, 0x01)
14
15             val restored = ThumbnailPreviewFiles.restoreLayerPreviews(
16                 outputDirectory = directory,
17                 previews = mapOf(42L to bytes),
18             )
19
20             val preview = requireNotNull(restored[42L])
21             assertEquals("42.webp", preview.name)
22             assertArrayEquals(bytes, preview.readBytes())
23         } finally {
24             directory.deleteRecursively()
25         }
26     }
27 }
```

146. app/src/test/java/com/wetinknext/engine/undo/TileCoordinateContractTest.kt

```
1 package com.wetinknext.engine.undo
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class TileCoordinateContractTest {
7     @Test
8     fun topCanvasTileStartsAtBottomGlRegion() {
9         val canvasTop = 0
10        val glBottom = canvasTop
11        assertEquals(0, glBottom)
12    }
13
14    @Test
15    fun bottomCanvasTileUsesItsCanvasYAsGlY() {
16        val canvasTop = 1792
17        val glBottom = canvasTop
18        assertEquals(1792, glBottom)
19    }
20 }
```

147. app/src/test/java/com/wetinknext/engine/undo/TileSnapshotTest.kt

```
1 package com.wetinknext.engine.undo
2
3 import org.junit.Assert.assertArrayEquals
4 import org.junit.Assert.assertEquals
5 import org.junit.Test
6
7 class TileSnapshotTest {
8     @Test
9     fun rawIdentityTileKeepsItsExactBytes() {
10         val bytes = byteArrayOf(0, 10, 20, 30, 40, 50, 60, 70)
11         val tile = TileSnapshot(
12             coord = TileCoord(2, 3),
13             pixelLeft = 512,
14             pixelTop = 768,
15             pixelWidth = 1,
16             pixelHeight = 2,
17             bytesPerPixel = TileSnapshot.BYTES_PER_PIXEL_RGBA8,
18             storedData = bytes,
19         )
20
21         assertEquals(bytes.size, tile.memorySize)
22         assertEquals(bytes.size, tile.rawSize)
23         assertArrayEquals(bytes, tile.decompress())
24     }
25
26     @Test
27     fun tileSizeIsTheSpecified256Pixels() {
28         assertEquals(256, TileSnapshot.TILE_SIZE)
29     }
30 }
```

148. app/src/test/java/com/wetinknext/engine/undo/UndoManagerTest.kt

```

1 package com.wetinknext.engine.undo
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Assert.assertFalse
5 import org.junit.Assert.assertNotNull
6 import org.junit.Assert.assertSame
7 import org.junit.Assert.assertTrue
8 import org.junit.Test
9
10 class UndoManagerTest {
11     @Test
12     fun undoAndRedoMoveTheSameEntryBetweenStacks() {
13         val manager = UndoManager()
14         val entry = entry(1L)
15         manager.push(entry)
16
17         assertTrue(manager.canUndo)
18         assertFalse(manager.canRedo)
19         assertSame(entry, manager.peekUndo())
20         assertTrue(manager.commitUndo(entry))
21         assertFalse(manager.canUndo)
22         assertTrue(manager.canRedo)
23         assertSame(entry, manager.peekRedo())
24         assertTrue(manager.commitRedo(entry))
25         assertTrue(manager.canUndo)
26         assertFalse(manager.canRedo)
27     }
28
29     @Test
30     fun aNewEntryClearsRedoHistory() {
31         val manager = UndoManager()
32         val first = entry(1L)
33         manager.push(first)
34         val undo = manager.peekUndo()
35         assertNotNull(undo)
36         assertTrue(manager.commitUndo(checkNotNull(undo)))
37         assertTrue(manager.canRedo)
38
39         manager.push(entry(1L))
40
41         assertFalse(manager.canRedo)
42         assertEquals(1, manager.undoCount)
43         assertEquals(0, manager.redoCount)
44     }
45
46     @Test
47     fun stepLimitEvictsTheOldestHistoryEntry() {
48         val manager = UndoManager(maxSteps = 2)
49         manager.push(entry(1L))
50         manager.push(entry(2L))
51         manager.push(entry(3L))
52
53         assertEquals(2, manager.undoCount)
54     }
55
56     @Test
57     fun rejectedCommitLeavesUndoHistoryUntouched() {
58         val manager = UndoManager()
59         val stored = entry(1L)
60         manager.push(stored)
61
62         assertFalse(manager.commitUndo(entry(2L)))
63         assertSame(stored, manager.peekUndo())
64         assertTrue(manager.canUndo)
65         assertFalse(manager.canRedo)
66     }
67
68     @Test
69     fun removingLayerPurgesOnlyThatLayersUndoAndRedoEntries() {
70         val manager = UndoManager()
71         val firstLayer = entry(1L)
72         val secondLayer = entry(2L)
73         manager.push(firstLayer)
74         manager.push(secondLayer)
75         assertTrue(manager.commitUndo(secondLayer))
76
77         manager.removeEntriesForLayer(1L)
78
79         assertFalse(manager.canUndo)
80         assertTrue(manager.canRedo)
81         assertSame(secondLayer, manager.peekRedo())
82
83         manager.removeEntriesForLayer(2L)
84
85         assertFalse(manager.canRedo)
86     }
87
88     @Test
89     fun entryKeepsItsOperationMetadata() {
90         val entry = UndoEntry(

```

```

91         layerId = 1L,
92         beforeTiles = emptyList(),
93         afterTiles = emptyList(),
94         operation = UndoOperationType.CLEAR_LAYER,
95         tag = "clear_layer",
96     )
97
98     assertEquals(UndoOperationType.CLEAR_LAYER, entry.operation)
99     assertEquals("clear_layer", entry.tag)
100 }
101
102 @Test
103 fun removingLayerEntriesDoesNotBlockOtherHistory() {
104     val manager = UndoManager()
105     val removedLayerEntry = entry(layerId = 10L)
106     val validEntry = entry(layerId = 20L)
107     manager.push(removedLayerEntry)
108     manager.push(validEntry)
109
110     manager.removeEntriesForLayer(10L)
111
112     assertTrue(manager.canUndo)
113     assertEquals(validEntry, manager.peekUndo())
114 }
115
116 @Test
117 fun fiveEntriesCanBeCommittedInOrder() {
118     val manager = UndoManager()
119     repeat(5) { index -> manager.push(entry(layerId = index.toLong())) }
120
121     assertEquals(5, manager.undoCount)
122     repeat(5) {
123         val entry = manager.peekUndo()
124         assertNotNull(entry)
125         assertTrue(manager.commitUndo(checkNotNull(entry)))
126     }
127
128     assertEquals(0, manager.undoCount)
129     assertEquals(5, manager.redoCount)
130 }
131
132 private fun entry(layerId: Long): UndoEntry = UndoEntry(
133     layerId = layerId,
134     beforeTiles = listOf(tile(0)),
135     afterTiles = listOf(tile(1)),
136 )
137
138 private fun tile(x: Int): TileSnapshot = TileSnapshot(
139     coord = TileCoord(x, 0),
140     pixelLeft = x,
141     pixelTop = 0,
142     pixelWidth = 1,
143     pixelHeight = 1,
144     bytesPerPixel = TileSnapshot.BYTES_PER_PIXEL_RGBA8,
145     storedData = arrayOf(1, 2, 3, 4),
146 )
147 }

```

149. build.gradle.kts

```
1 plugins {  
2     id("com.android.application") version "8.6.1" apply false  
3     id("org.jetbrains.kotlin.android") version "2.0.21" apply false  
4     id("org.jetbrains.kotlin.plugin.compose") version "2.0.21" apply false  
5     id("org.jetbrains.kotlin.plugin.serialization") version "2.0.21" apply false  
6 }
```

150. gradle.properties

```
1 org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
2 android.useAndroidX=true
3 android.overridePathCheck=true
4 kotlin.code.style=official
5 org.gradle.daemon=true
6 org.gradle.caching=true
```

151. gradle/wrapper/gradle-wrapper.properties

```
1 distributionBase=GRADLE_USER_HOME
2 distributionPath=wrapper/dists
3 distributionUrl=https\://services.gradle.org/distributions/gradle-8.7-bin.zip
4 networkTimeout=10000
5 validateDistributionUrl=true
6 zipStoreBase=GRADLE_USER_HOME
7 zipStorePath=wrapper/dists
```

152. gradlew

```
1 #!/bin/sh
2
3 APP_HOME=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd -P) || exit 1
4 JAVA_CMD="{JAVA_HOME:+$JAVA_HOME/bin/}java"
5 exec "$JAVA_CMD" -classpath "$APP_HOME/gradle/wrapper/gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain "$@"
```

153. gradlew.bat

```
1 @rem Minimal Gradle startup script for Windows.
2 @echo off
3 setlocal
4 set APP_HOME=%~dp0
5 if defined JAVA_HOME goto javaHome
6 java -classpath "%APP_HOME%gradle\wrapper\gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain %*
7 goto end
8 :javaHome
9 "%JAVA_HOME%\bin\java.exe" -classpath "%APP_HOME%gradle\wrapper\gradle-wrapper.jar" org.gradle.wrapper.GradleWrapperMain %*
10 :end
```

154. local.properties

```
1 sdk.dir=C:\\Users\\iick\\AppData\\Local\\Android\\Sdk
```

155. settings.gradle.kts

```
1  pluginManagement {
2      repositories {
3          google()
4          mavenCentral()
5          gradlePluginPortal()
6      }
7  }
8
9  dependencyResolutionManagement {
10     repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
11     repositories {
12         google()
13         mavenCentral()
14     }
15 }
16
17 rootProject.name = "WetInk Next"
18 include(":app")
```